

به نام آنکه جان را فکرت آموخت



معرفی درس:

طراحی پایگاه داده‌ها (۴۰۳۸۴)

مرتضی امینی

نیمسال دوم ۹۶-۹۷

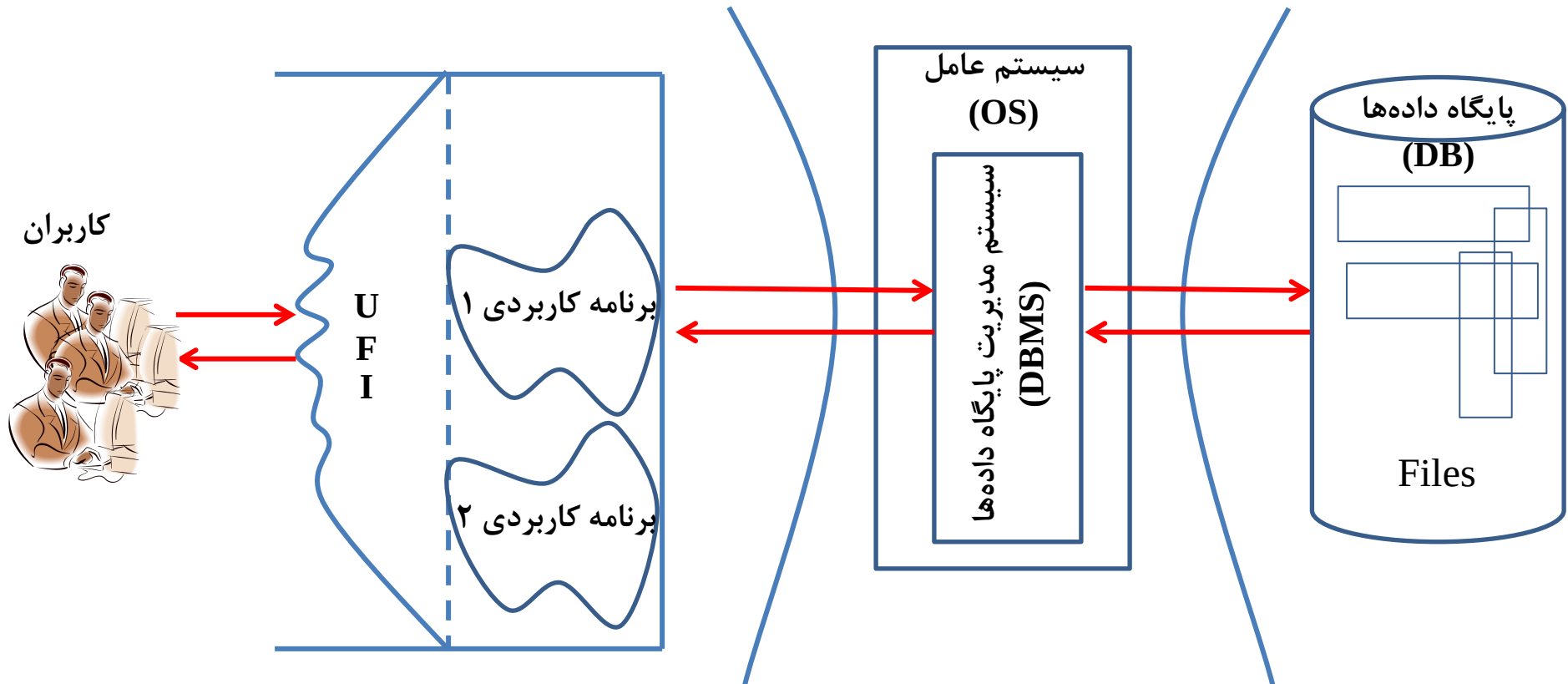
☐ نیازمند توسعه سیستم‌های اطلاعاتی یا برنامه‌های کاربردی برای استفاده از اطلاعات

☐ وجود حجم زیادی از داده‌ها و اطلاعات ذخیره شده  پایگاه داده‌ها

☐ نیازمند سیستم واسطی برای ذخیره، جستجو، بازیابی و به‌روزرسانی اطلاعات  سیستم مدیریت

پایگاه داده‌ها (سمپاد – DBMS)







❑ **سوال:** برای تولید یک سیستم پایگاهی در یک محیط عملیاتی چه باید کرد؟





امکانات مورد نیاز در ایجاد پایگاه داده‌ها

معرفی درس طراحی پایگاه داده‌ها

۵

سوال: در مدلسازی، طراحی و پیاده‌سازی پایگاه داده‌ها چه امکاناتی نیاز است؟

یک مدل داده‌ای برای طراحی منطقی
مدل رابطه‌ای و جدولی

یک روش و زبان مدلسازی داده‌ها
روش نمودار روابط موجودیت‌ها (ER)

یک زبان استاندارد برای تعریف، کنترل و انجام
عملیات پایگاهی
زبان SQL

یک زبان برای انجام عملیات
جبر رابطه‌ای و حساب رابطه‌ای

...

یک سیستم مدیریت پایگاه داده‌ها



۱- کلیات

□ تعریف پایگاه داده‌ها، مشی فایلینگ و مشی پایگاهی، عناصر محیط پایگاه داده، انواع معماری سیستم پایگاهی

۲- مدل‌سازی معنایی داده‌ها با روش ER و EER

□ نمودار ER و اجزای آن، انواع دام‌ها، تکنیک‌های تخصیص، تعمیم، تجزیه، ترکیب و تجمیع، ویژگی‌های روش مدل‌سازی معنایی

۳- آشنایی با ساختار داده‌ای جدولی (رابطه‌ای)

□ ساختار جدولی و اجزای آن، پایگاه داده جدولی و طراحی آن، زبان پایگاه داده جدولی (SQL)

۴- معماری سه سطحی پایگاه (پیشنهادی ANSI)

□ دید (نمای) ادراکی، دید داخلی، دید خارجی، تبدیلات بین سطوح، عملیات از دید خارجی و مشکلات آن، استقلال داده‌ای فیزیکی و منطقی

۵- سیستم مدیریت پایگاه داده‌ها (DBMS)

□ ریزفعالیت‌های ایجاد سیستم پایگاهی، مزایا و معایب تکنولوژی پایگاهی، وظایف، اجزا و رده‌بندی سمپاده‌ها، تیم مدیریت پایگاه داده‌ها (DBA)



۶- مفاهیم اساسی مدل داده رابطه‌ای

□ رابطه و مفاهیم مربوطه، میدان (دامنه)، انواع رابطه، رابطه‌های نرمال و غیرنرمال، انواع کلید در مدل رابطه‌ای

۷- اصول طراحی پایگاه داده‌های رابطه‌ای به روش بالا به پایین

□ تکنیک‌های تبدیل مدل‌سازی معنایی به طراحی منطقی

۸- اصول طراحی پایگاه داده‌های رابطه‌ای به روش سنتز

□ روش سنتز (نرمال‌ترسازی رابطه‌ها)، مفاهیمی از تئوری وابستگی، شرح فرم‌های نرمال، تجزیه مطلوب

۹- جامعیت در مدل رابطه‌ای

□ قواعد کاربری، مکانیزم‌های اعمال قواعد جامعیت کاربری، قواعد جامعیت موجودیتی و ارجاعی (C1 و C2)

۱۰- عملیات در پایگاه رابطه‌ای

□ جبر رابطه‌ای، حساب رابطه‌ای

نکته: یادگیری زبان SQL به عهده دانشجو است.

(در کلاس به شکل مختصر معرفی می‌شود)



□ مفاهیم بنیادی پایگاه داده‌ها نوشته سیدمحمدتقی روحانی رانکوهی، ویراست چهارم، ۱۳۹۰.

□ **An Introduction to Database Systems**, By C.J. Date, 8th Edition, 2003.

□ **Fundamental of Database Systems**, By R. Elmasri, 7th Edition, 2015.

□ **Database Systems**, By T. Connolly and C. Begg, 6th Edition, 2014.

□ **Database Management Systems**, By R. Ramakrishnan and J. Gehrke, 3rd Edition, 2002.

□ **Database System Concepts**, By A. Silberschartz, H.F. Korth and S. Sudarshan, 6th Edition, 2010.



- ☐ میان ترم (۶ نمره)
- ☐ پایان ترم (۷ نمره)
- ☐ تمرین‌های نظری (۴ نمره)
- ☐ پروژه عملی - مستمر در طی ترم (۳ نمره)
- ☐ کوئیزهای موردی و فعالیتهای کلاسی
- ☐ نمرات تشویقی - حل تمرینهای اضافی، انجام بهینه پروژه عملی، ...

نحوه توزیع نمرات تقریبی بوده و ممکن است تا پایان ترم بر حسب کیفیت هر یک از موارد تغییر نماید.

انتظار می‌رود دانشجویان در همه موارد حداکثر تلاش خود را داشته باشند.



پرسش و پاسخ ...

amini@sharif.edu

به نام آنکه جان را فکرت آموخت



بخش اول : مقدمه

مرتضی امینی

نیمسال دوم ۹۶-۹۷

(محتویات اسلایدها برگرفته از یادداشت‌های کلاسی استاد محمدتقی روحانی رانکوهی است.)

هر سیستم نرم‌افزاری از مجموعه‌ای از داده‌های ذخیره شده ممکن است استفاده کند.

در قالب تعدادی فایل (محیط فیزیکی ISR) در کجا؟ ← در یک سلسله مراتب حافظه

فرمت ثابت و از پیش تعیین شده دارد. well-formatted است،
 ساختمند (structured)
 نیم ساختمند (semi-structured)
 ناساختمند (un-structured)

آیا نیاز به تحمیل یک ساختار در اینها داریم؟ آیا واقعا داده ناساختمند داریم؟

انواع سیستم نرم‌افزاری:

بنیادی یا پایه (سیستم‌های عامل)

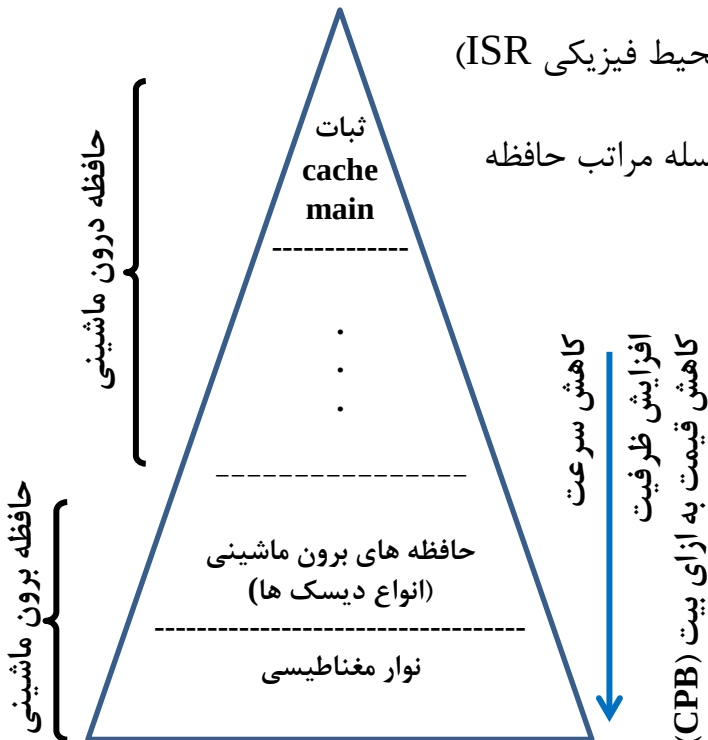
نیمه بنیادی (DBMS، DMS، کامپایلرها، اسمبلرها، و ...)

کاربردی (برنامه‌های کاربردی)

ابزاری: انواع toolهای خاص منظوره

Data Management System

Database Management System



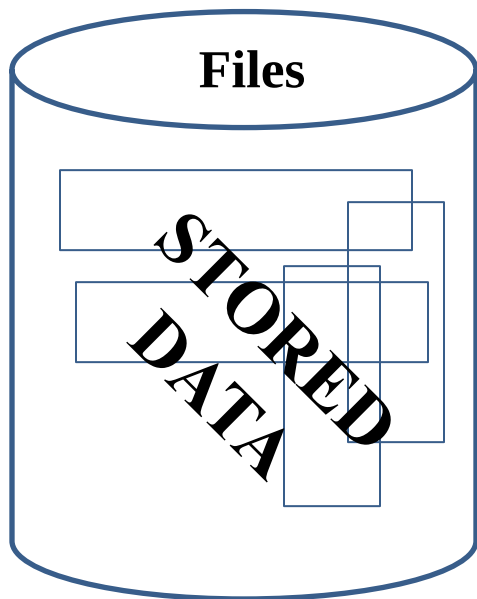


کنجکاوی: دلایل استفاده از این سلسله مراتب حافظه چیست؟

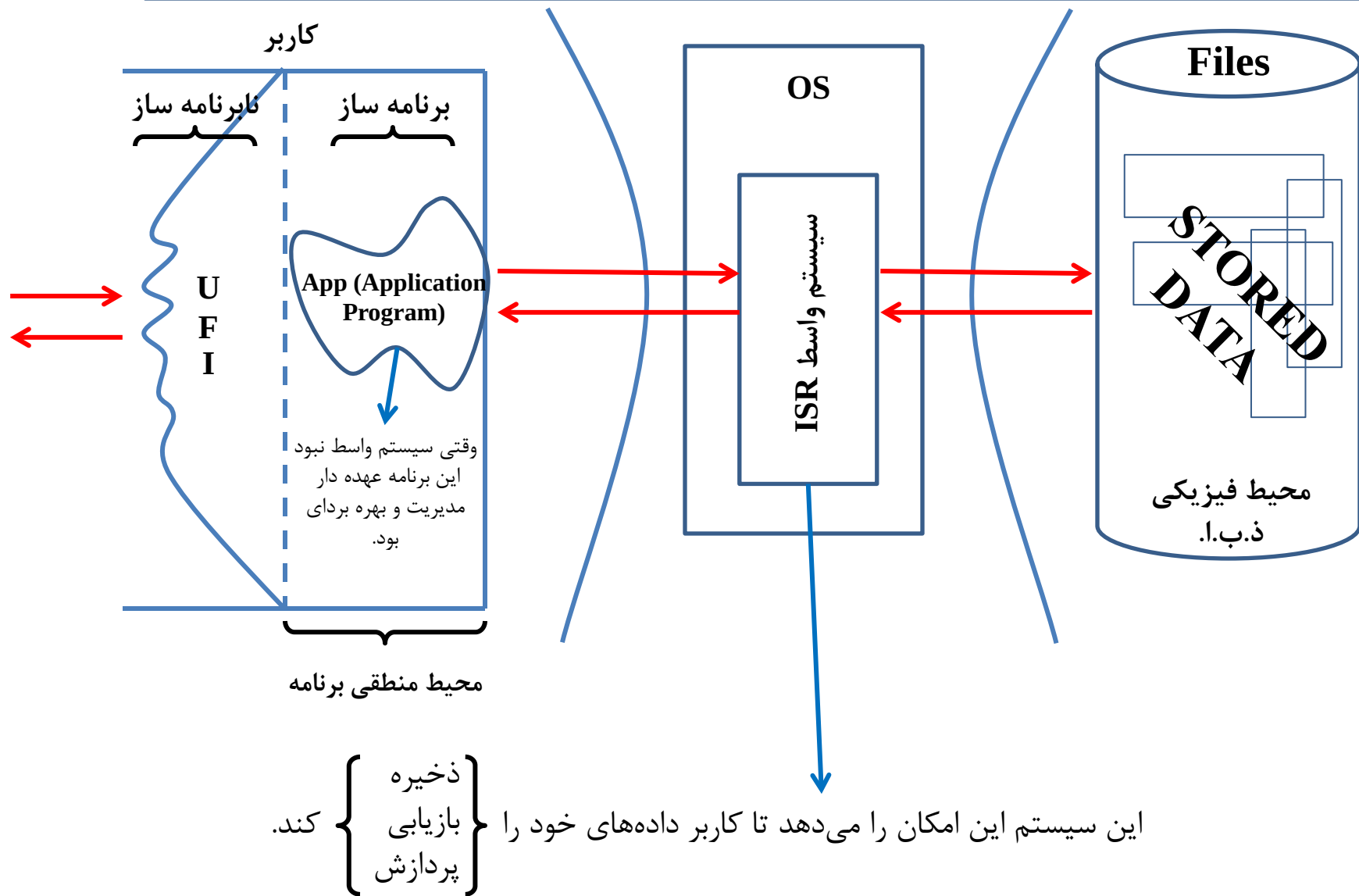
کنجکاوی: چه داده ای، برای چه مدتی، در کدامیک از مراتب سلسله مذکور قرار می گیرد؟

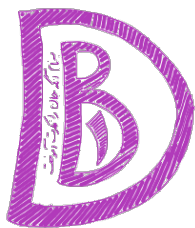
کنجکاوی: خصوصیات عمومی فایل ها چیست؟

□ محیط فیزیکی «ذ.ب.ا.» (ذخیره و بازیابی اطلاعات) یا ISR (Information Storage and Retrieval)



ISR: باید { ایجاد
مدیریت
بهره برداری } شود. ← نیاز به یک سیستم واسط ذ.ب.ا داریم.

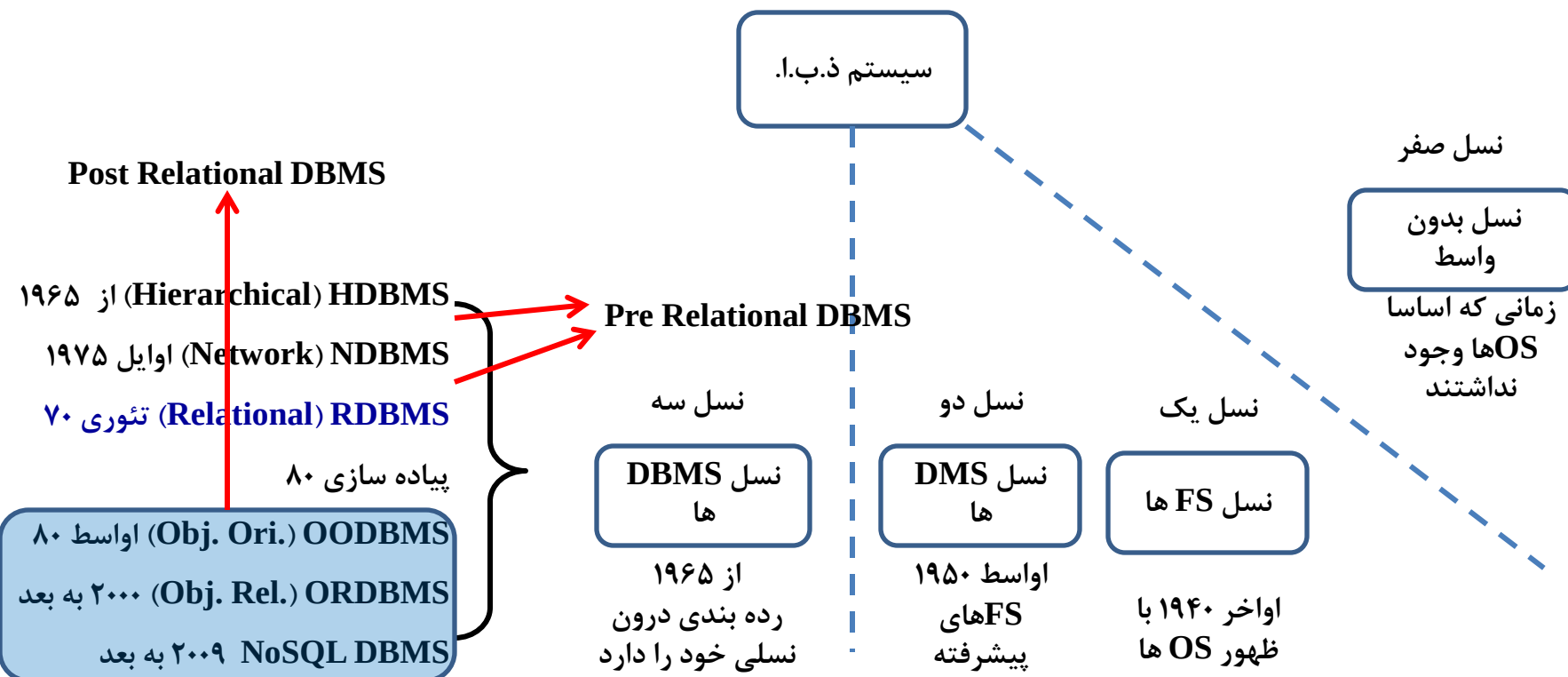


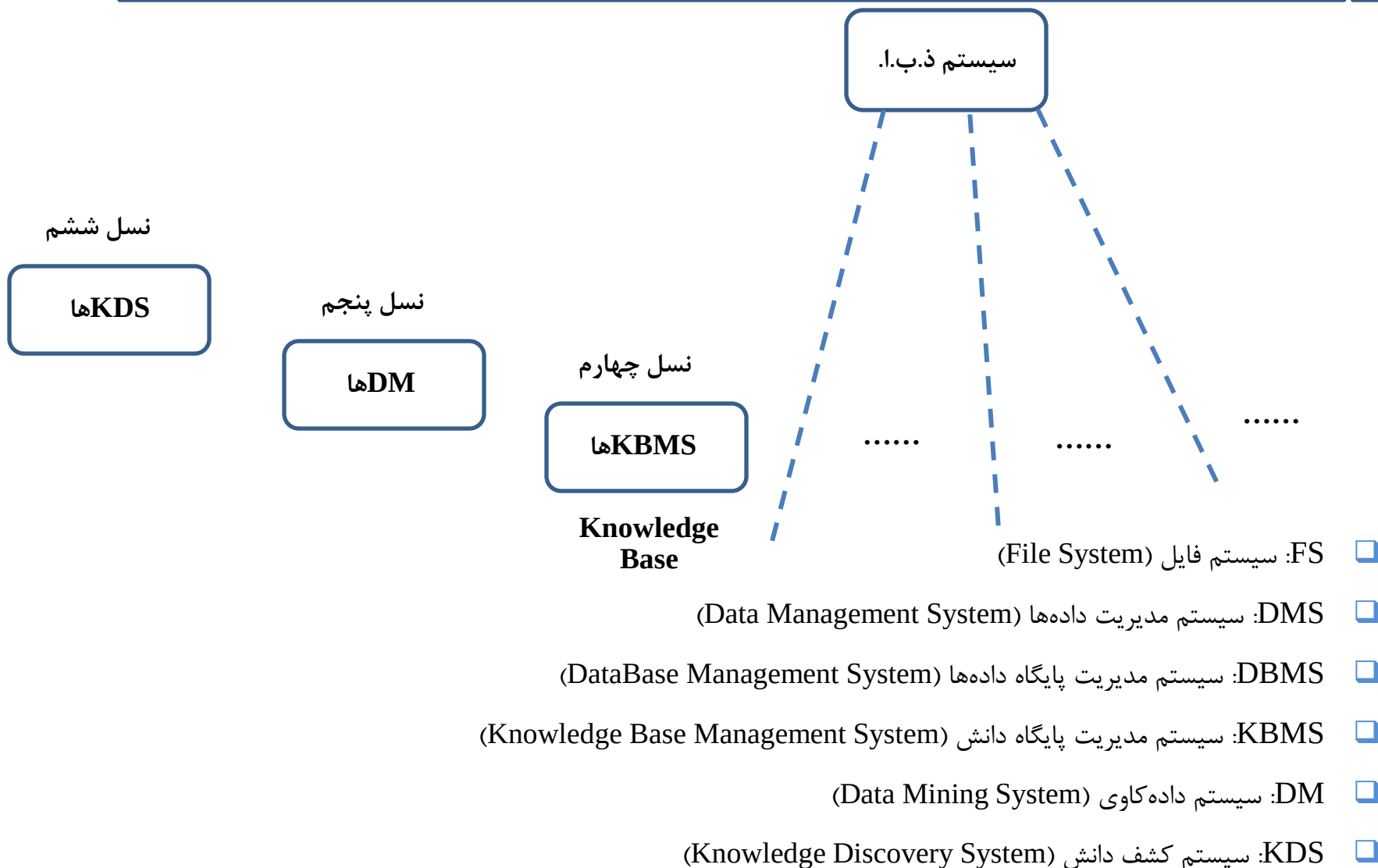
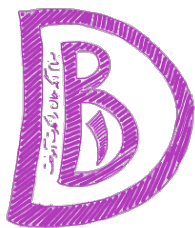


کنجکاوی: رده بندی از مفهوم کاربر ارایه کنید؟ به بیان دیگر گونه های دیگر کاربر کدامند؟

□ سیستم واسط "ISR" سیر تحول خاص خود را دارد :

□ ۶ نسل تکنولوژیک قابل بازیابی است (به طور کلی) [دیدگاه نرم افزاری]







□ در این نسل‌بندی، نسل بعدی نسل قبلی را منسوخ نمی‌کند. نسل بعدی نسل قبلی را تکمیل می‌کند و از آن استفاده می‌کند.

□ انواع نیازهای پردازشی، کنترلی، و عملیاتی سبب ایجاد نسل‌های سیستم «ذ.ب.ا.» شد.



داده (Data) □

□ تعریف اول ANSI: نمایش داده‌ها، پدیده‌ها، مفاهیم یا شناخته‌ها به طرزى صوری و مناسب برای برقراری ارتباط، تفسیر یا پردازش توسط انسان یا هر امکان خودکار

□ تعریف دوم ANSI: هر نمایشی اعم از کاراکتری (نویسه‌ای) یا کمیت‌های قیاسی که معنایی به آن قابل انتساب باشد (توسط انسان یا یک مکانیسم خودکار)

اطلاع (Information) □

□ تعریف دقیق و جامعی از مفهوم اطلاع وجود ندارد.

□ تعریف اول [LIPS92]: اطلاع، داده پردازش شده است.

□ تعریف دوم [روحا ۷۸-الف]: معنایی که انسان به داده منتسب می‌کند، از طریق قراردادهای شناخته شده‌ای که در نمایش داده به کار می‌روند.

□ برخی داده را همان مقدار واقعا ذخیره شده و اطلاع را معنای آن می‌دانند. بنابراین اطلاع دارای خاصیت اطلاع‌دهندگی و ارتباط‌دهندگی است، در حالیکه داده مجرد این خاصیت را ندارد.

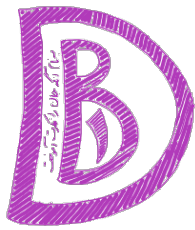


دانش (Knowledge) □

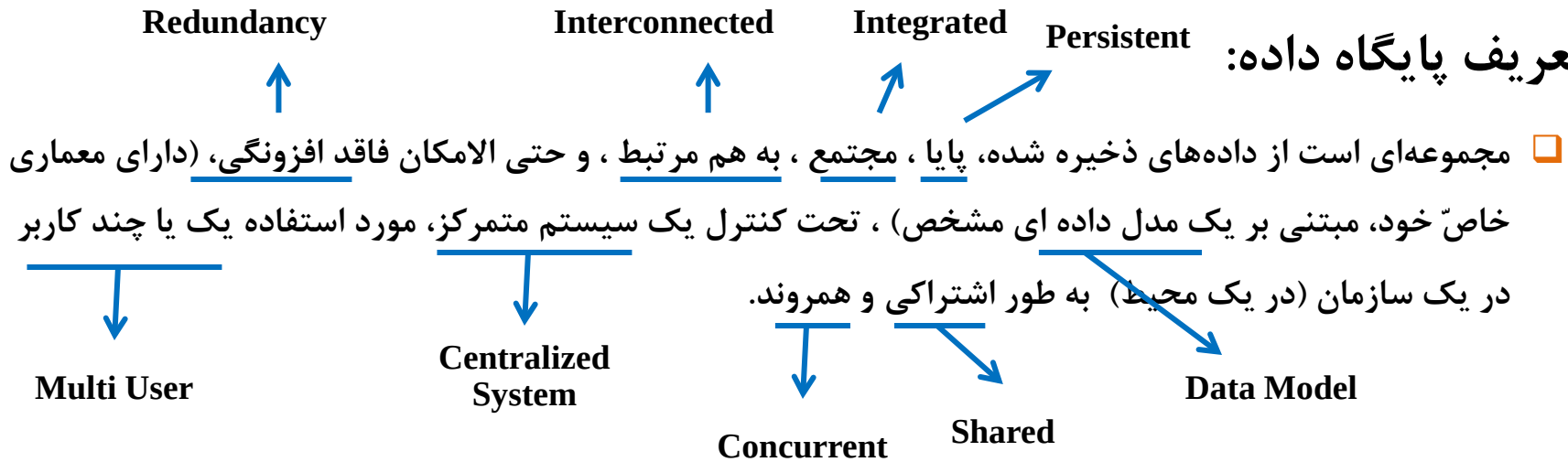
□ تعریف [FROS87]: دانش عبارت است از نمایش نمادین جنبه‌هایی از بخشی از جهان واقع (جهان موردنظر یا محیط مطرح)

▪ مثال: شنبه هوا بارانی است. حسن فرزند علی است.

□ تعریف دوم [روحا ۹۱]: دانش منطقی نوعی شناخت است که از یک مجموعه از اطلاعات بر اساس یک مجموعه از قواعد مشخص، معمولاً با روش استقراء حاصل می‌شود. حصول این شناخت می‌تواند توسط انسان یا یک سیستم خودکار انجام شود.



تعریف پایگاه داده: □



□ مثال کاربردی

□ محیط عملیاتی: دانشگاه



بخشی از جهان واقعی که قصد ایجاد سیستم برای آن را داریم.

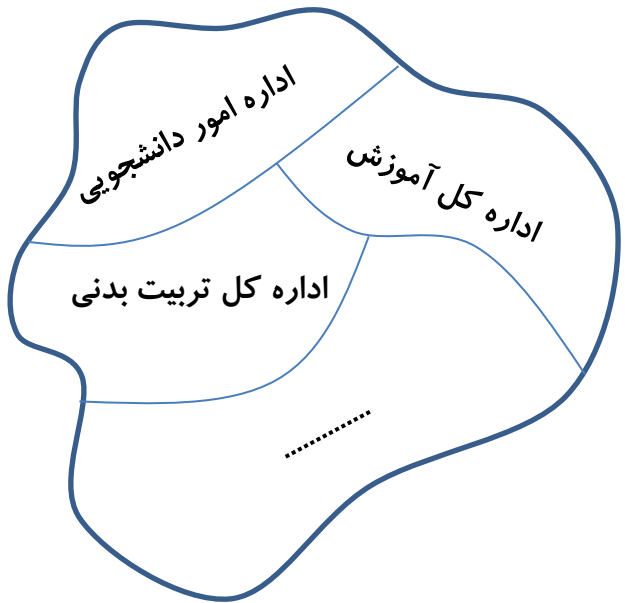


Micro Real World (خرد جهان واقع)
Mini World
Universe of Discourse (جهان مطرح)

□ نکته: هر محیط از تعدادی زیر محیط تشکیل شده است.

□ در هر محیط مجموعه‌ای از **نوع موجودیت‌ها** وجود دارند که نیازهای داده‌ای کاربران ناظر به آنهاست (یعنی به پردازی } داده‌ای

داده‌هایی در مورد آنها نیاز دارند).





❑ **نکته:** زیرمحیط‌های یک محیط معمولاً با هم اشتراک دارند در نوع موجودیت‌ها (Entity Type یا Object Type)

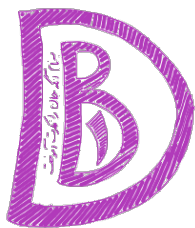
❑ مثال: در محیط دانشگاه دانشجو، استاد، درس، کلاس، و ...

❑ مثال: نوع موجودیت دانشجو در هر سه زیر محیط مطرح است.

❑ **مسئله (خواسته):** ایجاد سیستم(های) کاربردی برای این زیر محیط‌ها

❑ برای این منظور در اساس دو مَشی-روش (approach) وجود دارد. $\left. \begin{array}{l} \text{مشی فایلینگ [سنتی یا کلاسیک] یا ناپایگاهی} \\ \text{مشی پایگاهی Database Approach} \end{array} \right\}$

یعنی ممکن است مشی‌های بینابینی نیز وجود داشته باشد.



کارهای لازم در مشی فایلینگ به طور خلاصه:

توجه: این کارها معمولاً برای هر زیرمحیط به طور جداگانه انجام می شود. ← تعدادی سیستم کاربردی جدا

(نامجتمع) و بی ارتباط در یک محیط ...

- | | | |
|--|-----------------------------|--------------------------|
| ۱- مطالعه و شناخت محیط | } ← Requirement Engineering | ۱- تشخیص نیازهای داده‌ای |
| ۲- انجام مهندسی نیازها | | ۲- تشخیص نیازهای پردازشی |
| ۳- تعیین مشخصات سیستم کاربردی System Specification | | ۳- مستندسازی نیازها |
| ۴- انتخاب پیکربندی سخت افزار و نرم افزار H/S | | ۴- دریافت تایید سازمان |
| ۵- انتخاب یک FS و/یا DMS [سیستم واسطه ISR] | } | |
| ۶- طراحی تعدادی فایل (طبق مشخصات سیستم) | | |



۶-۱- تعیین فرمت رکورد

۶-۲- تعیین ساختار فایل

ساختار فایل: ساختاری که براساس آن فقره داده ها (رکوردها) در سطح منطقی [و/یا فیزیکی] با یکدیگر مرتبطند. ساختار فایل یک امکان برای نمایش ارتباط بین فقره داده‌هاست (Data Items) خواه در سطح نمایش منطقی باشد یا فیزیکی.

کنجکاوی: چند نوع ساختار فایل وجود دارد؟

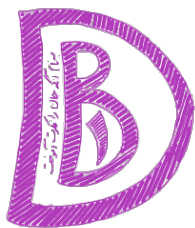
۶-۳- نحوه دسترسی به رکوردها - استراتژی دسترسی

۶-۴- اندازه فایل ها

۶-۵- میزان گسترش چه میزان باشد

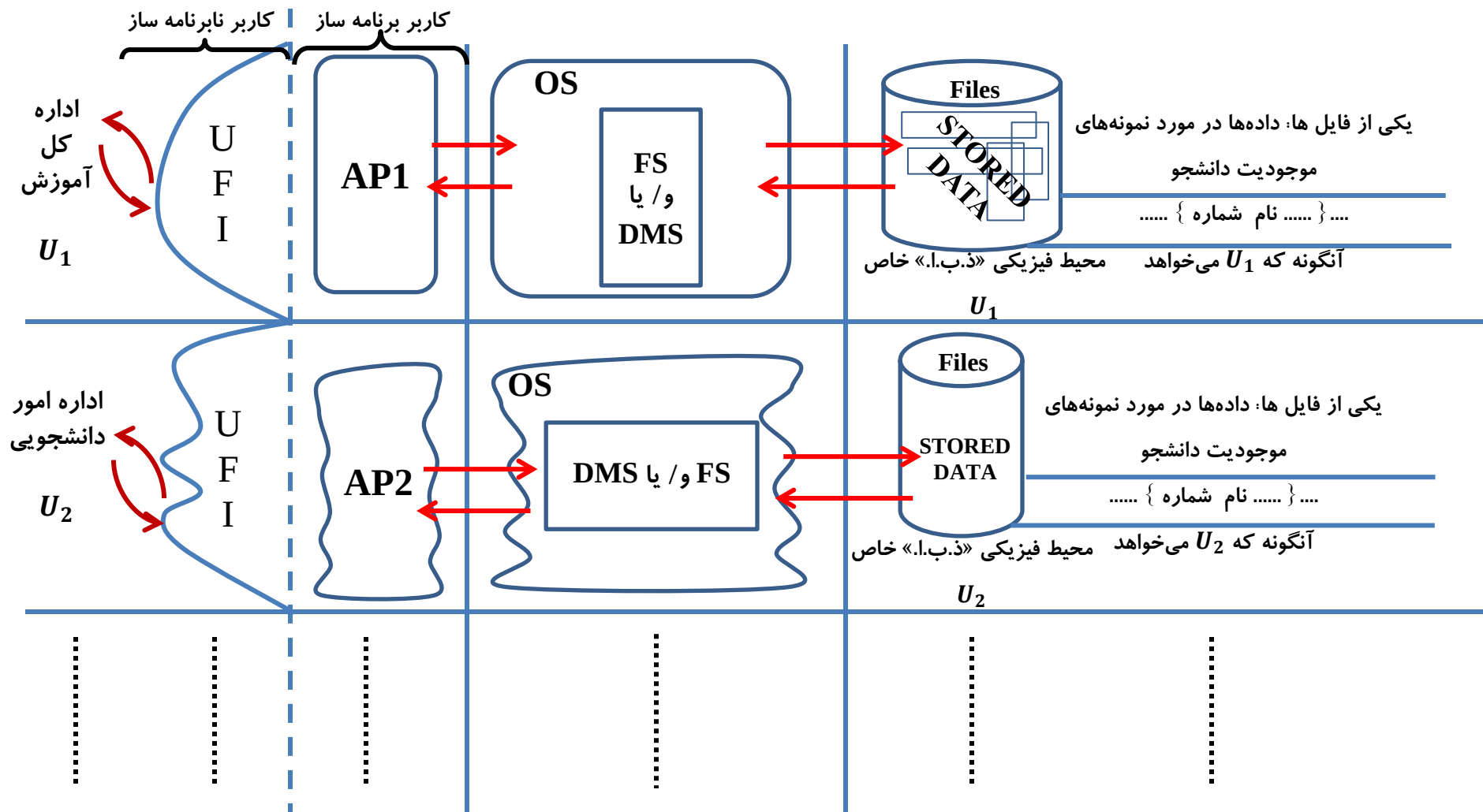
۶-۶- ارتباط با فایل های دیگر

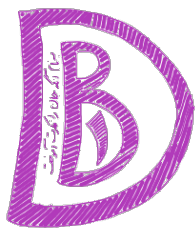
۶-۷- عملیات مجاز در فایل ها + کاربران



❑ کارهای لازم در مشی فایلینگ به طور خلاصه : (ادامه)

- ۷- طراحی واسطه‌های کاربری (UFI)
- ۸- طراحی تعدادی برنامه کاربردی (Application Program) [ضمن تعیین تراکنش(ها)]
- ۹- تولید برنامه‌های {
ایجاد
کنترل
پردازش
فایل‌ها
- ۱۰- ایجاد محیط فیزیکی «ذ.ب.ا.» به طور آزمایشی (برای داده‌های تست)
- ۱۱- ایجاد محیط فیزیکی «ذ.ب.ا.» با داده‌های واقعی و تست مرحله دوم
- ۱۲- ایجاد محیط فیزیکی واقعی با نصب، پیکربندی و ورود داده‌های اولیه (Data Entry)
- ۱۳- آغاز بهره‌برداری و نگهداری سیستم
- ۱۴- رفع معایب و بهینه‌سازی سیستم





❑ برخی از معایب مشی فایلینگ:

❑ وجود سیستم های نامجتمع در يك سازمان [محيط] و نامرتبط به هم

❑ عدم وجود يك سیستم کنترل متمرکز روی كل داده های سازمان

❑ وجود افزونگی زیاد

❑ خطر بروز ناسازگاری داده ها (Data Inconsistency) ← کنجکاوی: جنبه های بروز ناسازگاری کدامند؟

❑ عدم امکان اعمال ضوابط حفظ امنیت داده ها (Data Security)

❑ عدم امکان اشتراکی شدن داده ها (Data Sharing) [یا در حداقل و یا با دشواری]

❑ مصرف نابهبینه سخت افزار (به ویژه سخت افزار ذخیره ساز)

❑ وابسته بودن برنامه ها به جنبه های فایلینگ محیط ذخیره سازی، به گونه ای که اگر قرار باشد در فایلینگ

تغییراتی ایجاد شود، برنامه ها هم متناسباً باید تغییر یابد. (به طور مثال فرمت ساختار یا نحوه دسترسی

(Access Strategy) را تغییر دهیم)





□ افزونگی در معنای محدود (یعنی درون فایلی - intrafile redundancy - در مباحث فایلینگ)

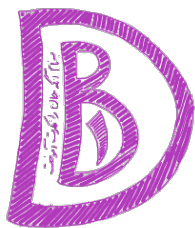
□ عبارت است از تکرار ذخیره سازی مقادیر (value) یک صفت یا بیش از یک صفت یک نوع موجودیت در فایل داده‌ای یا فایل کمکی آن.

□ این نوع افزونگی گونه‌هایی دارد:

۱- **طبیعی**: ناشی از ماهیت داده‌های محیط (مثل صفت رشته دانشجو که برای دانشجویان مختلف می‌تواند یکسان و در نتیجه تکراری باشد)

▪ **کنجکاوی**: برای کاهش مصرف حافظه در حالت افزونگی طبیعی چه باید کرد؟

۲- **تکنیکی**: ناشی از استفاده از یک تکنیک معمولا برای افزایش سرعت (مثل نمایه سازی [شاخص بندی
[Indexing])



افزونگی در معنای گسترده (یعنی برون‌فایلی - در مباحث پایگاه داده)

عبارت است از تکرار ذخیره‌سازی داده‌ها در مورد نمونه‌های یک یا بیش از یک نوع موجودیت از یک محیط.

این نوع افزونگی نه از نوع طبیعی و نه از نوع تکنیکی است بلکه ناشی از رهیافت انتخاب شده برای طراحی و تولید سیستم‌های کاربردی است.

به طور مثال تکرار اطلاعات دانشجویان در دو زیرسیستم اداره کل آموزش و زیرسیستم اداره امور دانشجویی.

نکته: افزونگی از نوع طبیعی و تکنیکی در پایگاه داده هم می‌تواند وجود داشته باشد.

دلایل بروز افزونگی در سیستم های ISR به ویژه سیستم های پایگاهی کدامند؟





تشکیل شده از تعدادی درایه (مدخل-entry)

نمایه متراکم dense

یک

یا

گروهی (به صورت

هر مدخل اشاره دارد به { از رکورد ها

چند سطحی)

نمایه نامتراکم Non-dense

آدرس مقدار

تکیه گاه (Anchor point)

مقدار یک صفت (معمولا کلید)



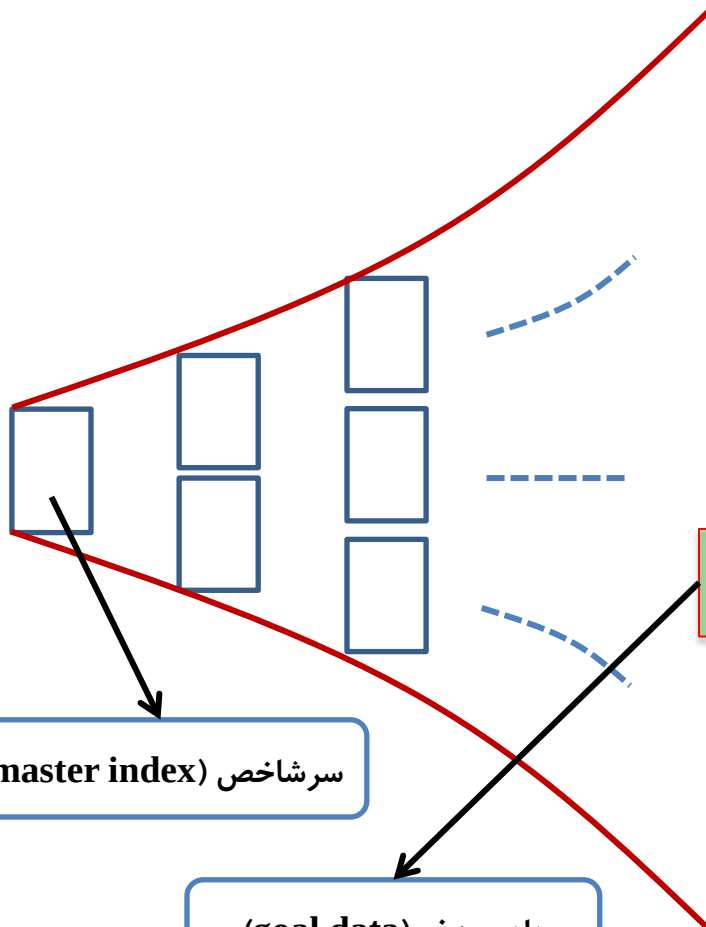
نمایه نامتراکم

نمایه متراکم

فایل نمایه سازی شده

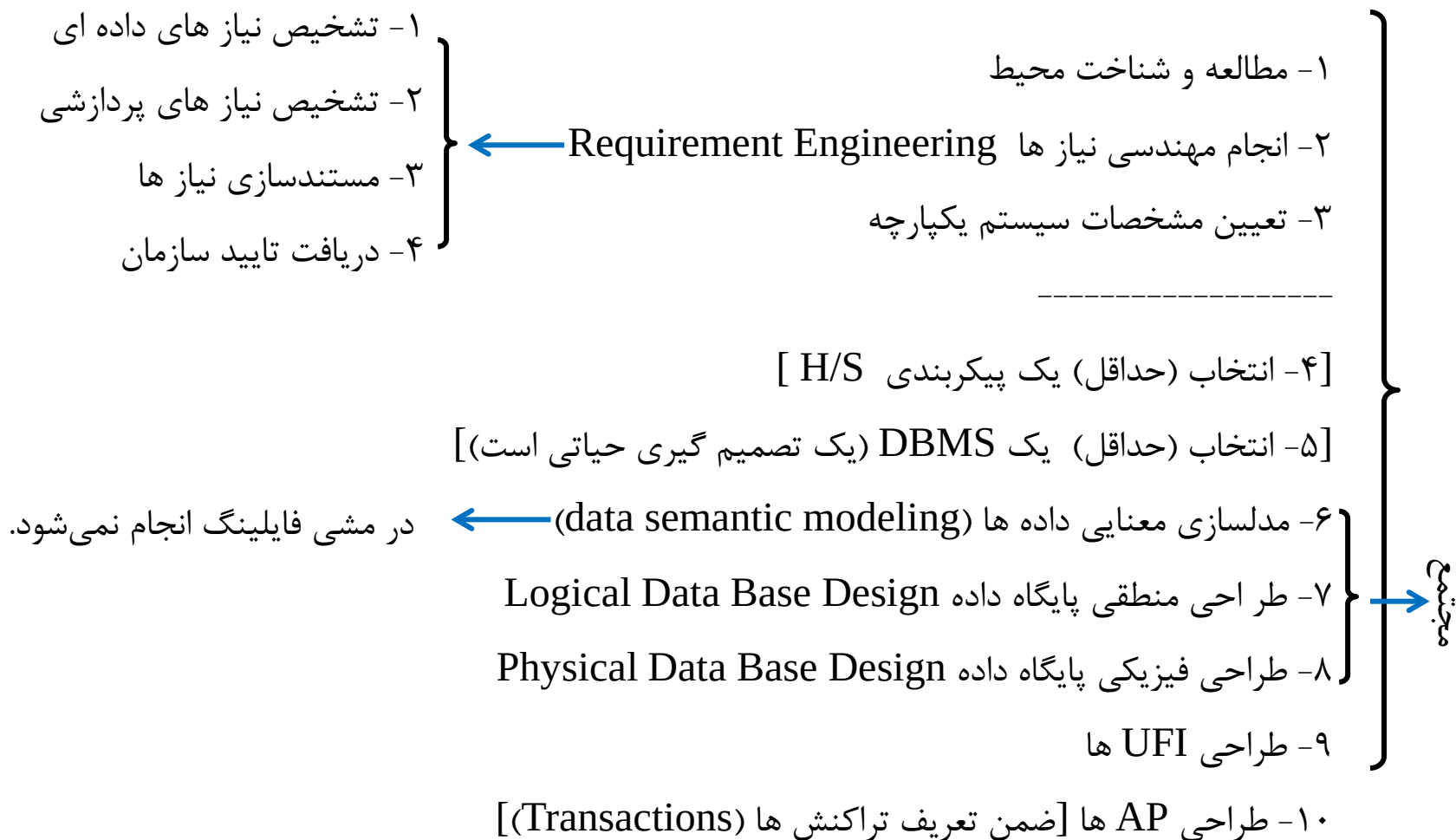
(چندسطحی معمولاً با ساختار B-Tree یا B⁺-Tree)

(زمان جستجو بالاست)



شماره	نام	رشته	...
۱۰۰	۱	نرم افزار	
۱۰۱	۲	نرم افزار	
۱۰۲	۳	سخت افزار	
۱۰۳	۴	نرم افزار	
۱۰۴	۵	سخت افزار	
۱۰۵	۶	نرم افزار	
⋮			
k		نرم افزار	
⋮			
۹۹۷	۸۹۸	سخت افزار	
۹۹۸	۸۹۹	سخت افزار	
۹۹۹	۹۰۰	نرم افزار	

□ کارهای لازم در انجام یک «پروژه پایگاهی»: (فعلا نه در جزئیات)



ادامه:....

مزایا و معایب جداسازی این دو دسته برنامه



تعریف و کنترل و عملیات در داده‌ها چیست؟

۱- از دیدگاه عملیات در داده‌ها

۲- از دیدگاه زبان‌های برنامه‌سازی

۱۱- تولید برنامه‌های تعریف (ایجاد) و کنترل DB

۱۲- تولید برنامه‌های عملیات در داده‌ها (پردازش داده‌ها)

۱۳- ایجاد محیط فیزیکی «ذ.ب.ا.» با داده‌های تستی و رفع اشکال‌ها (تست مرحله اول)

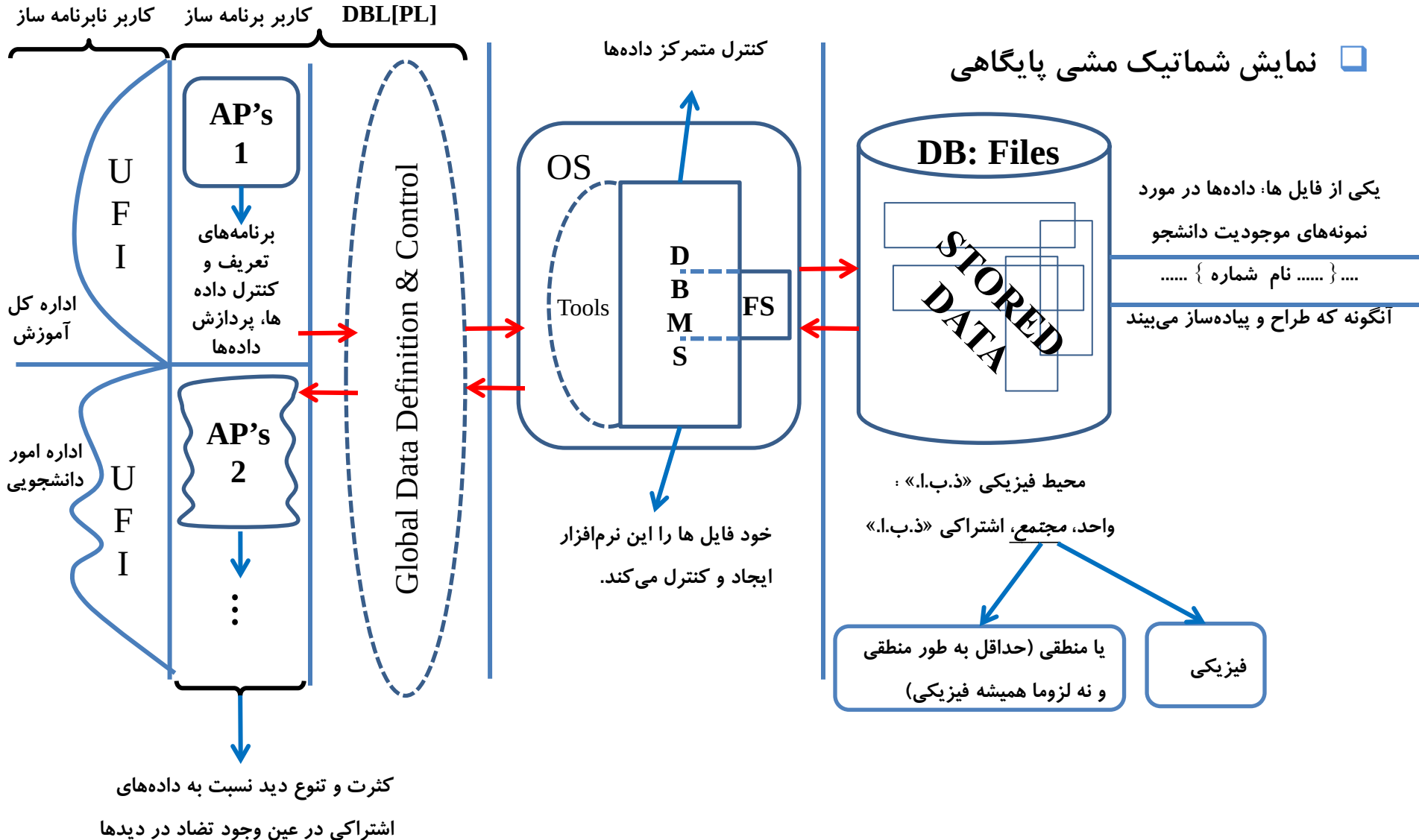
۱۴- ایجاد محیط فیزیکی «ذ.ب.ا.» با داده‌های واقعی اما حجم محدود و انجام تست مرحله دوم

۱۵- تنظیم سیستم پایگاهی (Data Base System Tuning) ← به طور مثال به منظور افزایش کارایی

۱۶- آغاز بهره‌برداری و نگهداری از سیستم

۱۷- در صورت لزوم، گسترش سیستم ← یکی از ویژگی‌های DBMS گسترش پذیری سیستم است.

۱۸- رفع معایب و بهینه‌سازی سیستم





چگونه از این کثرت دید می توان به آن «وحدت» داده رسید؟ ←

۱- خود نرم افزار DBMS

۲- معماری پایگاه داده

تمرین: مزایای مشی پایگاهی چیست؟ ← (طبق معلومات فعلی: عکس معایب مشی فایلینگ)

تمرین: چند سطح تعریف داده داریم؟



تراکنش Transaction:



□ دنباله ای از عملیات («قطعه برنامه») که معمولاً حد اقل یک عمل تغییردهنده (درج، حذف، به روزرسانی) در محیط ذخیره سازی داده ها انجام می دهد و باید یا به تمامی اجرا شود و یا اجرا نشده تلقی شود.

□ دارای خواص ACID (Atomicity Consistency Isolation Durability)

دوام (پایداری) انفراد و جدایی سازگاری یا همه یا هیچ

شرط سازگاری پایگاه داده در این مثال : $A+B$ ثابت باشد.



BEGIN TRANS

READ (A)

$A = A - 50$

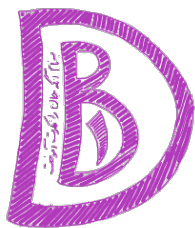
UPDATE (A)

READ (B)

$B = B + 50$

UPDATE (B)

END TRANS



❑ **اتمیک بودن (Atomicity):** یا کل تراکنش اجرا شود یا هیچ بخشی از آن اجرا نشود.

❑ اگر بخشی از یک تراکنش با شکست مواجه شود کل تراکنش شکست خورده تلقی شود و حالت پایگاه داده تغییری نکند.

❑ **سازگاری (Consistency):** هر تراکنش پایگاه داده را از یک حالت سازگار یا معتبر (منطبق با محدودیت‌های

جامعیتی تعریف شده) به یک حالت سازگار یا معتبر دیگر ببرد.

❑ معتبر بودن حالات توسط تعدادی محدودیت جامعیتی تعریف می‌شود. مثلاً مانده حساب برابر مجموع واریزها منهای مجموع برداشتها است.

❑ **جدا بودن (Isolation):** اگر چند تراکنش به صورت هم‌روند اجرا شوند، حالت پایگاه داده مشابه حالتی باشد که این

تراکنش‌ها به صورت سریال (پشت سر هم) اجرا شده باشند.

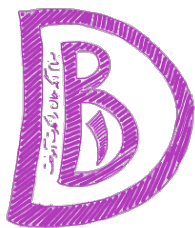
❑ **مانایی (Durability):** هر گاه که یک تراکنش خاتمه یافته اعلام شود، اثر آن بر روی پایگاه داده مانا باقی بماند

(حتی اگر سیستم دچار اختلال شود یا برق آن قطع شود یا هر مشکل دیگری پیش آید).



□ عناصر اصلی محیط پایگاهی:

- ۱- سخت افزار ←
 - ذخیره سازی
 - پردازشگر
 - ارتباطی (همرسانی) Data Communication
- ۲- نرم افزار
- ۳- کاربر
- ۴- داده



- رسانه اصلی: دیسک ترجیحا با تکنولوژی RAID
(Redundant Array of Inexpensive Disk)

- رسانه فرعی: نوار مغناطیسی [از جمله برای تولید نسخه های پشتیبان]

□ سخت افزار ذخیره سازی:

□ اغلب DBMS های امروزی تکنیک های تولید Back up را دارا هستند.



تکنیک های تولید نسخه
پشتیبان؟

سطوح مختلف Back up

- کامپیوتر های معمولی از هر رده [PC, main,...]

- اما ماشین های خاص DB هم داریم : DB Machines

□ سخت افزار پردازشگر:

- امکانات محلی: برای ارتباط دستگاه های جانبی با پردازنده

- امکانات شبکه ای: برای ایجاد شبکه در سیستم پایگاهی نامتمرکز

□ سخت افزار ارتباطی (همرسانی):

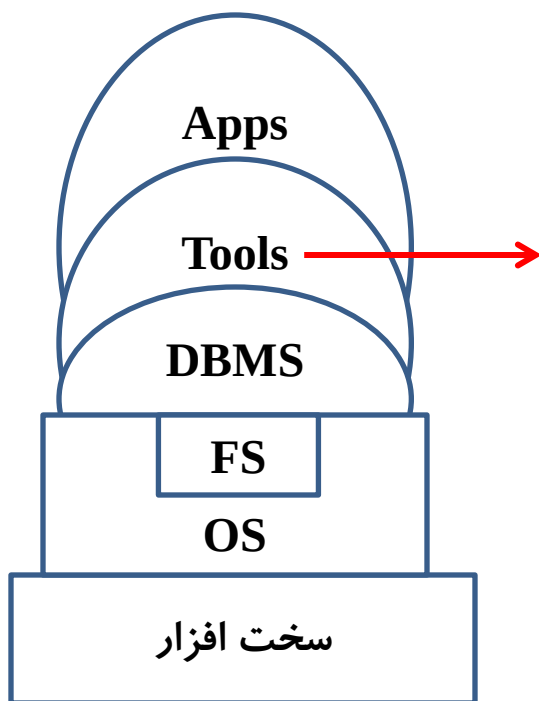
□ انواع نرم افزارهای مطرح در محیط پایگاهی:

□ سیستم عامل و سیستم فایل (FS و OS)

□ سیستم مدیریت پایگاه داده‌ها (DBMS)

□ ابزارها (Tools)

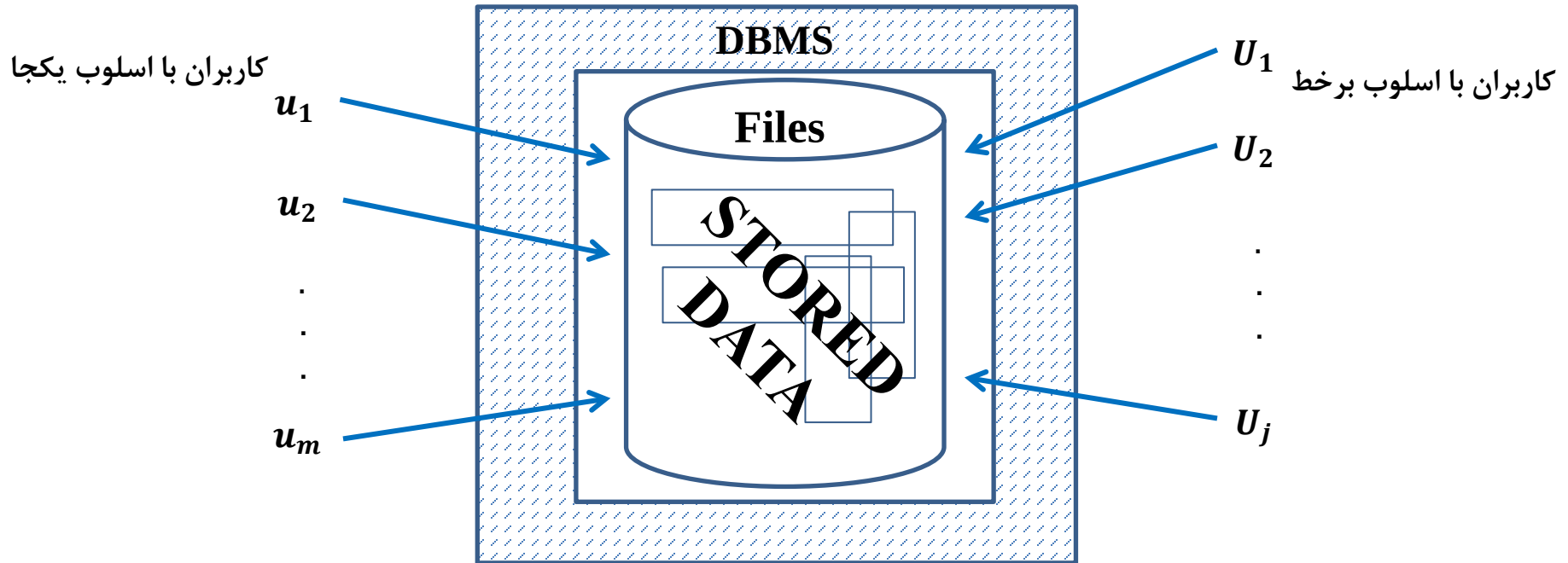
□ برنامه‌های کاربردی (Apps)

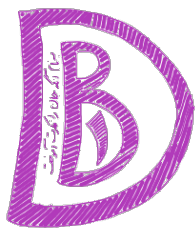


یا با خود DBMS می فروشند،
یا جداگانه خریداری می شود و به امکانات آن اضافه می شود.

→ تسهیلات نرم افزار

□ در معنای عام هر استفاده کننده از سیستم پایگاهی را **کاربر** گوئیم که انواع مختلفی دارد. کاربر می تواند انسانی و یا ماشینی باشد.





□ انواع کاربر از نظر اسلوب عملیاتی:

□ **Batch – یکجا** (تعدادی برنامه یا پرس و جو جمع آوری می شود و به صورت یکجا به سیستم داده می شود و جواب آن بر می گردد).

□ **Online – برخط – پیوسته** (یک برنامه یا پرس و جو به سیستم داده می شود، اجرا می شود، و جوابش برمی گردد).

□ **Interactive – تعاملی** – بسته به اینکه چه جوابی داده شود عمل دیگری از کامپیوتر درخواست می شود.

▪ Online لزوماً Interactive نیست اما Interactive لزوماً Online است.

□ سیستم پایگاهی به صورت پیش فرض چند کاربره (multi-user) است.



□ داده‌های ذخیره شده در یک سیستم پایگاهی عبارتند از:

□ داده‌های کاربران

□ داده‌های سیستمی

□ مباحث مرتبط با داده در محیط پایگاهی در ادامه درس مطرح می‌گردد.



☐ سوال: می‌خواهیم یک سیستم کاربردی پایگاهی ایجاد کنیم. بر اساس کدام معماری ایجاد کنیم؟

☐ در توصیف معماری یک سیستم باید مشخص کنیم که

☐ از چه مولفه‌هایی، از هر مولفه چند عدد و با چه کیفیتی تشکیل شده است،

☐ مولفه‌ها چگونه با هم ترکیب شده‌اند (جنبه ساختاری سیستم)،

☐ مولفه‌ها چگونه با یکدیگر در تعامل هستند (جنبه رفتاری سیستم).

☐ انواع معماری سیستم پایگاهی:

☐ معماری متمرکز

☐ معماری نامتمرکز

▪ معماری مشتری-خدمت‌گزار

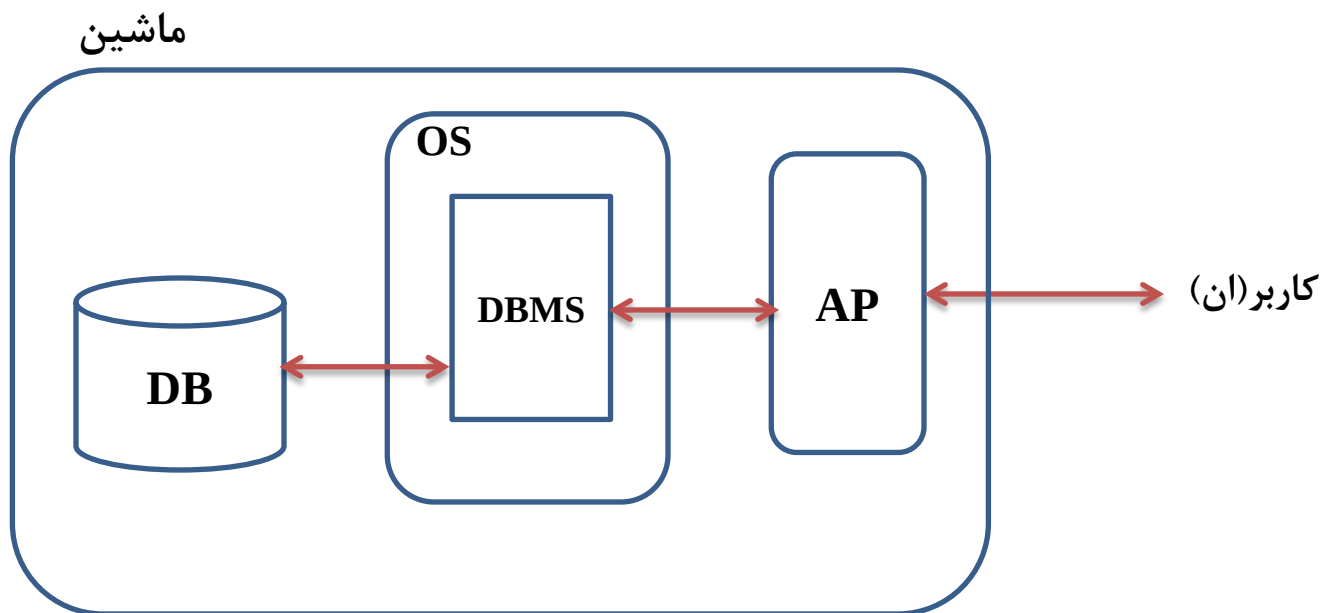
▪ معماری توزیع شده

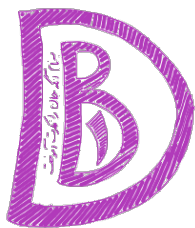
▪ معماری چندپایگاهی

▪ معماری با پردازش موازی

□ در این معماری یک پایگاه داده (متمرکز و مجتمع) روی یک سیستم کامپیوتری و بدون ارتباط با سیستم کامپیوتری دیگر ایجاد می‌شود.

□ معمولاً به صورت تک کاربری و برای کاربردهای کوچک و با امکانات محدود از این معماری استفاده می‌شود.





□ **دلیل:** دلیل اصلی استفاده از معماری مشتری-خدمتگذار (Client-Server): تقسیم وظایف سیستم

□ **تعریف:** هر ماشینی (فیزیکی یا منطقی) که خدمتی را به ماشین دیگر بدهد، **خدمتگذار** نامیده می‌شود.

نمونه‌هایی از انواع خدمتگذارها: DB Server, Message Server, Print Server, File Server



□ انواع معماری مشتری – خدمتگذار

□ معماری تک مشتری – تک خدمتگذار

□ معماری چند مشتری – تک خدمتگذار

□ معماری تک مشتری – چند خدمتگذار

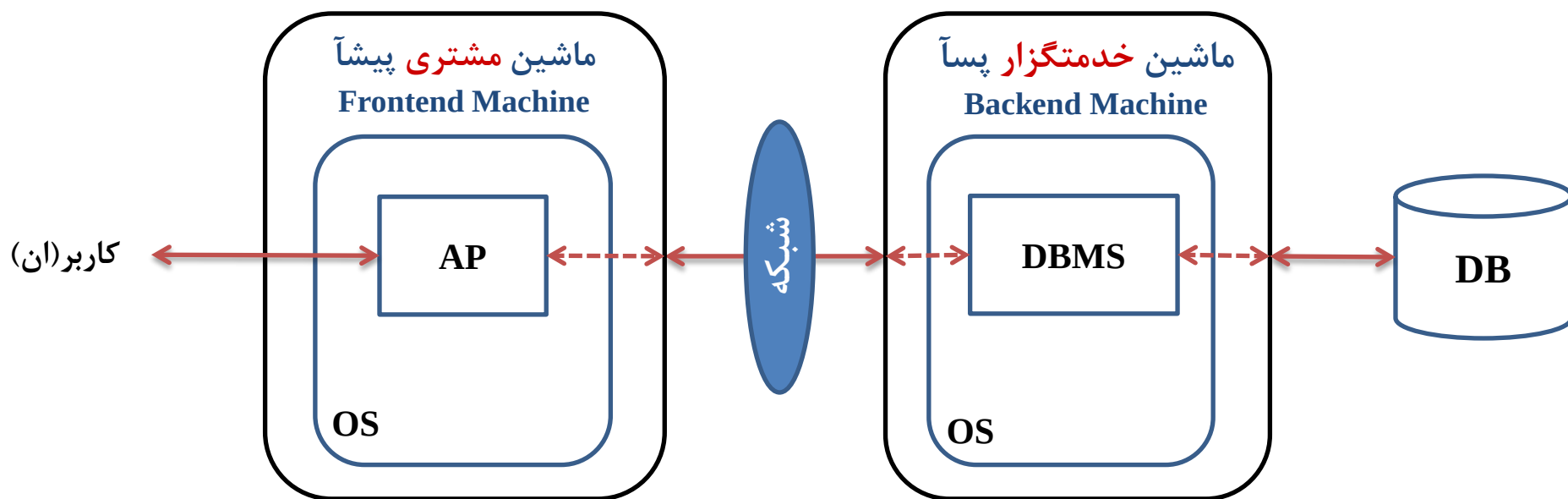
□ معماری چند مشتری – چند خدمتگذار

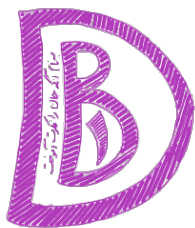
معمولا شامل دو سایت:

سایت مشتری: تمام برنامه‌های کاربردی در آن اجرا می‌شوند.

سایت خدمتگزار: تمام داده‌ها در آن ذخیره می‌شوند

به این معماری، **معماری دولایه (2-tier)** نیز گویند.





ماشین‌های ساده، ارزان و حتی بدون دیسک (thin client)

□ برخی مزایای معماری سه لایه نسبت به دو لایه:

□ گسترش پذیری بهتر

□ کارایی بالاتر

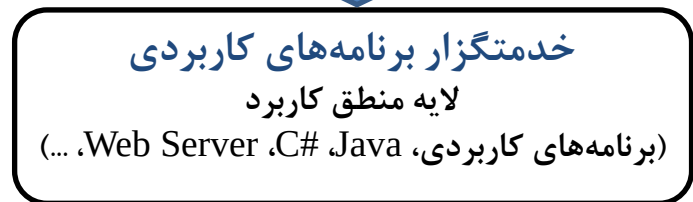
□ امنیت داده‌ای بیشتر (عدم ارتباط مستقیم مشتری‌ها با کارگزار داده)

□ قابلیت کاهش هزینه سخت افزاری (با استفاده از thin client)

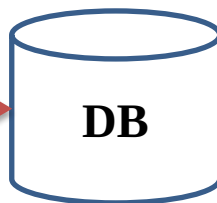
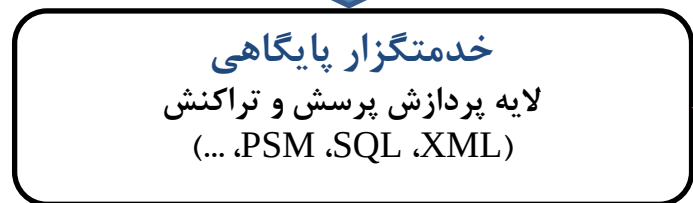
□ سهولت و هزینه کم بروزرسانی

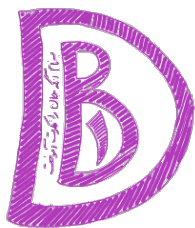


پروتکل HTTP



ODBC, JDBC, SQL, SQL/CLI





☐ سیستم‌های پایگاهی همزمان یا ناهمزمان ایجاد می‌شوند.

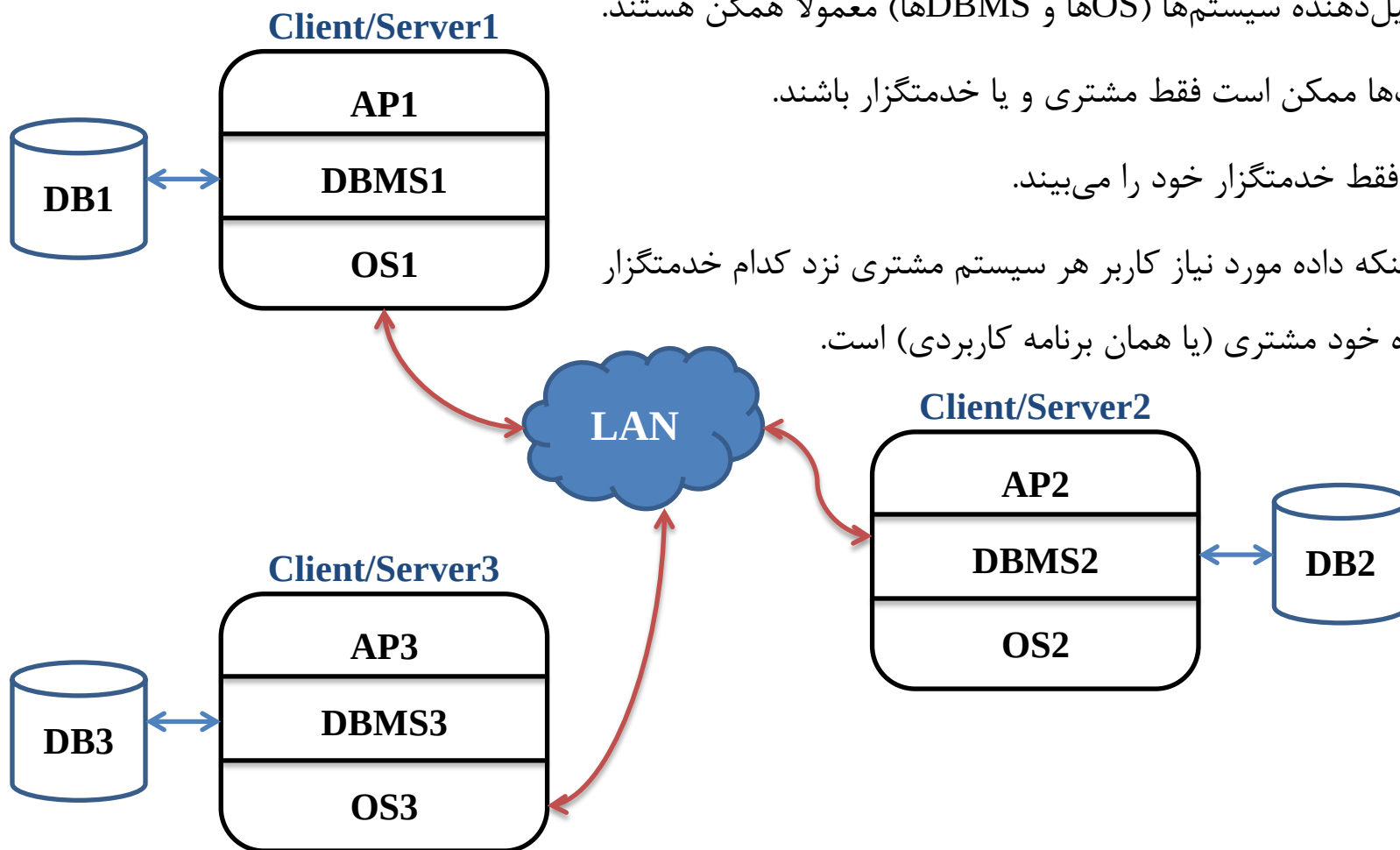
☐ اجزای تشکیل‌دهنده سیستم‌ها (OSها و DBMSها) معمولاً همگن هستند.

☐ برخی سایت‌ها ممکن است فقط مشتری و یا خدمتگزار باشند.

☐ هر مشتری فقط خدمتگزار خود را می‌بیند.

☐ مسئولیت اینکه داده مورد نیاز کاربر هر سیستم مشتری نزد کدام خدمتگزار

است برعهده خود مشتری (یا همان برنامه کاربردی) است.

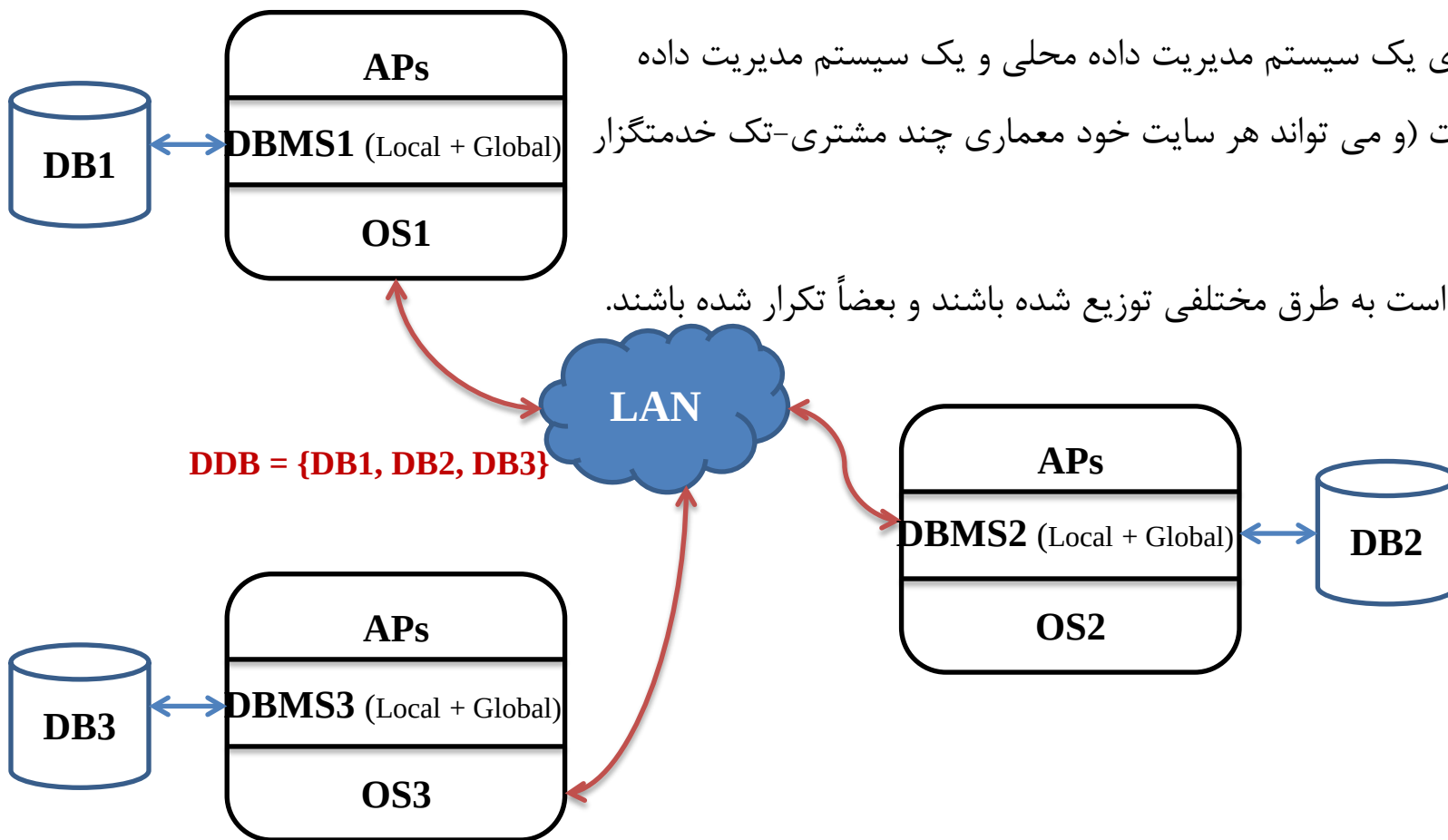


مجموعه‌ای است از چند پایگاه داده منطقاً یکپارچه (مجتمع)، ولی به طور فیزیکی توزیع شده روی یک شبکه کامپیوتری.

توزیع شدگی از دید برنامه‌ها و کاربران پایگاه داده پنهان است.

هر سایت دارای یک سیستم مدیریت داده محلی و یک سیستم مدیریت داده توزیع شده است (و می تواند هر سایت خود معماری چند مشتری-تک خدمتگذار داشته باشد).

داده‌ها ممکن است به طرق مختلفی توزیع شده باشند و بعضاً تکرار شده باشند.





پرسش و پاسخ ...

amini@sharif.edu

به نام آنکه جان را فکرت آموخت



بخش دوم : مدلسازی معنایی داده‌ها

مرتضی امینی

نیمسال دوم ۹۶-۹۷

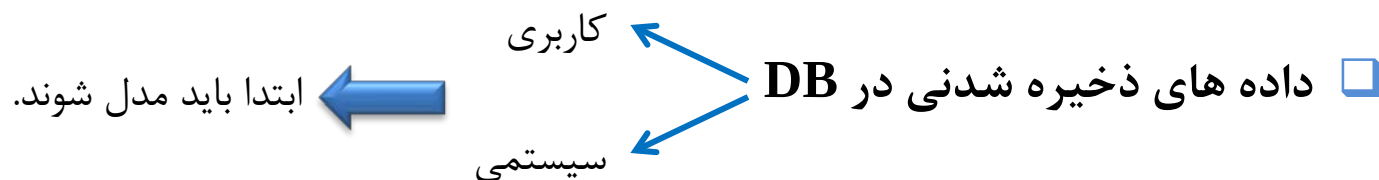
(محتویات اسلایدها برگرفته از یادداشت‌های کلاسی استاد محمدتقی روحانی رانکوهی است.)



مدلسازی معنایی داده‌ها (Semantic Data Modeling)

بخش دوم: مدلسازی معنایی داده‌ها

۲

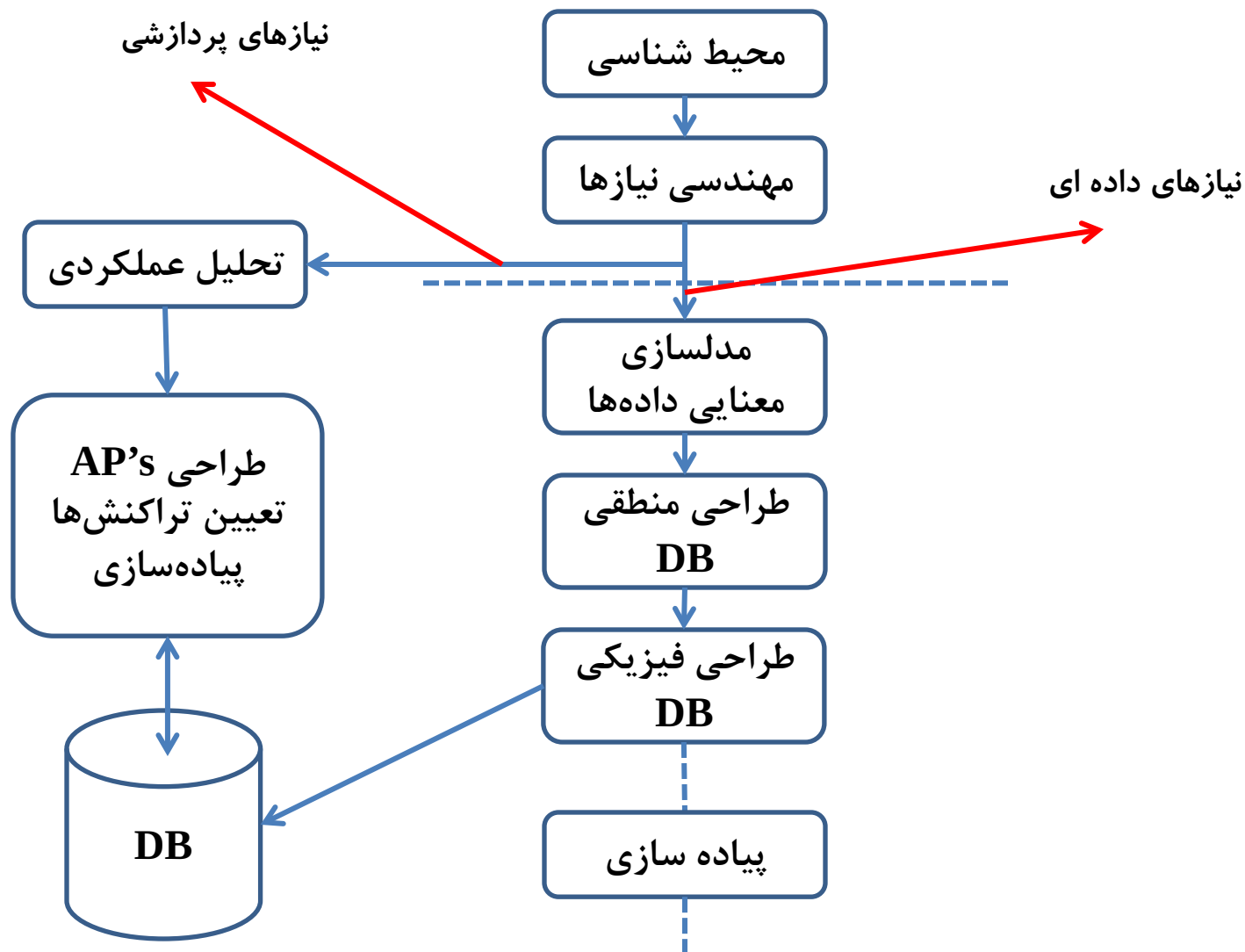


داده‌های کاربری

- موسومند به داده‌های عملیاتی (مثلاً اطلاعات دانشجویان، درس‌ها و اساتید در محیط عملیاتی دانشگاه)
- پایا هستند: بعد از اجرای برنامه کاربر کماکان در سیستم ماندگارند [حسب تعریف]
- لزوماً همان داده‌های ورودی/خروجی (I/O) نیستند. هر داده موجود در پایگاه داده لزوماً داده ورودی نیست و هر داده خروجی از پایگاه لزوماً در پایگاه ذخیره شده نیست (مانند داده‌های محاسبه شده از داده‌های موجود - مثلاً میانگین نمرات)

داده‌های سیستمی

- سیستم تولید می‌کند برای انجام وظایفش (مثلاً اطلاعات مربوط به جداول پایگاه داده و یا اطلاعات مربوط به ستونهای موجود در جداول)

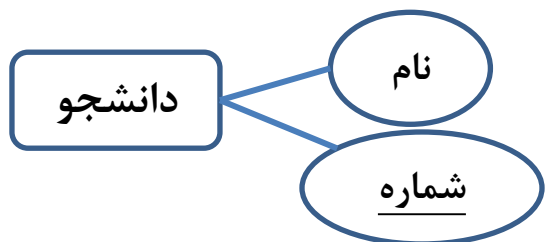




مدلسازی معنایی داده‌ها:

ارائه یک مدل کلی (در بالاترین سطح انتزاع) از داده‌های محیط با استفاده از مفاهیم انتزاعی و براساس معنایی که کاربر برای داده‌ها قائل است.

مفهوم انتزاعی: مفهومی است فراتر از سطح منطقی و طبعاً فراتر از سطح پیاده‌سازی



برای درک مفهوم انتزاع:



در سطح انتزاع

Student

<u>StudentID</u>	Name	...
V ₁	V ₂	...

در سطح منطقی

نمونه رکورد

بخش کنترلی V₁ V₂

در سطح پیاده‌سازی



□ برای مدلسازی نیاز به روش داریم:

□ روش رایج تر در دانش و تکنولوژی پایگاه داده

■ روش ER (Entity Relationship): ←
ER مبنايي }
ER گسترش یافته (Extended or Enhanced ER)

■ روش UML (Unified Modeling Language): خاص مدلسازی معنایی داده‌ها نیست بلکه برای

مدلسازی و طراحی سیستم های نرم‌افزاری است. لذا با آن می‌توان پایگاه داده را مدل کرد.



- سه مفهوم اساسی داریم:
- Entity Type نوع موجودیت
 - Attribute صفت (خصیصه - ویژگی)
 - Relationship Type نوع ارتباط

نمودار ER:

- نموداری است که موجودیت‌ها، صفات یا خصوصیات آنها و روابط بین آنها را توصیف می‌نماید.
- نموداری است که سه مفهوم اساسی نوع موجودیت، صفت و نوع ارتباط در آن نمایش داده می‌شوند.
- در واقع این نمودار امکانی است برای نمایش مدلسازی و اولین طرح پایگاه داده‌ها در بالاترین سطح انتزاع.
- برای رسم این نمودار به نمادهایی نیاز داریم. در این درس از نمادهای چن استفاده می‌شود.



بخش دوم: مدلسازی معنایی داده ها

[نام نوع موجودیت]

☐ نوع موجودیت

[نام نوع موجودیت]

☐ نوع موجودیت ضعیف

[نام نوع
ارتباط]

☐ نوع ارتباط

[نام نوع
ارتباط]

☐ نوع ارتباط موجودیت ضعیف با قوی

[نام نوع موجودیت]

[نام نوع
ارتباط]

☐ مشارکت نوع موجودیت در نوع ارتباط

[نام نوع موجودیت]

[نام نوع
ارتباط]

☐ مشارکت الزامی



نمادهای نمودار ER مبنایی (ادامه)

بخش دوم: مدلسازی معنایی داده ها

۸

[نام صفت]

صفت ☐

[نام صفت]

صفت شناسه اول ☐

[نام صفت]

صفت شناسه دوم (در صورت وجود) ☐

[نام صفت]

[نام صفت]

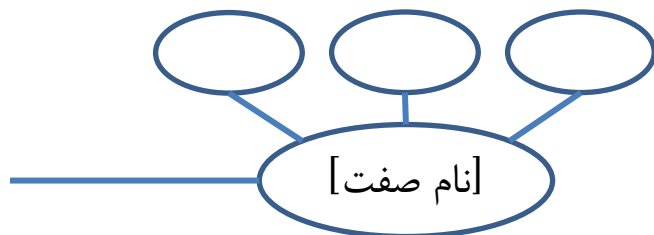
صفت شناسه مرکب (مثلا دو صفتی) ☐

[نام صفت]

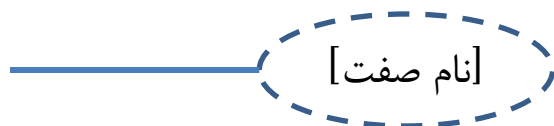
صفت چندمقداری ☐



بخش دوم: مدلسازی معنایی داده ها



☐ صفت مرکب



☐ صفت مشتق (مجازی یا محاسبه‌شدنی)



☐ چندی ارتباط



نوع موجودیت:

مفهوم کلی شیء، چیز، پدیده و به طور کلی آنچه از یک محیط که می خواهیم در موردش اطلاع داشته باشیم.

- خرد جهان واقع Micro Real World
- Mini World
- جهان مطرح (UOD) Universe of Discourse

۱- دانشجو
۲- درس
۳- استاد
۴- کارمند
...

محیط عملیاتی : دانشگاه



نوع موجودیت ها

تذکر: اولین قدم در مدلسازی معنایی تشخیص درست نوع موجودیت ها است.

در مثال فوق آیا دانشگاه یک نوع موجودیت در نظر گرفته می شود یا خیر؟

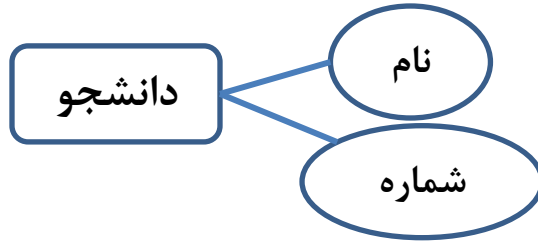




ER مبنایی – نوع موجودیت (ادامه)

۱۱

بخش دوم: مدلسازی معنایی داده ها



هر نوع موجودیت:

□ یک نام دارد.

□ یک معنا دارد.

□ مجموعه‌ای از صفات دارد (حداقل یکی).

نکته؟ در چه حالتی بهتر است نوع موجودیت تک صفتی را نوع موجودیت بگیریم؟ در چه حالتی نگیریم؟

نمونه‌هایی دارد (حداقل یک نمونه).
نکته؟ در چه حالتی نوع موجودیت تک نمونه‌ای را موجودیت در نظر می‌گیریم؟

ارتباط(هایی) با نوع موجودیت(های) دیگر دارد.
نکته؟ آیا نوع موجودیت ایزوله داریم؟

نوع موجودیت دو گونه است. ←
قوی (مستقل) Strong
ضعیف (وابسته) Weak

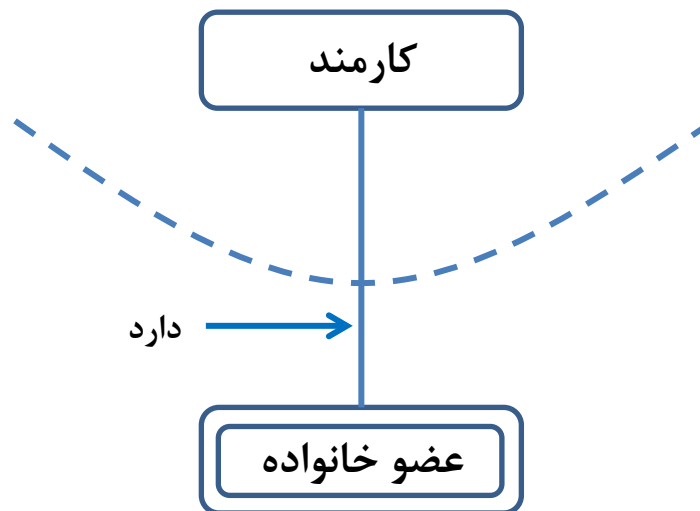


تعریف موجودیت قوی: ☐

- نوع موجودیت E را قوی گوئیم هرگاه خود مستقلاً در محیط مطرح باشد.

تعریف موجودیت ضعیف: ☐

- نوع موجودیت F را ضعیف نوع موجودیت E گوئیم هرگاه به آن «وابستگی وجودی» داشته باشد. (اگر E مطرح نباشد F هم مطرح نیست) به عبارتی F در مدلسازی دیده می شود به اعتبار E.
- تذکر: قوی و ضعیف بودن نسبی است.



عضو خانواده وابسته به نوع موجودیت کارمند است.





صفت: ☐

☐ خصیصه یا ویژگی نوع موجودیت و هر نوع موجودیت مجموعه‌ای از صفات دارد که حالت یا وضع آن را توصیف می‌کند.

☐ محیط عملیاتی: دانشگاه



☐ نوع موجودیت: درس

☐ صفات: شماره، نام، تعداد واحد، زمان برگزاری، تاریخ امتحان، نوع درس (پایه، تخصصی، اختیاری، ...)

سطح درس (کارشناسی، کارشناسی ارشد، دکترا)، ماهیت درس (نظری، عملی، ترکیبی)



بخش دوم: مدلسازی معنایی داده ها

هر صفت: ☐

☐ یک نام دارد.

☐ یک معنا دارد (معنای مشخص در حیطه معنایی مشخص).

☐ یک دامنه یا میدان (Domain) دارد.

محدودیت‌های صفت: ☐

صفت را مشخص می‌کند. و نه لزوماً نام صفت را.

نوع
طیف مقادیر

۱- محدودیت میدانی

۲- محدودیت نمایشی. **مثال:** قالب تاریخ yyyy/mm/dd

۳- محدودیت پردازشی ناشی از نوع صفت یا ناشی از قواعد محیط [غیر از آنچه ناشی از میدان است]

مثال: سن کاهش نمی‌یابد.

مثال: عدم جمع دو آدرس: محدودیت ناشی از میدان است.

۴- محدودیت وابستگی به یک صفت دیگر. **مثال:** وابستگی شمول به صفت دیگر $B\{values\} \subseteq A\{values\}$

۵- محدودیت یکتایی مقدار. **مثال:** شماره دانشجویی

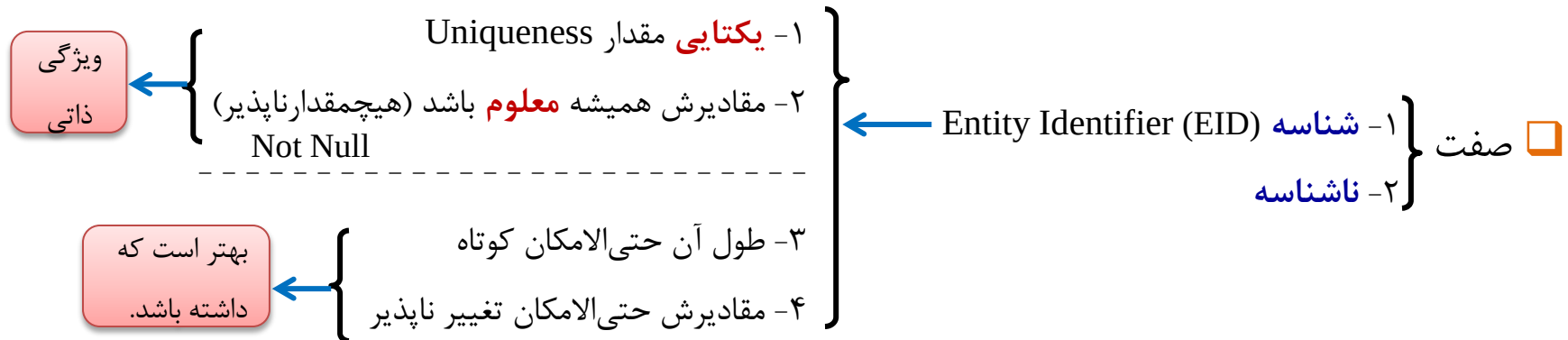
آیا صفت محدودیت‌های دیگری هم دارد؟





نمره دانشجو GR ← از دامنه Grade
 نوع : Real
 طیف مقادیر : $[0, \dots, 20]$

ردده بندی صفت: ☐



۱- **ساده – تجزیه ناپذیر**: از نظر معنایی در یک محیط مشخص – اگر صفت را تجزیه کنیم، خودِ تکه ها مقداری از صفت در آن محیط نشود. مثال: عنوان درس ☐ صفت

۲- **مرکب**: ترکیبی از چند صفت ساده (و می تواند ساختار سلسله مراتبی هم داشته باشد) مثال: آدرس (ترکیبی از استان، شهر، خیابان، ...)



 **توجه:** ساده یا مرکب بودن نسبی است و نه مطلق. بستگی به حیطه معنایی و کاربرد دارد. (مثال: آدرس از دید نشریه (ساده) یا از دید شهرداری (مرکب)).

اینکه صفت مرکب را در یک فیلد ذخیره کنیم یا اجزا را در فیلدهای مجزا به چه عواملی بستگی دارد؟



 **صفت**

۱- **تک مقداری:** به ازای یک نمونه از نوع موجودیت E، حداکثر یک مقدار می گیرد. **مثال:** نام درس

۲- **چند مقداری:** حداقل برای یک نمونه از نوع موجودیت E، بیش از یک مقدار. **مثال:** شماره تلفن استاد

 **توجه**

ساده – تک مقداری
مرکب – تک مقداری
ساده – چند مقداری
مرکب – چند مقداری

 **صفت**

۱- **هیچمقدار پذیر (Nullable یا Nullvalue):** مقدار صفت می تواند ناشناخته، ناموجود، تعریف نشده یا غیر قابل اعمال باشد. **مثال:** شماره تلفن دانشجو

۲- **هیچمقدار ناپذیر (Not nullable):** حتما مقدار صفت برای هر نمونه موجودیت باید معلوم باشد. **مثال:** شماره درس

مشکلات هیچمقدار؟ package ها با آن چه برخوردی دارند؟





- صفت 
- ۱- واقعی (Real): مقدار ذخیره شده در DB دارد. مثال: نمره درس
 - ۲- مجازی - مشتق (Virtual): مقدار ذخیره شده در DB ندارد، سیستم با پردازشی معمولاً محاسبه و مقدارش را در اختیار کاربر قرار می دهد. مثال: میانگین نمرات درس

 **تذکر:** اگر صفتی ماهیت محاسبه شوندگی داشته باشد لزوماً مجازی نیست و ممکن است برای افزایش سرعت و در صورتی که بسامد (فرکانس) ارجاع زیاد باشد مقدار ذخیره شده داشته باشد.



نوع ارتباط Relationship Type:

رابطه، اندرکنش و یا تعامل بین $N \geq 1$ نوع موجودیت $\leftarrow N = 1$ ارتباط با خود - بازگشتی (self-relationship) 

ارتباط نوع موجودیت‌های دانشجو و درس



▪ دانشجو درس را **انتخاب** می‌کند.

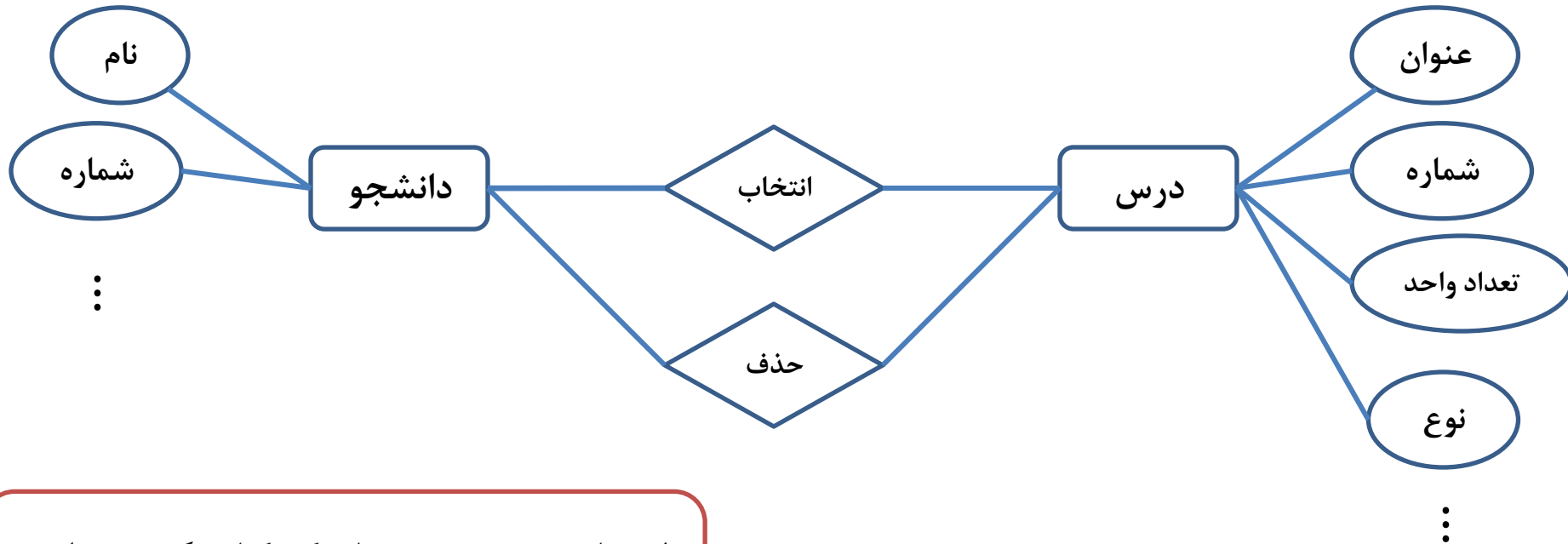
▪ دانشجو درس را **حذف** می‌کند.



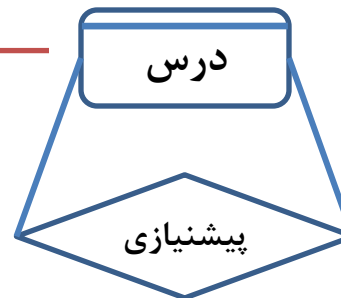
ER مبنایی - نوع ارتباط (ادامه)

۱۹

بخش دوم: مدلسازی معنایی داده ها



طرز نمایش نوع موجودیت زمانی که یکبار دیگر در نمودار ER آمده باشد. (به خاطر اجتناب از شلوغ شدن نمودار)



ارتباط موجودیت با خود : مثال



مفهوم پیشنیازی درس را به چند روش دیگر می توان مدل کرد؟





ER مبنایی – نوع ارتباط (ادامه)

۲۰

بخش دوم: مدلسازی معنایی داده ها

نوع ارتباط:

یک نام دارد.

یک معنا دارد.

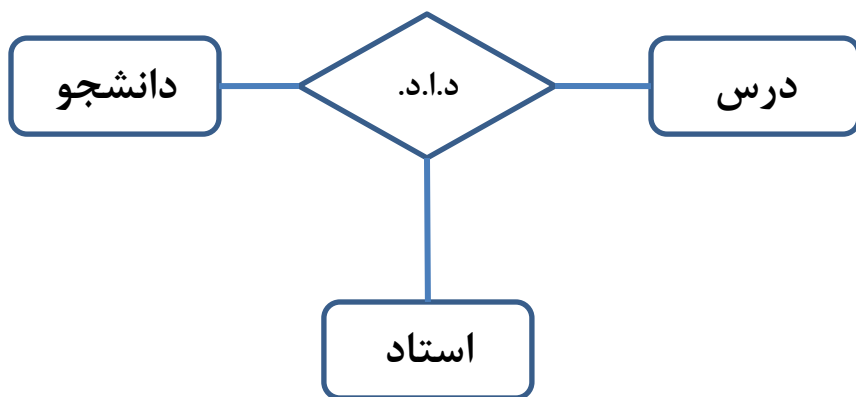
شرکت کنندگانی (participants) دارد ($N \geq 1$).

به تعداد شرکت کنندگان **درجه** (arity or degree) ارتباط گویند.

درجه یک و دو: مثال‌های پیش دیده



درجه سه: ارتباط درس، استاد، دانشجو



تذکر: در عمل به ندرت $N \geq 4$ پیش می‌آید.



❑ مشارکت نوع موجودیت E در نوع ارتباط R

❑ **الزامی** (کامل): هر نمونه از موجودیت E لزوماً در یک نمونه ارتباط R مشارکت دارد.

❑ **غیر الزامی** (ناقص): حداقل یک نمونه موجودیت E وجود دارد که در هیچ نمونه ارتباط R مشارکت ندارد.

❑ الزامی بودن مشارکت از محدودیت‌های معنایی محیط، ناظر به نوع ارتباط است.

هر دانشجو لزوماً درسی را انتخاب می‌کند ولی همه دروس لزوماً توسط دانشجویان انتخاب نمی‌شوند.

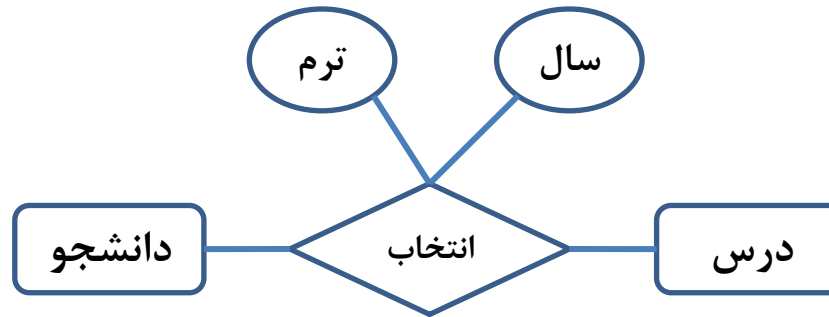


هر نوع ارتباط:

می تواند صفت(هایی)، موسوم به صفت(های) توصیفی داشته باشد.



دانشجوی X درس Y را در چه ترم و سالی انتخاب می کند؟



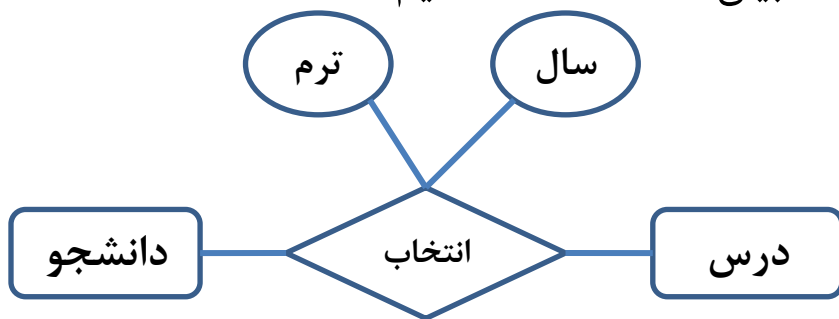
نکته مهم: هر نمونه ارتباط باید توسط نمونه موجودیت های شرکت کننده در آن ارتباط به طور یکتا

قابل شناسایی باشد [Silb2010].



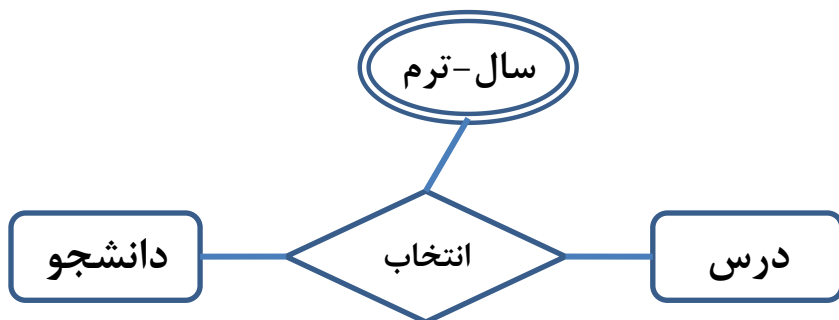
بخش دوم: مدلسازی معنایی داده ها

در مواردی که به ظاهر نتوانیم با نمونه موجودیت‌های شرکت کننده، یکتایی نمونه‌های یک ارتباط را تامین نماییم، می‌توانیم از صفت چندمقداری (برای رعایت نکته بیان شده) استفاده کنیم.



قابل درج نیست. چون ترکیب دانشجو و درس تکرار می‌شود و دیگر شناسه رابطه محسوب نمی‌شود.

دانشجو	درس	سال	ترم
۹۲۱۰۱۲۳۵	۴۰۳۸۴	۹۴-۹۳	۲
۹۲۱۰۱۲۳۵	۴۰۱۳۲	۹۵-۹۴	۱



قابل درج است؛ به عنوان مقادیر دیگر یک صفت مرکب چند مقداری.

دانشجو	درس	سال	ترم
۹۲۱۰۱۲۳۵	۴۰۱۳۲	۹۵-۹۴	۱
۹۲۱۰۱۲۳۵	۴۰۳۸۴	۹۴-۹۳	۲
		۹۵-۹۴	۱



چندی ارتباط Multiplicity یا Cardinality Ratio: ☐

تناظر
1:1
1:N
M:N

☐ چندی ارتباط بین دو نوع موجودیت E و F عبارت است از چگونگی تناظر بین

عناصر مجموعه نمونه‌های موجودیت E و عناصر مجموعه نمونه‌های موجودیت F.

☐ اگر دو نوع موجودیت E و F را در نظر بگیریم:

☐ در ارتباط یک به یک، یک نمونه از E حداکثر با یک نمونه از F ارتباط دارد و برعکس.

☐ در ارتباط یک به چند (از E به F)، یک نمونه از E با n نمونه از F ($n \geq 1$) و در صورت مشارکت

غیرالزامی، ($n \geq 0$) ارتباط دارد، ولی یک نمونه از F حداکثر با یک نمونه از E ارتباط دارد.

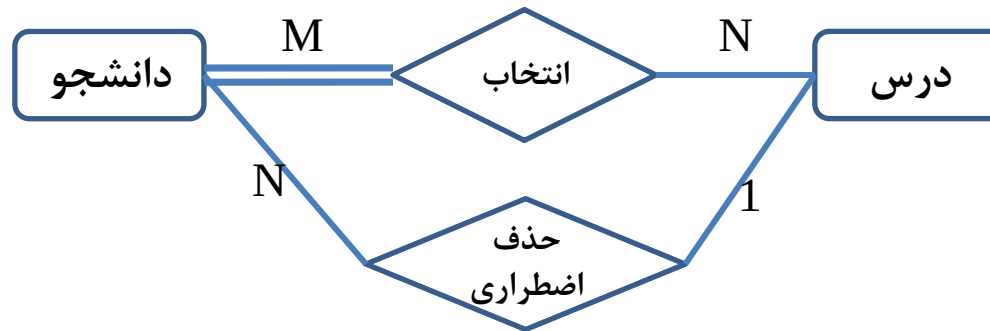
☐ در ارتباط چند به چند، یک نمونه از E با n نمونه از F ($n \geq 1$) ارتباط دارد و برعکس.

☐ **نکته:** چندی نوع ارتباط چندگانی ($m > 2$) عبارت است از تعداد نمونه‌های یک نوع موجودیت شرکت کننده

در آن نوع ارتباط، وقتی که تعداد نمونه‌های $m-1$ نوع موجودیت دیگر شرکت کننده در نوع ارتباط را ثابت فرض کنیم.

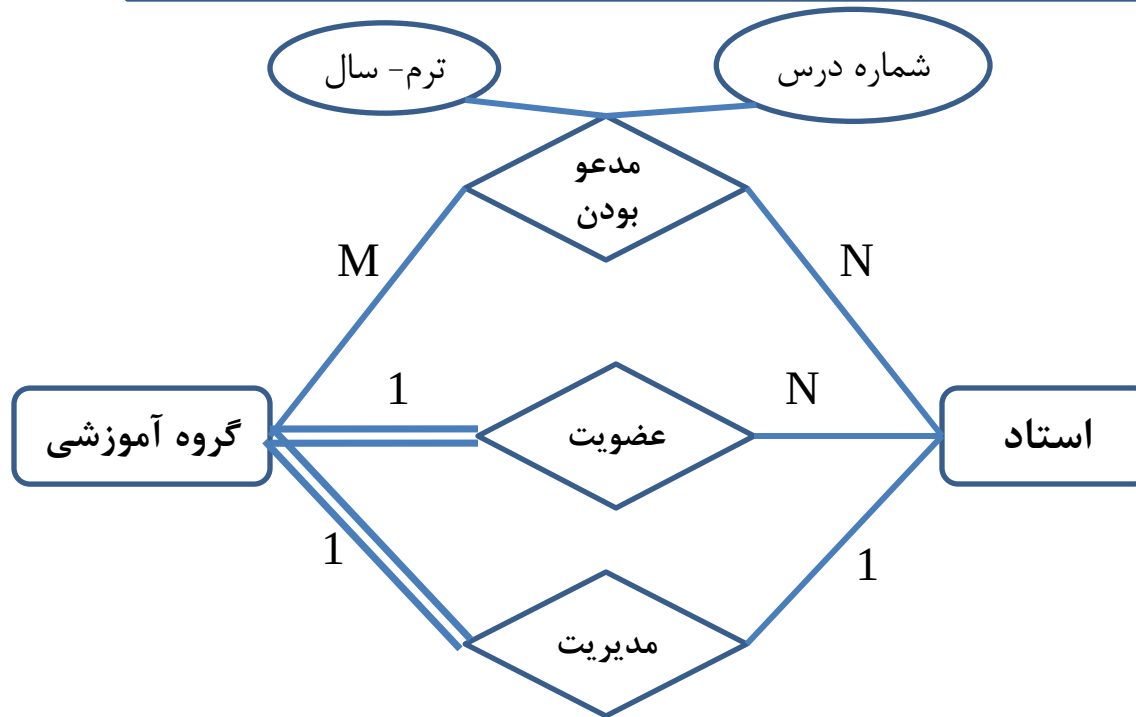


با فرض اینکه هر دانشجو چند درس می تواند انتخاب کند ولی فقط یک درس را می تواند حذف اضطراری کند، چندی ارتباطات به صورت زیر خواهد بود.

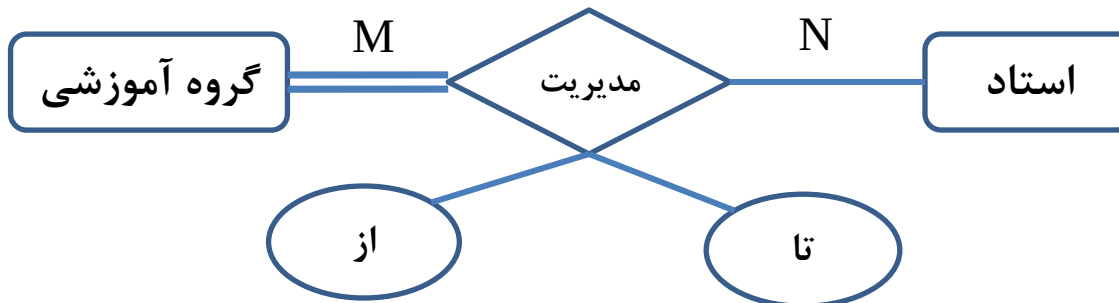




مثالی دیگر از چندی ارتباط



تذکر: اگر به ارتباط صفت هایی از جنس زمان بدهیم، چندی ارتباط می تواند بسته به قواعد معنایی محیط

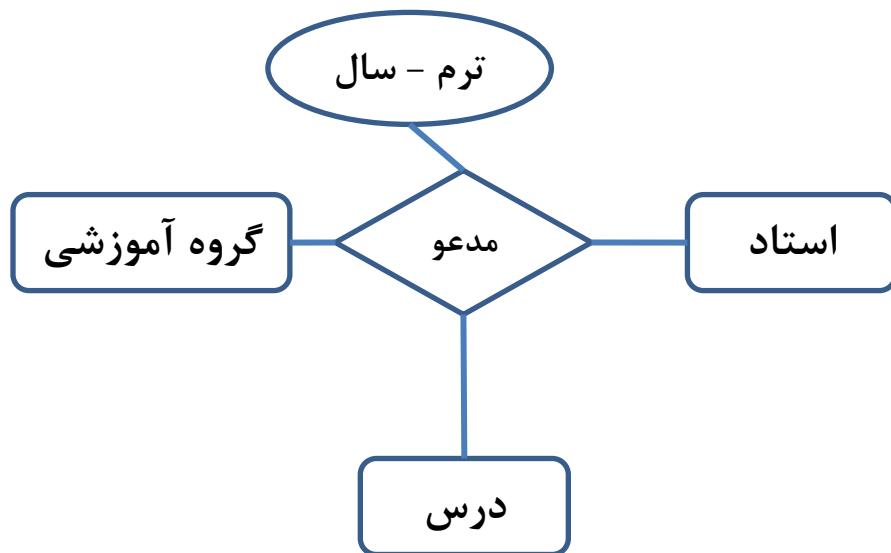
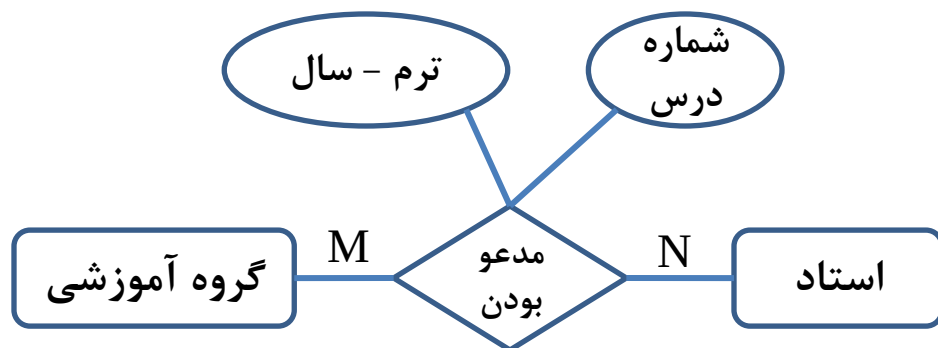


تغییر کند.



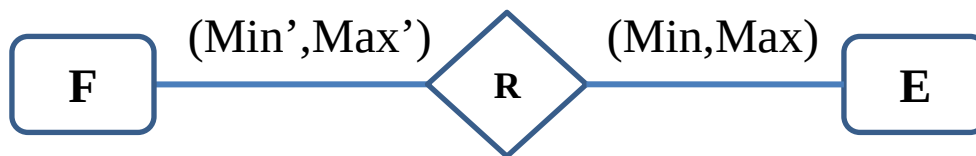
بخش دوم: مدلسازی معنایی داده ها

گونه‌های دیگر مدل کردن نوع ارتباط مدعو بودن چیست؟



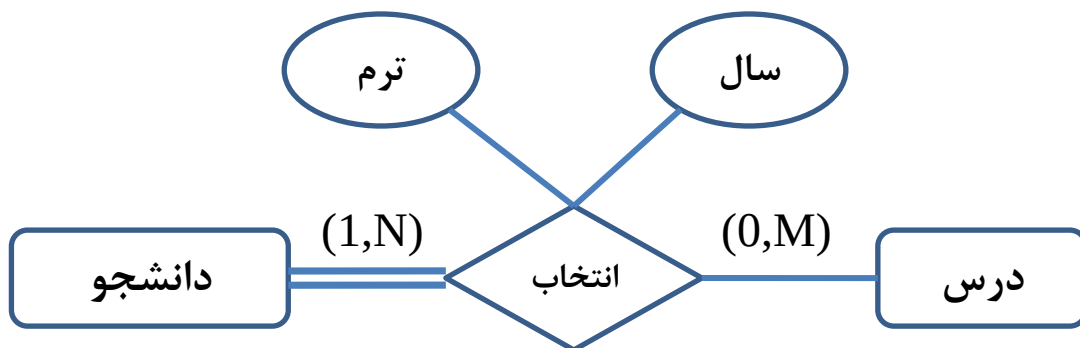
با استفاده از نوع ارتباط سه گانی:

تذکر: طرز دیگر نمایش چندی ارتباط



هر نمونه e از نوع موجودیت E باید حداقل در Min و حداکثر در Max نمونه از ارتباط R شرکت داشته باشد.

رابطه انتخاب درس توسط دانشجو

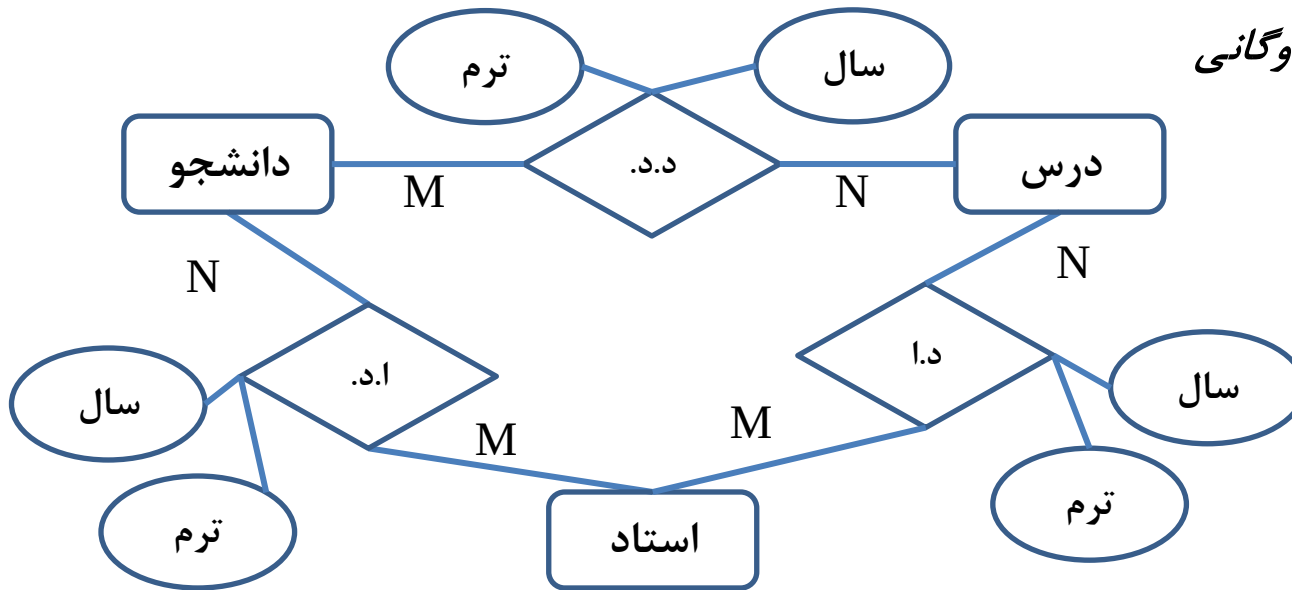


مزایای این روش نمایش چندی؟



نکته مهم در مورد ارتباط بین سه نوع موجودیت:

مدل یک: سه ارتباط دوگانی



سه فقره اطلاع:

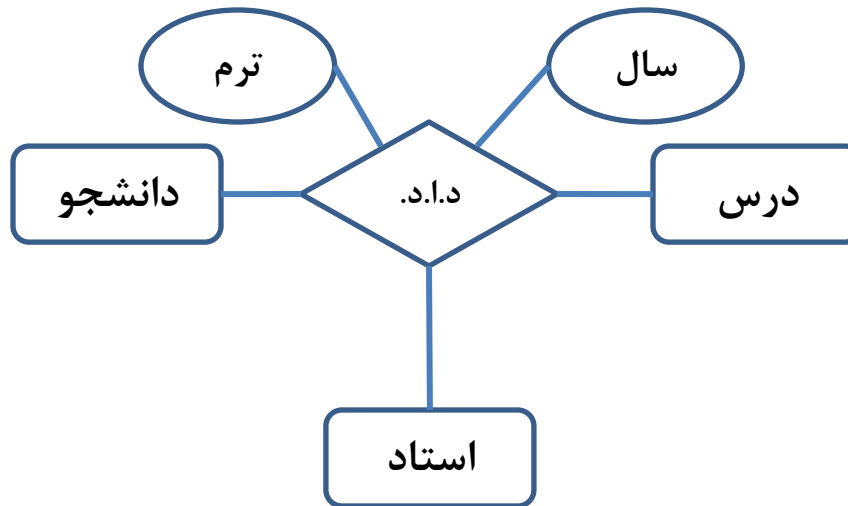
- دانشجو 's' درس 'c' را در ترم t1 سال y1 اخذ کرده است.
- استاد 'p' درس 'c' را در ترم t1 سال y1 ارایه کرده است.
- دانشجو 's' دانشجوی استاد 'p' در ترم t1 سال y1 است.

از این سه فقره اطلاع لزوماً همیشه **نمی توان** نتیجه گرفت که دانشجو 's' درس 'c' را با استاد 'p' گذرانده است.



بخش دوم: مدلسازی معنایی داده ها

□ مدل دوم: ارتباط سه گانی



□ در حالت سه ارتباط دوگانی اگر از فقره اطلاع های دوگانی، فقره اطلاع سه گانی را استنتاج کنیم در شرایطی که از لحاظ معنایی این استنتاج درست نباشد می گوییم دچار **دام پیوندی حلقه ای** شده ایم.

انواع دیگر دام چیست؟ (دام چندشاخه (چتری)، دام گسل (شکافت)، ...)



بخش دوم: مدلسازی معنایی داده ها

مثال: فعالیت هایی از محیط دانشکده

بعضی از نوع موجودیت های ممکن:

دانشجو

استاد

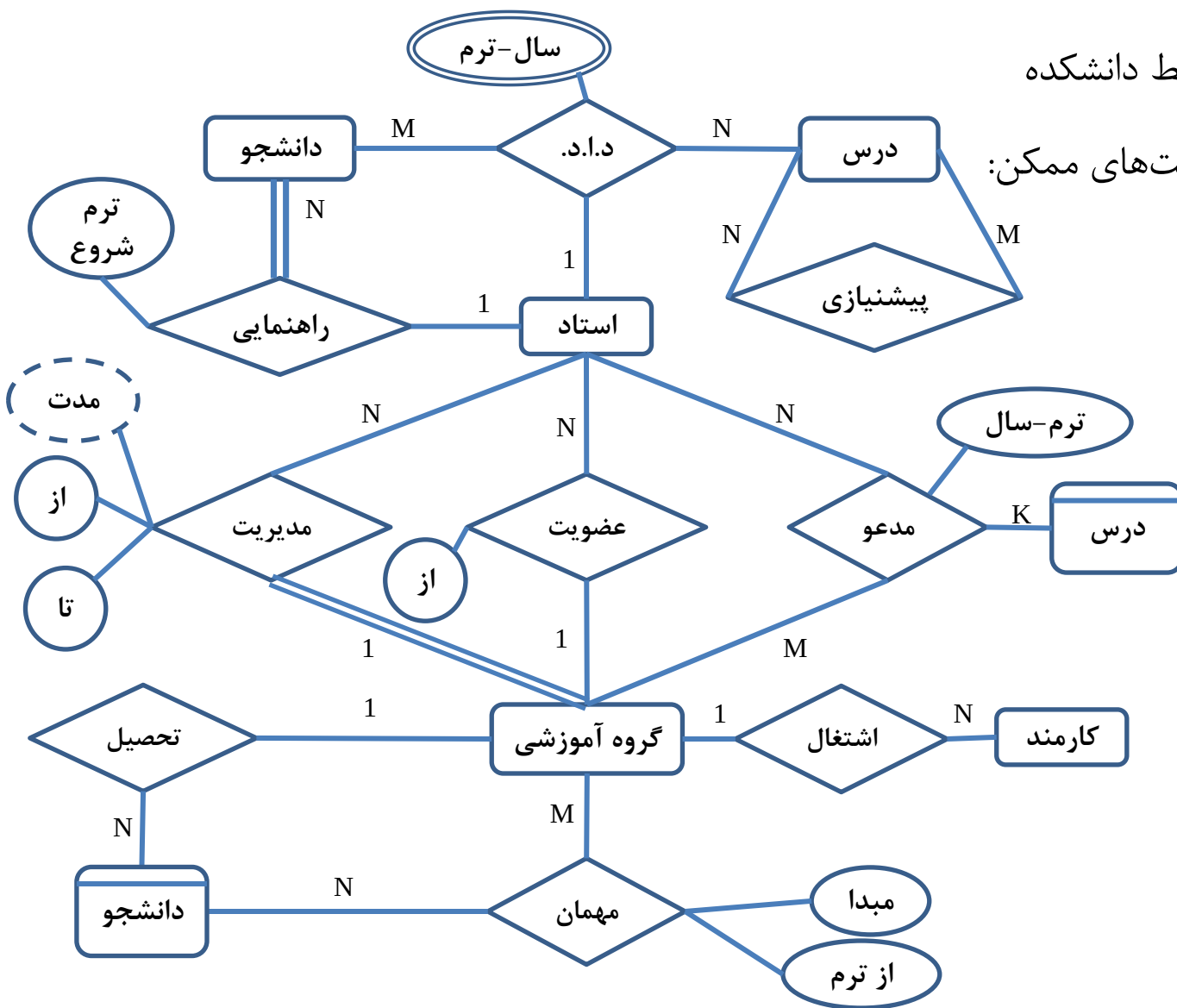
درس

کارمند

گروه آموزشی

کتاب

...

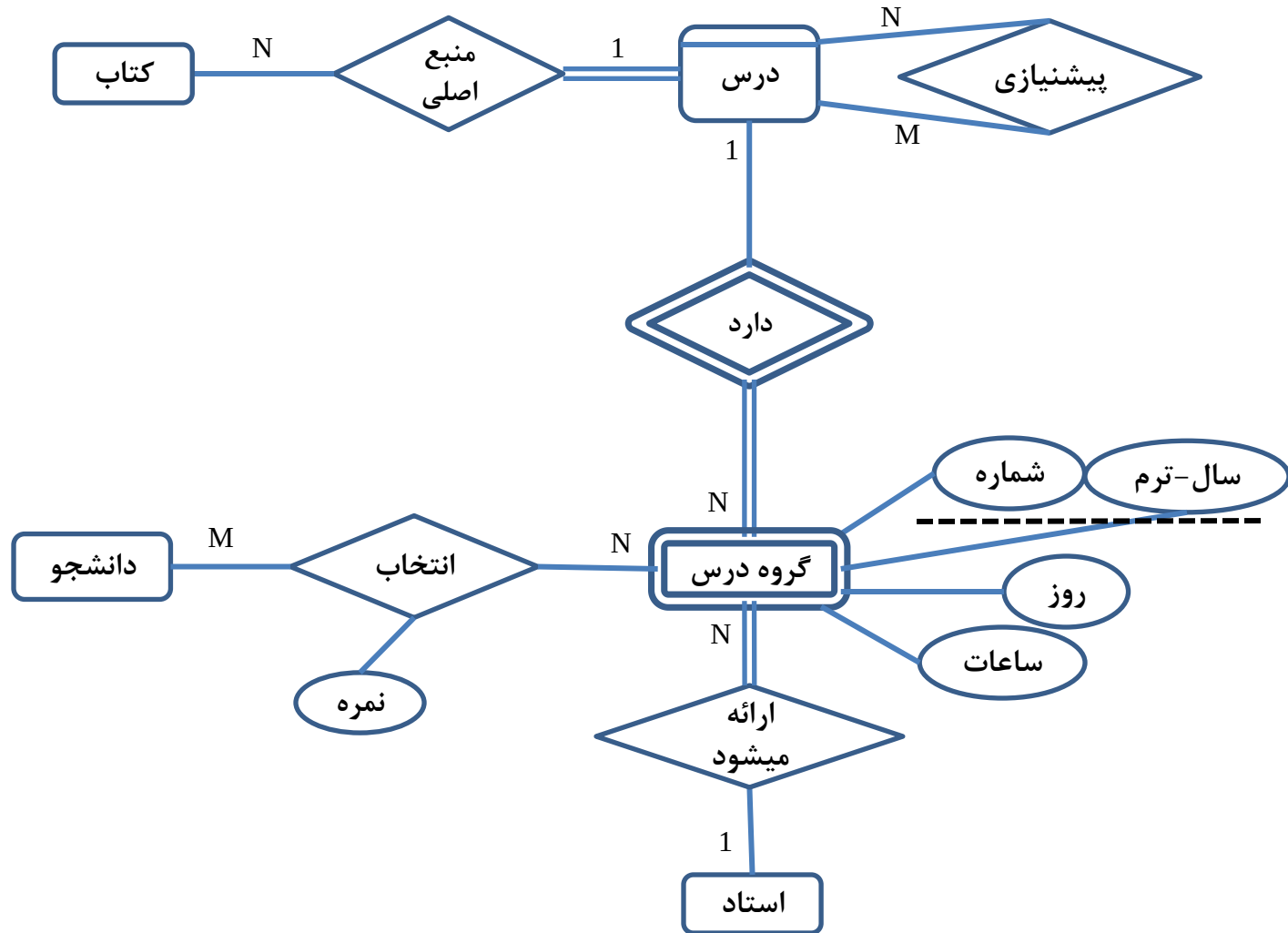




مثال: محیط دانشکده (ادامه)

بخش دوم: مدلسازی معنایی داده ها

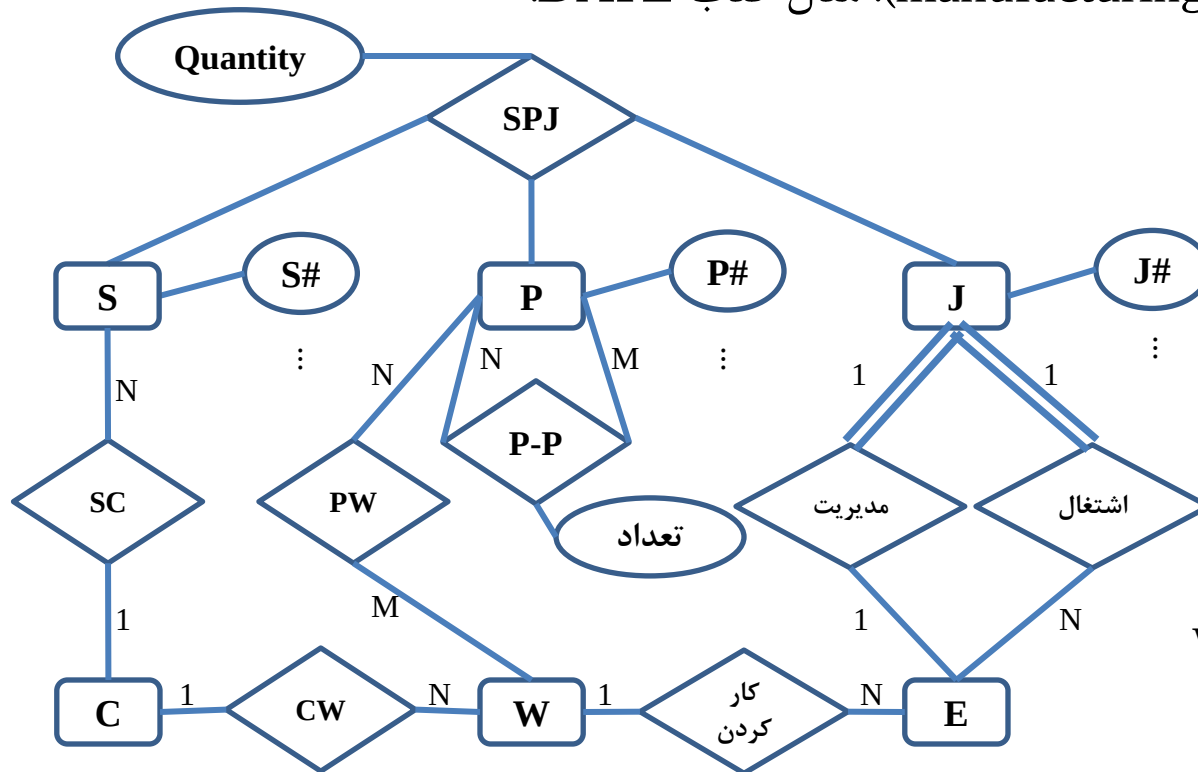
۳۲



مثال: محیط تولیدی-کارگاهی (manufacturing). مثال کتاب DATE.

نوع موجودیت ها:

- تولید کننده S ■ Supplier
- نوع قطعه P ■ Part
- پروژه J ■ Project
- کارمند/کارگر E ■ Employee
- شهر C ■ City
- انبار W ■ Warehouse





نوع موجودیت ضعیف:

□ نوع موجودیت F را ضعیف نوع موجودیت E گوئیم هرگاه F با E «وابستگی وجودی» داشته باشد. (یعنی اگر E در مدلسازی مطرح نشود، F هم مطرح نباشد). علاوه بر این نوع موجودیت ضعیف از خود شناسه ندارد.

□ طرز نمایش:



□ **تاکید:** قوی و ضعیف بودن نسبی است.

□ نوع ضعیف از خود شناسه ندارد. بلکه از خود می تواند یک **صفت ممیزه-جداساز** (Discriminator) یا به عبارت دیگر یک **کلید جزئی** (Partial Key) داشته باشد.

□ صفت ممیزه (کلید جزئی):

- صفتی که یکتایی مقدار دارد اما نه در تمام نمونه های نوع ضعیف بلکه در بین مجموعه تمام نوع ضعیف های وابسته به یک نمونه از نوع موجودیت قوی (به صورت نسبی یکتاست یا در ترکیب با شناسه موجودیت قوی یکتاست).
- در عمل اگر یک نوع موجودیت وابستگی وجودی به نوع موجودیت دیگر داشته باشد و از خود شناسه داشته باشد دیگر ضعیف دیده نمی شود.



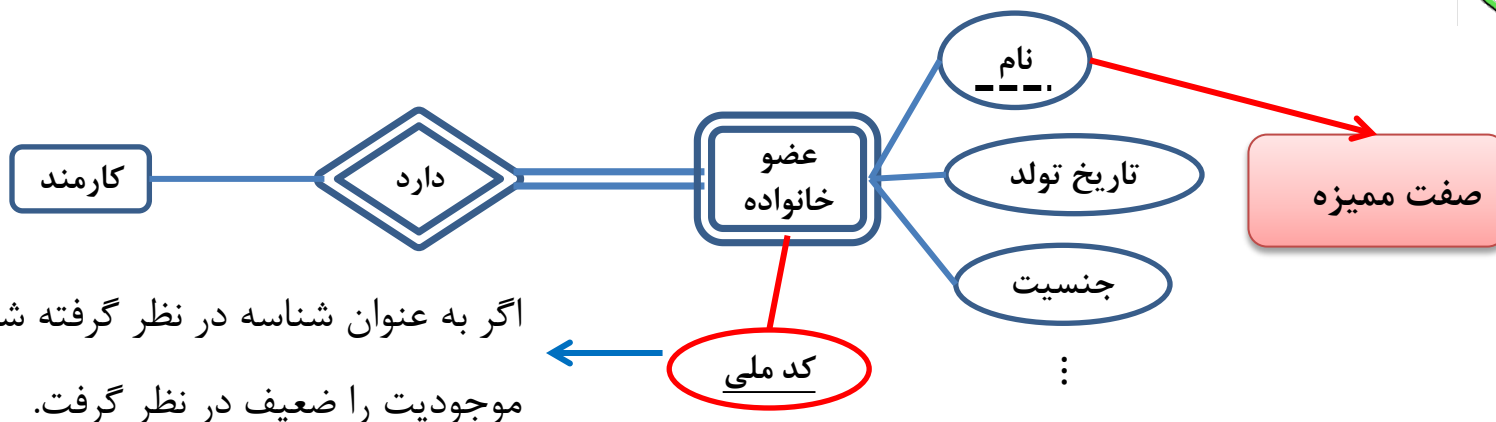
بحث تکمیلی : نوع موجودیت ضعیف (ادامه)

بخش دوم: مدلسازی معنایی داده ها

۳۵



عضو خانواده به عنوان یک موجودیت ضعیف



شماره کارمند	نام
۱۰۰	{ گلی سلی قلی }
۲۰۰	{ ناجی تاجی سلی }



بحث تکمیلی : نوع موجودیت ضعیف (ادامه)

بخش دوم: مدلسازی معنایی داده ها

۳۶

□ به ارتباط قوی-ضعیف، **ارتباط شناسا** (Identifying Relation) گویند.

□ مشارکت نوع ضعیف در ارتباط شناسا الزامی است.

□ چندی ارتباط معمولا $1:N$ (در حالت خاص $1:1$ تمرین: مثال قید شود).

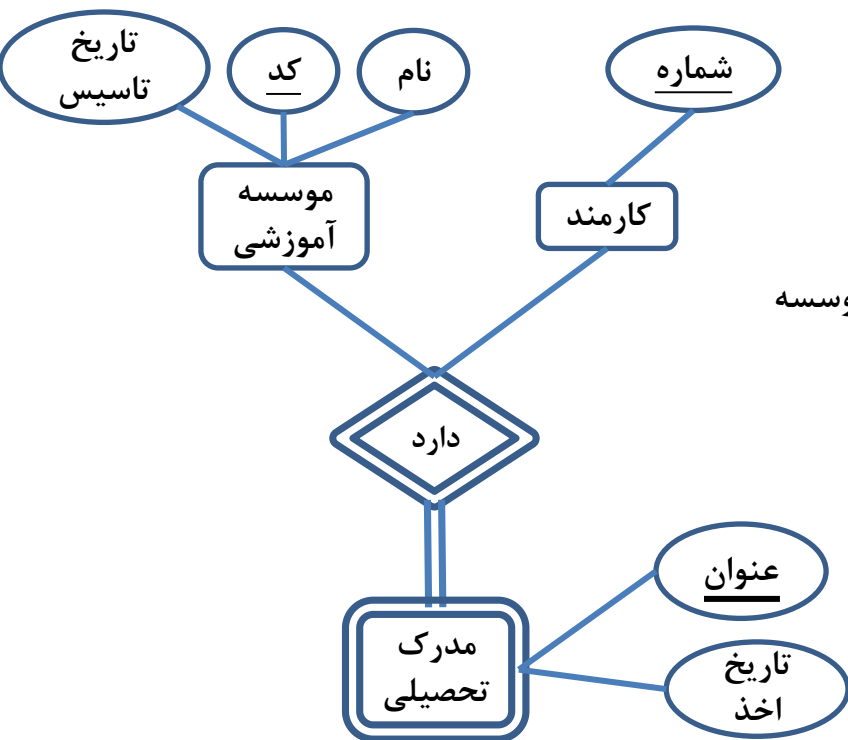


بحث تکمیلی : نوع موجودیت ضعیف (ادامه)

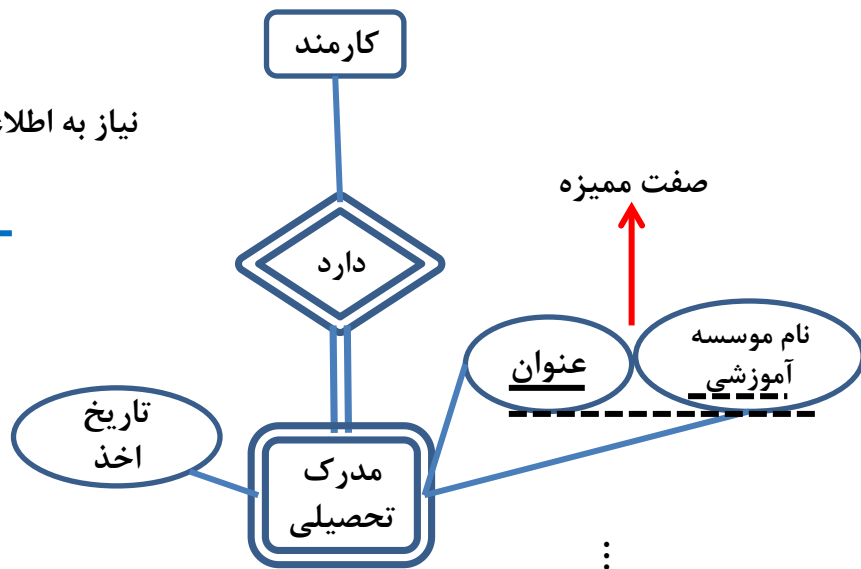
۳۷

بخش دوم: مدلسازی معنایی داده ها

□ درجه ارتباط شناسا معمولا ۲ و گاه بیشتر است.



نیاز به اطلاعات بیشتر از موسسه آموزشی

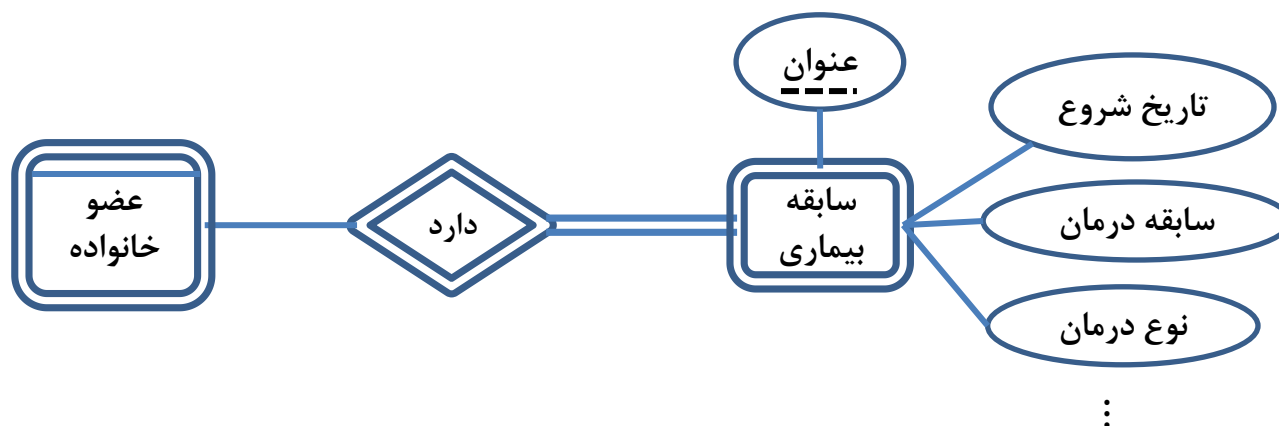


در اینجا **مدرک تحصیلی** وابستگی وجودی به بیش از یک موجودیت دارد.

آیا این محیط را می توان به گونه ای دیگر مدل کرد؟



□ نوع موجودیت ضعیف می تواند خود قوی برای نوع موجودیت ضعیف دیگر باشد.

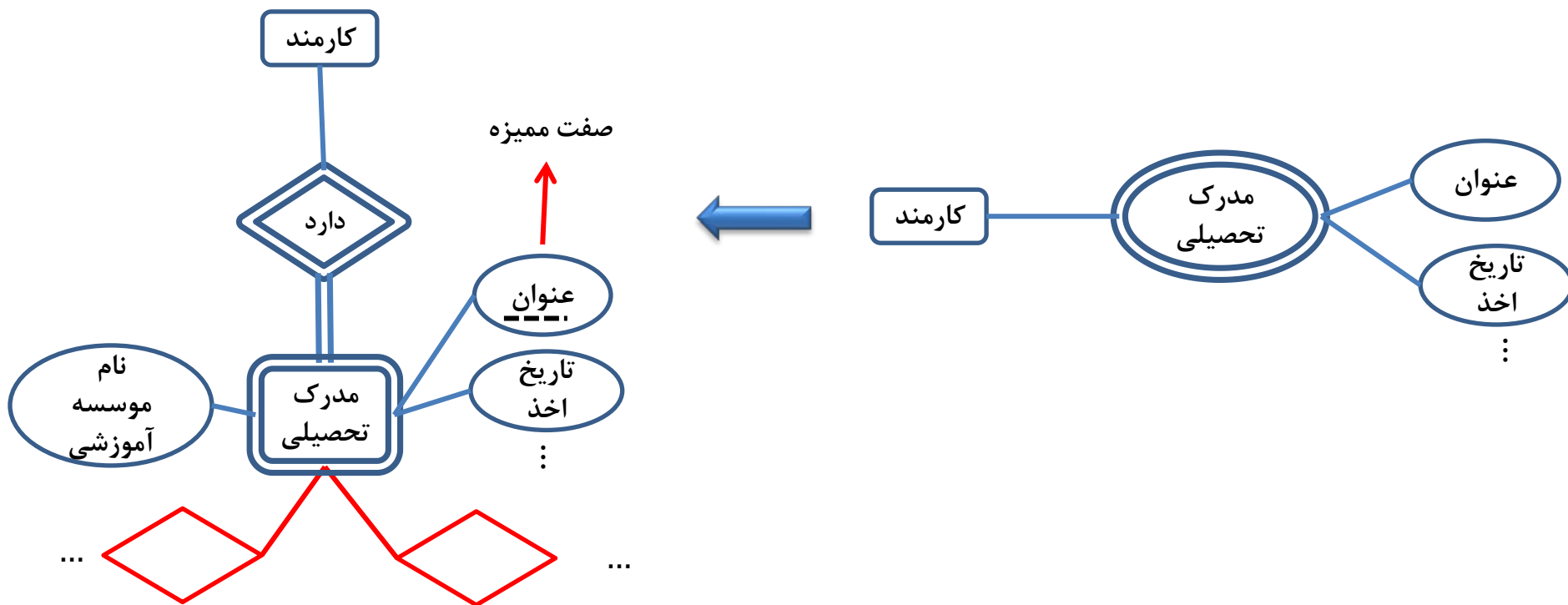


اگر بخواهیم برای کارمند سابقه بیماری اش را نگه داریم چه کارکنیم؟



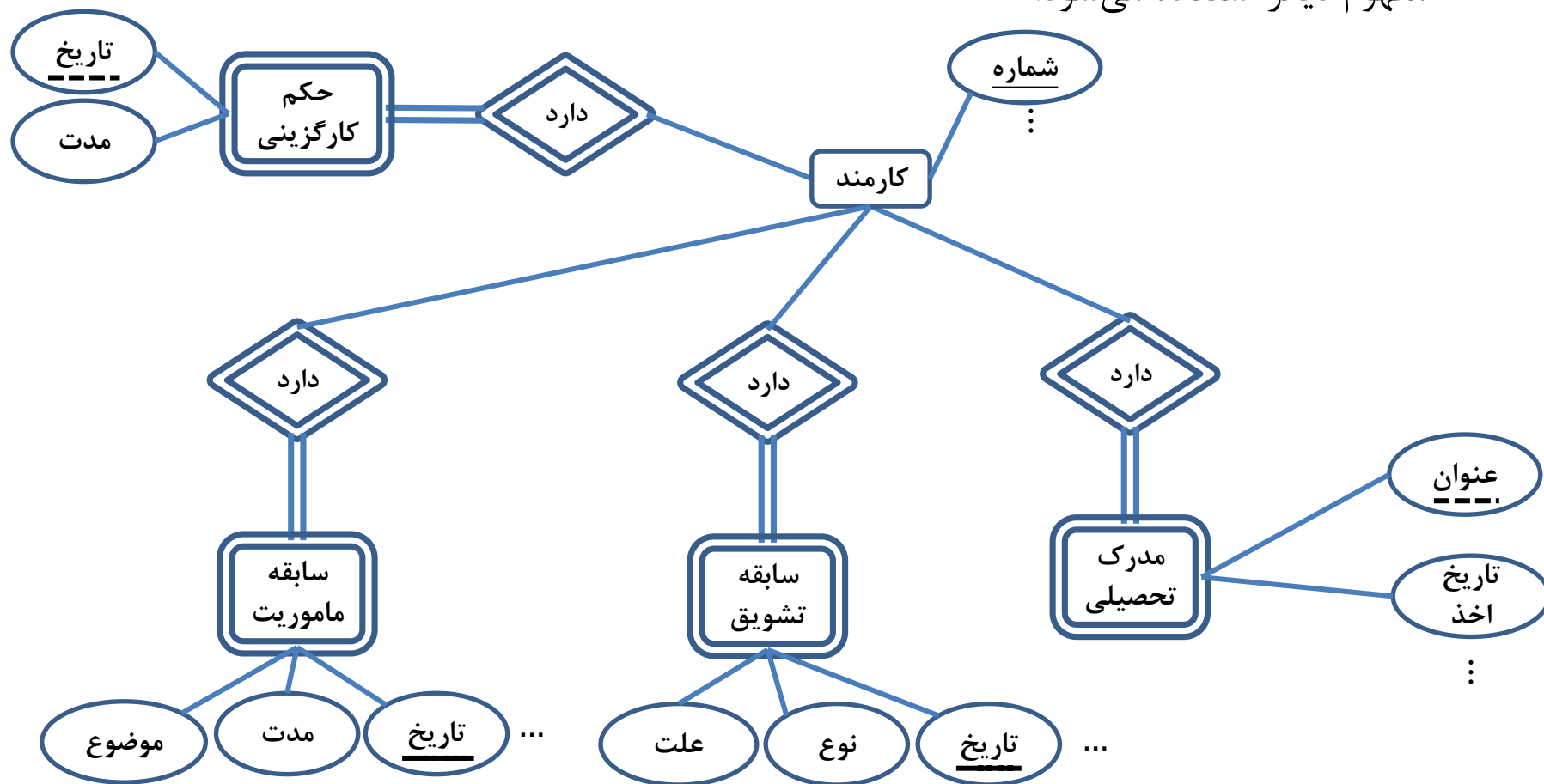
□ صفت چند مقداری (به خصوص مرکب) را همیشه می توان با مفهوم نوع موجودیت ضعیف مدل کرد (نمایش داد) اما عکس این تکنیک توصیه نمی شود.

□ **دلیل:** انعطاف پذیری مدل را از نظر گسترش پذیری کاهش می دهد، زیرا نوع ضعیف می تواند خود نوع ارتباطی داشته باشد با دیگر نوع موجودیت ها، اما وجود ارتباط با صفت معنا ندارد.



مفهوم نوع موجودیت ضعیف به ویژه برای مدل کردن پدیده‌های تکرار شونده (در زمان) و وابسته به

مفهوم دیگر استفاده می‌شود.



تبدیل ارتباط سه گانی به ارتباطات دوگانی

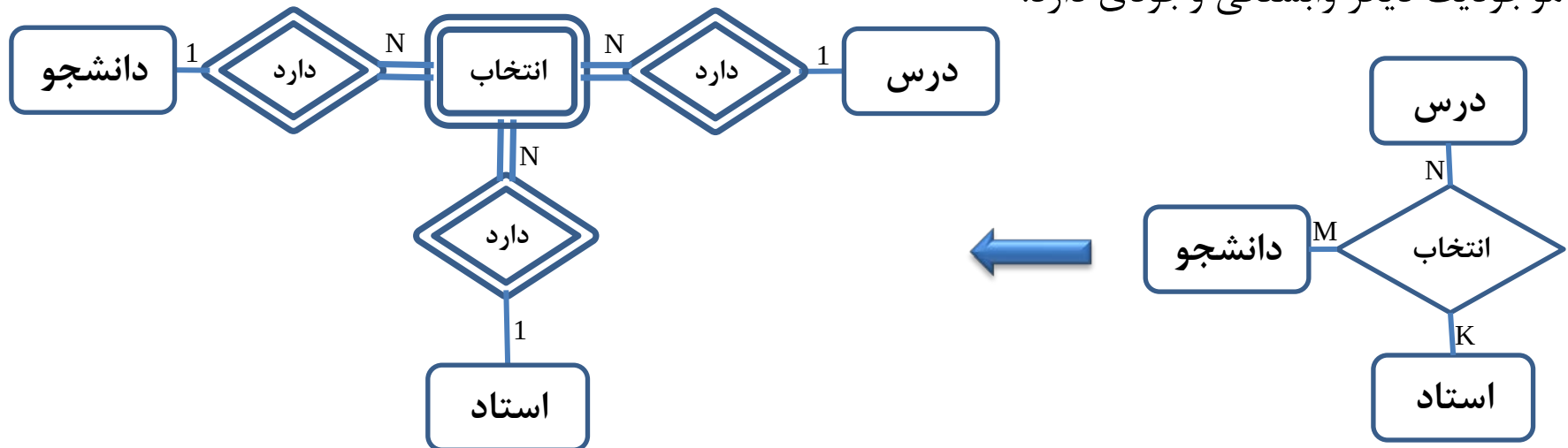
از مفهوم نوع موجودیت ضعیف می توان برای تبدیل یک ارتباط سه گانی (یا n -گانی) به ارتباطات دوگانی استفاده کرد.

اغلب ابزارهای طراحی مبتنی بر روش ER فقط ارتباطات دوگانی را پشتیبانی می کنند.

تبدیل رابطه سه گانه انتخاب به سه رابطه دوگانی. در اینجا موجودیت انتخاب صفت ممیزه ندارد و به هر



سه موجودیت دیگر وابستگی وجودی دارد.





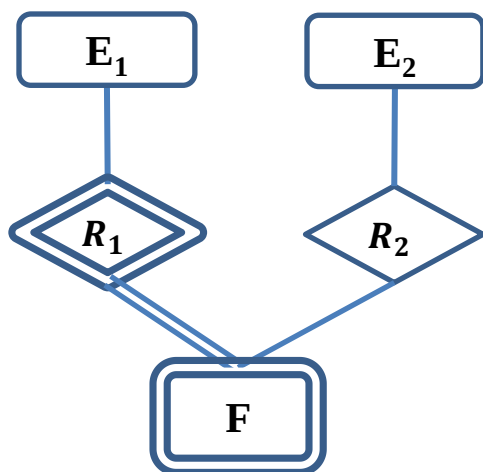
بحث تکمیلی : نوع موجودیت ضعیف (ادامه)

بخش دوم: مدلسازی معنایی داده ها

۴۲

□ یک نوع موجودیت ضعیف می تواند در یک نوع ارتباط دیگر با نوع موجودیت قوی دیگر شرکت داشته باشد.

رابطه بین گروه درسی و استاد در مثالهای پیشتر بیان شده.



مثالی دیگر از مطلب فوق بیاورید.





❑ مشکل تصمیم‌گیری در مورد اینکه یک مفهوم، نوع موجودیت در نظر گرفته شود یا صفت یا نوع ارتباط باید در یک فرآیند تدریجی در مدلسازی معنایی داده‌ها اصلاح شود.

❑ اگر یک مفهوم، صفت به نظر آید، آنرا **صفت** می‌گیریم، اما اگر به نوع موجودیت دیگری **ارجاع** داشته باشد، آن را یک **نوع ارتباط** در نظر می‌گیریم.

❑ اگر یک (چند) صفت به هم مرتبط (از لحاظ معنایی) در چند نوع موجودیت، **مشترک** باشند، آنها را به عنوان **صفات یک نوع موجودیت مستقل** منظور می‌کنیم.

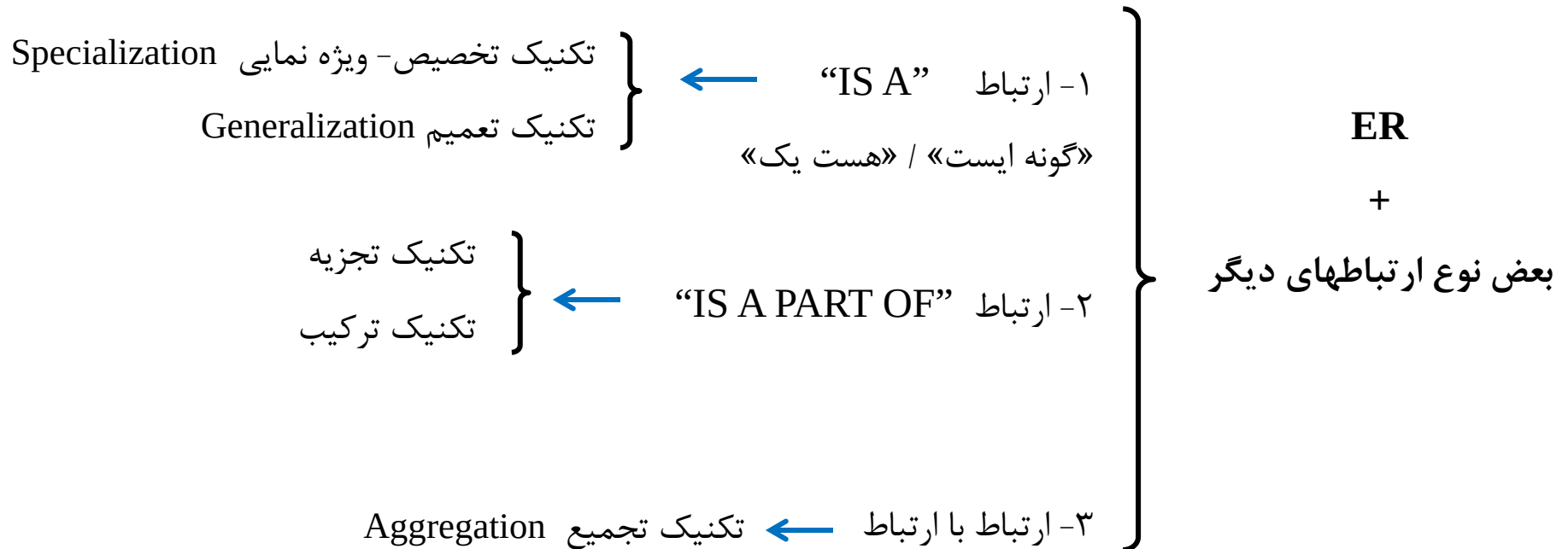
❑ اگر یک **نوع موجودیت**، تنها **یک** صفت داشته باشد و تنها با **یک** نوع موجودیت دیگر مرتبط باشد، آن را **صفت** در نظر می‌گیریم.

❑ اگر مجموعه‌ای از صفات مستقلاً قابل شناسایی نباشند، آن را به صورت **نوع موجودیت ضعیف** در نظر می‌گیریم.



Enhanced ER یا Extended ER :EER 

ER مبنایی کمداشت‌هایی دارد در نمایش بعضی نوع ارتباطها (که بعدا در حیطه شیء‌گرایی مطرح شد) 





□ ارتباط IS A: ارتباط بین یک نوع موجودیت عام است با نوع موجودیت (های) خاص آن که بر

زیرنوع
(SubType)

زبرنوع
(Supertype)

اساس یک ضابطه مشخص بازشناسی می شود.

صفت معرف

Defining Attribute

□ طرز نوشتن: “F IS-A E”

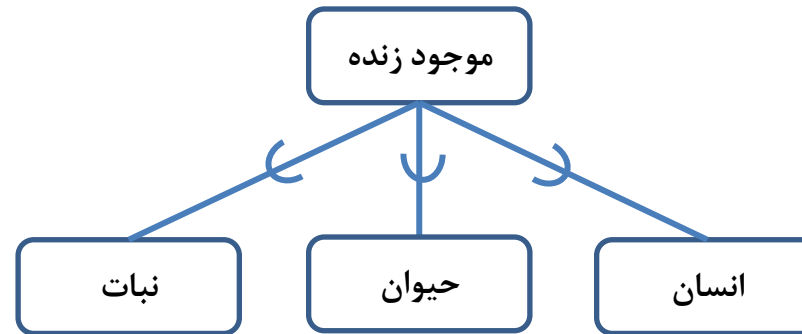
□ وقتی نوع های خاص یک نوع عام را بازشناسی می کنیم به آن تکنیک ویژه نمایی-تخصیص یا

Specialization گوییم.

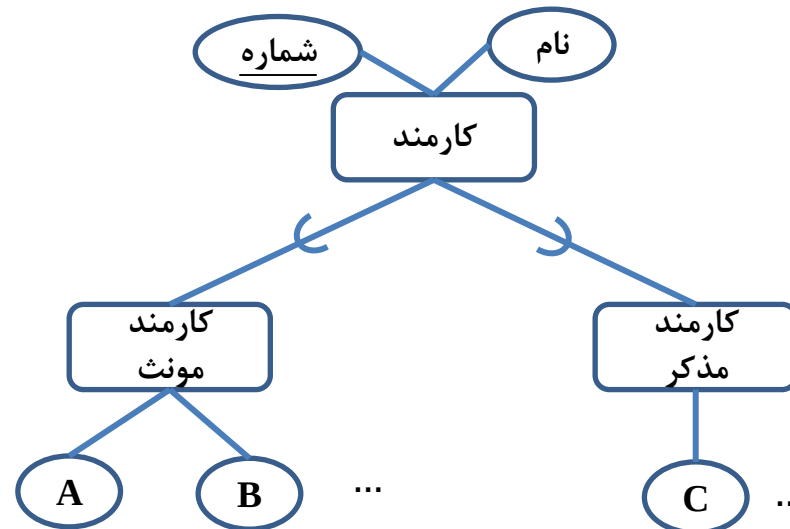
□ عکس این تکنیک را تعمیم یا Generalization گوییم.



انواع موجودات زنده



انواع کارمندان





نکات: 

 زیرنوع مجموعه صفاتی دارد مشترک در تمام زیرنوع‌ها

▪ در نتیجه زیرنوع تمام صفات (و روابط) زیرنوع را به ارث می‌برد (وراثت صفات از نوع ساختاری).

▪ مفهوم ارث‌بری با تکنیک ارتباط IS-A مدلسازی می‌شود.

▪ وراثت ممکن است ساختاری باشد یا رفتاری. در اینجا وراثت صفات، وراثتی ساختاری است.



 زیرنوع مجموعه صفات خاص خود را هم دارد [حداقل یک صفت]

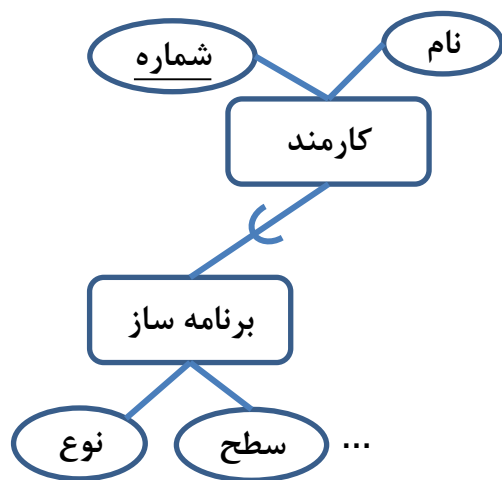
 اگر m تعداد شاخه‌های تخصیص منشعب از یک زیرنوع باشد داریم: $m \geq 1$



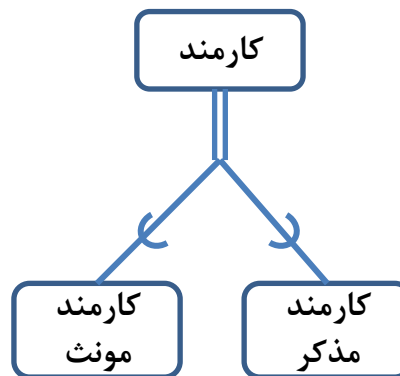
بخش دوم: مدلسازی معنایی داده ها

- ۱- کامل: تمام زیرنوع‌های (ممکن) زیرنوع در مدلسازی در نظر گرفته **می‌شوند**. بدین ترتیب هر نمونه از زیرنوع، جزء نمونه‌های حداقل یکی از زیرنوع‌ها است. **تخصیص** □
- ۲- ناقص: تمام زیرنوع‌های (ممکن) زیرنوع در مدلسازی در نظر گرفته **نمی‌شوند**. هر نمونه از زیرنوع لزوماً جزء نمونه‌های یکی از زیرنوع‌ها نیست.

تخصیص ناقص: براساس مهارت کارمند فقط برنامه‌سازان را جدا کرده‌ایم. ممکن است کارمندی باشد که برنامه‌ساز نباشد.

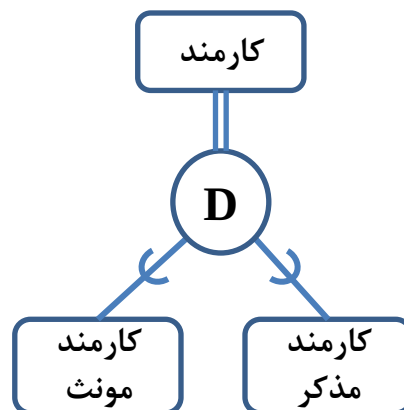


تخصیص کامل: هر نمونه کارمند یا مونث است یا مذکر.

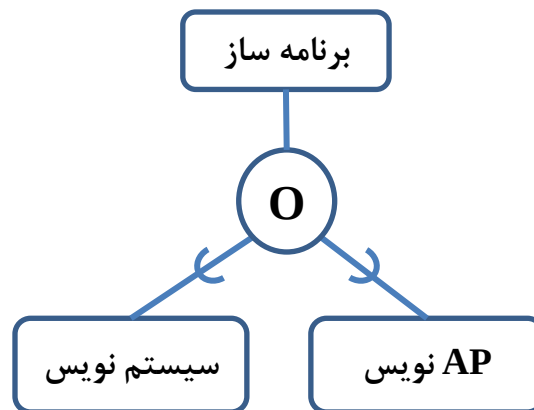




- تخصیص
- ۱- مجزا: هر نمونه از زیرنوع جزء مجموعه نمونه‌های حداکثر یک زیرنوع است.
- ۲- همپوشا: یک نمونه از زیرنوع جزء مجموعه نمونه‌های حداقل دو زیرنوع است.



تخصیص مجزا



تخصیص همپوشا



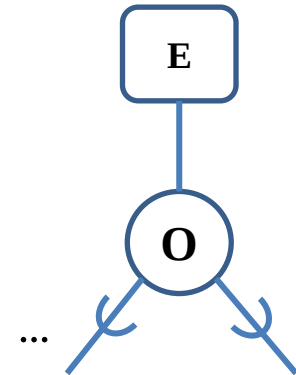
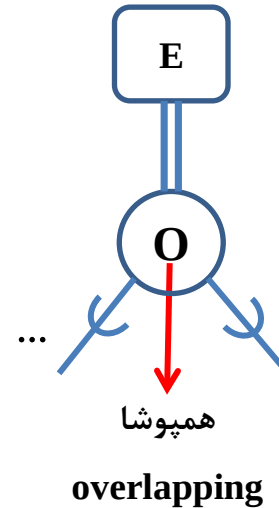
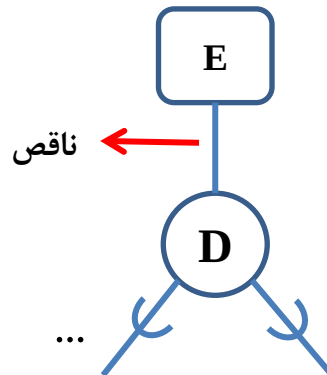
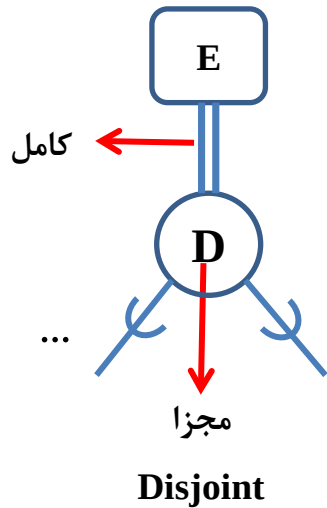


ارتباط “IS A” – تخصیص (ادامه)

بخش دوم: مدلسازی معنایی داده ها

۵۰

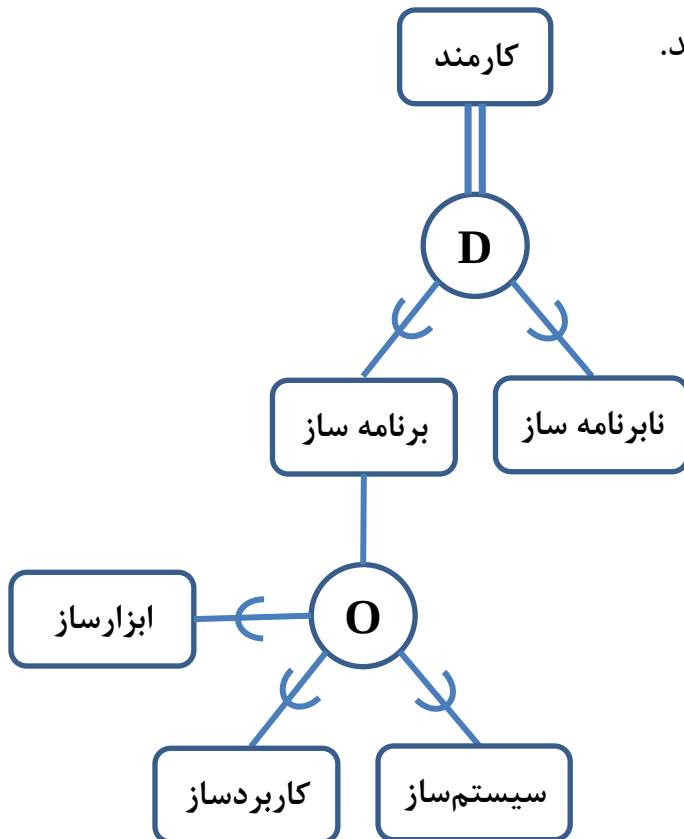
براساس این دو ویژگی چهارگونه تخصیص داریم: □



ادامه نکات: 

زیرنوع می تواند خود زیرنوع هایی داشته باشد. 

یعنی ژرفای (عمق) درخت تخصیص می تواند بیش از یک باشد. 



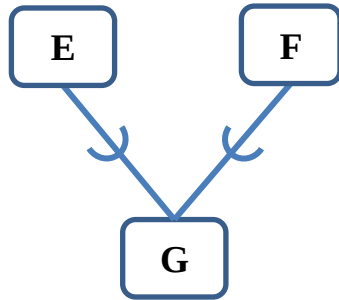


ارتباط "IS A" (ادامه)

۵۲

بخش دوم: مدلسازی معنایی داده ها

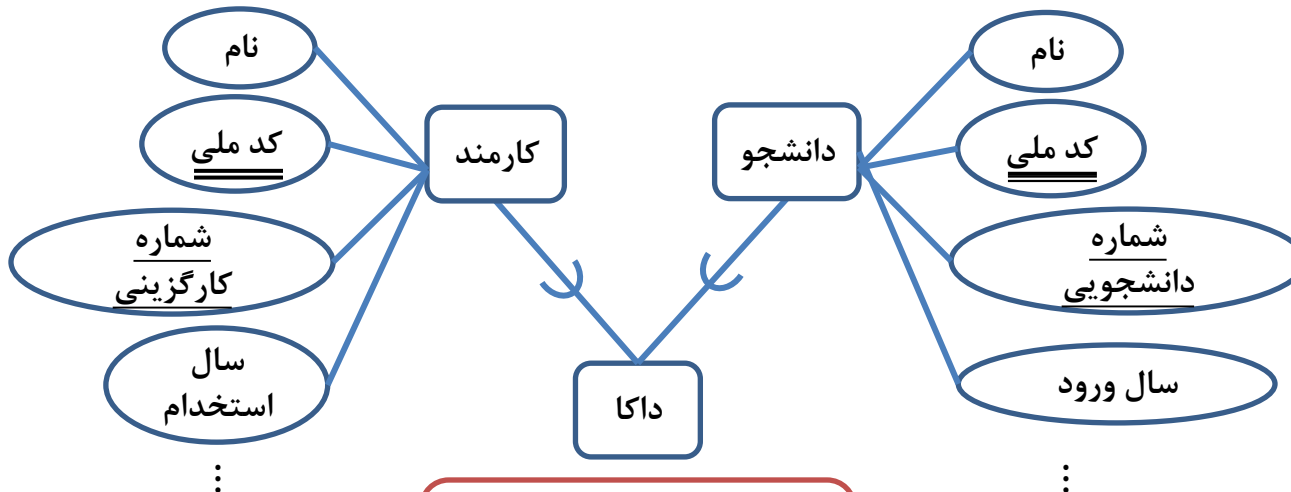
□ زیرنوع می تواند بیش از یک زبرنوع داشته باشد.



□ G صفات را هم از E و هم صفات F را به ارث می برد.

□ **وراثت چندگانه (Multiple Inheritance)** را می توان اینگونه مدل کرد.

آیا G می تواند از خود نیز صفاتی داشته باشد؟



کد ملی و نام را فقط یک بار
برای «داکا» محاسبه می کند.

ارث بری چندگانه



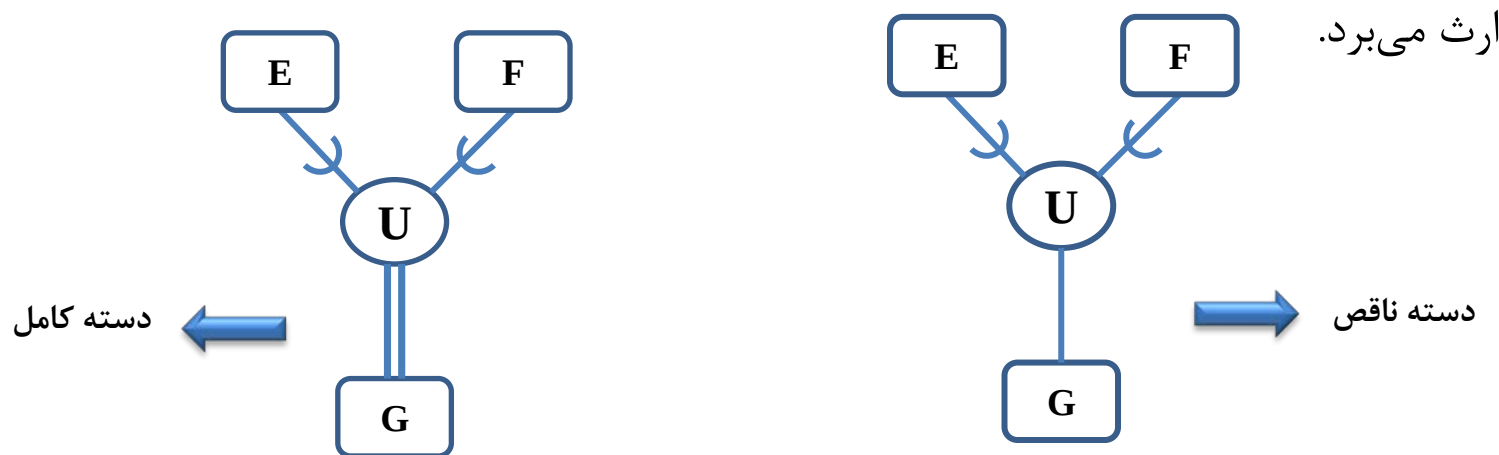
زیرنوع اجتماع (U-Type) یا Category «دسته» □

□ زیرنوع موجودیت G را زیرنوع U-Type زیرنوع های $E, F, \dots$ گوئیم هرگاه در مجموعه نمونه های G

نمونه هایی از $E, F, \dots$ وجود داشته باشد. در واقع نمایانگر اجتماعی از نمونه هایی از انواع مختلف است.

اگر همه نمونه ها ← دسته کامل
اگر بعض نمونه ها ← دسته ناقص

□ یک نمونه از زیرنوع اجتماع (دسته)، بسته به اینکه از نوع کدام زیرنوع باشد، صفات همان زیرنوع را به



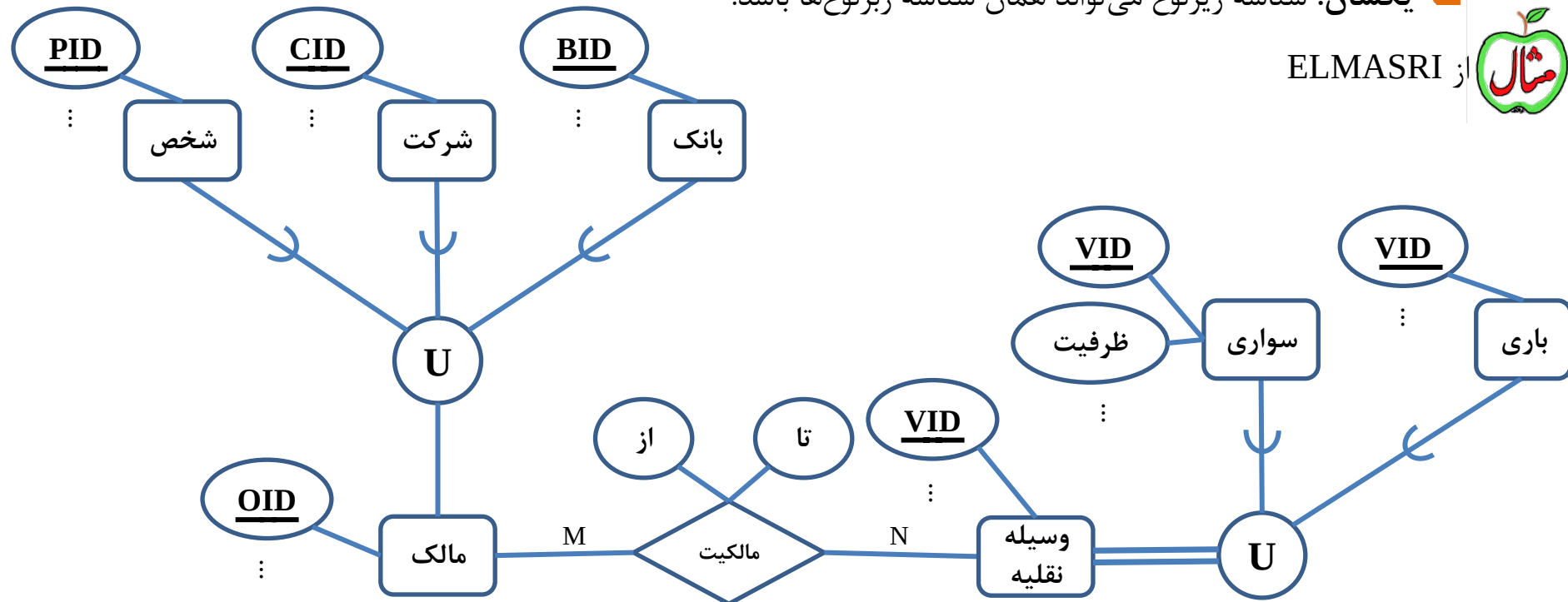


شناسه های زیرنوع ها می تواند از دامنه های متفاوت باشد.

متفاوت: شناسه زیرنوع شناسه ای است که خود باید در نظر بگیریم.

یکسان: شناسه زیرنوع می تواند همان شناسه زیرنوع ها باشد.

از ELMASRI



در چه صورت مدلسازی با U-Type را می توان با تکنیک تخصیص (ویژه‌نمایی) معمولی مدل کرد؟ در چه شرایطی کدام یک

بهتر است؟



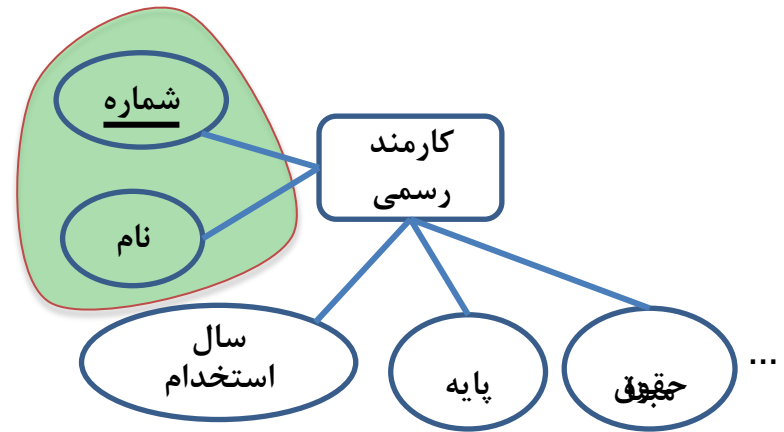
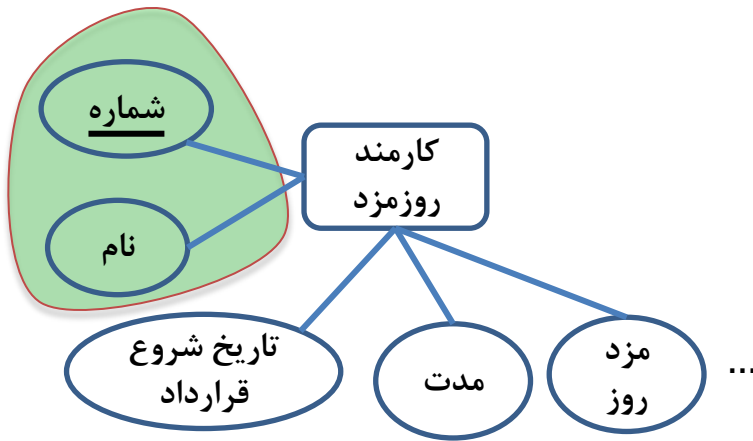


تمرین : برای محیط با مفاهیم زیر، هم با U-Type و هم بدون U-Type یک مدلسازی ارائه دهید:

- بانک - دانشگاه
- شخص (دانشجو - استاد - کارمند و متفرقه)
- حساب بانکی (کوتاه مدت - بلند مدت - قرض الحسنه و...)
- عملیات واریز - برداشت - انتقال وجه

تعمیم عبارت است از تشخیص یک نوع موجودیت جدید (در سطح انتزاع بالاتر) از روی [با داشتن] $n \geq 2$ نوع موجودیت از پیش دیده که ماهیتا از یک نوع باشند. (احيانا به منظور ادغام ERDهای جدا)

فرض: در یک مدلسازی یا در دو مدلسازی جدا برای دو زیر محیط:



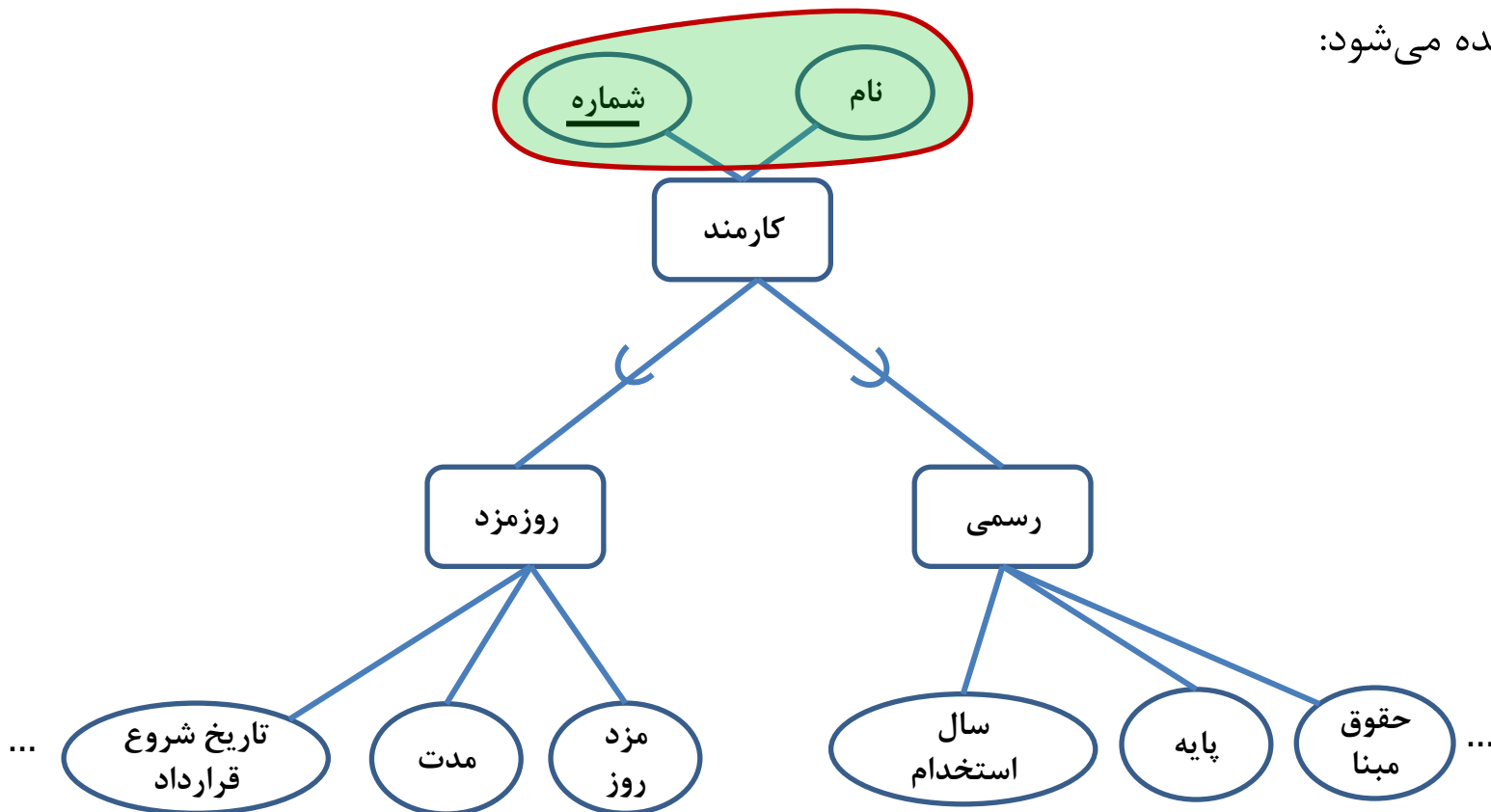


ادامه:



یک نوع موجودیت (کارمند) در سطح انتزاعی

بالاتر دیده می شود:





بخش دوم: مدلسازی معنایی داده ها

شرایط تعمیم: ☐

☐ داشتن شناسه مشترک [یعنی از یک دامنه]

☐ حداقل وجود دو نوع زیرنوع

☐ هرچه صفات مشترک بیشتر، تعمیم توجیه پذیرتر است [شرط لازم نیست ولی شرط ارجحیت است].

ارتباطها؟



ارتباط “IS-A-PART Of” یا “Has-A” یا “Contains”

بخش دوم: مدلسازی معنایی داده ها

۵۹

تعریف: ارتباط بین نوع موجودیت کل است با نوع موجودیت‌های جزء آن (تشکیل دهنده آن)

F is a part of E

E شامل F است.

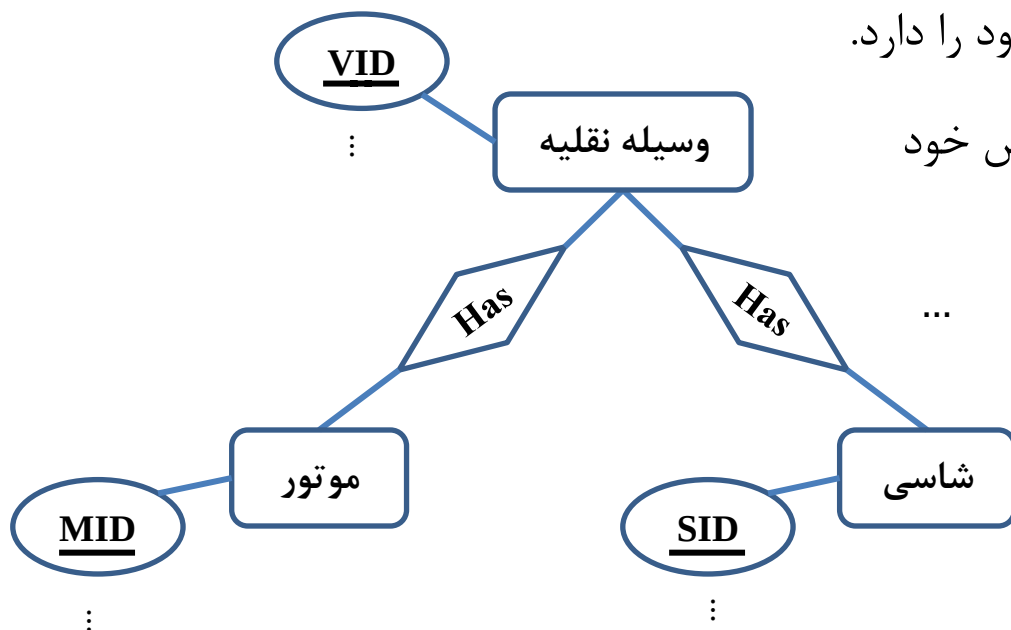
E دارد F.

نکته: نوع کل مجموعه صفات خاص خود را دارد.

نکته: نوع جزء هم مجموعه صفات خاص خود

را دارد [از جمله شناسه].

ارتباط شاسی و موتور با وسیله نقلیه





ارتباط “IS-A-PART Of” (ادامه)

بخش دوم: مدلسازی معنایی داده ها

۶۰

تفاوت های نوع ضعیف با نوع جزء: ☐

☐ نوع جزء از خود شناسه دارد ولی نوع ضعیف نه.

☐ با حذف نوع کل لزوماً نوع جزء حذف نمی شود (به عبارتی وابستگی وجودی لزوماً نداریم).

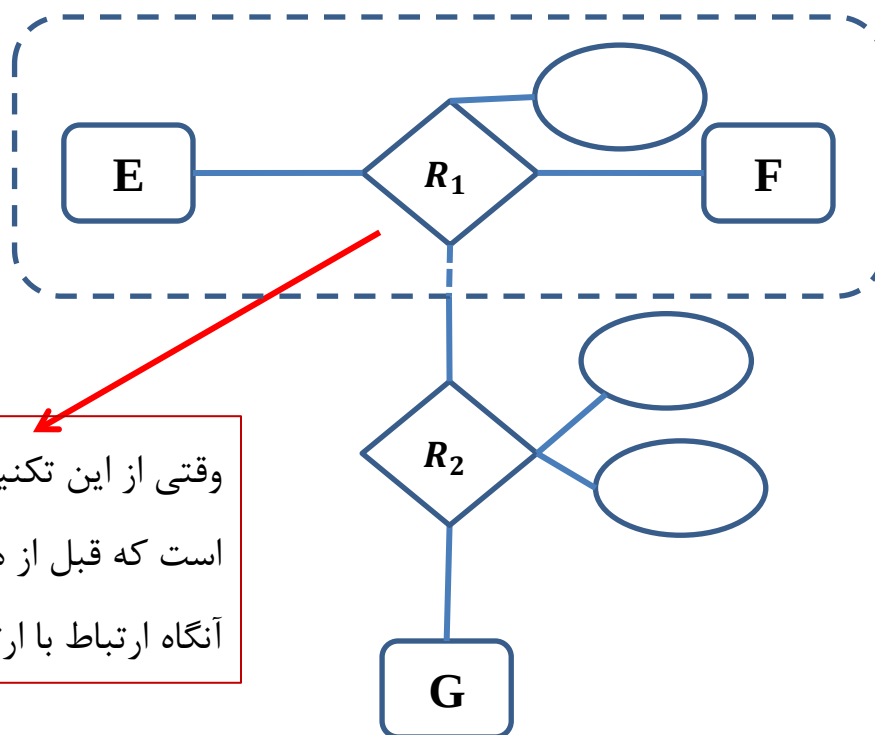
☐ ...؟

☐ در ارتباط “IS-A-PART Of” ← تکنیک تجزیه: دیدن نوع موجودیت های جزء از روی نوع موجودیت کل
تکنیک ترکیب: دیدن نوع موجودیت کل از روی اجزاء

❑ **تکنیک تجمیع (Aggregation):** دیدن $N \geq 1$ نوع موجودیت شرکت کننده در ارتباط R ، به صورت

یک نوع موجودیت انتزاعی: به منظور مدلسازی ارتباط با ارتباط (به ویژه زمانی که نوع ارتباط R صفاتی هم داشته باشد).

❑ ارتباط با ارتباط حیطه معنایی خاص خود را دارد.



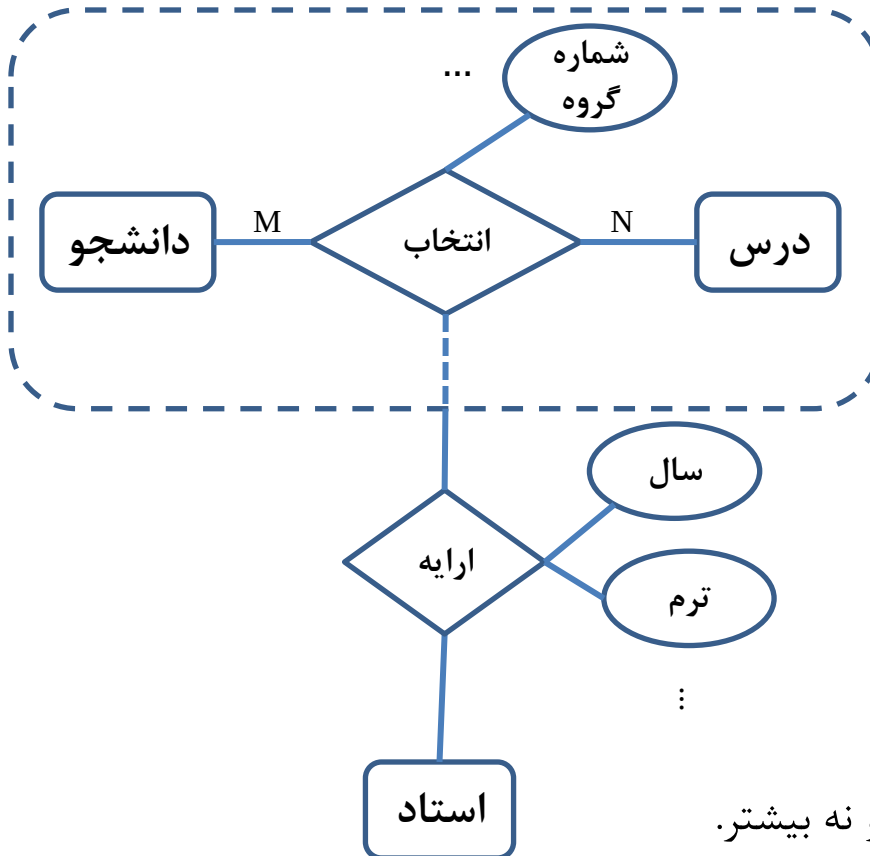
دیدن موجودیتهای دخیل
در ارتباط R_1 به صورت
یک موجودیت انتزاعی

وقتی از این تکنیک استفاده می شود، معنایش این
است که قبل از هر چیز به ارتباط R_1 نیاز است.
آنگاه ارتباط با ارتباط مطرح شده است.

بخش دوم: مدلسازی معنایی داده ها

معمولا از این تکنیک زمانی استفاده می شود که چندی ارتباط $M:N$ باشد.  چرا ؟

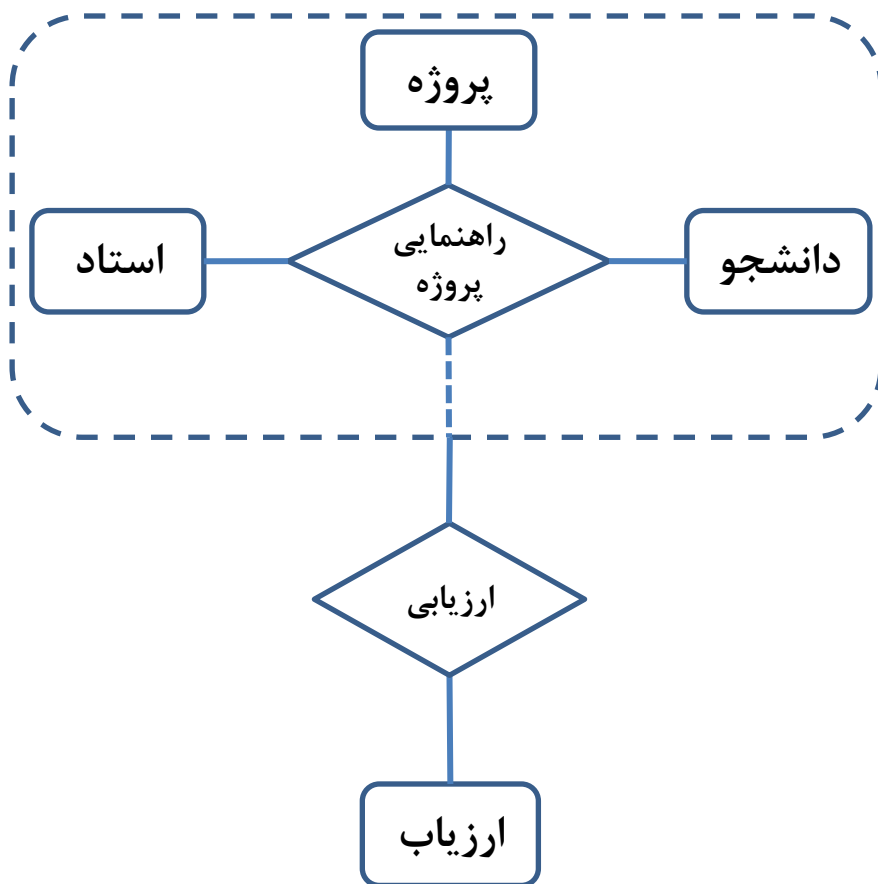
طرز دیگر مدلسازی برای محیط دانشجو - درس - استاد:



نکته: هر Aggregation برای یک ارتباط است و نه بیشتر. 



ارزیابی راهنمایی پروژه پژوهشی دانشجو توسط استاد





☐ نکات زیر بررسی شود:

☐ ویژگی های عمومی روش مدلسازی

☐ کمداشت های روش [E]ER

☐ تناظر بین مفاهیم روش [E]ER و روش UML [در نمودار رده Class diagram]



- ۱- مطالعه، تحلیل و شناخت محیط
- ۲- برآورد خواسته‌ها و نیازهای اطلاعاتی و پردازشی همه کاربران ذیربط محیط (مهندسی نیازها) و تشخیص محدودیت‌های معنایی و قواعد فعالیت‌های محیط
- ۳- بازشناسی نوع موجودیت‌های مطرح و تعیین وضع هر نوع موجودیت (قوی یا ضعیف بودن آن)
- ۴- تعیین مجموعه صفات هر نوع موجودیت، میدان و جنبه‌های هر صفت
- ۵- بازشناسی نوع ارتباط‌های بین نوع موجودیت‌ها، تشخیص الزامی بودن یا نبودن مشارکت در آنها و تشخیص چندی هر ارتباط
- ۶- رسم نمودار ER (یا EER) به صورت واضح، خوانا و حتی‌الامکان با کمترین افزونگی
- ۷- فهرست کردن پرسش‌هایی که پاسخ آنها از نمودار به دست می‌آید (بر حسب گزارش‌های مورد نیاز و کلا نیازهای داده‌ای کاربران)
- ۸- واری مدلسازی انجام شده، برای اطمینان از پاسخگو بودن به نیازهای کاربران.



□ گاه به علت وسعت محیط عملیاتی و تعدد کاربران آن لازم است مدلساز به ازای هر زیرمحیط و یا حتی یک کاربر نمودار ER رسم کند.

□ در این صورت نیازمند **ادغام و یکپارچه‌سازی نمودارهای ER** هستیم.

□ در ادغام چند نمودار ER باید به تعارض‌های (ماهیتا معنایی) بین نمودارها توجه کرد. از جمله موارد زیر:

□ مدل‌های نایکسان برای ایده واحد

□ تعارض در نامگذاری یک مفهوم (از لحاظ معنایی) واحد (دو موجودیت Car و Automobile برای اتومبیل)

□ تعارض معنایی دو مفهوم ظاهرا یکسان (دو موجودیت با عنوان Student؛ یکی به معنای دانشجو و دیگری به معنای دانش‌آموز)

□ تعارض در میدان صفت‌ها

□ تعارض در محدودیت‌ها

□ تحلیل این تعارض‌ها قبل از تصمیم‌گیری درباره ادغام ERها باید انجام شود.



پرسش و پاسخ ...

amini@sharif.edu

به نام آنکه جان را فکرت آموخت



بخش سوم : طراحی منطقی

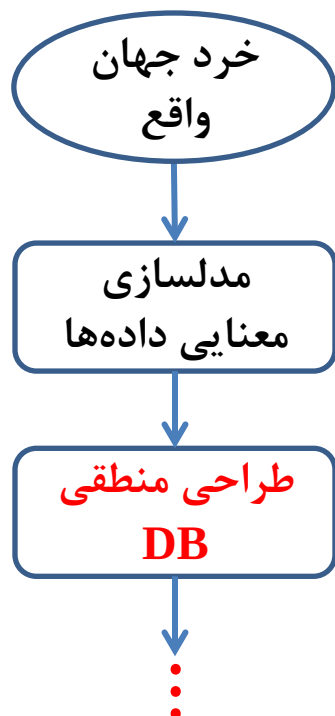
مرتضی امینی

نیمسال دوم ۹۶-۹۷

(محتویات اسلایدها برگرفته از یادداشت‌های کلاسی استاد محمدتقی روحانی رانکوهی است.)



بخش سوم: طراحی منطقی پایگاه داده‌ها



☐ مدلسازی داده‌ها می‌تواند در سطوح انتزاعی مختلفی صورت پذیرد.

☐ سطح پایین‌تر از سطح مدلسازی معنایی داده‌ها، سطح طراحی منطقی است.

☐ **سطح طراحی منطقی:** برای نمایش پایگاه داده‌ها در این سطح از مفاهیمی استفاده می‌شود که مستقل از مفاهیم محیط فایلینگ پایگاه داده‌ها است.



□ بحث مقدماتی: دیدگاه کاربردی [و نه تئوریک]

□ برای طراحی منطقی پایگاه داده‌ها (و همچنین عملیات در DB و کنترل DB) هم امکان خاصی لازم است: یک **مدل داده (DM)**، که شامل یک **ساختار داده (DS)** است.

□ مفاهیم مطرح در طراحی منطقی پایگاه داده‌ها

□ ساختار داده جدولی : TDS

□ پایگاه داده جدولی : TDB

□ زبان پایگاهی جدولی : TDBL



□ چرا ساختار داده (در معنای عام)؟

□ برای نمایش نوع موجودیت‌ها و ارتباط بین آنها در سطح
} منطقی
} فیزیکی

□ دلایل لزوم ساختار داده (DS) در حیطه پایگاهی:

- ۱- تامین کننده محیط فرافایلی (محیط انتزاعی)
- ۲- مبنا و چارچوب طراحی منطقی DB
- ۳- مبنا و چارچوب طراحی زبان پایگاه داده‌ها DBL
-
- ۴- مبنا و چارچوب طراحی خود DBMS
- ۵- ضابطه‌ای است برای مقایسه DBMS ها و ارزیابی آنها
- ۶- مبنایی است برای ایجاد و گسترش تکنیک‌های طراحی DB
- ۷- ...



□ DSها [در حیطه دانش و تکنولوژی DB]:

پیش رابطه‌ای (پکیج IMS و IDMS) ←	{	HDBMS	□ ۱- HDS [از Hierarchical DM]
		NDBMS	□ ۲- NDS [از Network DM]
	{	RDBMS	□ ۳- RDS [از Relational DM]
		ODDBMS	□ ۴- ODS [از Object DM]
		ORDBMS	□ ۵- ORDS [از Object Relational DM]

□ TDS- ساختار داده جدولی:

□ عنصر ساختاری اساسی در Relational Model (RM)، مفهوم **رابطه** است.

□ رابطه [Relation]: یک مفهوم ریاضی است ...

□ اما از دید کاربر [در عمل]: نمایش جدولی دارد.

▪ فعلا به جای ساختار داده رابطه‌ای (RDS) می‌گوییم ساختار داده جدولی (TDS).

(البته رابطه و جدول تفاوت‌های جدی با هم دارند که در مباحث بعدی درس بدان پرداخته می‌شود.)



اصطلاحات TDS:

□ نوع جدول ← { نام جدول
نام و نوع ستون ها } برای نمایش نوع { موجودیت و/یا
ارتباط

□ سطر ← برای نمایش نمونه { موجودیت
ارتباط

□ ستون ← برای نمایش صفت

عنصر ساختاری اساسی:

□ هر DS حداقل یک **عنصر ساختاری اساسی** دارد.

عنصری است که به کمک آن نوع موجودیت، نوع ارتباط، و یا هردو آنها را نمایش می‌دهیم.



□ TDS فقط یک عنصر ساختاری اساسی دارد : همان **نوع جدول**



TDB چیست؟ ☐

■ از دید کاربر

$$\left\{ \begin{array}{l} \text{طراح} \\ \text{پیاده ساز DB} \\ \text{برنامه ساز AP} \end{array} \right\}$$

■ از لحاظ نوع: مجموعه‌ای است از تعدادی **نوع جدول** (که آنها را طراحی می‌کنیم)

■ از لحاظ محتوای داده‌ای [در سطح نمونه]: مجموعه‌ای است از نمونه‌های متمایز یک [چند] **نوع سطر**

■ نوع سطر را همان نوع جدول مشخص می‌کند.



☐ صفت **شناسه** در نوع موجودیت‌ها، حکم **کلید** را در جدول دارد.

☐ مفهوم کلید در مدل داده جدولی تعریف نشده است و برگرفته از مفاهیم تعریف شده در مدل داده‌ای رابطه است.

[از دیدگاه کاربردی] **کلید** امکان دسترسی به تک نمونه (از یک نوع موجودیت یا نوع ارتباط) را فراهم می‌نماید. لذا مقدار آن در سطرهای جدول مبین موجودیت یا ارتباط، **یکتا** است.



[از دیدگاه کاربردی] یک یا چند صفت (ستون) تشکیل **کلید اصلی** را در یک جدول می‌دهند اگر مقادیر آن(ها) در سطرهای جدول **یکتا** و **معلوم** باشد.



☐ در مواقعی ممکن است بیش از یک کلید داشته باشیم. یکی از کلیدها که مقادیرش در همه سطرها معلوم است را کلید اصلی می‌گیریم (بقیه را با یکتا بودن مقادیر - با استفاده از UNIQUE در SQL - مشخص می‌نماییم).

☐ در معرفی مدل داده رابطه‌ای، با انواع کلید و تعاریف آنها آشنا می‌شوید.

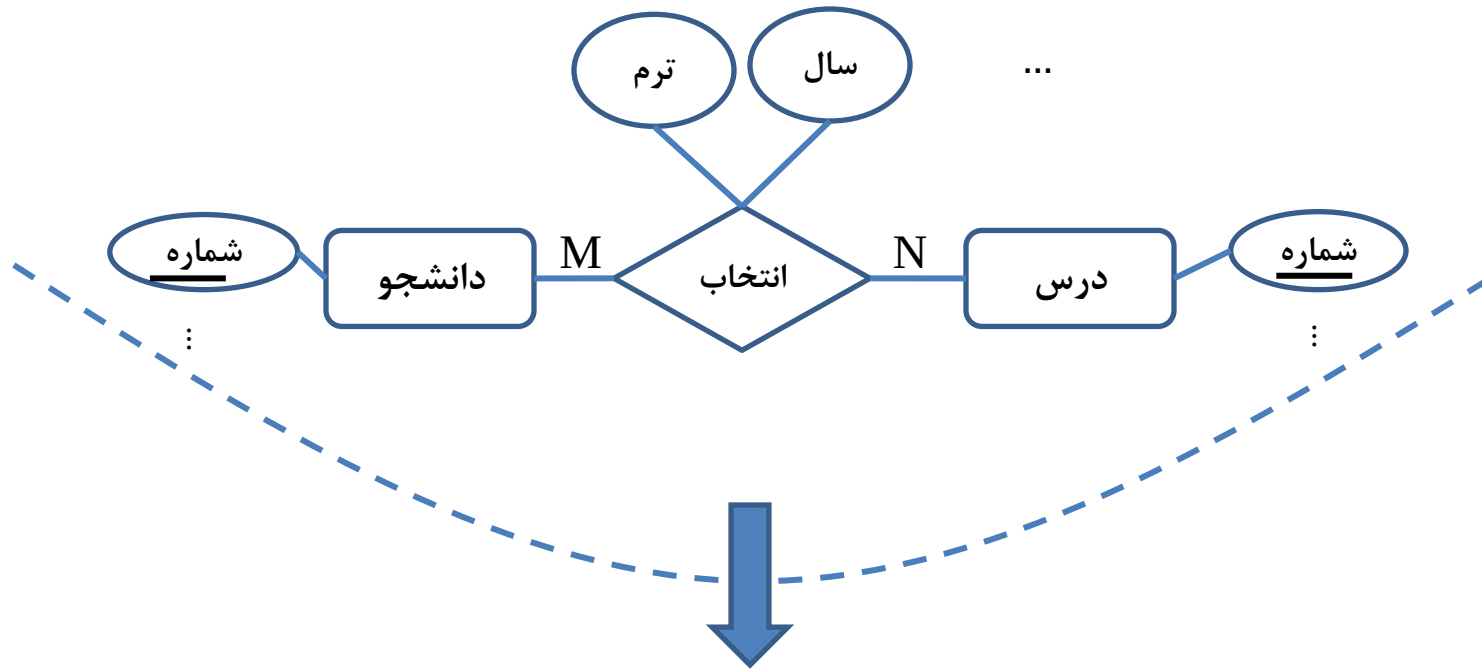


طراحی منطقی با TDS – رابطه چند به چند

۹

بخش سوم: طراحی منطقی پایگاه داده‌ها

چندی M:N



مساله: تبدیل به TDB [با TDS]

سه نوع جدول لازم داریم: ←
} برای هر نوع موجودیت یک نوع جدول
} برای نوع ارتباط M:N یک نوع جدول



طراحی منطقی با TDS – رابطه چند به چند (ادامه)

بخش سوم: طراحی منطقی پایگاه داده‌ها

۱۰

خط زیرین
نمایانگر کلید

STT	<u>STID</u>	STNAME	STLEV	STMJR	STDEID
	777	st7	bs	phys	d11
	888	st8	ms	math	d12
	444	st4	ms	phys	d11
	:	:	:	:	:

COT	<u>COID</u>	COTITLE	CREDIT	COTYPE	CEDEID
	:	:	:	:	:
	co3	programming	4	t (تئوری)	d13
	:	:	:	:	:



طراحی منطقی با TDS – رابطه چند به چند (ادامه)

بخش سوم: طراحی منطقی پایگاه داده‌ها

۱۱

طبق قواعد معنایی محیط ممکن است سال و ترم هم جزو کلید باشند.

(در واقع اگر صفت چند مقداری مرکب برای رابطه باشند، جزو کلید محسوب می‌شوند.)

STCOT



<u>STID</u>	<u>COID</u>	<u>TR</u>	<u>YR</u>
:	:	:	:
888	co2	1	87
888	co3	1	87
444	co2	1	87



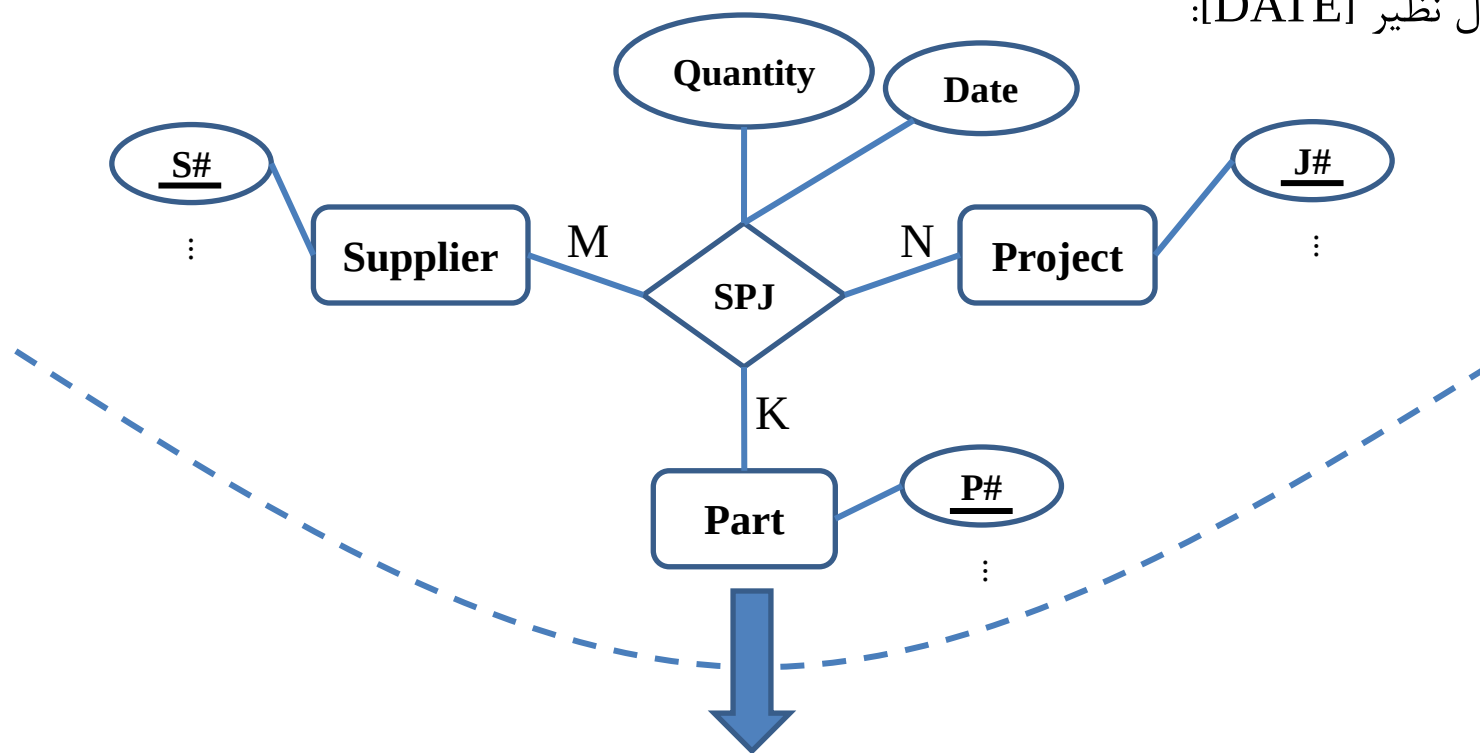
طراحی منطقی با TDS – رابطه چند به چند (ادامه)

بخش سوم: طراحی منطقی پایگاه داده‌ها

۱۲



مثال نظیر [DATE]:



مساله: تبدیل به TDB با TDS

چهار نوع جدول داریم: ←
} برای هر نوع موجودیت یک نوع جدول
} برای نوع ارتباط یک نوع جدول



طراحی منطقی با TDS – رابطه چند به چند (ادامه)

۱۳

بخش سوم: طراحی منطقی پایگاه داده‌ها

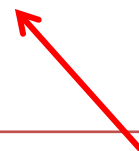
Supplier	<u>S#</u>	SNAME	CITY	...
	s1	...	c1	...
	s2	...	c1	...
	:	:	:	:

Part	<u>P#</u>	PNAME	CITY	...
	p1	...	c1	...
	p2	...	c2	...
	:	:	:	:

Project	<u>J#</u>	JNAME	CITY	...
	j1	...	c2	...
	j2	...	c1	...
	:	:	:	:

طبق قواعد معنایی محیط ممکن است تاریخ هم جزو کلید بشود.
(در واقع اگر صفت چند مقداری باشد، جزو کلید محسوب می‌شود.)

SPJ	<u>S#</u>	<u>P#</u>	<u>J#</u>	<u>Date</u>	QTY
	s1	p1	j1	d1	100
	s1	p1	j1	d2	50
	:	:	:	:	:



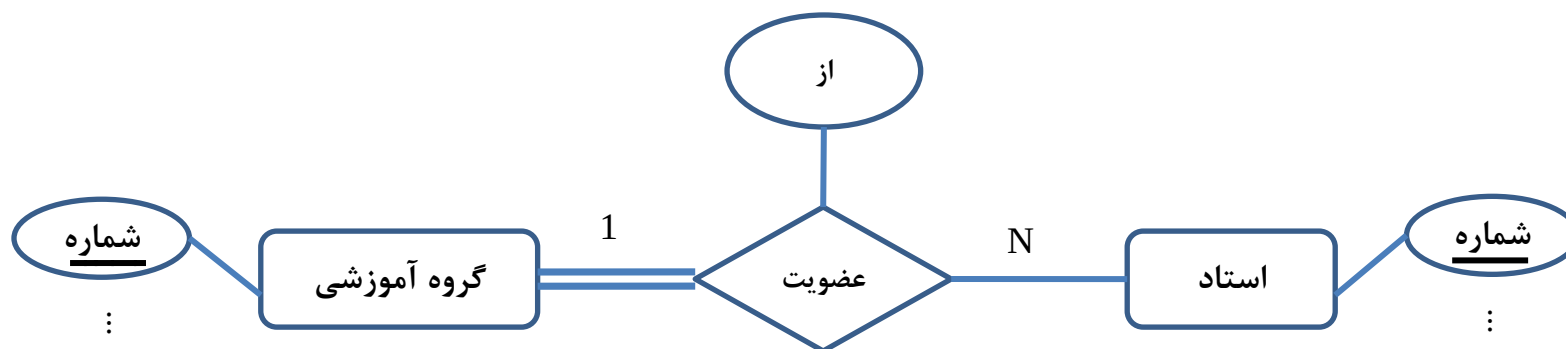


طراحی منطقی با TDS – رابطه یک به چند

بخش سوم: طراحی منطقی پایگاه داده‌ها

۱۴

مثال چندی 1:N



□ دو نوع جدول داریم: } یکی برای نوع موجودیت سمت 1
یکی برای نوع موجودیت سمت N و نیز خود ارتباط



طراحی منطقی با TDS – رابطه یک به چند (ادامه)

۱۵

بخش سوم: طراحی منطقی پایگاه داده‌ها

DEPT	<u>DEID</u>	DETITLE	...	DEPHONE
	D11	Phys	...	...
	D12	Math		
	:	:	:	:

PROF	<u>PRID</u>	PRNAME	RANK	...	FROM	<u>DEID</u>
	Pr100	...	استاد	...	d1	D13
	Pr200	...	استادیار	...	d2	D11
	Pr300	...	دانشیار	...	?	?

* ستون DEID در جدول PROF **کلید خارجی** است و با خط‌چین مشخص می‌شود.

کلید خارجی [کاربردی]: ستون c در جدول T2 کلید خارجی است هرگاه این ستون در جدول دیگری مانند T1 کلید اصلی باشد.



در چه حالاتی استفاده از سه نوع جدول قابل توجیه است؟





طراحی منطقی با TDS – رابطه یک به یک

بخش سوم: طراحی منطقی پایگاه داده‌ها

۱۶

چندی 1:1



□ دو نوع جدول داریم: } یکی برای نوع موجودیت سمت 1 غیرالزامی
یکی برای نوع موجودیت سمت 1 الزامی و نیز خود ارتباط



طراحی منطقی با TDS – رابطه یک به یک (ادامه)

بخش سوم: طراحی منطقی پایگاه داده‌ها

۱۷

DEPT	<u>DEID</u>	DETITLE	...	DEPHONE	<u>PRID</u>
	D11	Phys	...	...	...
	D12	Math	...	...	...
	:	:	:	:	:

یک طرز طراحی ممکن: ☐

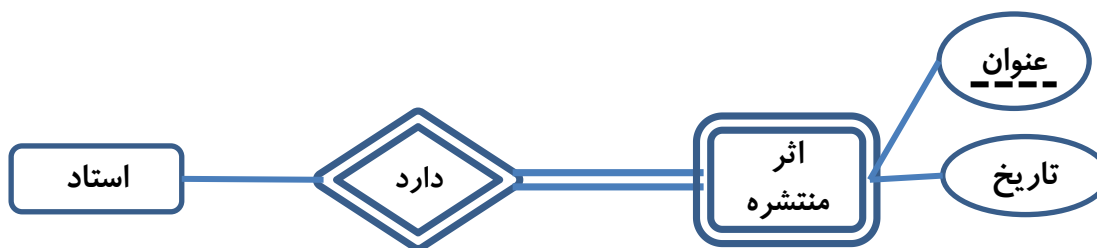
PROF	<u>PRID</u>	PRNAME	RANK	...
	Pr100	...	استاد	...
	Pr200	...	استادیار	...
	Pr300	...	دانشیار	...
	:	:	:	:

طرزهای دیگر طراحی؟





رابطه شناسا (رابطه موجودیت ضعیف)



□ دو نوع جدول داریم: ← یکی برای نوع موجودیت قوی } یکی برای نوع موجودیت ضعیف و رابطه (حاوی شناسه موجودیت قوی)



طراحی منطقی با TDS – رابطه شناسا (ادامه)

۱۹

بخش سوم: طراحی منطقی پایگاه داده‌ها

PROF	<u>PRID</u>	PRNAME	RANK	...
	Pr100	...	استاد	...
	Pr200	...	استادیار	...
	Pr300	...	دانشیار	...
	⋮	⋮	⋮	⋮

PUB	<u>PRID</u>	PTITLE	...	PDATE
	Pr100	Data Encryption...	...	...
	Pr100	Semantic Analysis of ...		
	⋮	⋮	⋮	⋮

* دو صفت PRID (کلید خارجی از جدول PROF) و TITLE، کلید اصلی جدول انتشارات را تشکیل می‌دهند.

حذف و بروزرسانی در جدول PROF چه تاثیری بر PUB باید داشته باشد.





□ دو نوع جدول داریم: ← یکی برای زیرنوع موجودیت (حاوی صفات خاص زیرنوع و شناسه زیرنوع) یکی برای زبرنوع موجودیت (حاوی صفات عام یا مشترک)



طراحی منطقی با TDS – رابطه IS-A (ادامه)

۲۱

بخش سوم: طراحی منطقی پایگاه داده‌ها

EMP	<u>EID</u>	ENAME	EBDATE	...	EPHONE
	E100	...	...	...	...
	E101				
	E102	...	...	...	...
	:	:	:	:	:

PROG	<u>EID</u>	LANG	...	LEVEL
	E100	C++	...	...
	E102	Java		
	:	:	:	:

* EID (کلید خارجی از جدول EMP) کلید اصلی جدول PROG نیز هست.

حذف و بروزرسانی در جدول EMP چه تاثیری بر PROG باید داشته باشد (و بالعکس)؟



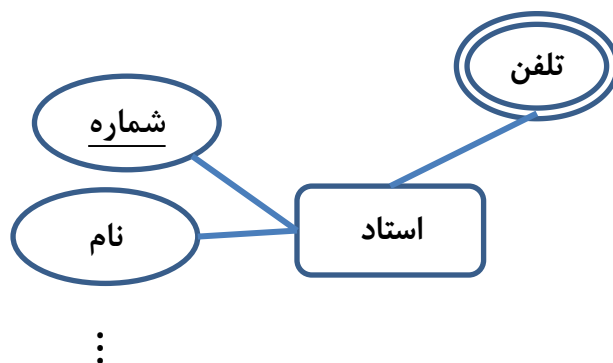


طراحی منطقی با TDS – صفت چندمقداری

بخش سوم: طراحی منطقی پایگاه داده‌ها

۲۲

صفت چندمقداری



□ دو نوع جدول داریم: ← } یکی برای نوع موجودیت (حاوی صفات تک‌مقداری)
یکی برای صفت (ساده یا مرکب) چندمقداری



طراحی منطقی با TDS – صفت چندمقداری (ادامه)

بخش سوم: طراحی منطقی پایگاه داده‌ها

۲۳

PROF

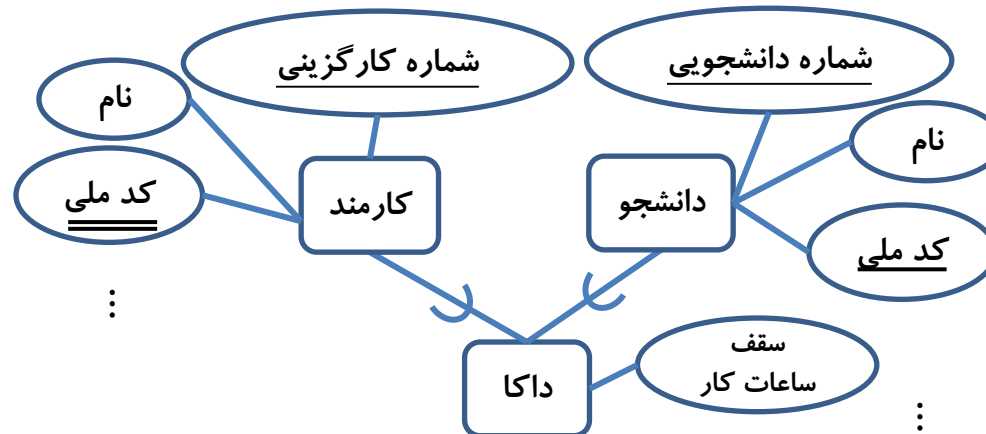
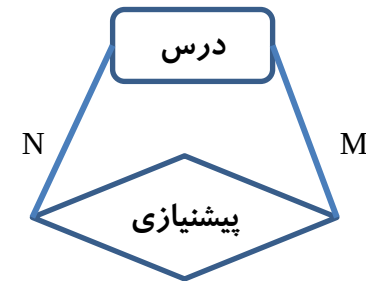
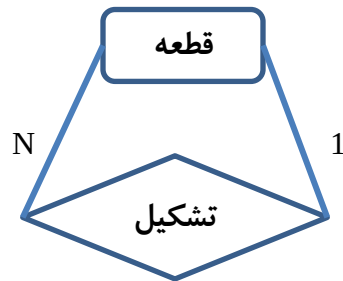
<u>PRID</u>	PNAME	RANK	...	...
Pr100	...	...	...	...
Pr101				
Pr102	...	...	...	...
⋮	⋮	⋮	⋮	⋮

PROFTEL

<u>PRID</u>	<u>TEL</u>
Pr100	09121234567
Pr100	02177889911
Pr101	09352348762
⋮	⋮

بخش سوم: طراحی منطقی پایگاه داده‌ها

تمرین: TDB را برای مدل‌سازی‌های زیر طراحی کنید.



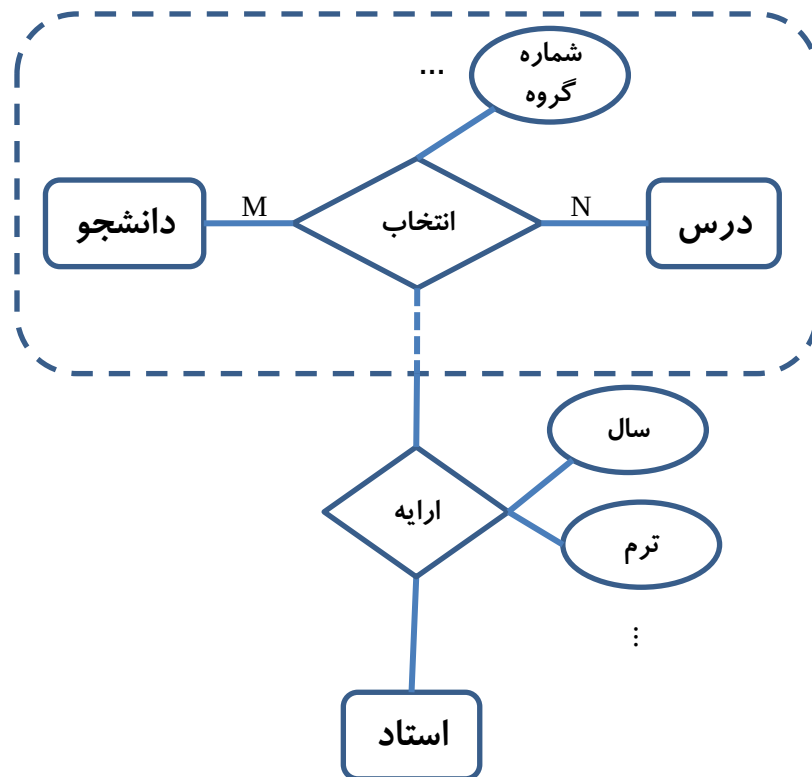
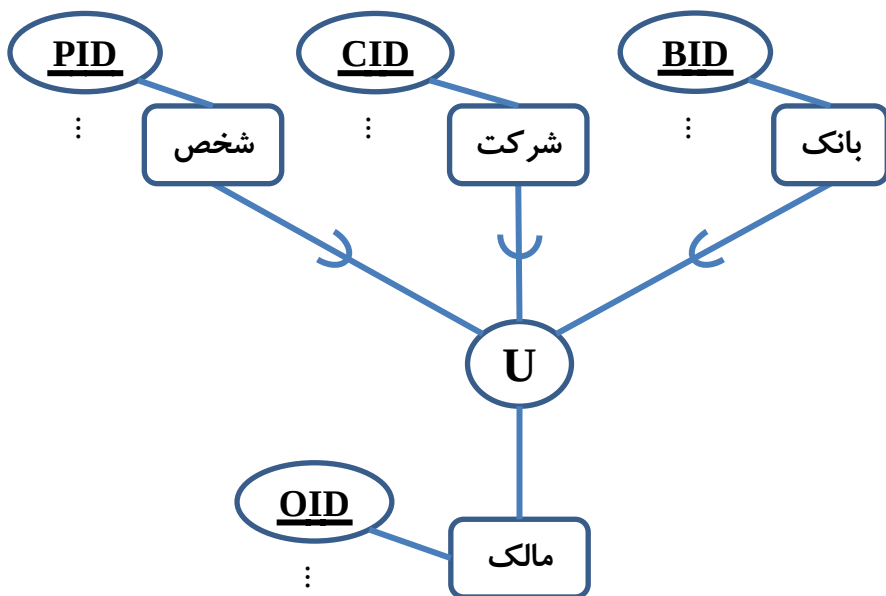


طراحی منطقی با TDS (ادامه)

۲۵

بخش سوم: طراحی منطقی پایگاه داده‌ها

تمرین: TDB را برای مدل‌سازی‌های زیر طراحی کنید.





پرسش و پاسخ ...

amini@sharif.edu

به نام آنکه جان را فکرت آموخت



بخش چهارم: مقدمات پیاده سازی و SQL

مرتضی امینی

نیمسال دوم ۹۶-۹۷

(محتویات اسلایدها برگرفته از یادداشت های کلاسی استاد محمدتقی روحانی رانکوهی است.)



بخش چهارم: مقدمات پیاده سازی و SQL

- برای پیاده سازی طراحی منطقی انجام شده در یک سیستم مدیریت پایگاه داده ها نیاز به یک زبان پایگاهی داریم.
- زبان SQL زبان استاندارد انجام عملیات پایگاهی در پایگاه داده های رابطه ای (از دیدگاه کاربردی: جدولی) است.

□ دستورهای (SQL) Structured Query Language

 Data Definition Language (DDL)
 Data Manipulation Language (DML)
 Data Control Language (DCL)

□ چند دستور از DDL

 CREATE TABLE ایجاد جدول
 DROP TABLE حذف جدول
 ALTER TABLE تغییر جدول

- **نکته:** در دستورات SQL در دو طرف مقادیر متنی یا رشته ای از single quote استفاده می شود (بسیاری از سیستم های پایگاه داده double quote را هم می پذیرند) ولی در اطراف مقادیر عددی چیزی قرار نمی گیرد.



تعریف و حذف پایگاه داده و شِما

بخش چهارم: مقدمات پیاده‌سازی و SQL

۳

☐ دستور تعریف پایگاه داده

CREATE DATABASE *DatabaseName*

☐ دستور حذف پایگاه داده

DROP DATABASE *DatabaseName*

☐ در اغلب سمپادها می‌توان در یک پایگاه داده چند شِما تعریف کرد.

☐ دستور تعریف و حذف شِما

CREATE SCHEMA *SchemaName*

DROP SCHEMA *SchemaName*

☐ شمای پایگاه داده‌ها عبارت است از تعریف (توصیف) ساختهای منطقی طراحی شده و نوعی برنامه است

شامل تعدادی دستور برای تعریف و کنترل داده‌ها.

☐ در واقع شِما شامل همه جداول، نوعها، دامنه‌ها، دیدها و محدودیتهای مرتبط با یک برنامه کاربردی است.



دستور تعریف جدول CREATE TABLE ☐

CREATE TABLE *TableName*

```
{ ( columnName dataType [NOT NULL | UNIQUE]
[DEFAULTL defaultOption][CHECK (searchCondition)] [, ...] ) }
[PRIMARY KEY (listOfColumns), ]
{[UNIQUE (listOfColumns),][, ...]}
{[FOREIGN KEY (listOfForeignKeyColumns)
REFERENCES ParentTableName [(listOfCandidateKeyColumns)],
[ON UPDATE referentialAction]
[ON DELETE referentialAction]][, ...]}
{[CHECK (searchCondition)][, ...]}
```

تعریف جدول‌ها: شمای پایگاه جدولی

می‌توان جدول را به صورت موقت نیز (با استفاده از CREATE TEMPORARY TABLE) ایجاد کرد. جدول ☐

موقت حاوی داده‌های ناپایا است و پس از اینکه برنامه کاربر (SQL Session) اجرایش تمام بشود، این جدول توسط سیستم حذف می‌شود.



□ انواع داده‌های قابل استفاده در تعریف ستون‌ها عبارتند از:

□ کاراکتری: CHAR(n), VARCHAR(n)

□ بیتی: BIT [VARYING] (n)

□ عددی: NUMERIC(p, q), REAL, INTEGER, SMALLINT, FLOAT(p),

DOUBLE PRECISION

□ زمانی: DATE, TIME, TIMESTAMP, INTERVAL



□ در برخی DBMS ها، نوع داده‌های خاصی پشتیبانی می‌شود که امکان ذخیره، بازیابی و پردازش داده‌های

از آن نوع را برای کاربر تسهیل می‌نماید. به طور مثال نوع داده جغرافیایی در PostgreSQL.



Default ☐ تعیین مقدار پیش‌فرض یک ستون

Not Null ☐ ستون ناهیچ‌مقدار

Unique ☐ یکتایی مقادیر ستون(ها)

Primary Key ☐ کلید اصلی (می‌توان تعدادی از ستونها را با یکدیگر به عنوان کلید اصلی تعریف کرد)

Foreign Key ... References ... ☐ کلید خارجی (می‌توان تعدادی از ستونها را با یکدیگر به عنوان

کلید خارجی تعریف کرد)

Check ☐ تعیین محدودیت مقداری برای مقادیر ستون



مثالی از تعریف جدول

بخش چهارم: مقدمات پیاده‌سازی و SQL

۷

شِمای پایگاه داده جدولی:



```
CREATE TABLE STT
( STID          CHAR(8) NOT NULL,
  STNAME        CHAR(25),
  STLEV         CHAR(12),
  STMJR         CHAR(20),
  STDEID        CHAR(4)
)
PRIMARY KEY (STID);
CHECK STMJR IN { 'bs', 'ms', 'doc', '???' }
```

```
CREATE TABLE COT
( COID          CHAR(6) NOT NULL,
  COTITLE       CHAR(16),
  CREDIT        SMALLINT,
  COTYPE        CHAR(1),
  CODEID        CHAR(4),
)
PRIMARY KEY (COID);
```

محدودیت صفتی (ستونی) [کلاز کنترلی]



مثالی از تعریف جدول (ادامه)

بخش چهارم: مقدمات پیاده‌سازی و SQL

۸

CREATE TABLE SCT

(STID CHAR(8) NOT NULL ,
COID CHAR(6) NOT NULL ,
TR CHAR(1) ,
YR CHAR(5) ,
GRADE DECIMAL(2 , 2)
)

PRIMARY KEY (STID, COID)

CHECK 0 >= GRADE AND GRADE <= 20

محدودیت صفتی (ستونی) [کلاس کنترلی]

FOREIGN KEY (STID) REFERENCES STT (STID)

ON DELETE CASCADE

ON UPDATE CASCADE

FOREIGN KEY (COID) REFERENCES COT (COID)

ON DELETE CASCADE

ON UPDATE CASCADE



دستور حذف جدول DROP TABLE

DROP TABLE *tablename* [CASCADE| RESTRICT]

CASCADE  باعث می‌شود که همه اشیاء وابسته به جدول (مانند دیدهای تعریف شده بر روی آن یا

محدودیت‌هایی مانند کلید خارجی وابسته به آن) نیز به صورت خودکار حذف شود.

RESTRICT  در صورت وجود دیگر اشیاء وابسته به جدول، از حذف آن جلوگیری می‌کند. پیش‌فرض

این دستور، RESTRICT است.



DROP TABLE SCT



□ دستور تغییر جدول ALTER TABLE

ALTER TABLE *tableName* اضافه کردن ستون، تغییر تعریف ستون، حذف ستون و ...

[ADD [COLUMN] *columnName* *dataType* [NOT NULL] [UNIQUE]

[DEFAULT *defaultOption*] [CHECK (*searchCondition*)]]

[DROP COLUMN *columnName* [RESTRICT | CASCADE]]

[ADD CONSTRAINT [*constraintName*]] *tableConstraintDefinition*

[DROP CONSTRAINT *constraintName* [RESTRICT | CASCADE]]

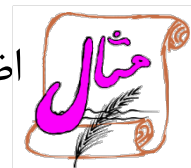
[ALTER COLUMN SET DEFAULT *defaultOption*]

[ALTER COLUMN DROP DEFAULT]

[RENAME COLUMN *columnName* TO *newColumnName*]

...

اضافه کردن ستون «وضعیت» به جدول اطلاعات دانشجو



ALTER TABLE STT

ADD COLUMN STATE CHAR(10)



بخش چهارم: مقدمات پیاده‌سازی و SQL

و نه دستورات

Data Manipulation (DM)

Data Definition (DD)

Data Controller (DC)

در شمای پایگاهی ← دستورات

این جدایی چه مزایایی دارد؟



سیستم با شمای پایگاهی چه می‌کند؟



اطلاعات موجود در آن را در جایی به نحوی ذخیره می‌کند. ← **در تعدادی جدول**

کاتالوگ سیستم

دیکشنری سیستم

مِتا داده‌ها

حاوی فراداده‌ها و داده‌های کنترلی در مورد داده‌های کاربران



مثالی از جدول‌های کاتالوگ:



SysTables

...	تعداد ستون	تاریخ	ایجاد کننده	نام جدول
	5	D1	C1	STT
	5	D2	C1	COT
	5	D2	C2	SCT
	⋮	⋮	⋮	⋮

جدولی که جدول‌ها را مدیریت می‌کند.

SysCols

...	طول	نوع	نام جدول	نام ستون
	8	CHAR	STT	STID
	25	CHAR	STT	STNAME
	⋮	⋮	⋮	⋮
	2,2	DEC	SCT	GR

جدولی که ستون‌ها را مدیریت می‌کند.



آیا برنامه ساز می تواند محتوای کاتالوگ را مستقیماً تغییر دهد؟ (با دستورات INSERT, DELETE, UPDATE)

تمرین: حداقل سه جدول دیگر برای کاتالوگ طراحی کنید.

تمرین: چه اطلاعاتی در کاتالوگ ذخیره می شود؟



❑ عملیات در TDB : دستورهای DML

SELECT

بازیابی

INSERT

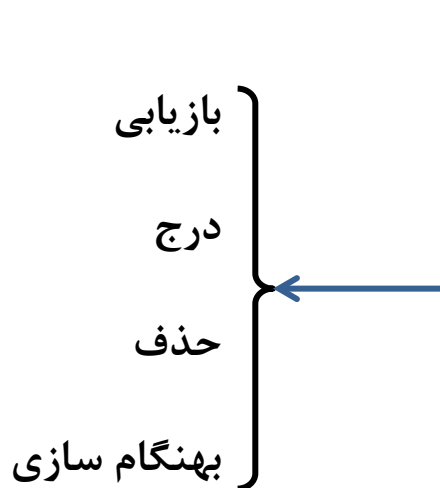
درج

DELETE

حذف

UPDATE

بهنگام سازی





دستور بازیابی SELECT ☐

```
SELECT [ALL | DISTINCT ] item(s) list  
FROM table(s) expression  
[WHERE condition(s)]  
[ORDER BY Col(s)]  
[GROUP BY Col(s)]  
[HAVING condition(s)]
```

☐ خروجی دستور SELECT یک جدول است.

☐ از DISTINCT برای حذف سطرهای تکراری در جدول نتیجه استفاده می‌شود.

☐ در شرط WHERE می‌توان از =، <>، <، >، <=، >=، BETWEEN، LIKE و IN استفاده کرد.



بخش چهارم: مقدمات پیاده‌سازی و SQL

برای تغییر نام

```
SELECT STT.STID AS SN,  
       STT.STNAME AS SName  
FROM   STT  
WHERE  STT.STMJR='phys'  
AND  
       STT.STLEV='bs'
```



```
SELECT STT1.STID AS SN,  
       STT1.STNAME AS SName  
FROM   STT AS STT1  
WHERE  STT1.STMJR='phys'  
AND  
       STT1.STLEV='bs'
```





مرتب سازی (ORDER BY) □

SELECT S.*

FROM S

ORDER BY SNAME یا 2

↓ ↓
نام ستون شماره ستون

■ پیش فرض صعودی: (Ascending)

■ نزولی (Descending): باید قید شود.

قابلیت‌های پیشرفته (Advanced features): □

SELECT S#, CITY

FROM S

WHERE SNAME

{
LIKE
NOT LIKE
}

{
'%N' → با N تمام شود
'M%' → با M شروع شود
'_ _ A _ _' → دقیقاً ۵ کاراکتر، کاراکتر سوم A
}



SELECT P#

FROM P

WHERE WEIGHT BETWEEN (5,15)

یا

WHERE WEIGHT >=5 AND WEIGHT <=15

BETWEEN



□ شماره قطعاتی را بدهید که وزن آنها بین ۵ و ۱۵ است.



NULL



SELECT S#, CITY

FROM S

WHERE STATUS

$\left\{ \begin{array}{l} \text{IS NULL} \\ \text{IS NOT NULL} \end{array} \right\}$

بررسی برخورد یک package با NULL؟





$tablename1$ op $tablename2$ [CORRESPONDING [BY {column, [, column ...]}]]

$$op \in \left\{ \begin{array}{l} \text{UNION [ALL]} \\ \text{INTERSECT [ALL]} \\ \text{EXCEPT [ALL]} \end{array} \right\}$$

☐ اگر از گزینه CORRESPONDING BY استفاده شود، عمل درخواست شده روی ستون‌های تصریح شده انجام می‌شود.

☐ اگر CORRESPONDING بدون BY استفاده شود، عمل درخواست شده روی ستون‌های مشترک انجام می‌شود.

☐ اگر از این گزینه استفاده نشود، عمل روی تمام ستون‌های دو جدول انجام می‌شود.

☐ شرط استفاده: برابری Heading: هم‌نامی و هم‌نوعی ستون(های) دو جدول

☐ **توجه:** تکراری‌ها در نتیجه اجرای عملگرهای جبر مجموعه‌ها حذف می‌شوند مگر آنکه از ALL استفاده شود.



```
SELECT S.S#,  
FROM S  
INTERSECT
```

شماره تهیه کنندگانی را بدهید که حداقل یک قطعه تولید می‌کنند.



```
SELECT SP.S#,  
FROM SP
```

```
SELECT SP.S#,  
FROM SP  
EXCEPT
```

تست سازگاری پایگاه داده‌ها: هر فردی که قطعه ای تولید کرده

باید یکی از افراد ثبت شده در جدول تولیدکنندگان باشد.



```
SELECT S.S#,  
FROM S
```

مدل دیگر



SP **EXCEPT** S Using S# یا Corresponding by S#



شماره تهیه کنندگانی را بدهید که هیچ قطعه‌ای تولید نمی‌کنند.



```
SELECT S.S#,  
FROM S  
EXCEPT  
SELECT SP.S#,  
FROM SP
```

تمرین: این مثال‌ها به طرز دیگر هم نوشته شود. □



Aggregation Functions ☐

← AVG ☐ میانگین

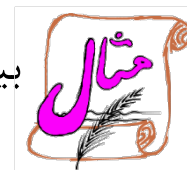
← MIN ☐ مینیمم

← MAX ☐ ماکزیمم

← SUM ☐ جمع

← COUNT(*) / COUNT ☐ تعداد عبارات ناهیچمقدار / تعداد کل سطرها

بیشینه وضعیت تهیه کنندگان در شهرهای c1 یا c2



```
SELECT MAX ( STATUS ) AS SMAX  
FROM S  
WHERE CITY='c1'  
OR  
CITY='c2'
```



تعداد انواع قطعات تولیدی توسط تولیدکنندگان



```
SELECT COUNT (DISTINCT P#) AS N1  
FROM SP
```

تعداد انواع قطعات قابل تولید



```
SELECT COUNT (*) AS N2  
FROM P
```

تعداد کل قطعات تولیدی توسط s2



```
SELECT SUM (QTY) AS N3  
FROM SP  
WHERE S# = 's2'
```

NULL و توابع جمعی؟ (در سه پکیج بررسی شود)





GROUP BY

□ سطرهای جدول داده شده در کلاز FROM را گروه بندی می‌کند، به نحوی که مقدار ستون(های) گروه‌بندی در گروه یکسان است.

تعداد کل قطعات تولیدی توسط هر تولیدکننده



```
SELECT S# AS SN , SUM (QTY) AS SQ  
FROM SP  
GROUP BY S#
```

SP
گروه‌بندی
شده

S#	P#	QTY
s1	p1	...
s1	p2	...
s1	p4	...
s2	p2	...
s2	p3	...
s3	p5	...
...	...	...



جدول جواب

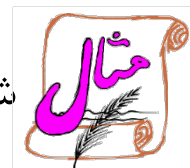
SN	SQ
s1	280
s2	100
s3	203
...	...



در کلاز SELECT نمی توان نام ستونی را آورد که در کلاز GROUP BY نباشد، غیر از ستون‌هایی که با توابع جمعی به دست آمده‌اند.

HAVING

 امکانی است برای دادن شرط یا شرایط ناظر به گروه سطرها



شماره تهیه‌کنندگانی را بدهید که بیش از ۱۰۰ قطعه تولید کرده‌اند.

SELECT S#

FROM SP

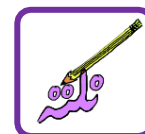
GROUP BY S#

HAVING SUM(QTY) > 100



تمرین: شماره دانشجویانی را بدهید که در ترم دوم سال ۹۴-۹۵ بیش از ۲۰ واحد گرفته باشند.

تمرین: شماره دانشجویانی را بدهید که در ترم دوم سال ۹۴-۹۵ بیش از ۷ درس گرفته باشند.



GROUP BY و HAVING در SQL افزونه‌اند، اما نوشتن QUERY بدون آنها پیچیده است.

HAVING بدون GROUP BY؟



به چند روش می‌توان یک کپی از جدول ساخت؟





روش اول

```
SELECT SNAME
FROM S, SP
WHERE SP.S# = S.S# AND SP.P# = 'p2'
```

شبیه سازی عملگر پیوند

قطعه 'p2' را بدهید:

تهیه کنندگان

نام



در جدول SP

در جدول S

```
SELECT T1.*, T2.*
FROM T1, T2
```

ضرب دکارتی در SQL



مکانیزم اجرا از دید برنامه‌ساز: □

- به ازای هر سطر جدول S، بررسی می‌کند که آیا S# آن در SP وجود دارد یا نه و P# آن سطر در SP، p2 است یا نه. اگر درست بود SNAME آن سطر جزو جواب است.



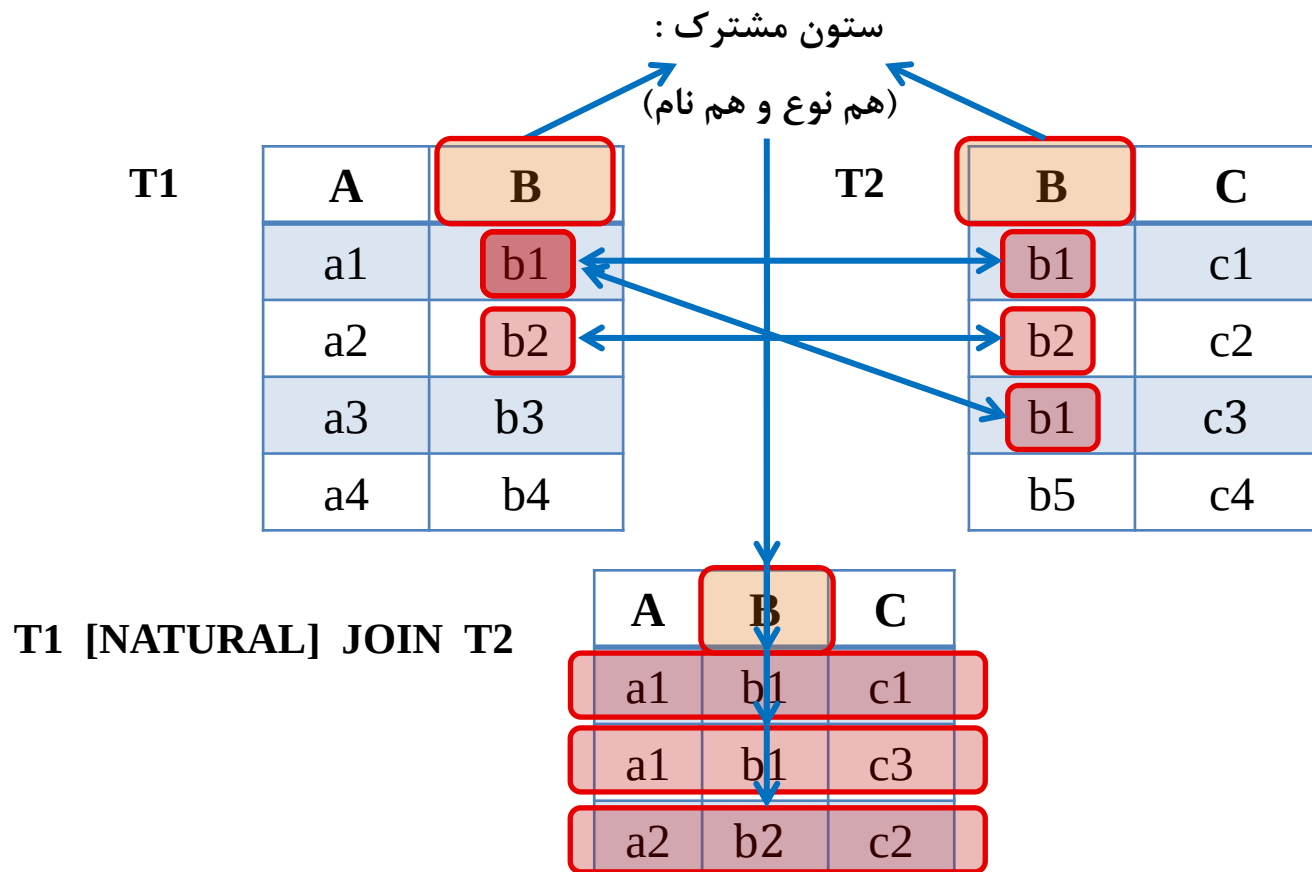
بازیابی از بیش از یک جدول – عملگر پیوند یا JOIN

بخش چهارم: مقدمات پیاده‌سازی و SQL

۲۹

پیوند: ارائه مقدماتی (غیر ریاضی) □

T1 [NATURAL] JOIN T2 □





بازیابی از بیش از یک جدول – عملگر پیوند یا JOIN (ادامه)

بخش چهارم: مقدمات پیاده‌سازی و SQL

۳۰

توضیح مقدماتی عملگر پیوند:

صرف نظر از جزئیات تئوریک، سطرهای دو جدول را که مقدار ستون(های) مشترکشان یکسان است،

به هم پیوند می‌زند.

روش دوم

```
SELECT SNAME
FROM S [NATURAL] JOIN SP
WHERE P# = 'p2'
```

نام تهیه کنندگان قطعه 'p2' را بدهید:



S

S#	SNAME	...
s1	sn1	...
s2	sn2	...
s3	sn3	...
s3	sn4	...
...	...	...

SP

S#	P#	QTY
s1	p1	100
s1	p2	120
s1	p3	500
s2	p1	50
...	...	...

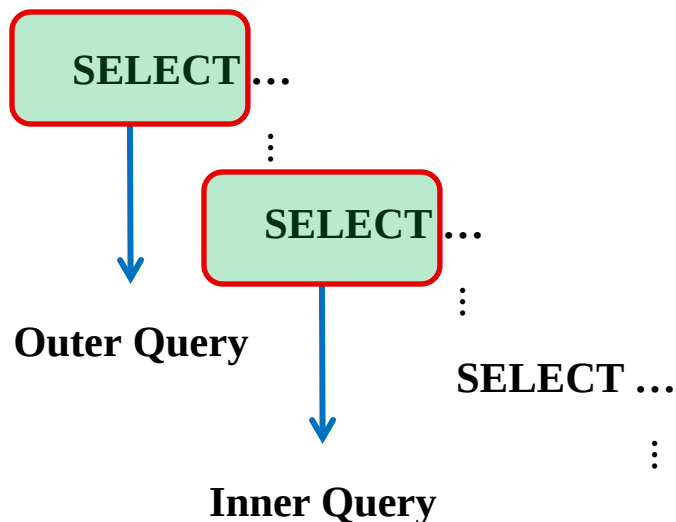
S [NATURAL] JOIN SP

S#	SNAME	...	P#	QTY
s1	sn1	...	p1	100
s1	sn1	...	p2	120
s1	sn1	...	p3	500
s2	sn2	...	p1	50
...	...	...	...	...



زیر پرسش یا SubQuery

یک SELECT است در درون SELECT دیگر. ← پرسش تو در تو



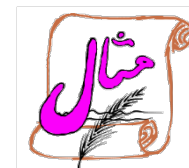


بازیابی از بیش از یک جدول – عملگر تعلق

بخش چهارم: مقدمات پیاده‌سازی و SQL

۳۲

IN و NOT IN: عملگر تعلق □



روش سوم

SELECT SNAME

FROM S

WHERE S# IN (SELECT S# FROM SP

WHERE P# = 'p2')

روش چهارم

یا
= ANY

روش پنجم

یا
= SOME

عملگر تعلق

مکانیزم اجرا: □

- سیستم ابتدا SELECT درونی را اجرا می‌کند، آنگاه به ازای هر سطر S بررسی می‌کند که S# در مجموعه جواب SELECT درونی هست یا نه.



بازیابی از بیش از یک جدول – پرسش های بهم بسته

بخش چهارم: مقدمات پیاده سازی و SQL

۳۳

 دو پرسش درونی و بیرونی (در یک پرسش تو در تو) را **بهم بسته (Correlated)** گوئیم هرگاه در کلاز WHERE پرسش درونی به ستونی از جدول موجود در کلاز FROM پرسش بیرونی، ارجاع داشته باشیم.

 **توجه:** نحوه اجرای پرسش های **بهم بسته** با طرز اجرای پرسش های **ناهم بسته** متفاوت است: در حالت بهم بسته، سیستم پرسش درونی را به ازای هر سطر از جدول پرسش بیرونی یک بار اجرا می کند.

روش ششم

SELECT SNAME

FROM S

WHERE 'p2' IN (SELECT P# FROM SP
WHERE SP.S# = S.S#)

روش هفتم

یا

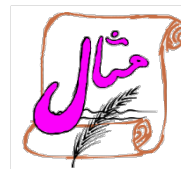
= ANY

روش هشتم

یا

= SOME

CORRELATED یا زیرپرسش بهم بسته





$$\text{theta} \in \left\{ \begin{array}{c} = \\ \neq \\ < \\ \leq \\ \geq \\ > \end{array} \right\} \quad \left\{ \begin{array}{ll} \text{theta} & \text{ANY} \\ \text{theta} & \text{SOME} \\ \text{theta} & \text{ALL} \end{array} \right\} \quad \square \text{ امکان}$$

شماره تهیه کنندگانی را بدهید که مقدار وضعیت آنها بیشینه نباشد.



1- SELECT S#

FROM S

WHERE STATUS < ANY (SELECT DISTINCT STATUS FROM S)

2- SELECT S#

FROM S

WHERE STATUS < (SELECT MAX (STATUS) FROM S)

چون جواب SELECT تک مقداری است نیازی به ANY نیست.



روش نهم



```
SELECT SNAME
FROM S
WHERE 0 < ( SELECT COUNT(*)
              FROM SP
              WHERE SP.S# = S.S#
                  AND
                  SP.P# = 'p2' )
```




NOT EXISTS و EXISTS ☐

☐ امکان بررسی وجود یا عدم وجود سطر در جدول بازگشتی

روش دهم

```
SELECT SNAME
FROM S
WHERE EXISTS ( SELECT *
                FROM SP
                WHERE SP.S# = S.S#
                AND
                SP.P# = 'p2' )
```



روش‌های دیگر؟





دستورهای INSERT, UPDATE, DELETE ☐

درج INSERT: ☐

INSERT INTO *table-name* [(*col1,col2, ...*)]
VALUES (*one row*) | *subquery*

بهنگام سازی UPDATE: ☐

UPDATE *table-name*
SET *col = value* | *expression* [, *col = value* | *expression*]...
:
WHERE *condition(s)* | *subquery*

حذف DELETE: ☐

DELETE FROM *table-name*
WHERE *condition(s)* | *subquery*



درج سطری (سطر کامل – سطر ناقص):



```
INSERT INTO STT  
VALUES ( '222' , 'st2' , 'IT' , 'bs' , 'D17' )
```

```
INSERT INTO STT  
VALUES ( '333' , 'st3' , null , 'ms' , null )
```

درج گروهی:



```
CREATE TEMPORARY TABLE T1  
( STN, .... )
```

اطلاعات دانشجویان مقطع کارشناسی ارشد

```
INSERT INTO T1  
( SELECT STT.*
```

رشته کامپیوتر در جدول موقت T1 درج شود.

```
FROM STT
```

```
WHERE STJ = 'comp'
```

```
AND
```

```
STL = 'ms' )
```



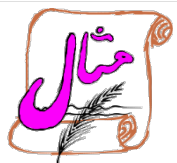
بهنگام سازی چند سطر:



تعداد واحد تمام درس های عملی گروه آموزشی D11 را برابر یک کن.

```
UPDATE COT
SET CREDIT = '1'
WHERE COTYPE = 'p' AND CODEID = 'D11'
```

بهنگام سازی در بیش از یک جدول:



```
UPDATE STT
SET STID = 91104444
WHERE STID = 91107777

UPDATE STCOT
SET STID = 91104444
WHERE STID = 91107777
```

اگر دستور دوم اجرا نشود؟





نمره دانشجویان گروه آموزشی D111 در درس 'com222' در ترم دوم سال ۹۴-۹۵ را ناتمام اعلان



```
UPDATE STCOT
```

```
SET STCOT.GRADE = 'U'
```

```
WHERE STCOT.TR = '2' AND STCOT.YRYR = '94-95'
```

```
AND STCOT.COID = 'COM222'
```

```
AND STID IN (SELECT STID
```

```
FROM STT
```

```
WHERE STT.STDEID = 'D111');
```



حذف تکدرس: درس com111 را برای دانشجوی 88104444 حذف کنید.



```
DELETE FROM STOCOT
```

```
WHERE STID = 88104444
```

```
AND
```

```
COID = 'COM111'
```

آیا این حذف باید انتشار یابد؟



حذف از بیش از یک جدول:



```
DELETE FROM DEPT
```

```
WHERE DEID = 'D333'
```

```
UPDATE STT
```

```
SET DEID = null
```

```
WHERE DEID = 'D333'
```



☐ مطالعه شود :

☐ پرسش بازگشتی (Recursive)

☐ SQL ادغام شده

☐ SQL پویا

☐ نوشتن رویه

☐ نوشتن تابع

☐ امکانات شیء- رابطه‌ای

☐ مدیریت تراکنش

☐ پرسش پارامتری شده (Parameterized Query)



پرسش و پاسخ ...

amini@sharif.edu

به نام آنکه جان را فکرت آموخت



بخش پنجم: معماری پایگاه داده‌ها

مرتضی امینی

نیمسال دوم ۹۶-۹۷

(محتویات اسلایدها برگرفته از یادداشت‌های کلاسی استاد محمدتقی روحانی رانکوهی است.)



نیاز به یک معماری واحد از دیدگاه **داده شناسانه** (و نه دیدگاه عملکردی یا دیدگاه مولفه-مبنا) که در آن

داده‌ها به گونه‌ای قابل فهم (مستقل از پیچیدگی‌های سطح سمپاد) به کاربر نمایش داده شود.

عدم وجود اتفاق نظر در چگونگی معماری پایگاه داده‌ها در سالهای آغازین ایجاد

پیشنهاد معماری سه سطحی از سوی ANSI / SPARC

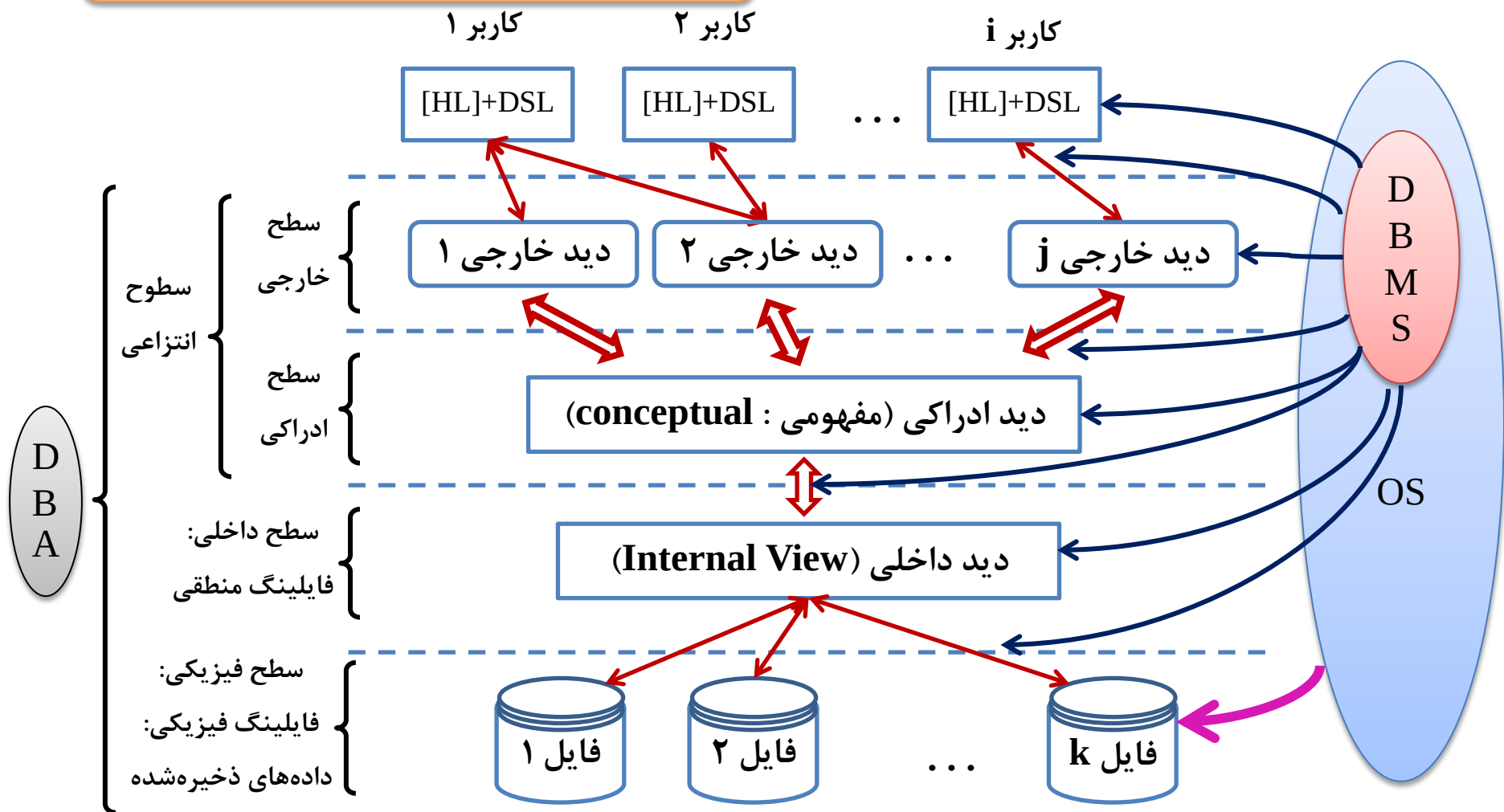
سه سطح معماری ANSI، در واقع سه سطح **تعریف و کنترل داده‌ها** است.

دو سطح در محیط انتزاعی و یک سطح در محیط فایلینگ منطقی.



نشان دهنده نگاشت (تبدیل) بین سطوح

معماری سه سطحی





□ اجزای معماری سه سطحی پایگاه داده‌ها:

- ۱- کاربر User
- ۲- زبان میزبان HL: مانند زبانهای جاوا، C# و PHP
- ۳- زبان داده‌ای فرعی (زیرزبان داده‌ای) DSL: زبانهای داده‌ای ادغام شده در زبانهای میزبان
-
- ۴- دید خارجی (نمای خارجی) → سطح خارجی
- ۵- دید ادراکی (فرایافتی یا مفهومی) → سطح ادراکی
- ۶- دید داخلی → سطح داخلی
-
- ۷- فایل‌های فیزیکی
- ۸- سیستم مدیریت پایگاه داده‌ها (کوته‌تر: سمپاد)
- ۹- مدیر پایگاه داده‌ها (DBA)



□ دید (نمای) ادراکی (فرایافتی یا مفهومی)

دید طراحی نسبت به داده های ذخیره شدنی (و نهایتا ذخیره شده) در پایگاه داده‌ها  دیدی جامع: دربرگیرنده نیازهای همه کاربران محیط ✓

✓ مطرح در محیط انتزاعی (فرافایلی) ← مبتنی بر یک ساختار داده مشخص

✓ طراحی با عنصر (عناصر) ساختاری اساسی

✓ پس از طراحی ← توصیف شود ← **شمای ادراکی (Conceptual Schema)**



 نوعی برنامه حاوی دستورات $\left. \begin{array}{l} \text{DDL} \\ \text{DCL} \end{array} \right\}$ و نه دستورات **DML**

✓ شمای ادراکی به سیستم مدیریت داده می شود و در کاتالوگ آن نگهداری می شود.



بخش پنجم: معماری پایگاه داده‌ها

CREATE TABLE S1 ...

CREATE TABLE S2 ...

CREATE TABLE S3 ...



□ دید ادراکی: جدول‌های مبنای S1 و S2 و S3

□ شمای ادراکی: تعریف جدول‌ها است.

□ به این جدول‌ها **جدول‌های مبنا** می‌گوییم.

□ اطلاعات شمای ادراکی به سیستم مدیریت داده می‌شود و در کاتالوگ آن نگهداری می‌شود.

کاتالوگ سیستم

در سیستم‌های جدولی: در تعدادی
جدول که خود سیستم ایجاد می‌کند.

در کجا ؟

چگونه ؟

کاتالوگ سیستم : متا داده‌ها (Data Dictionary : Meta Data)

✓ تمامی اطلاعات شمای ادراکی

□ حاوی : ✓ داده‌های کنترلی

✓ Data About Data





بخش پنجم: معماری پایگاه داده‌ها

جدول systables:



systables	نام جدول	ایجاد کننده	تاریخ ایجاد	تعداد ستون	کلید اصلی	...
	...	...	...	...	...	...

کاربر پیاده‌ساز : **CREATE TABLE STT ...**

سیستم : **INSERT INTO SYSTABLES**
VALUES ('STT' , 'c1' , 'd1' , 5 , 'STID' , ...)

کاربر پیاده‌ساز : **DROP TABLE STCOT ...**

سیستم : **DELETE FROM SYSTABLES**
WHERE TNAME = 'STCOT'



افزافه کردن ستون به یک جدول :  **ALTER TABLE STT** : کاربر پیاده‌ساز

ADD SADDRESS CHAR (80)

سیستم : **UPDATE SYSTABLES**

SET ColN = 6

WHERE TNAME = 'STT'

سیستم برای جدولی که تعداد ستون‌های آن تغییر می‌کند در سطح فایلینگ چگونه عمل می‌کند؟



آیا با دستور DELETE در سطح جدول‌های مبنا، جدول کاتالوگ تغییر می‌کند؟



DELETE FROM STT

WHERE STID='777'



□ دید (نمای) داخلی



دید خود DBMS [و نیز طراح پایگاه داده‌ها، در مرحله طراحی فیزیکی]، نسبت به داده‌های

ذخیره‌شده

- ✓ مطرح در سطح فایلینگ منطقی (و گاه مجازی)
 - ✓ مبتنی بر یک [یا چند] ساختار فایل ←
 - ✓ سطحی که فایل‌های منطقی پایگاه داده‌ها تعریف می‌شود.
- 1:1 پیش فرض (یک جدول : یک فایل)
1:N (چند جدول : یک فایل)
N:1 (یک جدول : چند فایل)

✓ تناظر بین «ساخت» های سطح ادراکی و «ساخت» های سطح داخلی

Table	TableFile
STT	STTFile
COT	COTFile
STCOT	STCOTFile
...	...

تناظر 1:1 بین ساخت‌های سطح ادراکی و سطح داخلی





بخش پنجم: معماری پایگاه داده‌ها

تعریف فایل‌ها
کنترل فایل‌ها

□ توصیف دید داخلی ← **شمای داخلی (Internal Schema)** ← دستورهای



 نوعی برنامه که توسط خود DBMS (و گاه براساس **اطلاعاتی** که طراح - پیاده‌ساز به سیستم می‌دهد) تولید می‌شود و شرح و وصف فایلینگ منطقی پایگاه داده‌ها است.

□ **توجه:** در شمای داخلی انواع رکوردها تعریف می‌شوند و دستورهای لازم جهت ایجاد فایل‌ها و کنترل آنها در این شمای وجود دارد.

TYPE STUDENT = **RECORD**
 STUDENT-ID : String ;
 STUDENT-NAME : String ;
 STUDENT-LEV : String ;
 STUDENT-MJR : String ;
 STUDENT-DEPT : String ;

شمای داخلی ساده شده در یک زبان شبه پاسکال





بخش پنجم: معماری پایگاه داده‌ها

اطلاعاتی که طراح-پیاده ساز به سیستم می‌دهد (مانند شاخص) در دید داخلی تاثیر می‌گذارد.

در سیستم‌های جدولی: خود سیستم روی کلید اصلی (PK) شاخص خودکار (Automatic Index) ایجاد می‌کند. (عمدتاً B-Tree)

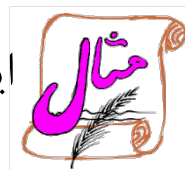
برای ایجاد شاخص روی دیگر ستون‌ها پیاده ساز باید درخواست کند.

ایجاد شاخص بر روی ستون STNAME که PK نیست:

```
CREATE INDEX SNX
```

```
ON STT ( STNAME )
```

خوشه‌بندی ؟ [CLUSTERED]





ویژگی‌های ستون شاخص؟

✓ تغییر ناپذیر (حتی الامکان)

✓ پرکاربرد در کلاز WHERE

✓ ... ؟

DROP INDEX SNX

حذف شاخص:



در سیستم چه اتفاقی می‌افتد؟

DROP TABLE
DROP INDEX

با اجرای دستور



مثالی از وضعیتی بیان کنید که براساس آن طراح-پیاده ساز تصمیم به ایجاد شاخص می‌گیرد.



دید منطقی DBMS نسبت به داده‌های ذخیره شده

۱۳

بخش پنجم: معماری پایگاه داده‌ها

- ✓ چه فایل‌هایی دارد
 - ✓ نگاشت سطح ادراکی به سطح داخلی
 - ✓ صفحات (Pages) فضای پایگاه داده کاربر
 - ✓ فرمت رکورد هر فایل [رکورد داخلی]
 - ✓ ساختار هر فایل
 - ✓ کلید(ها)
 - ✓ استراتژی دستیابی به رکوردها
 - ✓ توالی منطقی رکوردها در صفحات
 - ✓ اندازه جاری هر فایل
 - ✓ اندازه گسترش فایل
 - ✓ اطلاعات همگانی
 - ✓ ارتباط منطقی بین فایل‌ها
 - ✓
- BOF
- EOF
- R/W
- ⋮

می‌داند: جنبه‌های فایلینگ منطقی [مجازی]

DBMS □

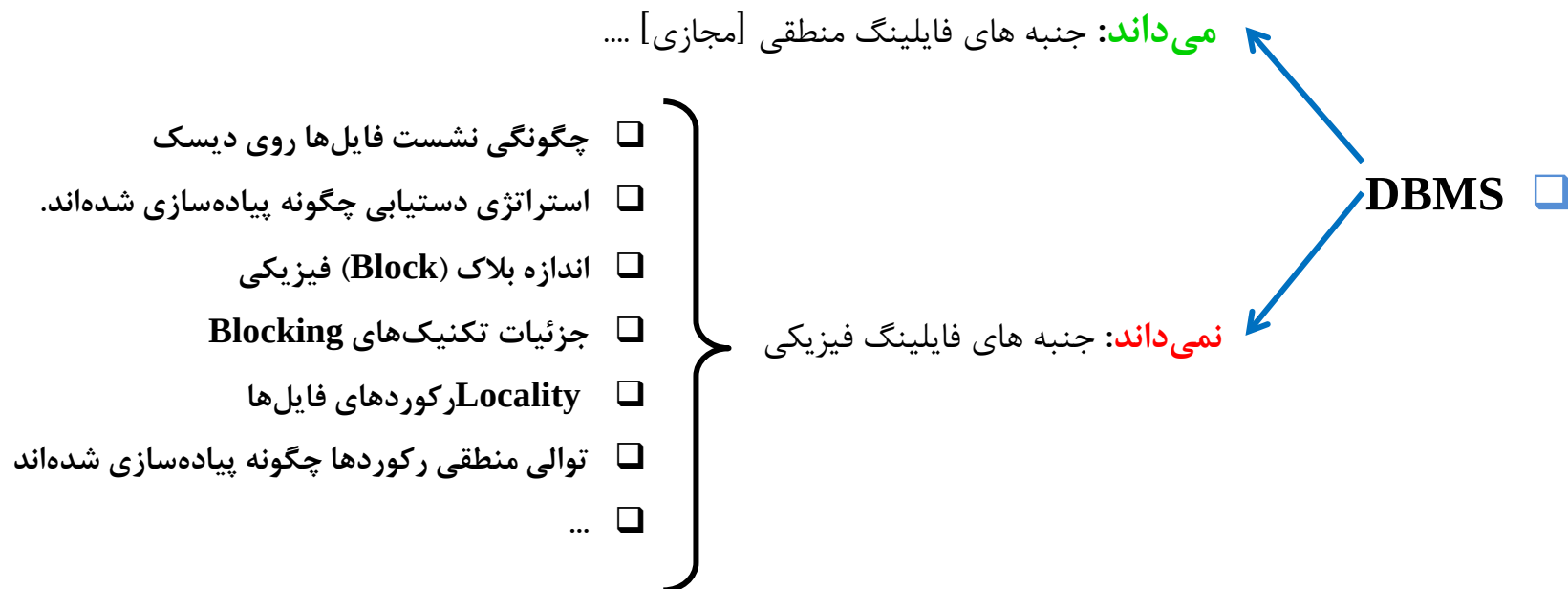
نمی‌داند: جنبه‌های فایلینگ فیزیکی



دید منطقی DBMS نسبت به داده‌های ذخیره شده (ادامه)

بخش پنجم: معماری پایگاه داده‌ها

۱۴



چيست و بر کدام یک از عملیات روی فایل‌ها تاثیر می‌گذارد؟





□ در بعضی از سیستم‌های مدیریت جدید، سیستم مدیریت، کل فضای پایگاه داده را به صورت **مجموعه‌ای از مجموعه صفحات** می‌بیند، یعنی نوعی **نمای مجازی** از داده‌های ذخیره شده در پایگاه داده دارد.

در سطح فایلینگ مجازی $DB = \{ Pages \}$



شماره صفحات	تعداد صفحات	نام جدول
p1 ... p10	10	STT
p15 ... p29	15	COT
P101 ... P1000	900	STCOT

SELECT STT.*

FROM STT

WHERE STID = '444'

DBMS : READ P1

(فرض کنید '444' در P1 است)



□ دید (نمای) خارجی



دید کاربر (برنامه ساز) خاص است نسبت به داده‌های ذخیره شده [مثلا دید یک AP نویس]

✓ دید جزئی (Partial): دربرگیرنده نیازهای داده‌ای یک کاربر مشخص [برای یک AP مشخص]

✓ مطرح در سطح انتزاعی ← مبتنی بر یک ساختار داده‌ای مشخص



آیا این ساختار داده همان ساختار داده سطح دید ادراکی است؟

✓ روی دید ادراکی طراحی و تعریف می‌شود.

✓ } یک کاربر ← چند دید متفاوت
چند کاربر ← یک دید مشترک



✓ توصیف دید خارجی ← **شِمای خارجی (External Schema)**



نوعی «برنامه» که کاربر سطح خارجی می‌نویسد، حاوی دستورات «تعریف داده‌ها» و **معدود** دستورات «کنترل داده‌ها» (**کجایی؟** چرا معدود؟)

✓ **شِمای خارجی** ← ذخیره در کاتالوگ



در سیستم‌های جدولی، دید خارجی خود نوعی جدول است، اما **مجازی (Virtual Table)** و نه ذخیره‌شده



دید خارجی در واقع **پنجره‌ای** است که از آن کاربر خارجی محدوده‌ی داده‌ای خود را می‌بیند و نه بیشتر.



بخش پنجم: معماری پایگاه داده‌ها

کاربر ۱

V1

STID STNAME

777 st7

444 st4

⋮

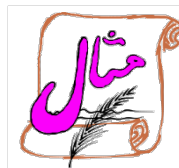
⋮

چند ستون از یک جدول

V2

CONUM COTITLE

دگرنامی ستون



STT

STID STNAME

777 st7

888 st8

444 st4

⋮

⋮

STLEV

bs

ms

bs

⋮

STMJR

phys

math

comp

⋮

STDEID

d11

d12

d14

⋮

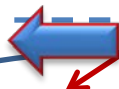
COT

COID ...

STCOT

STID COID ...

تناظر یک به یک



ST FILE

COT FILE

STCOT FILE



بخش پنجم: معماری پایگاه داده‌ها

کاربر ۲

v1

STID STNAME

777

st7

دید مشترک با کاربر ۱

444

st4

⋮

⋮



STT

STID

STNAME

STLEV

STMJR

STDEID

777

st7

bs

phys

d11

888

st8

ms

math

d12

444

st4

bs

comp

d14

⋮

⋮

⋮

⋮

⋮

COT

COID

...

STCOT

STID

COID ...

ST FILE

COT FILE

STCOT FILE



بخش پنجم: معماری پایگاه داده‌ها

کاربر ۳

v3

STNUM STNAME COTITLE TR YR



دید روی بیش از یک جدول

STT

STID STNAME STLEV STMJR STDEID

777 st7 bs phys d11

888 st8 ms math d12

444 st4 bs comp d14

⋮ ⋮ ⋮ ⋮ ⋮

COT

COID ...

STCOT

STID COID ...

ST FILE

COT FILE

STCOT FILE



بخش پنجم: معماری پایگاه داده‌ها

کاربر ۴

v4

STID	STNAM	TR	YR	AVG

صفت مجازی



STT				
STID	STNAME	STLEV	STMJR	STDEID
777	st7	bs	phys	d11
888	st8	ms	math	d12
444	st4	bs	comp	d14
⋮	⋮	⋮	⋮	⋮

COT	
COID	...

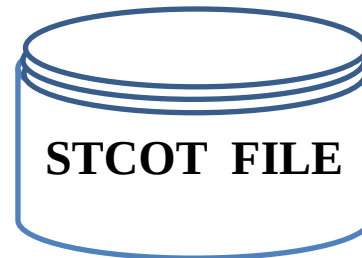
STCOT	
STID	COID ...



ST FILE



COT FILE



STCOT FILE



از این مثال‌ها نتیجه می‌گیریم که تعریف، طراحی و توصیف دید خارجی در سیستم‌های جدولی از پویایی بالایی برخوردار است.

یعنی انواع جدول‌های مجازی را می‌توان روی لایه‌های زیرین تعریف کرد.

تعریف شمای خارجی کاربر ۱ (با استفاده از مفهوم دید):

```
CREATE VIEW V1 [(STID, STNAME)]  
AS SELECT STT.STID, STT.STNAE  
FROM STT;
```

```
CREATE VIEW V2 [(SN, SJ, SL)]  
AS SELECT STID, STJ, STL  
FROM STT  
WHERE STJ != 'phys;  
[WITH CHECK OPTION]
```

شرط تعریف دید

در شرط تعریف دید می‌توان از نام ستونی که در محدوده دید نیست استفاده کرد (مثلاً با تعریف شرط بر روی ستون DEID از جدول STT در مثال فوق که در دید V2 نیامده است).



هر کاربر با اجازه Admin می‌تواند دید (View) خودش را داشته باشد (Sub-database).



دستور SELECT در متن دستور تعریف دید اجرایی نیست بلکه اعلانی است



یعنی هیچ داده‌ای بازیابی نمی‌شود و صرفاً برای اعلام محدوده داده‌ای کاربران است. □

تا آنجا که به تعریف دید مربوط است هر دستور SELECT معتبر با هر میزان پیچیدگی را می‌توان در



CREATE VIEW نوشت.

□ **تمرین:** مثال کاتالوگ پیش‌دیده را به نحوی گسترش دهید که اطلاعات (نه داده‌های) شمای داخلی و

شمای خارجی دیده شده را بتوان در آن ذخیره کرد (جدول دیگری برای کاتالوگ تعریف کنید که بتوان

این شماها را در آن ذخیره کرد).



نگاشت بین سطوح در عملیات سطح شمای خارجی

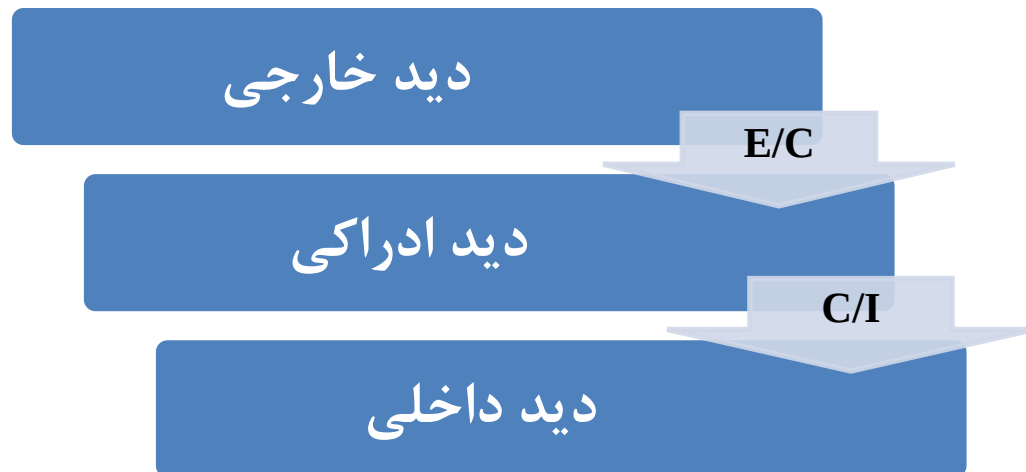
بخش پنجم: معماری پایگاه داده‌ها

۲۴

جهت انجام عملیات از دید خارجی نیاز به نگاشت یا تبدیل این عملیات بین سطوح مختلف است:

External to Conceptual Mapping: E/C ☐

Conceptual to Internal Mapping: C/I ☐



آیا تبدیل دیگری هم متصور است؟





بازیابی: کاربر حق دارد در محدوده دید خود عمل بازیابی انجام دهد.

❑ عملیات در شمای خارجی

درج

حذف

بروزرسانی

ذخیره‌سازی: به تشخیص Admin مجاز به انجام است.

❑ هر دستور [حکم] عمل‌کننده در شمای خارجی (روی دید خارجی)،

❑ تبدیل می‌شود به دستور(های) عمل‌کننده در شمای ادراکی (روی دید ادراکی)

❑ و سپس به قطعه برنامه‌ای عمل‌کننده در شمای داخلی (روی دید داخلی)

❑ و نهایتاً به عملیاتی در فایل‌های فیزیکی.



عملیات بازیابی: چون دید خارجی در سیستم‌های جدولی، به هر حال نوعی جدول است، برای بازیابی از همان دستور SELECT استفاده می‌کنیم.

```
SELECT V2.SN  
FROM V2  
WHERE SL='ms'
```

E/C

سیستم در نگاشت E/C، شرط یا شرایط داده شده در تعریف دید را AND می‌کند با شرط یا شرایط داده

شده در پرس‌وجوی روی دید. به این عمل، گاه **محاسبه دید** (View Computation) هم می‌گویند.

```
SELECT STT.STID  
FROM STT  
WHERE STL='ms'  
AND STJ != 'phys'
```

C/I



بخش پنجم: معماری پایگاه داده‌ها

به واحد رکورد

ناحیه پیام بافر سیستم

OPEN STFILE (R, SysBuf, MessageArea, ...)

LREAD STFILE ON STLINDEX.value='ms';

...

IF SysBuf.STJ != 'phys'

MOVE SysBuf.STID INTO UBuf[SN]

...

LOOP Control;

فایلینگ منطقی
در محیط

PSEEK

جستجوی فیزیکی

PREAD

خواندن فیزیکی

فایلینگ فیزیکی
در محیط

به واحد بلاک



❑ لزوماً از همه انواع دیدها نمی‌توان عملیات ذخیره‌سازی در DB انجام داد.

❑ همه انواع دیدها قابل بروزرسانی (Updatable) نیستند.

❑ محدودیتهایی هم در عمل و تاحدی در تئوری وجود دارد.

❑ دید از نظر قابلیت عملیات ذخیره‌سازی (بستگی دارد به ساختار دید و مکانیزم تعریف آن)

❑ پذیرا (Updatable): می‌توان از آنها عملیات ذخیره‌سازی انجام داد ولی گاه مشکلاتی دارند.

❑ ناپذیرا (Non Updatable): تبدیل E/C انجام شدنی نیست.

تعریف شده روی یک جدول مبنا

❑ دید

تعریف شده روی بیش از یک جدول مبنا ← در عمل ناپذیرا، اما در تئوری بعضی‌ها پذیرا هستند.



عملیات ذخیره‌سازی از دید تعریف شده روی یک جدول مبنا

بخش پنجم: معماری پایگاه داده‌ها

۲۹

□ دید تعریف شده روی یک جدول مبنا

□ دید دارای کلید جدول مبنا (Key Preserving) ← پذیرا (در عمل و تئوری) اما مشکلاتی هم دارد.

□ دید فاقد کلید جدول مبنا (Non Key Preserving) ← ناپذیرا

□ دید دارای ستون [صفت] مجازی (دیدهای آماری) ← ناپذیرا



عملیات ذخیره‌سازی از دید تعریف شده روی یک جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده‌ها

۳۰

دید حافظ کلید تعریف شده روی یک جدول مبنا ☐

V2	SN	SL	SJ
	888	ms	math
	444	bs	comp
	⋮	⋮	

STT	STID	STNAME	STL	STJ	STD
	777	st7	bs	phys	d11
	888	st8	ms	math	d12
	444	st4	bs	comp	d14
	⋮	⋮	⋮	⋮	⋮

```
CREATE VIEW V2 [(SN, SJ, SL)]
AS SELECT STID, STJ, STL
FROM STT
WHERE STJ != 'phys'
[WITH CHECK OPTION]
```



عملیات ذخیره سازی از دید تعریف شده روی یک جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده ها

۳۱

□ فرض بر مجاز بودن کاربر به انجام عمل داریم و لذا صرفاً شدنی بودن را بررسی می کنیم.

□ در دید حافظ کلید انجام عملیات سطری امکان پذیر است.

□ زیرا تناظر یک به یک بین سطرهاى دید و سطرهاى جدول مبنا برقرار است.

```
DELETE FROM V2  
WHERE SN='444'
```

E/C

```
DELETE FROM STT  
WHERE STID='444' AND STJ != 'phys'
```

حذف سطر در دید حافظ کلید



□ الان این سطر از جدول STT حذف می شود و اگر کاربر دیگری این سطر را در دیدش داشته باشد، دیگر به این سطر دسترسی ندارد.



عملیات ذخیره سازی از دید تعریف شده روی یک جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده ها

۳۲

بروزرسانی در دید حافظ کلید



UPDATE V2

SET SJ='IT'

WHERE SN='444'

E/C

UPDATE STT

SET STJ='IT'

WHERE STID='444' ' AND STJ != 'phys'



عملیات ذخیره سازی از دید تعریف شده روی یک جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده ها

۳۳



از نظر تئوریک درخواست زیر به دلیل **عدم رعایت محدودیت دید** باید رد شود.

UPDATE V2

SET SJ='phys'

WHERE SN='888'

□ در عمل: اگر از عبارت [with check option] استفاده کنیم، سیستم رد می کند، وگرنه درخواست

انجام می شود اما ...

E/C

UPDATE STT

SET STJ='phys'

WHERE STID='888' ' AND STJ != 'phys'

□ حال اگر بنویسیم:

SELECT V2.* FROM V2

□ سطر با کلید 888 دیگر در دید کاربر نمی آید!



عملیات ذخیره سازی از دید تعریف شده روی یک جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده ها

۳۴



```
INSERT INTO V2  
VALUES ('555', 'chem', 'bs')
```

E/C

```
INSERT INTO STT  
VALUES ('555', ?, 'chem', 'bs', ?)
```

❑ **عدم رعایت هیچ مقدار ناپذیری ستونهای نهان از دید:** اگر هر کدام از ستونهای نهان از دید کاربر، محدودیت هیچ مقدار ناپذیری داشته باشند، درخواست رد می شود.

❑ **عدم رعایت محدودیت یکتایی مقادیر کلید:** حال اگر به جای 555 بنویسیم 777، درخواست رد می شود (تبدیل E/C انجام نمی شود) به دلیل **عدم رعایت محدودیت یکتایی** مقادیر کلید.

❑ حال اگر به جای chem بنویسیم phys، همان پیش می آید که در مثال UPDATE دیدیم.



عملیات ذخیره سازی از دید تعریف شده روی یک جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده ها

۳۵

☐ دلایل رد شدن درخواست عمل ذخیره سازی در دید تک جدولی حافظ کلید:

☐ عدم رعایت محدودیت دید (در صورت وجود WITH CHECK OPTION در تعریف دید)

☐ عدم رعایت محدودیت یکتایی مقادیر کلید

☐ عدم رعایت محدودیت هیچ مقدار ناپذیری ستون های نهان

☐ ...



عملیات ذخیره سازی از دید تعریف شده روی یک جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده ها

۳۶

□ دید تعریف شده روی یک جدول مبنا و فاقد کلید

چون این دید فاقد کلید است، امکان انجام عملیات سطری وجود ندارد.



```
CREATE VIEW V3
```

```
AS SELECT STNAME, STJ
```

```
FROM STT
```

□ درخواست زیر انجام نمی شود، چون معلوم نیست کدام سطر از رابطه باید حذف شود. پس تبدیل E/C

ناممکن است، مگر اینکه بپذیریم این درخواست به صورت مکانیکی انجام شود؛ یعنی تمام سطرهای

حائز شرط داده شده (مجموعه ای از سطرها) حذف شوند.

```
DELETE FROM V3
```

```
WHERE STNAME='ali'
```

□ اگر کاربر این پیامد را بپذیرد مشکلی نیست، اما در عمل سیستم ها نمی پذیرند!

□ در دید V3 انجام INSERT نیز غیرممکن است (به دلیل عدم وجود مقدار کلید برای جدول مبنا).



عملیات ذخیره سازی از دید تعریف شده روی یک جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده ها

۳۷

حال اگر در تعریف V3، DISTINCT بزنیم چه پیش می آید؟



```
CREATE VIEW V3  
AS SELECT DISTINCT STNAME, STJ  
FROM STT
```

❑ فرقی نمی کند، باز هم همان مشکل پابرجاست:

```
DELETE FROM V3  
WHERE STNAME='a'
```

E/C

تبدیل می شود به حذف مجموعه ای از سطرها



عملیات ذخیره‌سازی از دید تعریف شده روی یک جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده‌ها

۳۸

□ دید تعریف شده روی یک جدول مبنا دارای ستون مجازی

□ این دیدها هم در عمل و هم در تئوری ناپذیرا هستند.



V4	PN	SQ
	P1	100
	P2	210
	P3	80

```
CREATE VIEW V4 (PN, SQ)
AS SELECT P#, SUM(QTY)
FROM SP
GROUP BY P#
```

SP	S#	P#	QTY
	S1	P1	100
	S1	P2	140
	S2	P3	80
	S2	P2	70



عملیات ذخیره‌سازی از دید تعریف شده روی یک جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده‌ها

۳۹

انجام عملیات سطری در دید V4 غیرممکن است. □

DELETE FROM V4

WHERE PN='p1'



سطری نیست، با نوعی تفسیر
می‌توان گفت که مجموعه‌ای
از سطرها را حذف می‌کند.

از لحاظ تئوریک هم دید V4 نباید پذیرا باشد. □

□ زیرا جدول V4 (که مجازی است) و جدول مبنای SP با هم تعارض معنایی (Semantic Conflict)

دارند. یعنی مسند بیانگر معنای رابطه V4 اساساً با مسند بیانگر رابطه SP تفاوت دارد.

[در بحث رابطه‌ای خواهیم دید که هر رابطه (جدول) یک معنا دارد و در اینجا این دو رابطه هیچ ربطی از نظر معنایی با هم ندارند].



عملیات ذخیره‌سازی از دید تعریف شده روی چند جدول مبنا

بخش پنجم: معماری پایگاه داده‌ها

۴۰

دیده‌های تعریف شده روی بیش از یک جدول ☐

☐ در عمل این دیده‌ها ناپذیرا هستند و دخالت خود برنامه‌ساز لازم است. البته به لحاظ تئوری در برخی موارد پذیرا هستند.

V5: T1 JOIN T2 دید پیوندی (پیوند طبیعی)

V6: T1 UNION T2

V7: T1 INTERSECT T2

V8: T1 EXCEPT T2

PK-PK: ستون پیوند در هر دو جدول PK است. در تئوری پذیرا و بدون مشکل

PK-FK: در تئوری پذیرا به شرط پذیرش پیامدها

FK-FK

(Non-Key) NK-NK

دید پیوندی ☐



عملیات ذخیره‌سازی از دید تعریف شده روی چند جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده‌ها

۴۱

دید پیوندی PK-PK ☐

V5 همان STT است اما این بار به صورت یک دید تعریف شده است.



V5	STID	STNAME	STL	STJ	STD
	777	st7	bs	phys	d11
	888	st8	ms	math	d12
	444	st4	bs	comp	d14
	:	:	:	:	:

```
CREATE VIEW V5
AS SELECT ST1.*, ST2.*
FROM ST1 JOIN ST2
```

ST1	STID	STNAME	STL
	777	st7	bs
	888	st8	ms
	444	st4	bs
	:	:	:

ST2	STID	STJ	STD
	777	phys	d11
	888	math	d12
	444	comp	d14
	:	:	:



عملیات ذخیره‌سازی از دید تعریف شده روی چند جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده‌ها

۴۲

یک دستور اجراشونده در شمای خارجی تبدیل می‌شود به دو دستور در شمای ادراکی. ☐

INSERT INTO V5

VALUES ('999', 'St9', 'chem', 'bs', 'D15')

E/C

INSERT INTO ST1

VALUES ('999', 'St9', 'bs')

INSERT INTO ST2

VALUES ('999', 'chem', 'D15')

عمل DELETE در این دید تبدیل می‌شود به دو عمل حذف از جدول‌های مبنایی زیرین و عمل UPDATE (بسته به ☐

ستونی که می‌خواهیم بروز کنیم) به یک یا دو عمل بهنگام‌سازی در جدول‌های زیرین تبدیل می‌شود.



عملیات ذخیره سازی از دید تعریف شده روی چند جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده ها

۴۳

دید پیوندی PK-FK ☐



CREATE VIEW V6

AS SELECT STT.STID, STT.NAME, STCOT.*

FROM STT JOIN STCOT

☐ درج در این دید تبدیل می شود به درج یک تاپل ناقص در STT به شرط آنکه شماره دانشجویی تکراری نباشد. ولی در STCOT حتما یک تاپل درج می شود.

INSERT INTO V6

VALUES ('9212345', 'Amir', '40638', 15)

E/C

INSERT INTO STT

VALUES ('9212345', 'Amir', ?, ?, ?)

INSERT INTO STCOT

VALUES ('9212345', '40638', 15)



عملیات ذخیره‌سازی از دید تعریف شده روی چند جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده‌ها

۴۴

حذف از این دید مشکل دارد. 

 اگر از هر دو جدول حذف شود، منجر به حذف داده‌های ناخواسته می‌شود.

 با حذف یک سطر از جدول STT، برای حفظ جامعیت ارجاعی نیز لازم است یک تعداد سطر دیگر از

STCOT حذف شود، مگر آنکه فقط از STCOT حذف کنیم و از STT سطر مربوطه را حذف نکنیم.

عمل بهنگام‌سازی نیز مساله مشابه حذف ممکن است داشته باشد. 

ستون پیوند در هیچ کدام کلید
نیست (نه اصلی و نه خارجی).

ستون پیوند در هر دو جدول
کلید خارجی است.

دید حاصل از پیوند FK-FK و دید حاصل از پیوند NK-NK چه رفتاری در عملیات ذخیره‌سازی دارند؟



انجام عملیات ذخیره‌سازی در این دیدها دارای عوارض بسیاری است که در عمل آنها را ناپذیرا می‌کند.



عملیات ذخیره‌سازی از دید تعریف شده روی چند جدول مبنا (ادامه)

بخش پنجم: معماری پایگاه داده‌ها

۴۵

□ دید حاصل از اجتماع، اشتراک، و تفاضل

□ این دیدها از لحاظ تئوری مشکلی در عملیات ذخیره‌سازی ندارند، هرچند نظرات مختلفی مطرح است.

عمل دید	درج	حذف	بهنگام‌سازی
$R_1 \cup R_2$	درج تاپل در R_1 و/یا R_2	حذف تاپل از R_1 و/یا R_2	بهنگام‌سازی تاپل در R_1 و/یا R_2
$R_1 \cap R_2$	درج تاپل در R_1 و R_2	حذف تاپل از R_1 و/یا R_2	بهنگام‌سازی تاپل در R_1 و R_2
$R_1 - R_2$	درج تاپل در R_1 (به شرط عدم وجود در R_2)	حذف تاپل در R_1	بهنگام‌سازی تاپل در R_1



□ موضوع دیدهای پذیرا در SQL استاندارد چندان روشن نیست. در SQL 2003 دیدهایی که تمام شرایط زیر را داشته باشند، قابل بهنگام‌سازی (درج، حذف و بروزرسانی) هستند.
[توجه: ممکن است برخی دیگر از دیدها هم قابل بهنگام‌سازی باشند.]

۱- عبارت تعریف‌کننده دید، یک عبارت SELECT ساده باشد (یعنی شامل عملگرهای JOIN، UNION، INTERSECT و EXCEPT نباشد).

۲- در عبارت SELECT گزینه DISTINCT وجود نداشته باشد.

۳- در کلاز FROM عبارت SELECT، فقط یک جدول وجود داشته باشد.

۴- جدول قید شده در کلاز FROM، یک جدول مبنا یا یک دید قابل بهنگام‌سازی باشد.



۵- در لیست نام ستون‌ها در عبارت `SELECT`، ستون‌های موردنظر باید در جدول مبنا متناظر داشته باشند و به یک ستون از جدول مبنا بیش از یک بار ارجاع وجود نداشته باشد. ضمناً حاوی ستون کلید باشد.

۶- در عبارت `SELECT`، کلاز `GROUP BY` و/یا کلاز `HAVING` وجود نداشته باشد.

۷- کلاز `WHERE` در عبارت `SELECT` حاوی کلاز `FROM` نباشد به گونه‌ای که در آن به همان

جدولی ارجاع داده شده باشد که در کلاز `FROM` ذکر شده در شرط ۴.

نتیجه اینکه عملاً دیدهایی که یک زیرمجموعه افقی-عمودی دارای کلید از یک جدول مبنا (یا از دید قابل

بهنگام‌سازی) باشند، قطعاً قابل بهنگام‌سازی هستند.

[توجه: به شرط رعایت محدودیت‌های جامعیتی مانند یکتایی کلید و هیچمقدارناپذیری]



معایب مفهوم دید: ☐

☐ محدودیت [و مشکلات] در عملیات ذخیره‌سازی

☐ فزونکاری (overhead) برای انجام تبدیل E/C (محاسبه دید). راه حل: استفاده از تکنیک دید ذخیره

شده



□ مزایای مفهوم دید خارجی:

□ فراهم‌کننده محیط انتزاعی فرافایلی برای کاربران با پویایی بالا

□ اشتراک داده‌ها (Data Sharing) ← داده‌ها یک بار ذخیره می‌شوند و کاربران بسته به نیاز خود از داده‌های ذخیره شده به صورت همروند استفاده می‌کنند.

□ تامین امنیت برای داده‌های زیرین. ← از طریق مفهوم داده مخفی (Hidden Data)، زیرا کاربر خارج از محدوده دید خود هیچ نمی‌بیند (داده‌های نهان امن هستند).

□ تامین‌کننده استقلال داده‌ای (مفهوم اساسی در تکنولوژی DB؛ هم مزیت و هم از اهداف مهم تکنولوژی DB).

□ امکانی است برای کوتاه‌نویسی یا ماکرونویسی پرسش‌ها.



تکنیک دید ذخیره شده [ساخته شده] (Materialized View)

در این تکنیک، دید در سیستم ذخیره می‌شود؛ یعنی دیگر مجازی نیست و جدول ذخیره شده است. تا در هر بار مراجعه به دید لازم نباشد تبدیل E/C انجام شود.

هدف: برای افزایش سرعت عملیات بازیابی.

شرط استفاده: در عمل از این تکنیک وقتی استفاده می‌کنیم که داده‌های ذخیره شده در جدول‌های مبنای زیرین حتی‌الامکان تغییر نکنند. به بیان دیگر، نرخ عملیات ذخیره‌سازی در جدول‌های زیرین پایین باشد. زیرا اگر جدول‌های زیرین تغییر کنند، تغییرات متناسباً در جدول‌های دید باید اعمال شوند و این خود سربار ایجاد می‌کند.

کاربرد: در برنامه‌های آماری، گزارش‌گیری‌ها و برنامه‌های داده‌کاوی (Data Mining)

دید ذخیره شده در SQL چگونه پیاده‌سازی می‌شود؟



☐ چه زمانی از مفهوم دید استفاده نمی‌کنیم؟

☐ هنگامی که سیستم تک‌کاربره باشد.

☐ هنگامی که به تشخیص admin برای افزایش کارایی سیستم، برخی برنامه‌ها را مستقیماً روی شمای ادراکی (جداول مبنایی) بنویسیم.

☐ هنگامی که کاربر نیازمند انجام عملیات ذخیره‌سازی باشد و از طریق دید امکان آن وجود نداشته باشد.



□ مفهوم استقلال داده‌ای [DI] (جدایی برنامه‌ها از داده‌ها):

□ مصونیت (تاثیرناپذیری) برنامه‌های کاربران [در سطح خارجی] در قبال تغییرات در سطوح زیرین معماری DB.

□ چرا نباید برنامه‌ها تغییر کنند؟

□ چون هر تغییر در برنامه‌ها، هزینه تولید و پشتیبانی و بازتولید برنامه‌ها را بالا می‌برد.

استقلال داده‌ای فیزیکی (PDI)

استقلال داده‌ای منطقی (LDI)

□ استقلال داده‌ای (DI)



استقلال داده‌ای فیزیکی (PDI) ☐

☐ مصونیت برنامه‌های کاربران در قبال تغییرات در شمای داخلی DB

☐ تغییرات در شمای داخلی شامل تغییر در جنبه‌های فایلینگ پایگاه

▪ ساختار فایل، طول رکورد، طرز ذخیره‌سازی فایل روی دیسک، گاه با دخالت طراح فیزیکی و گاه فقط توسط

.DBMS

☐ استقلال داده‌ای فیزیکی در پایگاه داده‌ها به طور کامل تامین است: زیرا کاربران با مفهوم دید کار

می‌کنند که اساساً در سطح فرافایلی مطرح است و برنامه‌ها درگیر جنبه‌های فایلینگ نیستند.



□ استقلال داده‌ای منطقی (LDI)

□ مصونیت برنامه‌های کاربران در قبال تغییرات در شمای ادراکی DB.

□ در سیستم‌های پایگاهی تا حد زیادی این استقلال تامین است ولی نه صددرصد.

□ تغییر در شمای ادراکی {
رشد پایگاه داده‌ها (DB Growth)
تغییر سازمان پایگاه داده‌ها [سازماندهی مجدد DB] (DB Restructuring)

□ **نکته:** تغییراتی که مورد بررسی قرار می‌دهیم، تغییراتی است که از داده‌ها و ساختار موجود **نمی‌کاهد**،

چرا که تغییرات کاهشی، قطعاً بر روی برنامه‌های سطح خارجی تاثیر می‌گذارد و استقلال داده‌ای حفظ نمی‌شود.



❑ چرا رشد DB: مطرح شدن نیازهای جدید

❑ اضافه شدن ستون(های) جدید به جدول(ها)

❑ ایجاد جدول‌های جدید

❑ استقلال داده‌ای منطقی (LDI) در قبال رشد DB، به کمک مفهوم دید تقریباً صددرصد تامین است، زیرا

کاربر دارای یک دید، خارج از محدوده آن دید هیچ نمی‌بیند.



شده است.

دیدهای پیش‌تر تعریف شده را روی جدول STT در نظر می‌گیریم. حال نیاز جدیدی برای کاربر مطرح

V1
STID STNAME

V2

V9 STID STADR

کاربر ۱

:

:

کاربر ۲

:

STT	STID	STNAME	STL	STJ	STD	STADR
	777	st7	bs	phys	d11	
	888	st8	ms	math	d12	
	444	st4	bs	comp	d14	
	:	:	:	:	:	

ALTER TABLE STT

ADD COLUMN STADR CHAR(70) این گسترش در سطح فایلینگ چگونه انجام می‌شود؟ ☐



□ آیا پیرو نیاز جدید کاربر در حد ستون، طراح همیشه جدول مبنا را گسترش می‌دهد؟

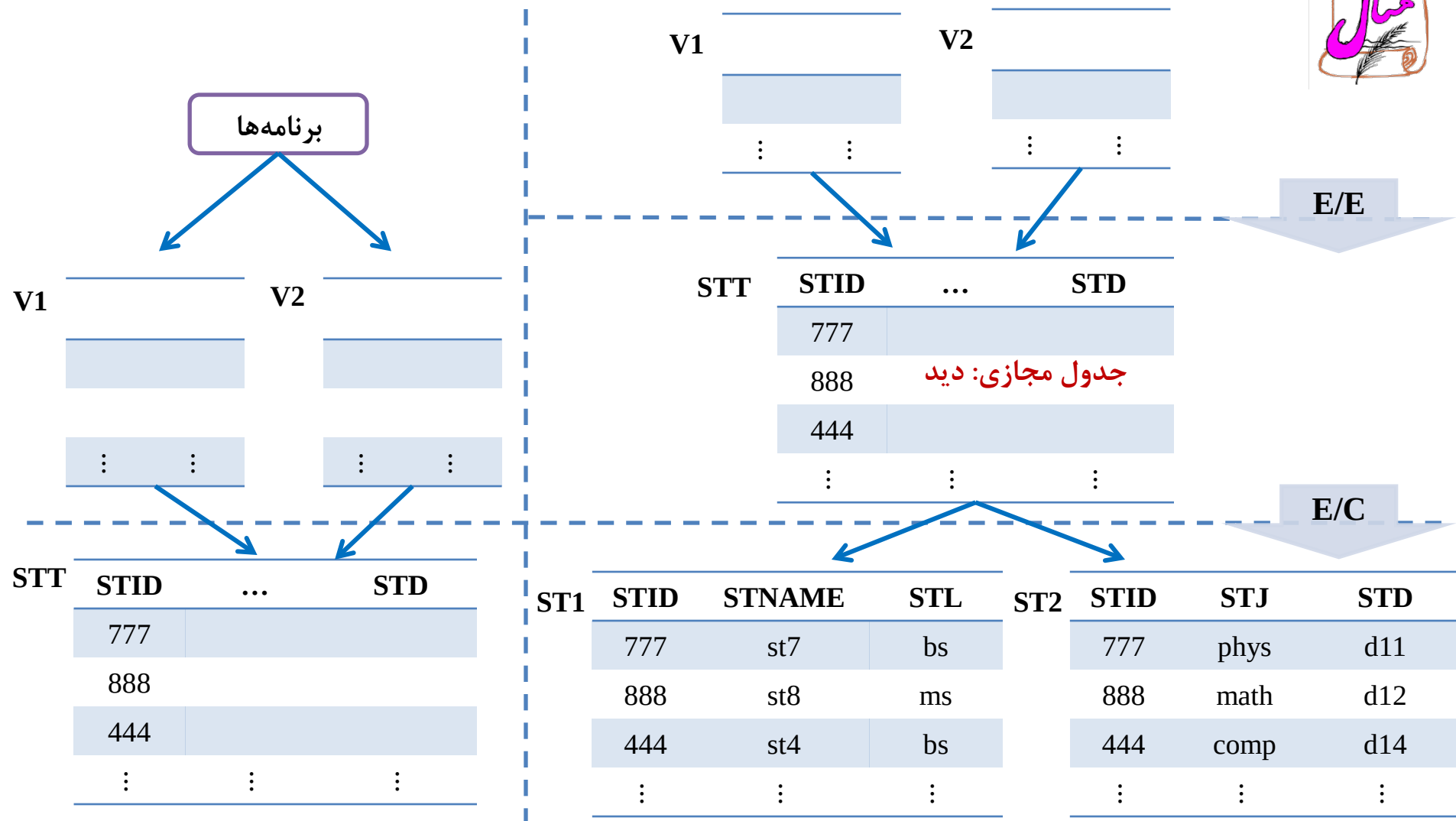
□ خیر، زیرا ممکن است آن ستون مجازی (محاسبه شدنی) باشد.

□ **سازماندهی مجدد DB** یعنی طراح به هر دلیلی طراحی منطقی DB را تغییر دهد. مثلاً یک جدول مبنای

موجود را به دو جدول، تجزیه عمودی کند و طبعاً شمای ادراکی هم تغییر می‌کند. می‌خواهیم ببینیم LDI در قبال این تغییر تا چه حد تامین است.

□ در این حالت، LDI به کمک مفهوم دید و امکان تعریف **دید روی دید** (View Definition on View)،

تا حدی تامین است.





```
CREATE TABLE ST1
```

```
(STID ...,
```

```
...
```

```
STL ...)
```

```
PRIMARY KEY STID
```

شمای جدید: ☐

البته در عمل مشکلات دیگری هم وجود دارد ☐

و صرفاً با این اعمال مشکل برطرف نمی‌شود.

```
CREATE TABLE ST2
```

```
(STID ...,
```

```
...
```

```
STD ...)
```

```
PRIMARY KEY STID
```

```
INSERT INTO ST1
```

```
(SELECT STID, STNAME, STL  
FROM STT)
```

```
INSERT INTO ST2
```

```
(SELECT STID, ..., STD  
FROM STT)
```

مهاجرت داده‌ها (Data Migration)



☐ دلایل این نوع تجزیه (که کلید در هر دو جدول باشد) چه می‌تواند باشد؟

☐ افزایش کارایی سیستم در رده فایلینگ با فرض 1-Table:1-File برای بعضی برنامه‌ها (مثلاً برنامه‌هایی

با فرکانس بالاتری نسبت به ستون‌های ST1 و با فرکانس پایین‌تری به ستون‌های ST2 ارجاع داشته

باشد، فایل‌ها را جدا می‌کند).

☐ توزیع داده‌ها در سایت‌ها وقتی پایگاه داده توزیع شده (DDB) داشته باشیم.

☐ کاهش حجم Null Value

☐ بهینه‌سازی طراحی (رجوع شود به بحث نرمال‌سازی رابطه‌ها)

☐ ...



□ با حذف جدول مبنای STT، دیدهای قبلاً تعریف شده روی آن نامعتبر می‌شوند و در نتیجه برنامه‌هایی که روی آنها کار می‌کردند، دیگر اجرا نمی‌شوند و LDI دیگر تامین نیست مگر اینکه طراح و پیاده‌ساز تدبیری

بیندیشد.



✓ جدول STT را با همان نام و ساختار به شکل یک دید تعریف می‌کنیم، با مکانیزم پیوند (دید روی دید):

```
CREATE VIEW STT
AS SELECT STID, ..., STD
FROM ST1 JOIN ST2
```

```
DROP TABLE STT
```

□ تعریف این دید وارد کاتالوگ سیستم می‌شود. ← دیدهای قبلاً تعریف شده معتبر می‌شوند (البته

در عمل ممکن است با مشکلاتی جزئی مواجه گردیم).



□ با این تدبیر، LDI برای برنامه‌هایی که بازیابی انجام می‌دهند، صددرصد تامین می‌شود، به قیمت افزایش سربار برای انجام تبدیل E/E علاوه بر E/C و C/I. زیرا از تکنیک **دید روی دید** استفاده کرده‌ایم.

□ اما LDI برای برنامه‌هایی که عملیات ذخیره‌سازی انجام می‌دادند، ممکن است تامین نباشد. زیرا این بار STT خود یک دید است و دیدها در عملیات ذخیره‌سازی عمدتاً مشکل دارند. ولی در این مثال خاص از نظر **تئوریک** مشکلی بروز نمی‌کند. ← چون STT یک دید پیوندی PK-PK است.



پرسش و پاسخ ...

amini@sharif.edu

به نام آنکه جان را فکرت آموخت



بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

مرتضی امینی

نیمسال دوم ۹۶-۹۷

(محتویات اسلایدها برگرفته از یادداشت‌های کلاسی استاد محمدتقی روحانی رانکوهی است.)



□ RDM مبنای تئوریک RDB و RDBMS

□ واضع مدل: F. Codd

□ مفاهیم زیر در طی سه بخش باقیمانده از این درس مرور می‌شوند:

□ رابطه (Relation)

□ دامنه (میدان)

□ رابطه نرمال و غیرنرمال

□ کلید در مدل رابطه‌ای

□ قواعد جامعیت رابطه‌ای

□ عملیات در RDB ← جبر رابطه‌ای

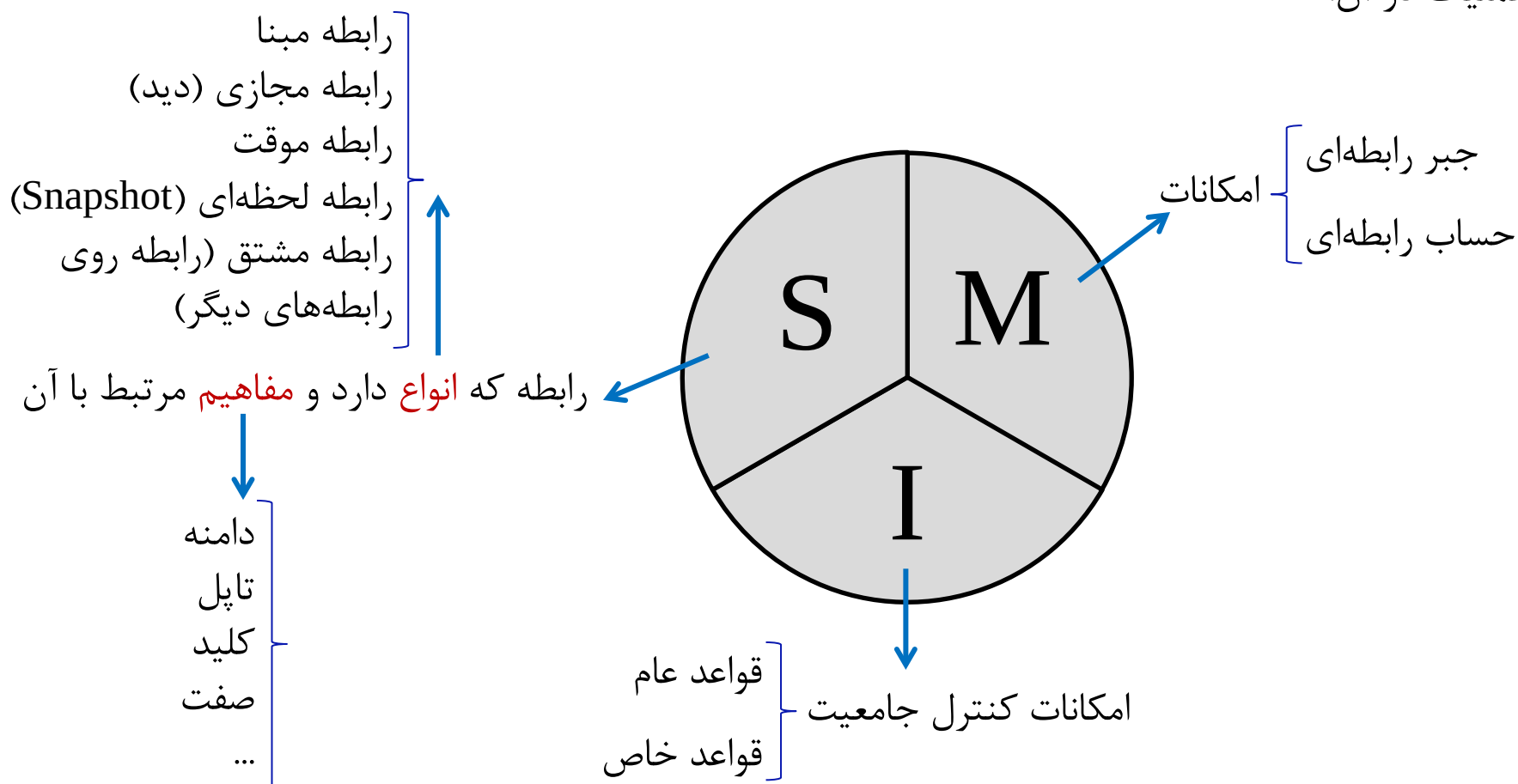
□ طراحی RDB ← حساب رابطه‌ای

روش بالا به پایین

روش نرمال‌ترسازی (سنتز)



✓ **مدل داده** مجموعه‌ای است از امکانات برای طراحی منطقی و تعریف پایگاه داده‌ها، کنترل آن و نیز انجام عملیات در آن.





در ریاضی: هر زیر مجموعه از ضرب کارتزین چند مجموعه



(۱) با فرض وجود m مجموعه از مقادیر موسوم به دامنه $[D_1, \dots, D_m]$:



رابطه R با صفات $A_1, \dots, A_m$ تعریف شده روی این m دامنه

مجموعه‌ای است از عناصر، هر یک به صورت $\langle d_{1i}, d_{2i}, \dots, d_{mi} \rangle$ موسوم به m -تاپل (m-tuple)

به نحوی که $d_{ji} \in D_j, \dots, d_{1i} \in D_1$



STUD (STID, STNAME, STJ, STL, STD)

777 st7 bs phys d11

⋮ ⋮ ⋮ ⋮ ⋮

444 st4 bs comp d14

یک تاپل ۵-تایی →



(۲) [Date] با فرض وجود m مجموعه از مقادیر موسوم به دامنه [میدان] $D_1, \dots, D_m$ نه لزوماً متمایز،

رابطه R تعریف شده روی این m دامنه:

- عنوان [سرآیند] (Heading): مجموعه‌ای است نامدار از اسامی صفات یعنی $\{A_1, \dots, A_m\}$ که با $R(A_1, \dots, A_m)$ نمایش داده می‌شود.
- بدنه [پیکر] (Body): مجموعه‌ای است از تاپل‌ها [همان مجموعه در تعریف اول].

رابطه دانشجو



STUD (STID, STNAME, STJ, STL, STD)

اصطلاح	m
رابطه یگانی	۱
رابطه دوگانی	۲
رابطه n گانی	n

□ **درجه رابطه:** کاردینالیتی عنوان یا تعداد صفات رابطه



مجموعه عنوان را با H_R یا $R(H)$ نیز نمایش می‌دهیم. به $R(H)$ ، Intention (ذات، جوهر یا **چکیده**) رابطه هم گفته می‌شود. ☐

$R(H)$ ثابت در زمان است. یعنی اگر مجموعه صفات را عوض کنیم، از نظر ریاضی یک رابطه دیگر است. ☐

همین $R(H)$ برای تعریف رابطه در سیستم کافی است. ☐



CREATE RELATEION STUD

(STID, STNAME, STJ, STL, STD)

هر رابطه یک معنا دارد، بیانگر واقعیتی از یک محیط مشخص. به عنوان مثال وقتی می‌گوییم رابطه STUD را داریم، معنایش این است که در خردجهان واقع، نوع موجودیتی با نام STUD و با صفات STID و STNAME و ... و STD وجود دارد. ☐



❑ **کاردینالیتی رابطه:** همان کاردینالیتی بدنه؛ تعداد تاپل‌ها (بزرگتر مساوی صفر؛ صفر در بدو تعریف)

❑ بدنه رابطه، متغیر در زمان است.

❑ به یک مقدار بدنه در یک لحظه مشخص instance گویند.

❑ به بدنه رابطه، Extension (**بسط** یا گسترده) یا حالت رابطه گویند.



مقدار رابطه‌ای. (۳) از نظر تئوری زبان‌های برنامه‌سازی [تشکیل شده است از یک متغیر رابطه‌ای و در هر لحظه از یک مقدار رابطه‌ای].

□ $R(H)$: متغیر رابطه‌ای، متغیری از جنس رابطه [RELVAR]

□ بدنه (r) : مقدار رابطه‌ای [RELVAL]

$R(A, B) \rightarrow$ متغیر رابطه‌ای

a_1	b_1
a_2	b_2
$\vdots$	$\vdots$
a_n	b_n

$\rightarrow$ یک مقدار رابطه‌ای
(در لحظه بعد ممکن است
مقدارش فرق کند).



□ تناظر بین مفاهیم رابطه‌ای و اصطلاحات جدولی

اصطلاح	مفهوم رابطه‌ای
جدول (صرفاً امکانی است برای نمایش مفهوم رابطه‌ای و تفاوت‌های متعددی با رابطه دارد.)	رابطه
سطر	تاپل
ستون	صفت
مقادیر مجاز ستون	دامنه
تعداد ستون‌ها	درجه
تعداد سطرها	کاردینالیتی
؟ (به معنایی که در مدل رابطه‌ای داریم، در بحث‌های جدولی مطرح نیست.)	کلید



مدل رابطه‌ای و مدل جدولی (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۱۰

ویژگی‌های رابطه: □

۱- صفات در عنوان رابطه نظم (مکانی) ندارند. [چون مجموعه است] $R(A, B) = R(B, A)$

در حالی که در جدول، ستون‌ها می‌توانند نظم مکانی داشته باشند.

در مدل رابطه‌ای، تنها راه ارجاع به صفت رابطه، نام صفت است.

۲- تاپل‌ها [در بدنه] نظم ندارند (مرتب نیستند) [چون مجموعه است].

۳- رابطه، تاپل تکراری ندارد [چون مجموعه است].

۴- **فرض:** تمام صفات رابطه (نرمال)، تک مقدار هستند [ارجوع شود به مفهوم رابطه نرمال] (این ویژگی دلیل تکنیکی دارد و از ذات رابطه نتیجه نمی‌شود). یعنی در هر تاپل دقیقاً یک مقدار برای هر صفت وجود دارد.

□ در RM هیچ یک از مفاهیم فایلینگ مطرح نیستند (مثل نظم، فیلد، رکورد، اشاره‌گر، آدرس که در

سطح طراحی و فایلینگ فیزیکی مطرح است).



تفاوت‌های مفهوم رابطه و اصطلاح جدول

۳ ویژگی اول رابطه، ۳ تفاوت

۴- در رابطه $m \geq 0$ (درجه)، یعنی از نظر تئوری رابطه می‌تواند از نظر درجه، صفر باشد.

۵- نمایش دقیق عنوان رابطه به صورت زیر است حال آنکه عنوان جدول چنین نیست.

عنوان رابطه مجموعه‌ای است از دوتایی‌های $\langle \text{دامنه: صفت} \rangle$ $R(H): \{ \langle D_1: A_1 \rangle, \langle D_2: A_2 \rangle, \dots \}$

۶- نمایش دقیق تاپل رابطه به صورت زیر است حال آنکه سطر در جدول چنین نیست.

تاپل مجموعه‌ای است از دوتایی‌های $\langle \text{صفت، مقدار} \rangle$ $TUPLE: \{ \langle A_1: V_1 \rangle, \langle A_2: V_2 \rangle, \dots \}$



مفهوم دامنه (میدان)

 مجموعه‌ای است نامدار از مقادیر هم نوع، که حداقل یک صفت از رابطه، از آن **معنا**، **نوع** و **مقدار** می‌گیرد.

 معادل است با مفهوم Data Type در تئوری انواع.

 دامنه‌هایی که یک رابطه روی آن‌ها تعریف می‌شود، لزوماً متمایز نیستند.

مفروض $R(H)$

(لزوماً چنین نیست که $(D_i \neq D_j \Rightarrow A_i \neq A_j)$ if $A_i \in H, A_j \in H$)



□ **تمرین:** مثالی از یک رابطه ۵-تایی که

□ دو صفت آن از یک دامنه باشد.

□ سه صفت آن از یک دامنه باشد.

□ اگر m درجه رابطه و n تعداد دامنه‌ها باشد، داریم: $n \leq m$.

□ برای تعریف یک رابطه در سیستم رابطه‌ای، از لحاظ تئوریک، ابتدا باید دامنه‌هایش را تعریف کرد.



مثالی از شمای پایگاه رابطه‌ای

CREATE DOMAIN SN	CHAR(8) DEFAULT '00000000'
CREATE DOMAIN SNAME	CHAR(20) DEFAULT 'noname'
CREATE DOMAIN SJ	CHAR(4) DEFAULT '?...?'
CREATE DOMAIN SL	CHAR(3) DEFAULT '?...?'
CREATE DOMAIN SD	CHAR(4) DEFAULT '?...?'
CREATE DOMAIN CN	CHAR(6) DEFAULT '?...?'
CREATE DOMAIN GRADE	DEC(2, 2) DEFAULT '?...?'

(در مدل تئوریک)

```
CREATE RELATEION STT
(STID DOMAIN SN,
STNAME DOMAIN SNAME,
STJ DOMAIN SJ,
STL DOMAIN STL,
STD DOMAIN SD)
```

تذکر: توجه شود که در عمل این دستور را نداریم و به جای آن دستور **CREATE TABLE** داریم.

CREATE RELATION COT

CREATE RELATION STCOT ...



دستورات زیر در SQL مطالعه شود.



CREATE DOMAIN ☐

ALTER DOMAIN ☐

DROP DOMAIN ☐

مزایای مفهوم دامنه از دیدگاه مهندسی نرم‌افزار بررسی شود.





□ رابطه نرمال (بهنجار - عادی Flat Relation):

رابطه‌ای که تمام صفات آن تک‌مقداری (حداکثر دارای یک مقدار در هر تاپل) باشند.



□ رابطه غیر نرمال (Nested Relation):

رابطه‌ای که حداقل یک صفت آن چندمقداری باشد.



□ **توجه:** تعریف زیر درست نیست:

□ رابطه‌ای نرمال است که مقادیر تمام صفات آن اتمیک (تجزیه نشدنی یا ساده) باشند.

□ **تذکر:** ساده یا مرکب بودن صفت نقشی در نرمال بودن و نبودن آن ندارد.



صفت چندمقداری ساده

NNCOPRECO (COID , PRECOID)

COID	PRECOID
c01	$\begin{Bmatrix} c11 \\ c17 \\ c08 \end{Bmatrix}$
c02	$\begin{Bmatrix} c03 \\ c09 \end{Bmatrix}$
c03	c10

یک تاپل

COPRECO (COID , PRECOID)

COID	PRECOID
c01	c11
c01	c17
c01	c08
c02	c03
c02	c09
c03	c10

یک تاپل

تبدیل به
رابطه نرمال



صفت چندمقداری مرکب
P# , QTY

NNSP (S# , **PQTY**)

SP (S# , P# , QTY)

S#	PQTY
s1	$\left\{ \begin{array}{ll} p1 & 100 \\ p2 & 90 \\ p3 & 50 \end{array} \right\}$
s2	$\left\{ \begin{array}{ll} p1 & 60 \\ p2 & 90 \end{array} \right\}$
s3	p1 150

یک تاپل



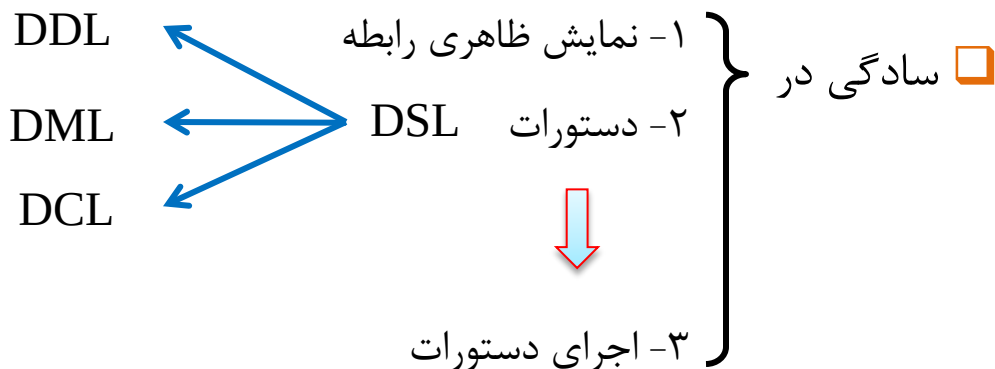
تبدیل به رابطه
نرمال

S#	P#	QTY
s1	p1	100
s1	p2	90
s1	p3	50
s2	p1	60
s2	p2	90
s3	p1	150

یک تاپل



□ دلیل نرمال بودن رابطه در RM:



مثال □ برای درک موارد ۲ و ۳

$I_1: \langle s4, p4, 40 \rangle$: در هر دو رابطه NNSP و SP منجر می شود به درج «تاپل

در رابطه» با همان دستور ساده «درج کن تاپل را».

□ درج کن

$I_2: \langle s2, p3, 30 \rangle$: با همان دستور ساده درج می شود در SP و نه NNSP.



رابطه نرمال و غیر نرمال (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۲۰

ادامه مثال ☐

$I_1 : \text{INSERT INTO } \begin{Bmatrix} \text{NNSP} \\ \text{SP} \end{Bmatrix}$
 $\text{TUPLE (S4 , P4 , 40);}$

$I_2 : \text{INSERT INTO SP}$
 $\text{TUPLE (S2 , P3 , 30);}$

امکان پذیر

$I_2 : \text{INSERT INTO NNSP}$
 $\text{TUPLE (S2 , P3 , 30);}$

امکان ناپذیر

☐ دلیل: تاپلی با کلید S2 وجود دارد.

برای درج I_2 در NNSP منطقاً چه باید کرد؟



✓ در رابطه غیر نرمال دستورات ساده‌ی تاپلی کار نمی‌کنند.



مزایا و معایب رابطه نرمال و غیر نرمال

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۲۱

نوع رابطه	مزایا	معایب
نرمال	سادگی (۱- ... ۲- ... ۳-...) تقارن صفات (پیاده‌سازی در سطح فایلینگ ساده‌تر) (نقش تمام صفات در عبارت WHERE وقتی که شرط جستجو را با theta می‌دهیم، یکسان است، زیرا همه تک‌مقداری‌اند. SELECT.... FROM WHERE A<(=)(>) 'Single Value' چنین تقارنی در رابطه غیرنرمال وجود ندارد.)	طولانی شدن کلید افزونگی (ادراکی یا منطقی) (این نوع افزونگی که در مرحله طراحی پیدا شده ممکن است منجر به افزونگی فیزیکی بشود یا نشود؛ بستگی دارد به نحوه پیاده‌سازی رابطه در سطح فایلینگ. اگر تناظر یک به یک باشد، که هر تاپل هم با یک رکورد پیاده‌سازی شود، افزونگی فیزیکی نیز پیش می‌آید.) سنگین و زمانگیر کردن کار طراحی منطقی پایگاه داده‌ها کاهش سرعت بازیابی در بعضی از پرسش‌ها
غیرنرمال	[عکس معایب رابطه نرمال]	پیچیدگی (۱- ... ۲- ... ۳-...) عدم تقارن صفات



مزایا و معایب رابطه نرمال و غیر نرمال (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۲۲

❑ در عمل با کلید طولانی چه باید کرد؟

❑ از یک کلید ساختگی استفاده می‌کنیم؛ یعنی یا خودمان به صورت دستی و یا خود سیستم به صورت

خودکار به هر سطر یک شماره می‌دهد.

این تکنیک چه مزایا و چه معایبی دارد؟





❑ اصطلاح **کلید**، یک اصطلاح عام است و گونه‌هایی دارد:

۱- سوپرکلید (اَبَر کلید): SK

۲- کلید کاندید (کلید نامزد): CK

۳- کلید اصلی: PK

۴- کلید بدیل: AK

۵- کلید خارجی: FK



کلید در مدل رابطه‌ای – سوپر کلید

۲۴

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

رابطه $R(A_1, A_2, \dots, A_m)$ را در نظر می‌گیریم. □

H_R

سوپر کلید (Super Key) □

نمایشگر مقادیر
صفات S از تاپل t_i

هر زیر مجموعه $S \subseteq H_R$ که یکتایی مقدار داشته باشد.

اگر t_i و t_j دو تاپل دلخواه و متمایز از R باشند و $t_i(S) \neq t_j(S)$ آنگاه S یک سوپر کلید است. □

اگر N تعداد SK های رابطه R باشد، $N \geq 1$ است، زیرا در بدترین حالت خود H سوپر کلید می‌شود. □

چون بدنه، مجموعه است و تاپل تکراری نداریم.

$$1 \leq N \leq 2^m - 1$$

کاربرد سوپر کلید: □

در عمل، فاقد کاربرد مستقیم، در تئوری در بحث طراحی. □

در SQL: با UNIQUE محدودیت یکتایی مقدار را اعمال می‌کنیم. □



کلید در مدل رابطه‌ای – کلید کاندید

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۲۵

کلید کاندید (Candidate Key) ☐

هر زیرمجموعه $K \subseteq H_R$ که دو ویژگی داشته باشد:



۱- یکتایی مقدار

۲- کاهش ناپذیری (Irreducibility) یا کمینگی (Minimality)

- $K \subseteq H_R$ کاهش ناپذیر است هرگاه هر زیرمجموعه محض از K ، خود یکتایی مقدار نداشته باشد.
- هر زیرمجموعه از H_R به نحوی که یک صفت را از آن حذف کنیم دیگر یکتایی مقدار نداشته باشد.



رابطه	کلید کاندید
STT	STID
COT	COID
STCOT	(STID, COID)
S	S#
P	P#
SP	(S#, P#)



کلید در مدل رابطه‌ای – کلید کاندید (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۲۶

□ CKها بر اساس قواعد معنایی محیط به دست می‌آیند.

دو حالت مختلف:



شماره ملی شماره پروژه شماره کارمند
EMP PROJ (E#, J#, ENC, ...)
 CK CK

□ هر کارمند در بیش از یک پروژه می‌تواند شرکت داشته باشد.

EMP PROJ (E#, J#, ENC, ...)
 CK CK

□ هر کارمند در حداکثر یک پروژه می‌تواند شرکت داشته باشد.



کلید در مدل رابطه‌ای – کلید کاندید (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۲۷

□ خصوصیات کلید کاندید:

□ هر SK, CK هم هست ولی عکس این مطلب صادق نیست.

□ هر رابطه حداقل یک CK دارد، زیرا در بدترین حالت، خود H_R می‌شود CK.

□ رابطه می‌تواند بیش از یک CK داشته باشد.

□ رابطه R حداکثر چند CK دارد؟

□ بیشترین تعداد CK زمانی است که به اندازه نصف تعداد صفات رابطه در CK شرکت کنند.

□ CKهای رابطه می‌توانند همپوشا باشند، یعنی حداقل در یک صفت مشترک باشند ولی زیرمجموعه

یکدیگر نمی‌توانند باشند. (چرا؟)

□ بنابراین اگر رابطه از درجه m باشد، بیشترین تعداد CK: $C_n^m = \frac{m!}{n!(m-n)!}$ به نحوی که $n = \left\lfloor \frac{m}{2} \right\rfloor$.



کلید در مدل رابطه‌ای – کلید کاندید (ادامه)

۲۸

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

☐ **نقش کلید کاندید:** تضمین‌کننده عملیات تاپلی (و نه مجموعه‌ای) یا امکان ارجاع به تک تاپل در رابطه را فراهم می‌نماید.

☐ هر زیرمجموعه از CK، یک SK است (تفاوتشان در این است که CK با کمترین تعداد صفات یکتایی مقدار را می‌دهد).

☐ CK(های) رابطه باید به سیستم معرفی شوند.



CREATE RELATEION EMPROJ

(E# ... NOT NULL,

J# ... NOT NULL,

ENC ... NOT NULL)

CANDIDATE KEY (E#, J#)

CANDIDATE KEY (J#, ENC)

☐ **تذکر:** تئوری این را می‌گوید ولی در عمل، سمپادها نمی‌پذیرند و صرفاً می‌توان یک CK را به عنوان PK در آنها تعریف کرد.



کلید اصلی (Primary Key) ☐

کلید اصلی (PK) یکی از CKها است به انتخاب طراح. 

☐ در عمل با عبارت PRIMARY KEY تعریف می‌شود.

ضوابط انتخاب کلید اصلی: ☐

۱- شناسه رایج در محیط باشد.

۲- مقادیرش همیشه معلوم باشد (نه هر CK، آنکه به عنوان PK انتخاب می‌شود)

۳- کوتاه‌تر بودن طول

۴- حتی‌الامکان مقادیرش تغییر نکند.



کلید در مدل رابطه‌ای – کلید اصلی (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۳۰

□ دلایل لزوم انتخاب کلید اصلی:

۱- دلیل تاریخی: PK مفهوم آشنا تر برای طراحان است.

۲- ایجاد شاخص اتوماتیک روی PK.

۳- در بحث جامعیت DB: چون محدودیت هیچ مقدارناپذیری را اگر به همه CKها بدهیم خیلی محدود

کننده است. کلید CKای که این محدودیت را روی آن اعمال می کنند می شود PK.

□ اصالت مفهومی در مدل رابطه‌ای با کلید کاندید (CK) است.



کلید در مدل رابطه‌ای – کلید بدیل

۳۱

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

کلید بدیل (Alternate Key) ☐

به هر کلید کاندید (CK) غیر از کلید اصلی (PK)، کلید بدیل (AK) گویند.



☐ در عمل متناظر ندارد.

☐ اگر N تعداد AKهای رابطه R باشد، داریم $N \geq 0$.



ممکن است فقط یک CK داشته باشیم که آن هم می‌شود PK و دیگر AK نداریم.



کلید در مدل رابطه‌ای – کلید خارجی

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۳۲

کلید خارجی (Foreign Key)

□ **در عمل:** $T_2.C$ در T_2 ، کلید خارجی است هرگاه در T_1 ، کلید اصلی باشد.

□ **در تئوری:** صفت (ساده یا مرکب) $R_2.A_i$ در R_2 کلید خارجی است، هرگاه در R_1 (نه لزوماً متمایز از

R_2)، کلید کاندید (CK) باشد.

□ صفت (صفات) کلید خارجی باید **هم‌میدان** با صفت (صفات) کلید کاندید باشد و معمولاً هم‌نام با کلید

کاندید است، ولی گاه لازم می‌شود که نام دیگری داشته باشد.



رابطه	کلید خارجی	دلیل: CK در
STCOT	STID	STT
STCOT	COID	COT
SPJ	S#	S
SPJ	P#	P
SPJ	J#	J



کلید در مدل رابطه‌ای – کلید خارجی (ادامه)

۳۳

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

□ اگر N تعداد FKهای رابطه R باشد، داریم $N \geq 0$.

□ معرفی کلید خارجی با عبارت FOREIGN KEY انجام می‌شود.

□ نقش **کلید خارجی**: برای نمایش ارتباطهای صریح بین نوع موجودیت‌ها (و در نتیجه بین نمونه‌های آنها) به

کار می‌رود. منظور از ارتباط صریح، ارتباطی است که در مدل ER با لوزی مشخص شده است.



$S(\underline{S\#}, \dots)$ $P(\underline{P\#}, \dots)$
CK CK

$SP(\overset{FK}{\underline{S\#}}, \overset{FK}{\underline{P\#}}, \dots)$
CK

$SCOT(\overset{FK}{\underline{\text{STID}}}, \overset{FK}{\underline{\text{COID}}}, \dots)$
CK



کلید در مدل رابطه‌ای – کلید خارجی (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۳۴

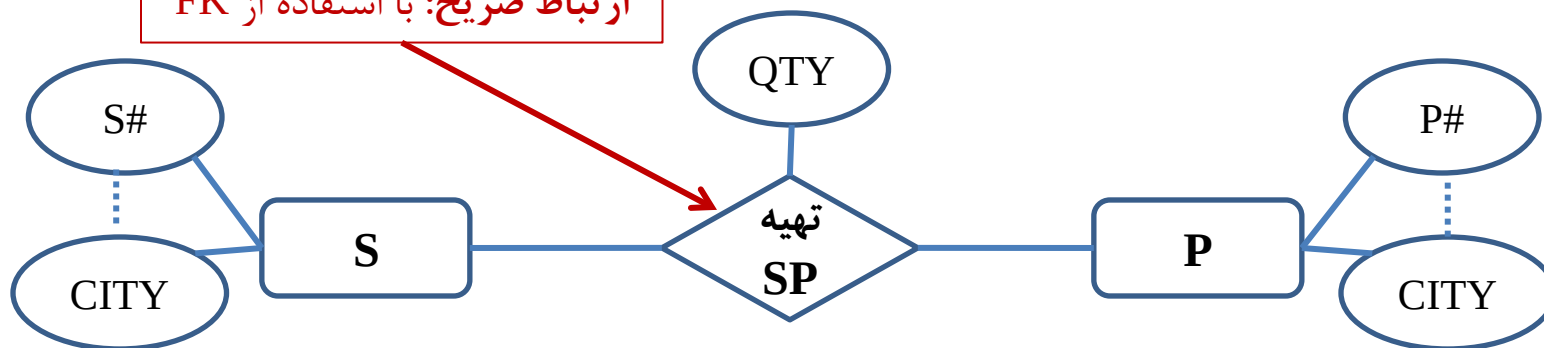
□ آیا FK تنها امکان نمایش ارتباط است یا امکان دیگری هم وجود دارد؟

□ FK تنها امکان نیست.

□ وجود هر صفت مشترک [هم دامنه و در عمل، هم نام (نه لزوماً)]، در عنوان مثلاً دو رابطه، نمایشگر

نوعی ارتباط است بین دو نوع موجودیت که با آن دو رابطه نمایش داده‌ایم.

ارتباط صریح: با استفاده از FK



→ S (S#, ..., CITY)
→ P (P#, ..., CITY)
→ SP (S#, P#, ...)

ارتباط ضمنی: از طریق هر صفت مشترک؛
صفت هم‌معنا (از یک میدان) و نه لزوماً هم‌نام

ارجاع



بحث تکمیلی: کلید خارجی – گراف ارجاع

۳۵

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

مفهوم گراف ارجاع

FK امکانی است برای ارجاع از یک رابطه به رابطه‌ای دیگر

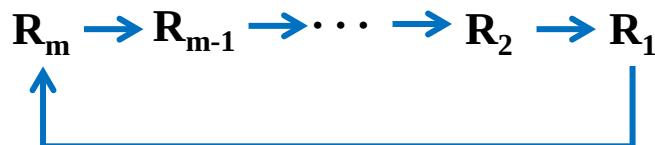
هر مقدار معلوم FK، امکانی است برای ارجاع مقداری، از تاپل (هایی) از رابطه (هایی) به تاپلی از رابطه (هایی).

تاریخچه گراف ارجاع امکانی است برای نمایش ارجاعات بین رابطه‌ها که در آن هر گره، نمایانگر یک رابطه و هر یال جهت‌دار، نمایانگر ارجاع از یک رابطه (حاوی کلید خارجی) به رابطه دیگر (حاوی کلید کاندید) است.

$P \leftarrow SP \rightarrow S$



شکل کلی مسیر ارجاع:



با این ارجاع می‌شود چرخه ارجاع

مسیر ارجاع می‌تواند چرخه‌ای باشد.



□ چرخه ارجاع می‌تواند تک‌رابطه‌ای باشد و این در صورتی است که یک رابطه خود ارجاع (Self-Referencing) داشته باشیم.

□ هنگامی که FK تعریف می‌کنیم باید معنایش را نیز بگوییم.

چرخه ارجاع بین دو رابطه کارمند و اداره.



شماره کارمند مدیر اداره

DEPT (D#, DTITLE, ..., E#)

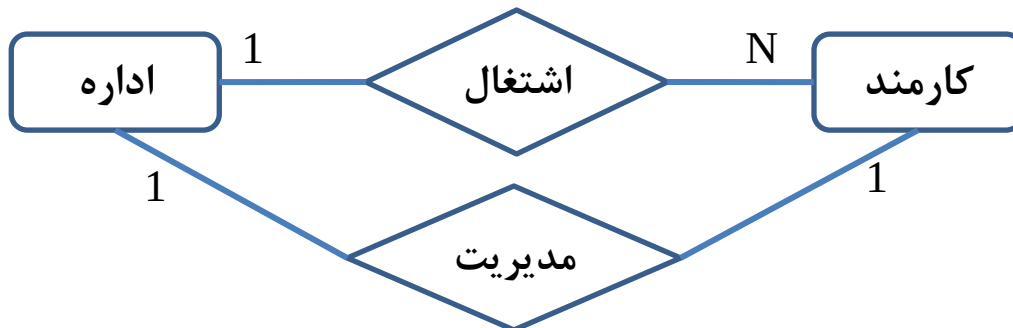
Unique

شماره اداره محل کار

EMPL (E#, ENAME, ..., D#)

DEPT $\longleftrightarrow$ EMPL

□ بر اساس کدام مدل‌سازی این طراحی انجام شده است؟





چرخه ارجاع تک‌رابطه‌ای کارمند با خودش.



شماره مدیر

EMPL (E#, ENAME, ENC, ..., EPHONE, EMANAGER#)



EMPL

نکته‌های مثال اخیر: ☐

☐ مثالی است از حالتی که در آن R1 و R2 در تعریف FK، لزوماً متمایز نیستند.

☐ رابطه EMPL به خود رجوع کننده (خود ارجاع) است.

☐ اگر m درجه EMPL باشد و n تعداد دامنه‌هایش باشد، داریم: $n \leq m-1$

☐ لزوم دگر نامی شماره کارمندی مدیر، چون عنوان رابطه (Heading)، مجموعه‌ای از نام صفات است.

☐ **تمرین:** این طراحی بر اساس کدام مدل‌سازی انجام شده است؟



بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

چرخه ارجاع سه رابطه‌ای



PROF (PRID, PRNAME, ..., DEID)

دانشکده استاد

DEPT(DEID, DTITLE,, UNID)

UNIV(UNID, UNAME, ..., UNPRESNUM)

شماره استادی رئیس دانشگاه



تمرین: این طراحی بر اساس کدام مدل‌سازی انجام شده است؟

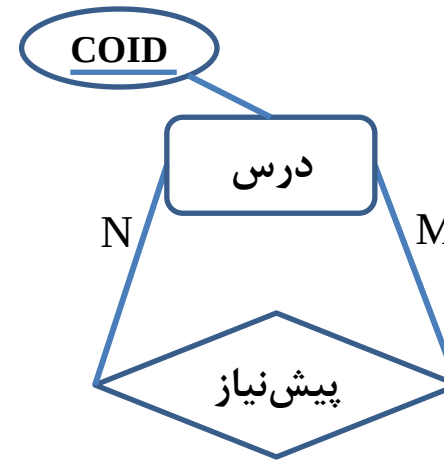


صرف وجود دور در ERD چرخه ارجاع ایجاد نمی‌شود.



COT (COID, ...)

COPRECO(COID, PRECO)



□ در چه وضعی چرخه ارجاع پدید می‌آید؟

□ باید به چندی ارتباطها توجه شود.



جامعیت پایگاه داده‌ها (DB Integrity) □



صحت و سازگاری داده‌های ذخیره شده در پایگاه داده‌ها

جنبه‌های کیفی داده (Data Quality Features)

□ مسئولیت کنترل جامعیت DB با RDBMS است.

□ بر اساس اطلاعاتی که کاربر [تیم طراح - پیاده‌ساز] به سیستم می‌دهد.

← قواعد یا محدودیت‌های جامعیتی (Integrity Rules/Constraints)

□ IRها [ICها] با استفاده از دستورات زبان پایگاهی به سیستم داده می‌شوند.

← اعلانی: قواعد به نحوی اعلان می‌شوند.

← اجرایی: قواعد در یک رویه به سیستم داده می‌شوند.



- ☐ هر DBMS باید بتواند جامعیت پایگاه داده‌ها را کنترل و تضمین کند.
- ☐ **دلیل:** زیرا همیشه ممکن است عواملی سبب نقض جامعیت شوند. از جمله:
 - ☐ اشتباه در برنامه‌های کاربردی (به ویژه اشتباهات معنایی)
 - ☐ اشتباه در وارد کردن داده‌ها
 - ☐ وجود افزونگی کنترل نشده
 - ☐ اجرای همروند تراکنشها به گونه‌ای که داده نامعتبر یا ناسازگار ایجاد شود.
 - ☐ خرابی‌های سخت‌افزاری و نرم‌افزاری که منجر به بروز ناسازگاری یا نقض صحت داده‌ها می‌شود.



□ اعمال قواعد جامعیتی (IRها) یا محدودیت‌های جامعیتی (ICها) برای سیستم **سربار** دارد.

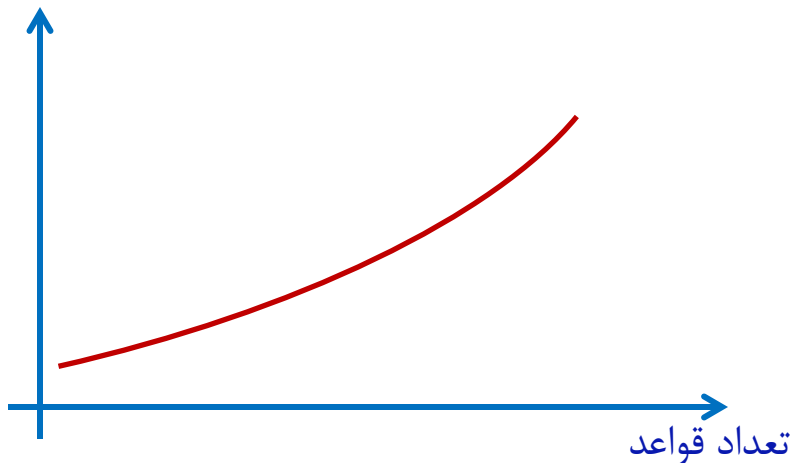
□ منشأ سربار (دلایل بروز سربار) در DBMS

□ انجام نگاشت‌ها (ناشی از معماری)

□ اعمال و کنترل قواعد جامعیتی

□ اعمال ضوابط و کنترل‌های امنیت داده‌ها در سطح DBMS

کار سیستم





□ IRها [ICها] در مدل رابطه‌ای

۱- قواعد [محدودیت‌های] عام: ناوابسته به داده‌های محیط: فراقواعد (MetaRules)

۲- قواعد [محدودیت‌های] خاص: وابسته به داده‌های محیط: قواعد کاربری (User Defined)

یا قواعد فعالیت‌های محیط (Business Rules)

□ قواعد عام در مدل رابطه‌ای

□ قاعده C1: جامعیت موجودیتی

□ قاعده C2: جامعیت ارجاعی



قواعد عام در مدل رابطه‌ای – قاعده جامعیت موجودیتی C1

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۴۴

□ قاعده (محدودیت) C1 – قاعده جامعیت موجودیتی (Entity IR)

□ ناظر است به PK.

□ هیچ جزء تشکیل دهنده PK نباید هیچ مقدار (Null) داشته باشد.

□ دلیل:

✓ PK عامل تمییز تاپل‌ها است.

✓ تاپل در مدل رابطه‌ای نمایشگر نمونه موجودیت است.

✓ PK عامل تمییز نمونه موجودیت‌ها است و تضمین کننده

دستیابی به تک موجودیت

۱- محدودیت یکتایی مقدار (با UNIQUE

□ مکانیزم اعمال C1: اعلان PK به سیستم کنترل می کند ← فقط این محدودیت کنترل می شود)

۲- محدودیت هیچ مقدار ناپذیری



قواعد عام در مدل رابطه‌ای – قاعده جامعیت ارجاعی C2

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۴۵

□ قاعده (محدودیت) C2 – قاعده جامعیت ارجاعی (Referential IR)

□ ناظر است به FK.

□ اگر $R_2.A_i$ در R_2 ، کلید خارجی باشد، مقدار A_i در هر تاپل از R_2 باید در R_1 مقدار قابل انطباق (Matchable Value) داشته باشد.

□ به عبارت دیگر باید هر مقدار معلوم A_i در R_2 ، در R_1 نیز وجود داشته باشد. یعنی در عمل می‌تواند در R_2 مقدار آن Null باشد (البته اگر جزء تشکیل‌دهنده کلید R_2 نباشد).

□ دلیل نیاز به این قاعده:

- FK عامل ارجاع است؛ ارجاع به نمونه موجودیت (ارجاع مقداری و نه ارجاع از طریق اشاره‌گر).
- در واقعیت نمی‌توان به نمونه موجودیت ناموجود ارجاع داد.



قواعد عام در مدل رابطه‌ای – قاعده جامعیت ارجاعی C2 (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۴۶



STT (STID, ...)

777

888

444

STCOT (STID, COID, ...)

777 CO1

... ...

444 CO4

INSERT INTO STCOT

VALUES ('999', 'CO9', ...)

چون برای 999 مقدار قابل انطباق در STT وجود ندارد، پس این درخواست رد می‌شود. □



قواعد عام در مدل رابطه‌ای – قاعده جامعیت ارجاعی C2 (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۴۷

□ برای اعمال قاعده C2 در مدل رابطه‌ای لازم است:

۱- معرفی FKها به سیستم

۲- دادن گراف ارجاع

۳- مشخص کردن روش اعمال در عملیات حذف و

به‌هنگام‌سازی مقدار کلید اصلی

(در درج روش خاصی لازم نیست و در صورت عدم

وجود تاپل مرجع، درخواست رد می‌شود.)

CREATE TABLE STCOT

(STID CHAR(6) NOT NULL

COID CHAR(6) NOT NULL

TR CHAR(1)

YR CHAR(5)

GR DEC(2, 2))

CHECK (0 <= GR <= 20)

PRIMARY KEY (STID, COID)

FOREIGN KEY STID REFERENCES STT (STID)

ON DELETE CASCADE

ON UPDATE CASCADE

FOREIGN KEY COID REFERENCES COT (COID)

ON DELETE CASCADE

ON UPDATE CASCADE

۲- گراف ارجاع

۳- روش اعمال (انتشار عمل)

۱- معرفی FK



قواعد عام در مدل رابطه‌ای – قاعده جامعیت ارجاعی C2 (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۴۸

□ روش‌های اعمال C2 در حذف (بعضاً در به‌هنگام‌سازی):

۱- روش CASCADE: انتشاری یا تسلسلی

در این روش با حذف (بروزرسانی) تاپل مرجع، تمام تاپل‌های رجوع کننده به آن حذف (بروزرسانی) می‌شوند.

هر چه گراف ارجاع سنگین‌تر باشد، کار سیستم در اینجا بیشتر است.
DELETE FROM STT
WHERE STID='444'

منجر می‌شود به

DELETE FROM STCOT
WHERE STID='444'

۲- روش RESTRICTED: روش منوط به ... (یا مشروط به ...) یا روش تعویقی

در این روش اگر بخواهیم تاپل مرجع را حذف کنیم، ابتدا باید تاپل‌های ارجاع کننده به آن حذف شوند.



قواعد عام در مدل رابطه‌ای – قاعده جامعیت ارجاعی C2 (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۴۹

□ روش‌های اعمال C2 در حذف (و بعضاً در به‌هنگام‌سازی):

۳- روش SET TO NULL: روش هیچ‌مقدارگذاری یا Nullifying

در این روش با حذف تاپل مرجع، FK در تاپل‌های رجوع کننده Null می‌شود به شرط آنکه FK جزء سازنده PK نباشد.

۴- روش SET TO DEFAULT: روش درج پیش‌فرض

در این روش، با حذف تاپل مرجع، FK با مقدار پیش‌فرض جاگذاری می‌شود به شرط آنکه FK جزء سازنده PK نباشد و تاپلی با مقدار پیش‌فرض برای PK در رابطه مرجع وجود داشته باشد.



قواعد عام در مدل رابطه‌ای – قاعده جامعیت ارجاعی C2 (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۵۰

□ روش‌های اعمال C2 در حذف (و بعضاً در به‌هنگام‌سازی):

۵- روش NO ACTION: عدم اقدام

برای این روش دو پیشنهاد داده شده است:

۵-۱- عدم اقدام مطلق: مثلاً مجاز نبودن عمل حذف تاپل مرجع و نمایش خطا.

۵-۲- انجام عمل خواسته شده و نه اقدام دیگر: تاپل مرجع حذف بشود ولی اقدام دیگری موقتاً انجام نشود. در

این مورد طراح-پیاده‌ساز می‌پذیرد که **موقتاً** (معمولاً تا پایان یک تراکنش و نه بعد از آن) محدودیت C2 نقض شود.

□ در حالت وجود **چرخه ارجاع** کدام روش انجام شدنی است؟

□ نمی‌توان روش RESTRICTED را در حالت کلی اعمال کرد. با روش CASCADE هم ممکن است

تاپل‌های ناخواسته حذف شود.

□ در این مواقع NO ACTION و یا SET TO NULL بر حسب شرایط می‌تواند راهگشا باشد.



❑ قواعد خاص در مدل رابطه‌ای:

❑ محدودیت دامنه‌ای (میدانی)

❑ محدودیت صفتی

❑ محدودیت رابطه‌ای

❑ محدودیت پایگاهی



قواعد خاص در مدل رابطه‌ای (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۵۲

محدودیت دامنه‌ای (میدانی)

 این محدودیت ناظر است به دامنه، مشخص‌کننده نوع و طیف مقادیر دامنه

 در همان دستور CREATE DOMAIN اعلان می‌شود.

دستور ایجاد دامنه '?...?' **CREATE DOMAIN** GRADE DEC(2, 2) **DEFAULT**

نام محدودیت (اختیاری) **CONSTRAINT** GRADECONST

CHECK VALUE BETWEEN (0, 20)



DROP DOMAIN GRADE دستور حذف دامنه



□ محدودیت صفتی [استونی]

□ این محدودیت ناشی می‌شود از محدودیت دامنه‌اش

□ صفت می‌تواند محدودیت‌های دیگری هم داشته باشد، به شرطی که ناقض محدودیت دامنه‌ای‌اش نباشد.

محدودیت‌های ناظر به صفت:



۱- صفت نمره باید بین ۰ تا ۲۰ باشد.

۲- صفت سن کاهش نمی‌یابد (محدودیت پردازشی).

محدودیت ۱، یک **محدودیت وضعیتی** است ولی محدودیت ۲، یک **محدودیت گذاری** است.



□ محدودیت صفتی را چگونه می‌توان به سیستم اعلان کرد؟

۱- با تعریف دامنه‌اش اعلان می‌شود.

۲- در همان دستور CREATE TABLE با عبارت CHECK اعلان می‌شود.

جدول انتخاب درس



CREATE TABLE STCOT

(STID ...

COID ...

TR ...

GR ...)

CHECK (0 <= GR <= 20)

۳- با ASSERTION اعلان می‌شود. (در اسلایدهای بعدی معرفی می‌شود)

۴- با TRIGGER به سیستم داده می‌شود. (در اسلایدهای بعدی معرفی می‌شود)



محدودیت رابطه‌ای ☐

☐ ناظر است به تاپل‌های یک رابطه (درون رابطه‌ای Intra-relational).

☐ حیطه اعمالش یک رابطه است و مقادیر مجاز یک متغیر رابطه‌ای را مشخص می‌کند.

☐ باید در هر عملی که بر روی رابطه انجام می‌شود (که منجر به تغییر در متغیر رابطه‌ای می‌گردد)

کنترل شود.

تهیه‌کنندگان ساکن شهر C2 نمی‌توانند مقدار وضعیت بیش از ۱۵ داشته باشند.





محدودیت پایگاهی

 ناظر است به تاپل‌های بیش از یک رابطه که به نحوی با هم ارتباط معنایی [منطقی] دارند.

مثال  رابطه بین جداول STT و STCOT

یا رابطه بین جداول S و SP

مثال  دانشجوی رشته کامپیوتر نمی‌تواند درس آمار و احتمال را از گروه آموزشی D13 (دانشکده ریاضی)

انتخاب کند. رابطه‌های دخیل: STT، COT و STCOT

مثال  تهیه‌کننده ساکن شهر C7 با وضعیت کمتر از ۱۵، نمی‌تواند قطعه آبی رنگ با وزن بیش از ۱۰ گرم به تعداد بیش از ۱۰۰ عدد تهیه کند.

 محدودیت‌های رابطه‌ای و پایگاهی چگونه اعمال می‌شوند؟

▪ با ASSERTION (اعلانی)

▪ با TRIGGER (اجرایی)



اظهار – ASSERTION ☐

☐ امکانی است اِعلانی برای بیان محدودیت‌های رابطه‌ای و پایگاهی [و صفتی]

CREATE ASSERTION *name*
CHECK *condition(s)*

☐ در قسمت *condition(s)* می‌توان یک شرط ساده، یک عبارت بولی شامل چند شرط و نیز یک عبارت **SELECT** معتبر نوشت (همانطور که بعد از عبارت **WHERE** نوشته می‌شود).

☐ دستور حذف اظهار

DROP ASSERTION *name*



امکانات بیان محدودیت‌ها – اظهار (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۵۸

با این اظهار، محدودیت یکتایی مقادیر صفت کد ملی STNATID اعلان می‌شود.



```
CREATE ASSERTION UNC-CHECK  
CHECK (UNIQUE(SELECT STNATID FROM STT))
```

با این اظهار این محدودیت که «جمع واحدهای انتخابی دانشجو در هر ترم-سال نباید بیش از ۲۰ واحد

باشد»، اعلان می‌شود.



```
CREATE ASSERTION TOTCRED-CHECK  
CHECK (NOT EXISTS (SELECT STID  
FROM COT JOIN STCOT  
GROUP BY (STID, TR, YR)  
HAVING SUM(CREDIT) > 20) )
```



امکانات بیان محدودیت‌ها – اظهار (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۵۹

همه دانشجویان دانشکده مهندسی کامپیوتر (CE) باید درس مبانی برنامه‌سازی (با کد ۴۰۱۱۱) را اخذ کرده باشند.



```
CREATE ASSERTION ELEM-CHECK
CHECK (NOT EXISTS
      ( SELECT * FROM STT
        WHERE DEPT='CE' AND
          NOT EXISTS
            ( SELECT * FROM STCOT
              WHERE STCOT.STID = STT.STID
                AND STCOT.COID='40111' ) ) )
```



TRIGGER – [راه‌انداز]

 امکانی است اجرایی برای اعمال محدودیت‌های [صفتی،] رابطه‌ای و پایگاهی قبل یا بعد از بروز یک

رویداد و یا به جای یک رویداد (معمولاً تغییر دهنده داده‌ها).

```
CREATE TRIGGER name
{BEFORE | AFTER | INSTEAD OF}
{INSERT | DELETE | UPDATE OF columnlist
ON tablename
[REFERENCING { OLD ROW | NEW ROW | OLD TABLE | NEW TABLE } AS name ]
[FOR EACH {ROW | STATEMENT}]
{(WHEN condition(s)
SQL Procedure
}}
```

 مفهوم نظری TRIGGER: مفهوم قاعده فعال [مفهوم محوری است در ADBMS ها]



ساختار (قاعده ECA): if **E**vent on **C**ondition, then **A**ction


Insert
Delete
Update



امکانات بیان محدودیت‌ها – رهانا (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۶۱

□ با FOR EACH ROW بعد از بروز رویداد در هر سطر عبارت رهانا اجرا شود.

□ با FOR EACH STATEMENT فقط یک بار پس از بروز رویداد (با هر تعداد سطر متاثر از آن)، عبارت رهانا اجرا شود.

این رهانا این محدودیت را که «حقوق کارمند هیچگاه کاهش نمی‌یابد» اعمال می‌کند.



```
CREATE TRIGGER EMP-PAY-TRIG
```

```
BEFORE UPDATE OF EMPSAL
```

```
ON EMPL
```

```
REFERENCING OLD AS OEMPL, NEW AS NEMPL
```

```
FOR EACH ROW
```

```
(WHEN OEMPL.EMPSAL > NEMPL.EMPSAL
```

```
SIGNAL SQL State '7005' ('salary cannot be decreased')
```

```
)
```



این رهانا باعث حفظ سازگاری در جدول PROF می‌شود تا همواره صفت SALAUG حاوی آخرین میزان افزایش حقوق استاد باشد.



```
CREATE TRIGGER EMP-PAY-TRIG
  AFTER UPDATE OF PSALARY
  ON PROF
  REFERENCING OLD AS OPROF, NEW AS NPROF
  FOR EACH ROW
  (UPDATE PROF
    SET SALAUG=NPROF.PSALARY – OPROF.PSALARY
    WHERE PROF.PID=OPROF.PID
  )
```

اگر بیش از یک عبارت باشد، آنها را داخل BEGIN و END قرار می‌دهیم.



از کاربردهای رهانا، استفاده از آن در انجام عملیات ذخیره‌سازی از دید خارجی است (به خصوص در سمپادهایی که از عملیات در دید خارجی پشتیبانی نمی‌کنند).

STT1 (STID, NAME, MAJOR, LEVEL) STT2 (STID, DEPT, BDATE, NATID)

CREATE VIEW CE-STT

AS SELECT STID, NAME, MAJOR

FROM STT1 JOIN STT2

WHERE DEPT='CE' AND LEVEL='BS'

CREATE TRIGGER INS-VIEW-TRIG

INSTEAD OF INSERT ON CE-STT

REFERENCING NEW AS NST

FOR EACH ROW

BEGIN

INSERT INTO STT1 VALUES (NST.STID, NST.NAME, NST.MAJOR, 'BS')

INSERT INTO STT1 VALUES (NST.STID, 'CE', NULL, NULL)

END



امکانات بیان محدودیت‌ها – رهانا (ادامه)

بخش ششم: مفاهیم اساسی مدل داده رابطه‌ای

۶۴

این رهانا باعث اعمال قاعده C2 در عمل حذف می‌شود.



```
CREATE TRIGGER DEL-TRIG  
  BEFORE DELETE  
  ON COT  
  REFERENCING OLD AS OCOT  
  FOR EACH ROW  
  (DELETE FROM STCOT  
    WHERE STCOT.COID=OCOT.COID )
```

مطالعه مثالهای بیشتر از اظهار و رهانا در یادداشتهای تکمیلی ☐



پرسش و پاسخ ...

amini@sharif.edu

به نام آنکه جان را فکرت آموخت



بخش هفتم: عملیات در پایگاه داده رابطه‌ای

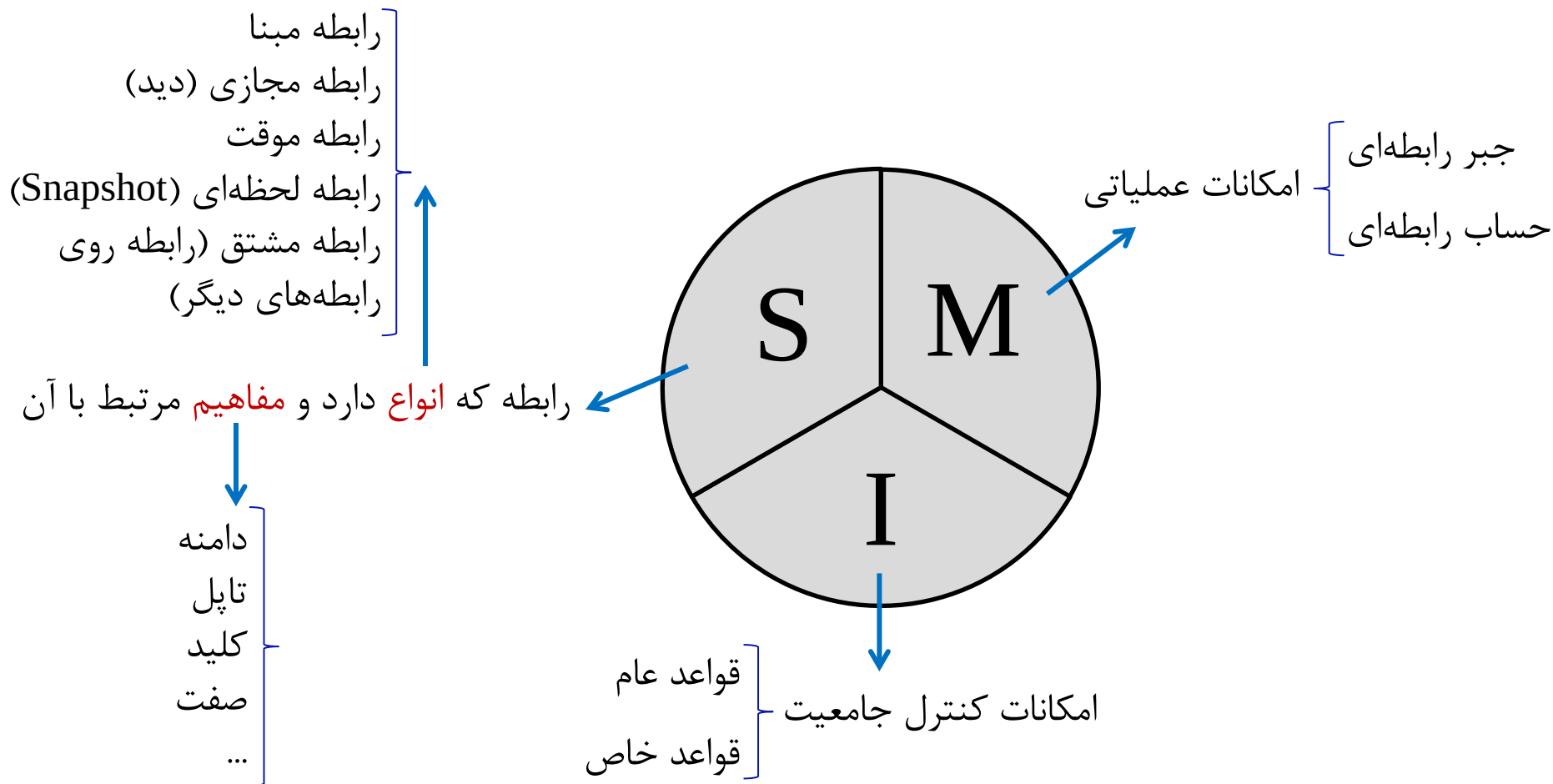
مرتضی امینی

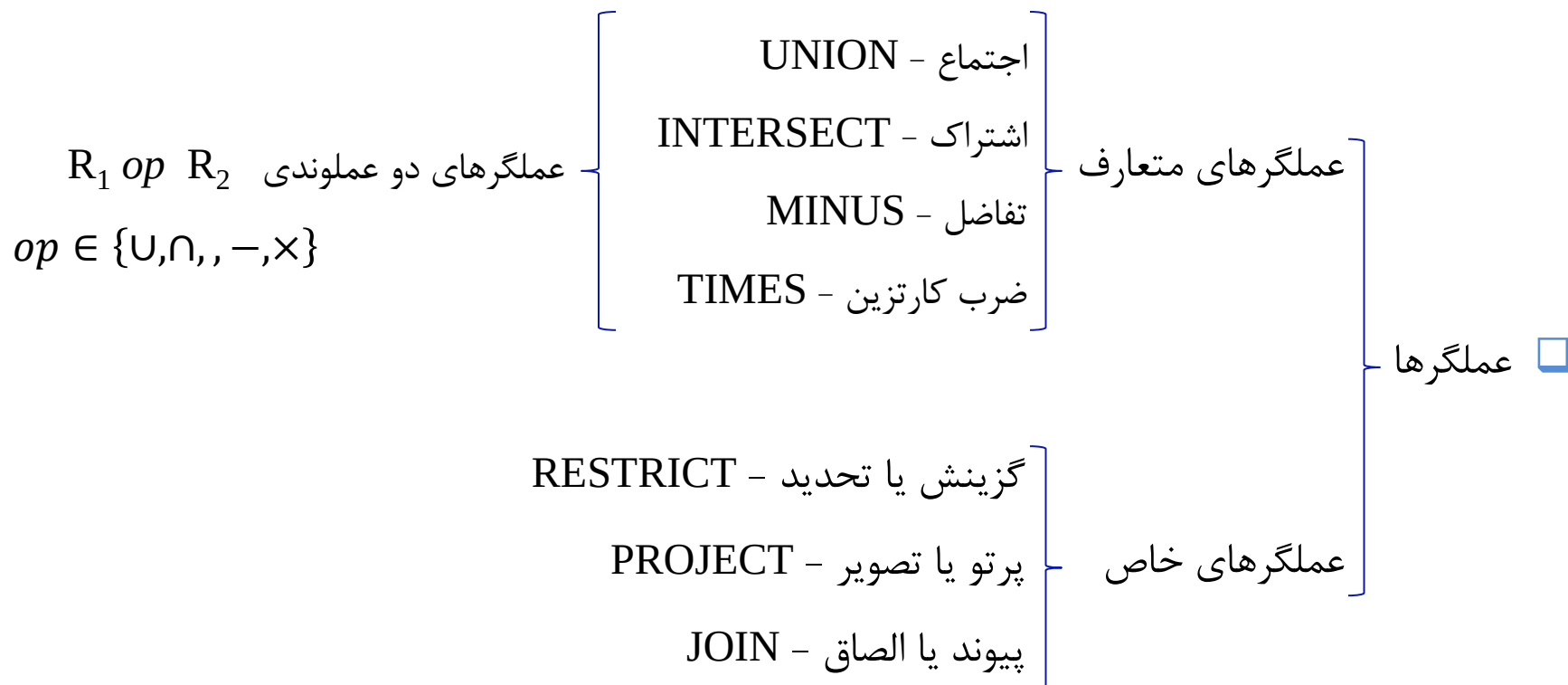
نیمسال دوم ۹۶-۹۷

(محتویات اسلایدها برگرفته از یادداشت‌های کلاسی استاد محمدتقی روحانی رانکوهی است.)



بخش هفتم: عملیات در پایگاه داده رابطه‌ای







□ **خاصیت بسته بودن:** حاصل ارزیابی هر عبارت جبر رابطه‌ای معتبر، باز هم یک رابطه است (که تاپل تکراری ندارد).

□ برای سه عملگر $\cup$ ، $\cap$ و $-$ ، باید عملوندها نوع-سازگار (Type Compatible) باشند:

□ **پیش شرط:** $H_{R_1} = H_{R_2}$

□ $R_3 = R_1 \text{ op } R_2 \longrightarrow H_{R_3} = H_{R_1} = H_{R_2} \quad \text{op} \in \{\cup, \cap, -, \dots\}$

□ بدنه نتیجه، حاصل انجام هر یک از اعمال اجتماع، اشتراک و یا تفاضل دو مجموعه بدنه است.

□ در عملگر ضرب کارتزین (TIMES):

□ **شرط:** در عنوان دو رابطه نباید صفت هم‌نام وجود داشته باشد. $H_{R_2} \cap H_{R_1} = \emptyset$

□ عنوان رابطه نتیجه برابر است با $H_{R_2} \cup H_{R_1}$ و بدنه نتیجه برابر ضرب کارتزین دو مجموعه بدنه است.

□ TIMES در SQL چگونه شبیه‌سازی می‌شود؟



عملگر گزینش یا تحدید - RESTRICT

نماد ریاضی: σ_c

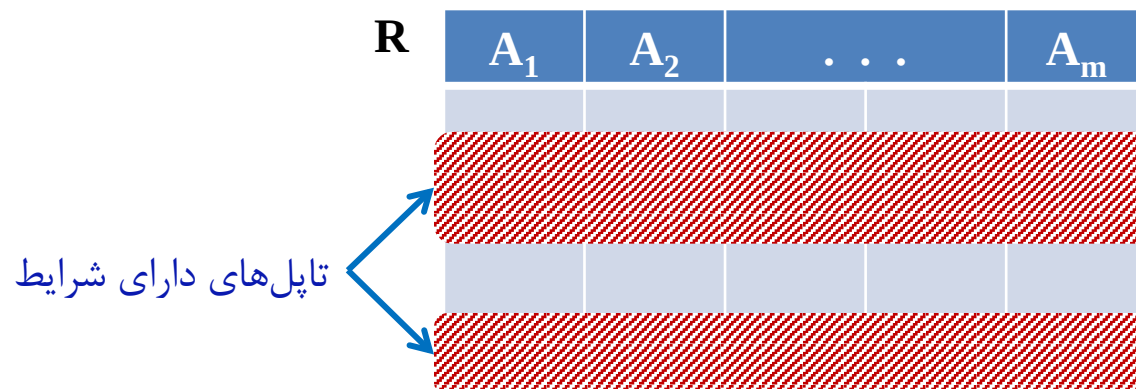
شرط یا شرایط گزینش

یک عبارت بولی تشکیل شده از شرطهای ساده به صورت $(A_i \text{ theta } A_j)$ یا $(A_i \text{ theta literal})$ که در آن theta یکی از عملگرهای $=, \neq, <, >, \leq, \geq$ است و literal یک مقدار ثابت است.

شکل کلی: $\sigma_c(R)$ یا $R \text{ WHERE } c$ یا **RESTRICT R WHERE c**

تک عملوندی: Monadic

عملکرد (در نمایش جدولی رابطه): زیرمجموعه‌ای افقی می‌دهد. عملگر تاپل (ها) یاب





بخش هفتم: عملیات در پایگاه داده رابطه‌ای

مشخصات کامل دانشجویان رشته فیزیک دوره کارشناسی را بدهید.



$\sigma_{STJ='phys' \wedge STL='bs'}(STT)$

```
SELECT STT.*  
FROM STT  
WHERE STJ='phys' AND STL='bs'
```

وقتی در شرط C (یا کلاز WHERE) بخشی از کلید را با شرط تساوی داده باشیم.

اگر $R' = \sigma_c(R)$ باشد آنگاه $CK_{R'} \subseteq CK_R$.





□ عملگر گزینش جابجایی پذیر است، یعنی:

$$\sigma_{c_1}(\sigma_{c_2}(R)) = \sigma_{c_2}(\sigma_{c_1}(R)) = \sigma_{c_1 \wedge c_2}(R)$$

□ عبارتهای جبری معادل:

$$R \text{ WHERE } (C_1 \text{ AND } C_2) \equiv (R \text{ WHERE } C_1) \text{ INTERSECT } (R \text{ WHERE } C_2) \quad \square$$

$$R \text{ WHERE } (C_1 \text{ OR } C_2) \equiv (R \text{ WHERE } C_1) \text{ UNION } (R \text{ WHERE } C_2) \quad \square$$

$$R \text{ WHERE NOT } C \equiv R \text{ MINUS } (R \text{ WHERE } C) \quad \square$$



PROJECT پرتو - PROJECT

□ نماد ریاضی: Π

□ شکل کلی: $\Pi_{\langle L \rangle}(R)$ یا $(R)[L]$ یا **PROJECT R OVER (L)**
لیست صفات پرتو

□ تک عملوندی: Monadic

□ عملکرد (در نمایش جدولی رابطه): زیرمجموعه عمودی می‌دهد. ← عملگر ستون(ها) یاب

R	A_1	...	A_i	...	A_j	...	A_m



بخش هفتم: عملیات در پایگاه داده رابطه‌ای

□ عملگر پرتو **تکراری‌ها** را حذف می‌کند. ← چون جواب رابطه است، پس یک مجموعه است و عضو تکراری ندارد.

شماره و رشته تمام دانشجویان را بدهید.



$\Pi_{\langle \text{STID}, \text{STJ} \rangle}(\text{STT})$

SELECT STID, STJ **FROM** STT

شماره دانشجویانی که درسی انتخاب نکرده‌اند.



$R := \Pi_{\langle \text{STID} \rangle}(\text{STT}) - \Pi_{\langle \text{STID} \rangle}(\text{STCOT})$

شماره و مقطع تحصیلی دانشجویان رشته IT را بدهید.



$\Pi_{\langle \text{STID}, \text{STL} \rangle}(\sigma_{\text{STJ}='IT'}(\text{STT}))$



□ اگر $R' = \Pi_{\langle L \rangle}(R)$ باشد آنگاه:

□ اگر $CK_R \subseteq L$ آنگاه $CK_{R'} = CK_R$.

□ اگر نه در بدترین حالت $CK_{R'} = L$.

کجای؟ □ اگر $R' = R_1 \text{ op } R_2$ و $op \in \{\cup, \cap, -, \times\}$ ، آنگاه $CK_{R'} = ?$.

□ SELECT در SQL استاندارد، در حالت کلی ترکیبی از دو عملگر RESTRICT و PROJECT است.



□ عملگر پرتو گسترش یافته – EXTENDED PROJECT

□ نماد ریاضی: Π

□ شکل کلی: $\Pi_{\langle F_1, F_2, \dots, F_n \rangle}(\mathbf{R})$

لیست صفات و یا توابع حسابی پرتو ←

□ این عملگر امکان می‌دهد تا در لیست صفات پرتو، از توابع حسابی استفاده شود و صفت (صفاتی) با

مقادیر حاصل از اجرای تابع (توابع) در رابطه جواب داشت.

رابطه‌ای با صفات شماره دانشجو، شماره درس و نمره دانشجو در درس، تغییر یافته با فرمول



$G := 1.2 * \text{GRADE}$ بدهید.

$\Pi_{\langle \text{STID}, \text{COID}, (1.2 * \text{GRADE}) \text{ RENAME AS } G \rangle}(\text{STCOT})$



□ عملگر تغییر نام - RENAME

□ نماد ریاضی: ρ

□ شکل کلی: $\rho_R(E)$

← نام رابطه حاصل از عبارت جبر رابطه‌ای E

□ این عملگر برای نامیدن رابطه حاصل از یک عبارت جبر رابطه‌ای به کار می‌رود.

□ عملکرد: $\rho_R(E)$ رابطه حاصل از عبارت جبر رابطه‌ای E را با نام R برمی‌گرداند.

□ از عملگر RENAME برای دگرنامی صفت هم می‌توان استفاده کرد (مشابه آنچه در مثال اسلاید قبل

آمد). مثلاً با دستور $R \text{ RENAME } A_i \text{ AS } B_j$ ، به صفت A_i از R ، نام دیگر B_j داده می‌شود.



□ عملگر پیوند JOIN (مدل ریاضی عمومی)

□ نام عمومی: Theta Join

□ نماد ریاضی: $\bowtie_{Cond(s)}$
← شرط پیوند

$$\left\{ \begin{array}{l} R_1 (A_1, A_2, \dots, A_n) \\ R_2 (B_1, B_2, \dots, B_m) \end{array} \right.$$

□ فرض: دو رابطه R_1 و R_2 نام صفت مشترک ندارند.

□ شکل کلی: $R_1 \bowtie_C R_2$ یا $R_1 \theta\text{-JOIN}_C R_2$ یا فقط $R_1 \text{ JOIN}_C R_2$

$$\left. \begin{array}{ll} \text{EQUI-JOIN} & = \\ \text{NOT EQUI-JOIN} & \neq \\ \text{LESS THAN-JOIN} & < \\ \text{LESS EQUI-JOIN} & \leq \\ \text{GREATER THAN-JOIN} & > \\ \text{GREATER EQUI-JOIN} & \geq \end{array} \right\} \text{Theta} \quad \square$$



شرط ساده پیوند (C): $R_1.A_i \text{ theta } R_2.B_j$

صفات پیوند

که باید هم‌دامنه و هم‌نام باشند.

چون نتیجه JOIN رابطه است و در heading صفت تکراری نباید وجود داشته باشد.

نکته: اگر صفات پیوند هم‌نام باشند، حداقل یکی را باید دگرنامی کرد (به دلیل وجود این راه حل،

حساسیتی در وجود صفت مشترک نداریم).

در حالت کلی شرط پیوند می‌تواند به صورت زیر باشد که در آن $C_1, \dots, C_n$ قالب بالا (قالب شرط ساده

پیوند) را دارند. $\langle c_1 \rangle \text{ AND } \langle c_2 \rangle \text{ AND } \dots \text{ AND } \langle c_n \rangle$

$\langle R1.A1 = R2.B1 \rangle \text{ AND } \langle R1.A2 = R2.B2 \rangle$





مشخصات کامل جفت تهیه‌کننده-قطعه از یک شهر را بدهید.



$R_1 := S \bowtie_{S.CITY=P.PCITY} (P \text{ RENAME CITY AS PCITY})$

S (S#, SNAME, STATUS, CITY)

S1	C1
S2	C2
S3	C3
S4	C4
S5	C5
S6	C6

P (P#, ... , W, CITY)

P1	5	C1
P2	6	C2
P3	4	C1
P4	7	C4
P5	10	C5

R₁ (S#, ..., CITY, P#, ... , W, PCITY)

S1	C1	P1	5	C1
S1	C1	P3	4	C1
S2	C2	P2	6	C2
S3				
S4	C4	P4	7	C4
S5	C5	P5	10	C5
S6				

تا پل پیوندشده ندارد.

تا پل پیوندشده ندارد.



بخش هفتم: عملیات در پایگاه داده رابطه‌ای

$$R_3 = R_1 \bowtie_C R_2 \quad \square \text{ عملکرد:}$$

$$H_{R_3} = H_{R_1} \cup H_{R_2}$$

■ در بدنه R_3 تاپل‌های پیوندشده از دو رابطه قرار دارند.

□ خصوصیات:

■ $R_1 \bowtie_C R_2 = R_2 \bowtie_C R_1$ چون صفات در heading رابطه نظم مکانی ندارند.

■ $R_1 \bowtie_C R_2 = \sigma_C(R_1 \times R_2)$ حاصل Theta-Join در حالت عمومی، زیرمجموعه‌ای افقی از

ضرب کارتیزین است که در آن تاپل‌هایی از حاصلضرب که حائز شرط پیوند هستند حضور دارند.

⊆ وقتی در شرط پیوند، تساوی بخشی از کلید هر دو رابطه را داده باشیم.

اگر $R' = R_1 \bowtie_C R_2$ باشد، آنگاه $CK_{R'} \subseteq CK_{R_1} \cup CK_{R_2}$



□ در عمل: دستور INNER JOIN در SQL، پیاده‌سازی این نوع از پیوند است.



گونه‌های خاص عملگر پیوند – پیوند طبیعی

بخش هفتم: عملیات در پایگاه داده رابطه‌ای

۱۷

پیوند طبیعی (Natural Join)

 گونه‌ای از پیوند است که دو ویژگی دارد:

۱- Theta =

۲- صفات پیوند یک بار در جواب می‌آیند. (صفت یا صفات پیوند باید هم‌نام هم باشند).

$$R_2 := S \bowtie_{S.CITY=P.CITY} P$$



$R_2 (S\#, \dots, CITY, P\#, \dots, W)$

S1	C1	P1	5
S1	C1	P3	4
S2	C2	P2	6
S4	C4	P4	7
S5	C5	P5	10



گونه‌های خاص عملگر پیوند – پیوند طبیعی (ادامه)

۱۸

بخش هفتم: عملیات در پایگاه داده رابطه‌ای

□ اگر صفت مشترک [هم‌نام و هم‌دامنه] یک صفت باشد، نیازی به قید کردن نیست.

اما اگر بیش از یک صفت باشد، باید صفت یا صفات پیوند را قید کنیم.

اگر قید نکنیم، پیوند روی تساوی مقادیر تمام صفات مشترک انجام می‌شود.

$$R_1: (A, B, C)$$

$$R_2: (A, F, C)$$

$$R' = R_1 \bowtie R_2$$

$$R': (A, B, C, F)$$

□ اگر $H_{R_1} \cap H_{R_2} = \emptyset$ ، آنگاه $R_1 \bowtie R_2 = R_1 \times R_2$.

□ اگر $H_{R_1} = H_{R_2}$ ، آنگاه $R_1 \bowtie R_2 = R_1 \cap R_2$.

□ **در عمل:** دستور NATURAL JOIN در SQL، پیاده‌سازی این نوع از پیوند است که پیوند را روی همه صفات

مشترک (بدون قید مشخص نمودن صفات پیوند) انجام می‌دهد.



نیم‌پیوند (Semijoin)

 در شکل عمومی با هر Theta نوشته می‌شود.

 نماد: $\bowtie_C$ (در چپ تعریف شده)

 مدل ریاضی: $R_3 := R_1 \bowtie_C R_2 = \Pi_{\langle H_{R_1} \rangle}(R_1 \bowtie_C R_2)$

 عملکرد:

$$H_{R_3} = H_{R_1} \quad \blacksquare$$

 در بدنه R_3 : تاپل‌های پیوند شدنی از رابطه چپ



گونه‌های خاص عملگر پیوند – نیم‌پیوند (ادامه)

بخش هفتم: عملیات در پایگاه داده رابطه‌ای

۲۰

$R_3 := S \bowtie_{S.CITY=P.PCITY} (P \text{ RENAME CITY AS PCITY})$



$R_3 (S\#, \dots, CITY)$

S1	C1
S2	C2
S4	C4
S5	C5

کاربرد این عملگر چیست؟



تمرین: عملگر نیم‌پیوند در SQL شبیه‌سازی شود. 



گونه‌های خاص عملگر پیوند – برون‌پیوند

بخش هفتم: عملیات در پایگاه داده رابطه‌ای

۲۱

برون‌پیوند (Outer Join)

 Theta هر چیزی می‌تواند باشد.

 سه گونه دارد:

$\bowtie_C$ Left O. J. - ۱

$\bowtie_C$ Right O. J. - ۲

$\bowtie_C$ Full O. J. - ۳

 عملکرد $R_4 := R_1 \bowtie_C R_2$:

$$H_{R_4} = H_{R_1} \cup H_{R_2} \quad \blacksquare$$

▪ در بدنه R_4 : تاپل‌های پیوند شدنی از دو رابطه و

تاپل‌های پیوندناشدنی از رابطه چپ گسترش یافته با هیچ مقدار (Null Value)

 **در عمل:** دستور LEFT/RIGHT/FULL OUTER JOIN در SQL ، پیاده‌سازی این نوع از پیوند

است.



گونه‌های خاص عملگر پیوند – برون‌پیوند (ادامه)

بخش هفتم: عملیات در پایگاه داده رابطه‌ای

۲۲

$$R_4 := S \bowtie P$$



$R_4 (S\#, \dots, CITY, P\#, \dots, W)$

S1	C1	P1	5
S1	C1	P3	4
S2	C2	P2	6
S4	C4	P4	7
<u>S5</u>	<u>C5</u>	<u>P5</u>	<u>10</u>
S3	C3	?	?
S6	C6	?	?

کلید R_4 (CK_{R_4}) چیست؟ بی‌تردید کلید اصلی ندارد.



مشکل Outer Join:

۱- از نظر ریاضی رابطه نیست، چون کلید اصلی ندارد.

۲- مصرف حافظه زیاد

این عملگرها در عمل چه کاربردی دارند؟



آیا عملگرهای Outer Join خاصیت جابجایی دارند؟





نیم تفریق (Semi Minus) □

$$R_1 \text{ SEMIMINUS } R_2 = R_1 \text{ MINUS } (R_1 \text{ SEMIJOIN } R_2) \quad \square$$

عملکرد □

$$H_{R_5} = H_{R_1} \quad \blacksquare$$

در بدنه R_5 : تاپل‌های پیوند نشدنی از رابطه چپ $\blacksquare$

بخش هفتم: عملیات در پایگاه داده رابطه‌ای

عملگر تقسیم (Divide)

مفروضند رابطه‌های:

$$\left[\begin{array}{l} R_1(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m) \\ R_2(B_1, B_2, \dots, B_m) \end{array} \right]$$

X Y

شرط عمل:

$$R_3(X) := R_1(X, Y) \div R_2(Y) \longrightarrow H_{R_2} \subseteq H_{R_1} \quad \blacksquare$$

عملکرد:

$$H_{R_3} = X = H_{R_1} - H_{R_2} - 1$$

۲- در بدنه R_3 : بخش X از تاپلهایی از R_1 که تمام مقادیر Y از R_2 باشند.



$$R_1(S\#, P\#) \div R_2(P\#) = R_3(S\#)$$

S1	P1	P1	S1
S1	P2	P2	
S1	P3	P3	
S2	P1		
S2	P2		
S3	P1		

$$R_1(S\#, P\#) \div R_4(P\#) = R_5(S\#)$$

S1	P1	P1	S1
S1	P2	P2	S2
S1	P3		
S2	P1		
S2	P2		
S3	P1		



☐ ضرب و تقسیم جبر رابطه‌ای لزوماً عکس هم نیستند.

☐ **تمرین:** عملگر تقسیم را در SQL شبیه‌سازی کنید.

☐ **تمرین:** Q3 و Q4 (صفحه A-3 از یادداشتهای تکمیلی سری II) را بدون استفاده از عملگر DIVIDE

بنویسید.



□ عملگر گسترش – EXTEND

□ صفت یا صفاتی را به عنوان (heading) یک رابطه اضافه می‌کند. حاصل، رابطه دیگری است.

EXTEND STUD ADD STADDRESS

STUD (STID, ..., STD, STADDRESS)

□ در SQL با ALTER TABLE پیاده‌سازی شده ولی ALTER ستون(هایی) را به همان جدول اضافه می‌کند.

□ با این عملگر می‌توانیم یک ستون محاسبه‌شدنی به رابطه اضافه نماییم.



عملگر تلخیص – SUMMARIZE ☐

☐ تاپل‌های رابطه را گروه‌بندی می‌کند به نحوی که مقدار صفت (صفات) گروه‌بندی در هر گروه یکسان باشد؛ معمولاً با یک یا چند تابع جمعی استفاده می‌شود.

☐ این عملگر در SQL با GROUP BY پیاده‌سازی شده است.

SUMMARIZE STCOT BY (STID) ADD AVG(GRADE) AS AVER

☐ برای این پرسش‌ها، اول عنوان (Heading) رابطه جواب را تعیین می‌کنیم.

☐ به جای AVG می‌توانیم از توابع جمع و یا گروهی دیگر مانند MIN (حداقل)، MAX (حداکثر)، SUM (جمع) و یا COUNT (شمارشگر تاپل‌ها) استفاده کنیم.



عملگر GROUP ☐

عملگر GROUP پیشنهاد Date است، برای تبدیل رابطه نرمال به غیرنرمال (در SQL، NEST است).

عکس آن UNGROUP (در SQL، UNNEST) است.

SP GROUP (P#, QTY) AS NNPQTY

NNSP (S#, NNPQTY[P#, QTY])

S1	P1	50
	P2	70
	P3	60
S2	P1	100
	P2	150

با استفاده از UNGROUP، رابطه نرمال SP را می‌توانیم مجدداً به دست آوریم.

NNSP UNGROUP NNPQTY



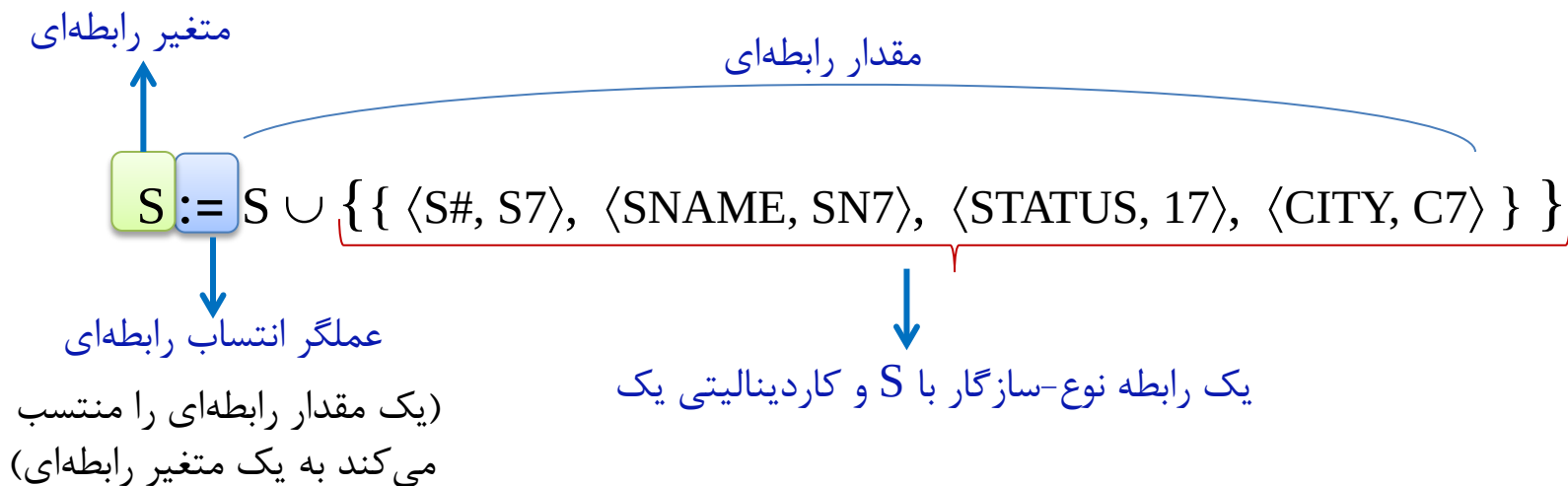
عملیات ذخیره‌سازی با جبر رابطه‌ای

۳۰

بخش هفتم: عملیات در پایگاه داده رابطه‌ای

از لحاظ تئوریک می‌توان عملیات ذخیره‌سازی را هم با عملگرهای جبر رابطه‌ای انجام داد.

عمل	عملگر
درج	$\cup$
حذف	$-$
به‌هنگام‌سازی	$\begin{matrix} \text{اول} \\ - \\ \text{بعد} \end{matrix} \cup$





مقایسه دو رابطه

□ دو رابطه R_1 و R_2 مقایسه‌شدنی (قابل قیاس) هستند، هر گاه نوع-سازگار باشند ($H_{R_2} = H_{R_1}$)

□ در مقایسه رابطه R_1 با R_2 ، بدنه R_1 با بدنه R_2 مقایسه می‌شود از نظر هم مجموعه‌گی، زیرمجموعه‌گی و زیرمجموعه‌گی

$$\Pi_{\langle \text{STID} \rangle}(\text{STT}) * \Pi_{\langle \text{STID} \rangle}(\text{SCR})$$

$$* \in \{ \subset, \supset, \subseteq, \supseteq, =, \neq \}$$

□ پاسخ عمل مقایسه: یا T یا F. به طور مثال در رابطه فوق:

▪ اگر $\supset$ باشد، پاسخ T است اگر حداقل یک دانشجو باشد که درسی انتخاب نکرده باشد.

▪ اگر $\subset$ باشد، پاسخ T است اگر حداقل در یک عمل ذخیره‌سازی در این DB قاعده جامعیت C2 رعایت

نشده باشد (حذف از دانشجو و یا درج در انتخاب درس).



- جبر رابطه‌ای **زبانی** است از نظر رابطه‌ای **کامل** (Relational Completeness) یعنی هر رابطه معتبر متصور از مجموعه رابطه‌های ممکن را می‌توان به کمک یک عبارت جبر رابطه‌ای بیان کرد.
- جبر رابطه‌ای ضابطه تشخیص کامل بودن زبان‌های رابطه‌ای است.
- اگر هر رابطه‌ای را که با جبر رابطه‌ای می‌توان نشان داد، با زبانی مدعی کامل بودن رابطه‌ای بتوان نشان داد، آن زبان از نظر رابطه‌ای **کامل** است.

□ کاربردهای جبر رابطه‌ای:

□ عملیات بازیابی

□ عملیات ذخیره‌سازی

□ تعریف انواع رابطه‌های مشتق (رابطه مجازی، لحظه‌ای و ...) مثال: تعریف دید (View) در SQL





□ برای نوشتن یک پرسش (Query)، اصولاً به ترتیب زیر باید مشخص کنیم که:

۱- از چه رابطه‌هایی استفاده کنیم.

۲- از چه عملگرهایی استفاده کنیم (حتی‌الامکان با کمترین تعداد عملگر)

۳- چه ترتیبی از عملگرها را استفاده کنیم.

□ مثال‌هایی از کاربرد جبر رابطه‌ای را در عملیات در RDB (در یادداشتهای تکمیلی سری II) (صفحه A-1

و A-2) مطالعه نمایید.



☐ **حساب رابطه‌ای** شاخه‌ای است از منطق ریاضی، منطق مسندات.

☐ حساب رابطه‌ای و جبر رابطه‌ای معادلند. یعنی هر رابطه‌ای را که بتوان با یک عبارت جبر رابطه‌ای نوشت، می‌توان با عبارتی از حساب رابطه‌ای هم نوشت و برعکس.

☐ حساب رابطه‌ای حالت **توصیفی** دارد ولی جبر رابطه‌ای حالت **دستوری** دارد.

↓
Prospective

دستورات عملیاتی به سیستم می‌دهیم.

↓
Descriptive

به کمک عبارات منطقی، شرایط ناظر
به رابطه را برای سیستم توصیف می‌کنیم.

☐ حساب رابطه‌ای هم ضابطه تشخیص زبان‌های رابطه‌ای کامل است.



متغیر تاپلی (Tuple Variable) یا متغیر طیفی (Range Variable):

متغیری است که مقادیر آن تاپل‌های یک رابطه است (هر لحظه یک تاپل).

RANGVAR SX RANGES OVER S;

RANGVAR PX RANGES OVER P;

RANGVAR SPX RANGES OVER SP;

RANGVAR C2X RANGES OVER (S WHERE CITY='C2');



طیف مقادیرش تاپل‌هایی از S است که شرط را داشته باشند.



سورها (Quantifiers)

 سور وجودی $\text{EXISTS } X (F)$: حداقل یک مقدار برای متغیر X وجود دارد به نحوی که به ازای آن، فرمول F به درست ارزیابی شود.

 سور همگانی (عمومی) $\text{FOR ALL } X (F)$: به ازای تمام مقادیر متغیر X ، فرمول F به درست ارزیابی می‌شود.

با فرض اینکه X از مجموعه اعداد صحیح مثبت مقدار می‌گیرد.



$\text{EXISTS } X (X < 10)$ حاصل ارزیابی: TRUE

$\text{FOR ALL } X (X < 10)$ حاصل ارزیابی: FALSE



□ **یادآوری:** بین این دو سور روابط زیر وجود دارد.

$$\text{FOR ALL } X (F) = \text{NOT EXISTS } X (\text{NOT } F)$$

$$\text{EXISTS } X (F) = \text{NOT } (\text{FORALL } X (\text{NOT } F))$$

$$\text{FORALL } X (F) \Rightarrow \text{EXISTS } X (F)$$

$$\text{NOT EXISTS } X (F) \Rightarrow \text{NOT FORALL } X (F)$$

□ بر اساس روابط فوق می‌توان روابط پیچیده دیگری را نیز استنباط کرد مانند روابط هم ارزی زیر:

$$\text{FORALL } X (F \text{ AND } G) = \text{NOT EXISTS } X (\text{NOT}(F) \text{ OR } \text{NOT}(G))$$

$$\text{FORALL } X (F \text{ OR } G) = \text{NOT EXISTS } X (\text{NOT}(F) \text{ AND } \text{NOT}(G))$$

$$\text{EXISTS } X (F \text{ OR } G) = \text{NOT FORALL } X (\text{NOT}(F) \text{ AND } \text{NOT}(G))$$

$$\text{EXISTS } X (F \text{ AND } G) = \text{NOT FORALL } X (\text{NOT}(F) \text{ OR } \text{NOT}(G))$$



یک فرمول خوش ساخت (WFF) به صورت زیر تعریف می‌شود:



□ اگر R یک رابطه و T یک تاپل یا متغیر تاپلی تعریف شده روی R باشد، آنگاه $R(T)$ یک فرمول اتمی است.

$[R(T)]$ یعنی، T یک عنصر (تاپلی) از R است.

□ اگر T_i یک متغیر تاپلی روی رابطه R و A یک صفت از R باشد و T_j یک متغیر تاپلی بر روی S و B یک

صفت از S باشد، آنگاه $T_i.A \text{ theta } T_j.B$ یک فرمول اتمی است (theta یکی از عملگرهای متعارف مقایسه‌ای است).

□ $T_i.A \text{ theta } C_2$ و $T_j.B \text{ theta } C_1$ و $C_1 \text{ theta } C_2$ نیز که در آن C_i یک مقدار ثابت است، فرمول اتمی هستند.

□ اگر F_1 و F_2 فرمول باشند، آنگاه $(F_1 \text{ AND } F_2)$ ، $(F_1 \text{ OR } F_2)$ ، $\text{NOT}(F_1)$ نیز فرمول هستند.

□ اگر F یک فرمول و T یک متغیر تاپلی باشد، آنگاه $\text{EXISTS } T(F)$ و $\text{FORALL } T(F)$ نیز فرمول هستند.



اگر X یک متغیر تاپلی روی رابطه $R(A_1, A_2, \dots, A_n)$ باشد در اینصورت شکل کلی **عبارت حساب رابطه‌ای** بدین صورت است:

(target-items) [**WHERE** F]

که در آن target-items فهرستی از صفات متغیر تاپلی X به صورت $X.A_1, X.A_2, \dots, X.A_n$ و F یک فرمول خوش ساخت است.



- ST.STID شماره تمام دانشجویان در رابطه STT
- ST.STID **WHERE** ST.STDEID='D11' شماره دانشجویان گروه آموزشی D11
- (ST.STID, ST.STL) **WHERE EXISTS** STCO (ST.STID=STCO.STID **AND** STCO.COID='COM11')

شماره دانشجویی و مقطع تحصیلی آنهایی که درس COM11 را انتخاب کرده‌اند.



حساب رابطه‌ای – عبارت حساب رابطه‌ای (ادامه)

بخش هفتم: عملیات در پایگاه داده رابطه‌ای

۴۰



☐ SX.S# شماره همه تهیه کنندگان

☐ SX.SNAME **WHERE** SX.CITY='C2' **AND** SX.STATUS> 15

نام تهیه کنندگان شهرستان C2 که وضعیت آنها بزرگتر از 15 باشد.

☐ مثال‌های بیشتر در کتاب‌های مرجع و یادداشتهای تکمیلی سری II.



پرسش و پاسخ ...

amini@sharif.edu

به نام آنکه جان را فکرت آموخت



بخش هشتم: طراحی پایگاه داده رابطه‌ای

مرتضی امینی

نیمسال دوم ۹۶-۹۷

(محتویات اسلایدها برگرفته از یادداشت‌های کلاسی استاد محمدتقی روحانی رانکوهی است.)



در طراحی پایگاه داده‌های رابطه‌ای باید موارد زیر را مشخص نمود:

- ☐ مجموعه‌ای از رابطه‌ها
- ☐ کلید(های) کاندید هر رابطه
- ☐ کلید اصلی هر رابطه
- ☐ کلیدهای خارجی هر رابطه (در صورت وجود)
- ☐ محدودیت‌های جامعیتی ناظر بر هر رابطه

روشهای طراحی RDB: ☐

طراحی با روش بالا به پایین (Top-Down) }
طراحی با روش سنتز [انرمال ترسازی رابطه‌ها]



روش طراحی بالا به پایین

ابتدا مدلسازی داده‌ها را (با روش [E]ER یا UML) انجام می‌دهیم و سپس مدلسازی را به مجموعه‌ای از رابطه‌ها تبدیل می‌کنیم.

روش طراحی سنتز رابطه‌ای (نرمال ترسازی)

ابتدا مجموعه صفات خرد جهان واقع را مشخص می‌کنیم. سپس با تحلیل قواعد و محدودیت‌های ناظر به صفات و تشخیص وابستگی‌های بین آنها، صفات را متناسباً با هم سنتز می‌کنیم (نوعی گروه‌بندی) تا به مجموعه‌ای از رابطه‌های نرمال دست یابیم.

در عمل روش ترکیبی استفاده می‌شود، یعنی ابتدا روش بالا به پایین، سپس نرمال ترسازی.



□ نمایش صحیح و واضح از خردجهان واقع باشد.

□ تمام داده‌های کاربران قابل نمایش باشد و همه محدودیت‌های (قواعد) جامعیتی منظور شده باشد.

□ کمترین افزونگی

□ کمترین هیچمقدار

□ کمترین مشکل در عملیات ذخیره‌سازی

□ بیشترین کارایی در بازیابی

تأمین چهار ویژگی آخر به صورت همزمان، در عمل ناممکن است!



□ تبدیل نمودار [E]ER به مجموعه‌ای از رابطه‌های نرمال (و نه لزوماً در نرمال‌ترین صورت) در طراحی RDB.

نهایتاً طراح تصمیم می‌گیرد چند رابطه داشته باشد و عنوان (Heading) هر رابطه چه باشد.

□ در نمودار مدلسازی معنایی داده‌ها، حالات متعدد داریم، که در ادامه به آنها می‌پردازیم.

□ **فرض:** تا اطلاع ثانوی، همه صفات ساده‌اند و موجودیت‌ها ضعیف نیستند.



حالت ۱: طراحی ارتباط چند به چند

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۶

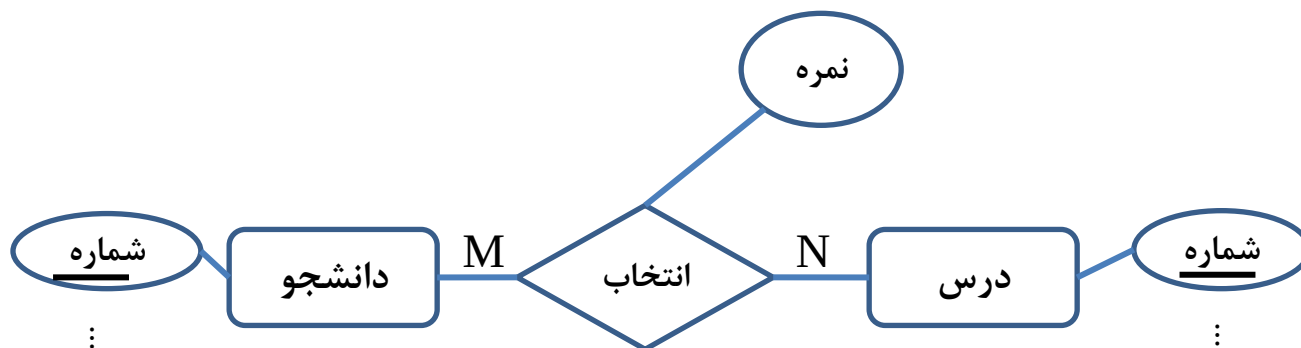
حالت ۱

درجه ارتباط: $n=2$

چندی ارتباط: $M:N$

سه رابطه لازم است.

طراحی در این حالت با کمتر از سه رابطه، افزونگی و هیچ‌مقداری زیادی پدید می‌آورد.



STUD (STID,)

COR (COID,)

SCR (STID, COID, GR)



حالت ۱: طراحی ارتباط چند به چند (ادامه)

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۷

تعمیم حالت ۱

□ درجه: $n > 2$

□ ابتدا فرض می‌کنیم چندی رابطه $M:N:P:\dots$ است.

□ $n+1$ رابطه طراحی می‌کنیم.

□ سپس بررسی می‌کنیم که آیا محدودیت خاصی روی چندی ارتباط بین بعض موجودیت‌ها وجود دارد.

□ اگر بله، این محدودیت را در مرحله نرمالترسازی دخالت می‌دهیم. ← تعداد رابطه‌ها ممکن است

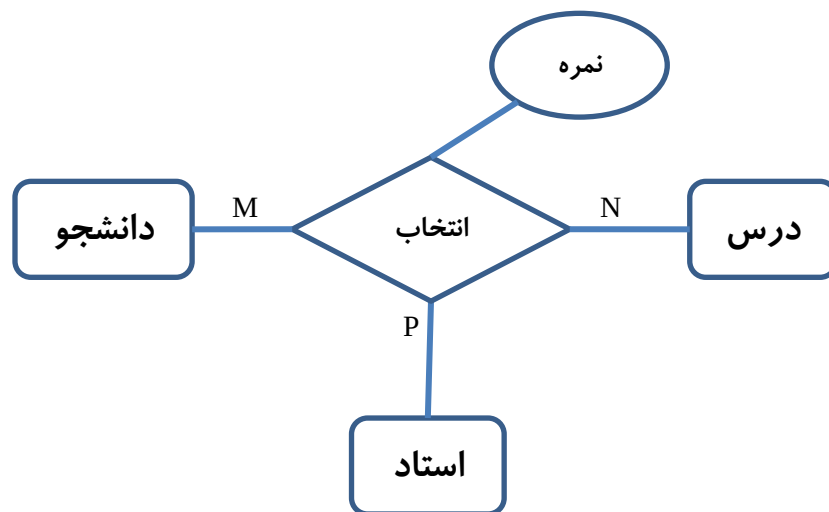
بیش از $n+1$ شود.



حالت ۱: طراحی ارتباط چند به چند (ادامه)

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۸



STUD (STID,)

COR (COID,)

PROF (PRID,)

SCP (STID, COID, PRID, GR)

فرض برای محدودیت: یک استاد فقط یک درس را تدریس می‌کند (البته در این مورد، چندی رابطه دقیق مدل نشده که این محدودیت لحاظ نشده است).

در این صورت باید رابطه SCP را به دو رابطه (یا بیشتر) تجزیه عمودی کنیم.

این محدودیت را در مرحله دوم طراحی (در مباحث آتی) دخالت می‌دهیم.



حالت ۲: طراحی ارتباط یک به چند

بخش هشتم: طراحی پایگاه داده رابطه‌ای

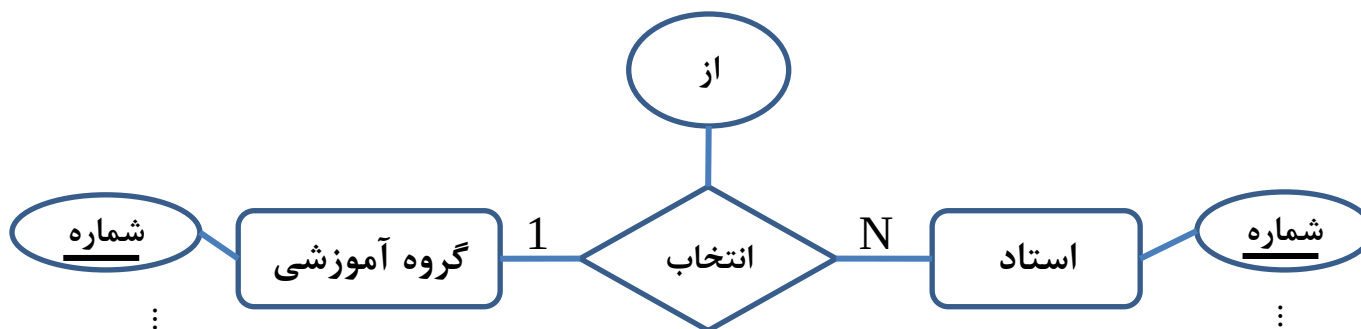
۹

حالت ۲

درجه ارتباط: $n=2$ ☐

چندی ارتباط: $1:N$ ☐

دو رابطه لازم است. رابطه سمت 1 به رابطه سمت N، FK می‌دهد (بیرون از کلید اصلی).



DEPT (DEID, DTID,, DPHONE)

PROF (PRID, PRNAME,, PRANK, DEID, FROM)



حالت ۲: طراحی ارتباط یک به چند (ادامه)

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۱۰

□ در چه وضعی طراحی این حالت با سه رابطه قابل توجیه است؟

۱- وقتی که مشارکت سمت N در ارتباط غیرالزامی باشد (درصد مشارکت کمتر از ۳۰ درصد) و تعداد

استاد زیاد باشد، برای کاهش مقدار Null، رابطه نمایشگر ارتباط را جدا می‌کنیم.

۲- فرکانس ارجاع به خود ارتباط بالا باشد و به صفات دیگر با فرکانس پایین‌تری احتیاج باشد.

۳- تعداد صفات خود ارتباط زیاد باشد و باعث زیاد شدن درجه ارتباط PROF شود.

□ اگر مشارکت سمت N الزامی باشد، باید این محدودیت معنایی را از طریق هیچمقدارناپذیر بودن صفت کلید

خارجی (با استفاده از NOT NULL) در رابطه نمایانگر نوع موجودیت سمت N ، اعلام کرد.



حالت ۳: طراحی ارتباط یک به یک

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۱۱

حالت ۳

درجه ارتباط: $n=2$

چندی ارتباط: 1:1

با دو / یا سه / یا یک رابطه طراحی می‌کنیم.



در صورت طراحی با **دو** رابطه، رابطه مربوط به نوع موجودیت با مشارکت الزامی، FK می‌گیرد.

COUR (COID,, BKID)

BOOK (BKID,, BKPRICE)



حالت ۳: طراحی ارتباط یک به یک (ادامه)

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۱۲

□ وقتی با **سه** رابطه توجیه دارد که مشارکت طرفین غیرالزامی باشد، تعداد شرکت کنندگان (نمونه‌ها) در ارتباط زیاد باشد، درصد مشارکت در رابطه ضعیف (کمتر از ۳۰٪) باشد و نیز ملاحظات در مورد فرکانس ارجاع.

□ وقتی با **یک** رابطه توجیه دارد که تعداد صفات موجودیت‌ها کم باشد، مشارکت طرفین الزامی باشد و فرکانس ارجاع به ارتباط کم باشد.

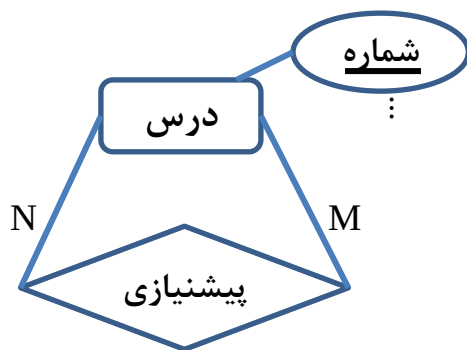


حالت ۴: طراحی ارتباط خود ارجاع چند به چند

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۱۳

حالت ۴



حالت خاص حالت اول

درجه ارتباط: $n=1$

چندی ارتباط: $M:N$

دو رابطه لازم است.

COUR (COID, ...)

COPRECO (COID, PRECOID) $\longrightarrow$ بیش از یک صفت از رابطه، از یک دامنه هستند.

COUR $\longleftarrow$ **COPRECO** **گراف ارجاع:**

نتیجه: صرف وجود ارتباط با خود، چرخه ارجاع ایجاد نمی‌شود. باید به چندی ارتباط توجه کنیم.



حالت ۵: طراحی ارتباط خود ارجاع یک به چند

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۱۴

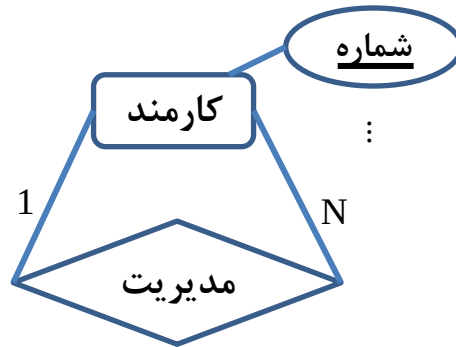
حالت ۵

حالت خاص حالت دوم ☐

درجه ارتباط: $n=1$ ☐

چندی ارتباط: $1:N$ ☐

یک رابطه لازم است.



در این رابطه چه نکاتی وجود دارد؟ ☐

EMPL (EMID, ENAME,, EPHONE, EMGRID)

 **گراف ارجاع:** **EMPL** ☐

برنامه‌ای در SQL بدهید که سطح (مدیریتی) تمام مدیران در سلسله مدیریت را بدهد (با استفاده از تکنیک ☐

(Recursion)

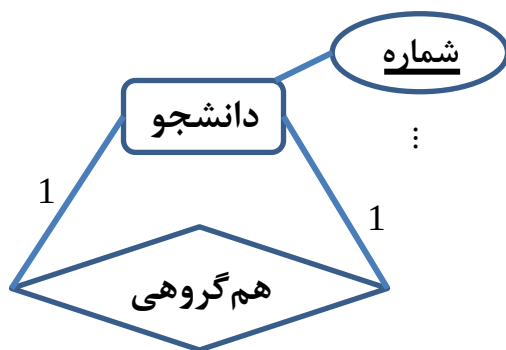


حالت ۶: طراحی ارتباط خود ارجاع یک به یک

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۱۵

حالت ۶



□ حالت خاص حالت سوم

□ درجه ارتباط: $n=1$

□ چندی ارتباط: 1:1

با یک یا دو رابطه طراحی می‌کنیم.

□ اگر مشارکت در هم‌پروژگی زیاد نباشد، از مدل II استفاده می‌کنیم.

(I) **STPROJST** (STID, STNAME, ..., JSTID)
P.K. C.K.

(II) **STUD** (STID, STNAME, ...,)

STJST (STID, JSTID)
C.K. C.K.

□ در STJST هر یک از صفات می‌توانند کلید اصلی باشند.

□ آیا طرز دیگری هم برای طراحی وجود دارد؟



حالت ۷: طراحی موجودیت ضعیف

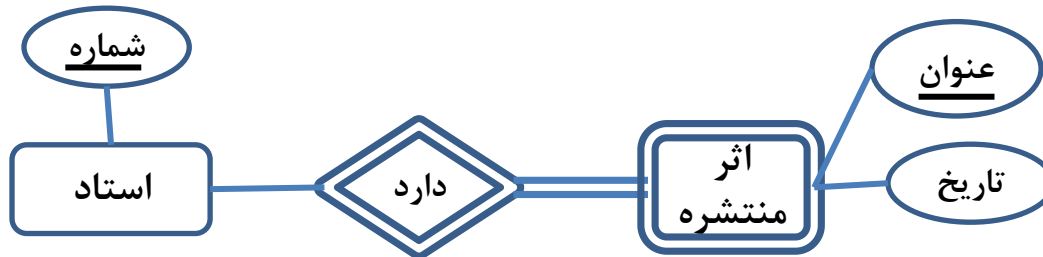
۱۶

بخش هشتم: طراحی پایگاه داده رابطه‌ای

حالت ۷

موجودیت ضعیف داریم. □

دو رابطه لازم است؛ یکی برای نوع موجودیت قوی، یکی برای نوع موجودیت ضعیف و ارتباط شناسا. رابطه نمایشگر موجودیت ضعیف از موجودیت قوی FK می‌گیرد که در ترکیب با صفت ممیزه می‌شود PK.



PROF (PRID, PRNAME,)

PRPUB (PRID, PTITLE, PTYPE,)

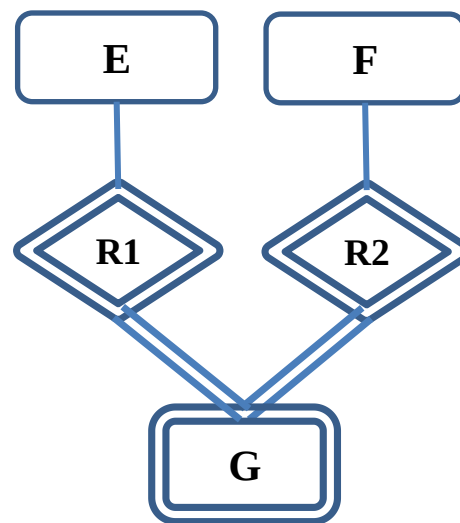
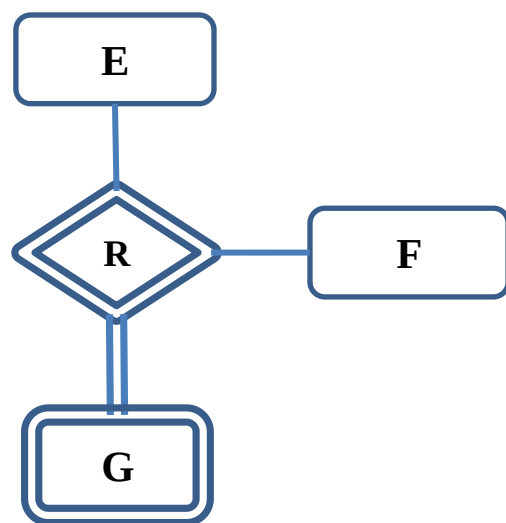


حالت ۷: طراحی موجودیت ضعیف (ادامه)

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۱۷

تمرین: رابطه‌های لازم برای مدل‌های داده‌ای زیر طراحی شود. □



در این حالات، کلید رابطه G از ترکیب کلید رابطه‌های E و F (و در صورت وجود صفت ممیزه G) حاصل می‌گردد.



حالت ۸: طراحی صفت چندمقداری

۱۸

بخش هشتم: طراحی پایگاه داده رابطه‌ای

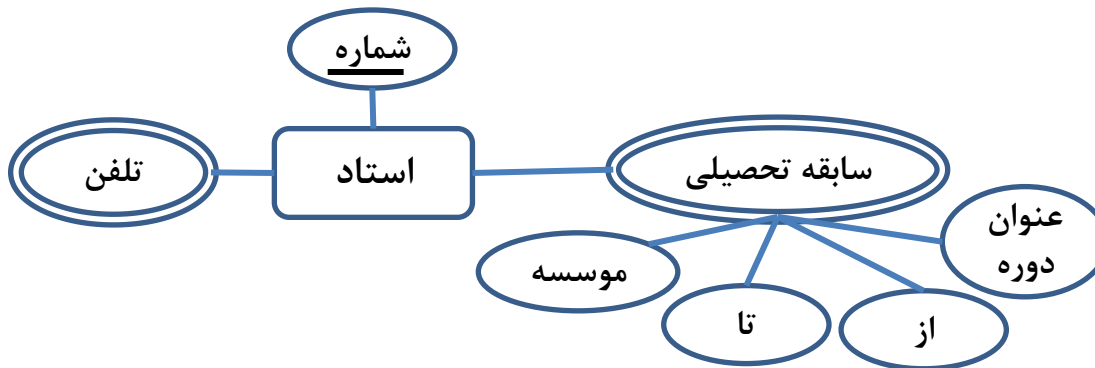
حالت ۸

□ وجود یک صفت چندمقداری برای یک نوع موجودیت.

□ دو تکنیک دارد:

[تکنیک عمومی] یک رابطه برای خود نوع موجودیت و یک رابطه برای هر صفت چندمقداری.

(بنابراین اگر نوع موجودیت E ، صفت چندمقداری داشته باشد، $m+1$ رابطه داریم.



PROF (PRID, PRNAME,)

PRTEL (PRID, PHONE)

✓ رابطه نمایشگر صفت چندمقداری از نوع

موجودیت اصلی FK می‌گیرد داخل کلید.



حالت ۸: طراحی صفت چندمقداری (ادامه)

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۱۹

□ در مدلسازی، موجودیت ضعیف به صفت چندمقداری ارجحیت دارد ولی تکنیک عمومی طراحی آنها مثل هم است.

PRHIS (PRID, TTL, FROM, TO, INSTNAME,)

□ اشکال تکنیک عمومی: اگر برای نوع موجودیت اصلی اطلاعات کامل بخواهیم، باید عمل JOIN انجام دهیم که می‌تواند زمانگیر باشد.



حالت ۸: طراحی صفت چندمقداری (ادامه)

۲۰

بخش هشتم: طراحی پایگاه داده رابطه‌ای

[در شرایط خاص] طراحی با یک رابطه (فرض: یک صفت چندمقداری): یک رابطه برای خود نوع موجودیت و صفت چندمقداری.

□ با فرض مشخص بودن حداکثر تعداد مقداری که صفت چندمقداری می‌گیرد، به همان تعداد صفت در رابطه در نظر می‌گیریم.

فرض: هر استاد حداکثر سه شماره تلفن دارد.



PRTELTEL (PRID, PRNAME, PRRANK, PHONE1, PHONE2, PHONE3)

□ مزیت این تکنیک: JOIN لازم ندارد.

□ عیب این تکنیک: هیچمقدار (Null) در آن زیاد است، اگر تعداد کمی از استادان، سه شماره تلفن داشته باشند.



حالت ۹

وجود ارتباط IS-A بین دو نوع موجودیت. ☐

چهار تکنیک دارد: ☐

۱- فرض: نوع موجودیت E، n زیرنوع دارد.

n+1 رابطه طراحی می‌کنیم. یک رابطه برای زیرنوع و یک رابطه برای هر یک از زیرنوع‌ها.

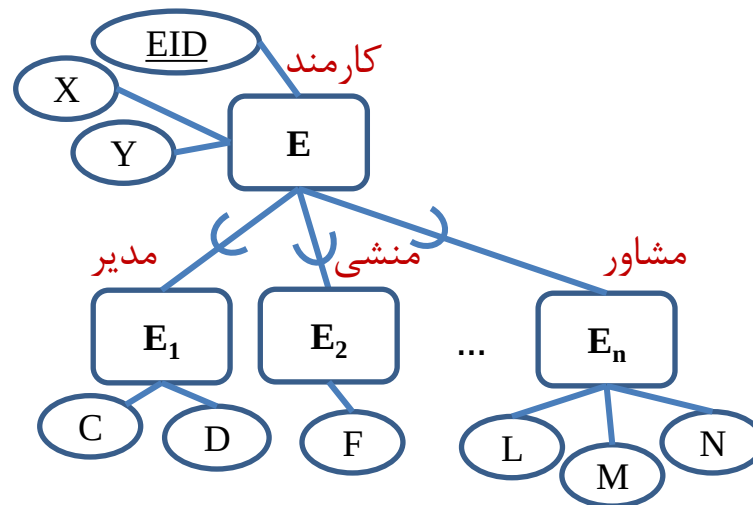
E (EID, X, Y)

E1 (EID, C, D)

E2 (EID, F)

...

En (EID, L, M, N)





حالت ۹: طراحی ارتباط IS-A (ادامه)

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۲۲

□ مزیت این تکنیک: شرط خاصی از نظر نوع تخصیص ندارد (تکنیک‌های دیگری که مطرح می‌شود، همگی

برای شرایط خاص هستند).

□ عیب این تکنیک: اگر بخواهیم در مورد یک زیرنوع، اطلاعات کامل به دست آوریم، باید JOIN کنیم.



۲- طراحی با n رابطه: برای زیرنوع، رابطه‌ای طراحی نمی‌کنیم. بنابراین صفات مشترک باید در رابطه نمایشگر هر زیرنوع وجود داشته باشد.

□ شرط لازم: باید تخصیص کامل باشد. اگر نباشد، بخشی از داده‌های محیط قابل نمایش نیستند.

E1 (EID, X, Y, A, B)

E2 (EID, X, Y, F)

...

En (EID, X, Y, L, M, N)

□ مزیت نسبت به تکنیک اول: برای به دست آوردن اطلاعات کامل زیرنوع‌ها نیازی به JOIN نیست.

□ نکته: در این تکنیک، لزوماً افزونگی پیش نمی‌آید. اگر تخصیص هم‌پوشا باشد میزانی افزونگی پیش می‌آید.



۳- طراحی فقط با یک رابطه، با استفاده از صفت نمایشگر نوع زیرنوع‌ها

□ شرط استفاده از این تکنیک: تخصیص مجزا باشد؛ یعنی یک نمونه کارمند، جزء نمونه‌های حداکثر یک زیرنوع باشد.

E (EID, X, Y, A, B, F, L, M, N, TYPE)

100 x1 y1 a1 b1 ? ? ? ? مدیر

200 x2 y2 ? ? ? l2 m2 n2 مشاور

□ مزیت این تکنیک: برای به دست آوردن اطلاعات کامل زیرنوع‌ها نیازی به JOIN نیست.

□ عیب این تکنیک: هیچ مقدار (Null) زیاد دارد و درجه رابطه زیاد است.



۴- طراحی فقط با یک رابطه، با استفاده از آرایه بیتی؛ هر بیت نمایشگر نوع یک زیرنوع. در واقع برای

نمایش هر نمونه موجودیت، بسته به اینکه در مجموعه نمونه‌های کدام زیرنوع باشد، بیت مربوطه‌اش را ۱ می‌کنیم.

□ شرط استفاده از این تکنیک: وقتی تخصیص هم‌پوشا باشد (سایر شرایط همانها که در تکنیک ۳ گفته

شد).

E (EID, X, Y, A, B, F, L, M, N, $\overbrace{\text{TB1, TB2, ..., TBn}}^{\text{آرایه بیتی}}$)

↑ ↑ ↑
مدیر منشی مشاور

100	x1	y1	1	0	0
-----	----	----	---	---	---

200	x2	y2	0	1	0
-----	----	----	---	---	---



حالت ۱۰: طراحی ارث‌بری چندگانه

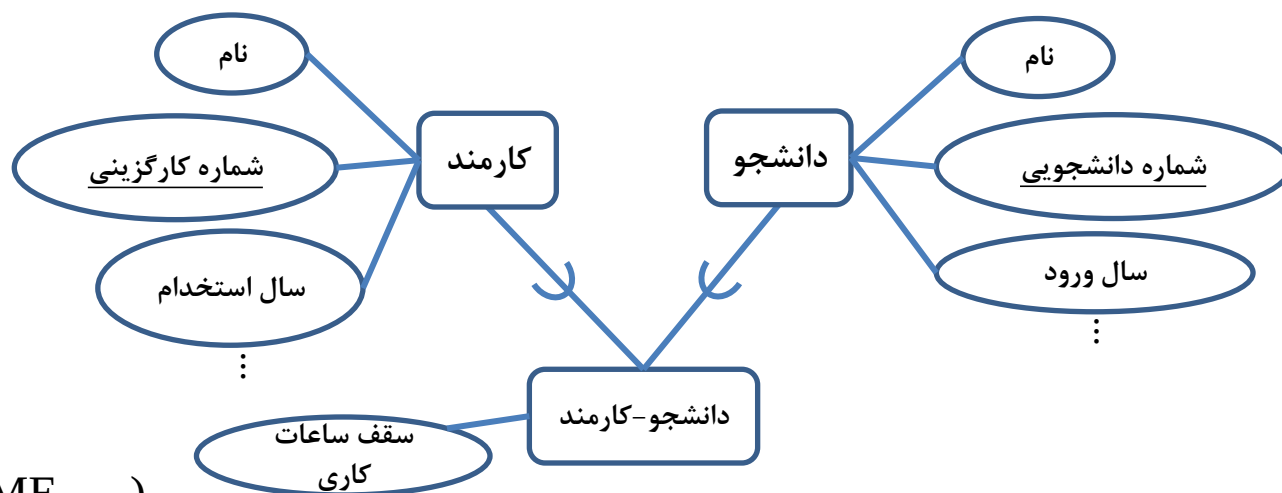
۲۶

بخش هشتم: طراحی پایگاه داده رابطه‌ای

حالت ۱۰

وجود ارث‌بری چندگانه بین یک زیرنوع با چندزیرنوع

اگر زیرنوع، n زیرنوع داشته باشد، رابطه نمایشگر زیرنوع حداقل n کلید کاندید دارد. کلید کاندید با ارجاع بیشتر کلید اصلی انتخاب می‌شود.



STUD (STID, STNAME, ...)

EMPL (EID, ENAME, ...)

STEM (STID, EID, MAXW)

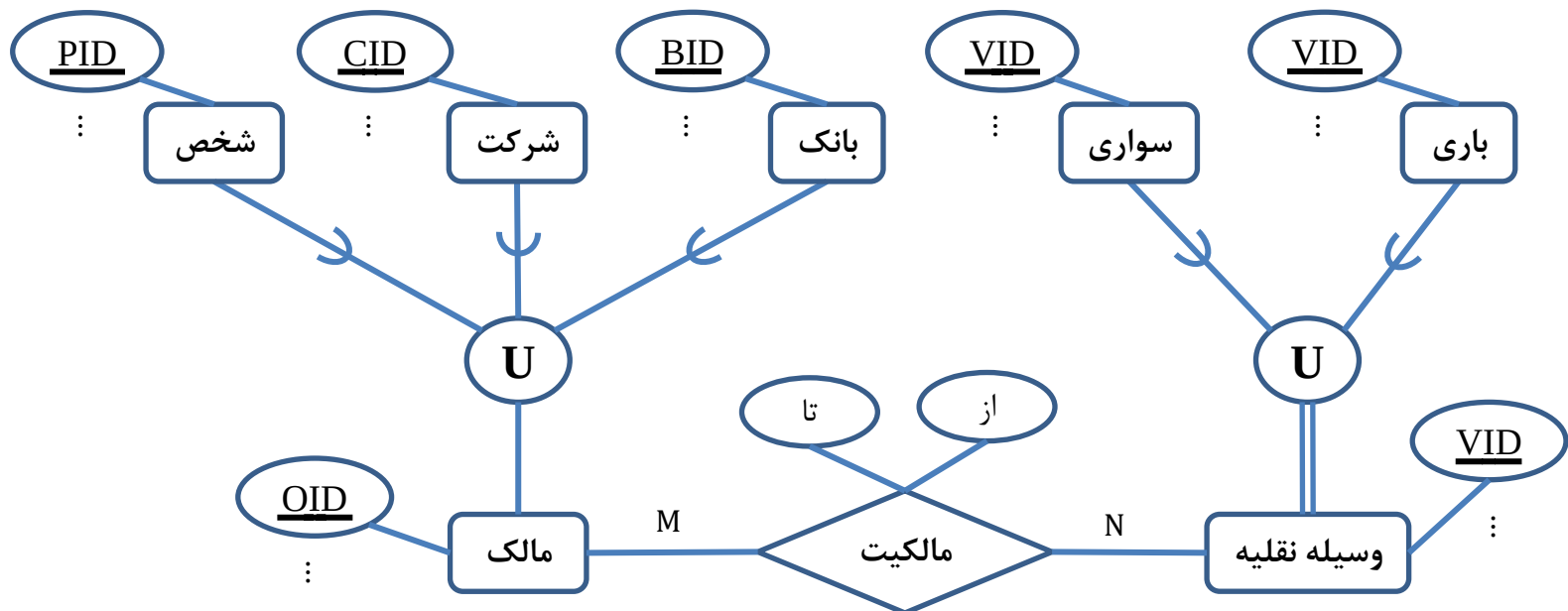
آیا ممکن است برای زیرنوع اصلاً رابطه طراحی نکنیم؟



حالت ۱۱

□ نوع موجودیت E، زیرنوع U-Type (دسته یا Category) n زیرنوع است.

n+1 رابطه طراحی می‌کنیم.





حالت ۱۱: طراحی زیرنوع اجتماع (ادامه)

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۲۸

□ $n+1$ رابطه

□ اگر شناسه زبرنوع‌ها از دامنه‌های متفاوت باشد، رابطه نمایشگر زیرنوع، FK می‌دهد به رابطه‌های نمایشگر زبرنوع‌ها، خارج از کلید.

□ اگر شناسه زبرنوع‌ها از یک دامنه باشد (و مقادیر شناسه در همه نمونه‌های زبرنوع‌ها یکتا باشد)، کلید

رابطه نمایشگر زیرنوع، همان کلید رابطه‌های نمایشگر زبرنوع‌ها است. **PERS** (PID, ..., OID)

COMP (CID, ..., OID)

BANK (BID, ..., OID)

چون دامنه کلیدهای زبرنوع‌ها یکسان نیست، خودمان کلید ساختگی می‌گذاریم. ← **OWNER** (OID, ...)

VEHIC (VID, ...)

OWNS (OID, VID, F, T, ...)

SAVARY (VID, N, ...)

BARY (VID, T, ...)

آیا طرز طراحی دیگری وجود دارد.





حالت ۱۲: طراحی ارتباط IS-A-PART-OF

بخش هشتم: طراحی پایگاه داده رابطه‌ای

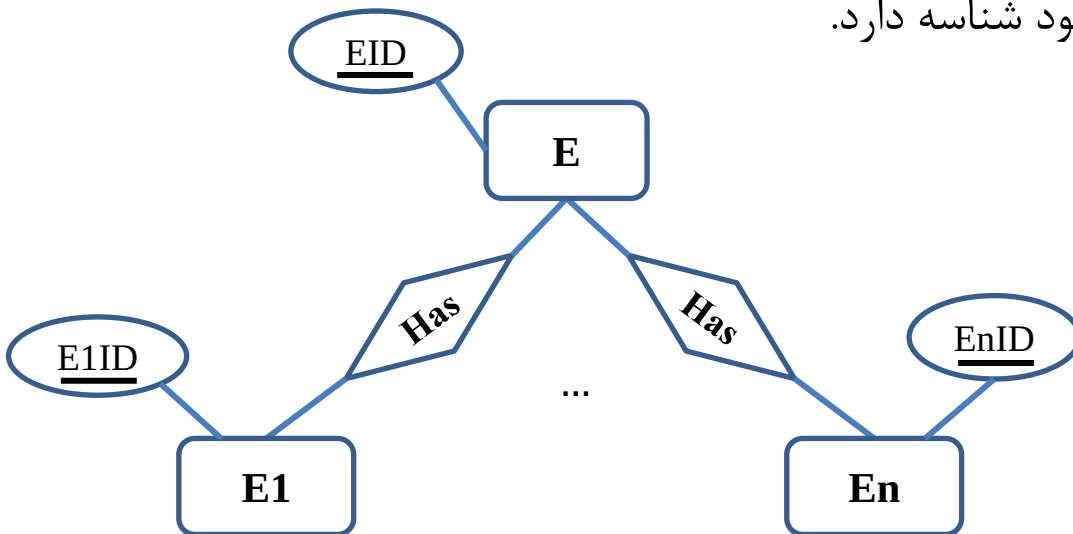
۲۹

حالت ۱۲

وجود ارتباط IS-A-PART-OF

اگر نوع موجودیت کل، n نوع موجودیت جزء داشته باشد، تعداد $n+1$ رابطه طراحی می‌کنیم.

توجه داریم که نوع موجودیت جزء از خود شناسه دارد.



$E(\underline{EID}, \dots)$

$E1(\underline{E1ID}, \underline{EID}, \dots)$

$\dots$

$En(\underline{EnID}, \underline{EID}, \dots)$

آیا طرز طراحی دیگری وجود دارد؟ در چه شرایطی؟





حالت ۱۳: طراحی تکنیک Aggregation

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۳۰

حالت ۱۳

استفاده از تکنیک Aggregation در مدلسازی

ابتدا نوع موجودیت انتزاعی (بخش درون مستطیل خط‌چین) را طراحی می‌کنیم (با توجه به درجه و چندی ارتباط). سپس بخش بیرون آن را (باز هم با توجه به چندی ارتباط و درجه آن).

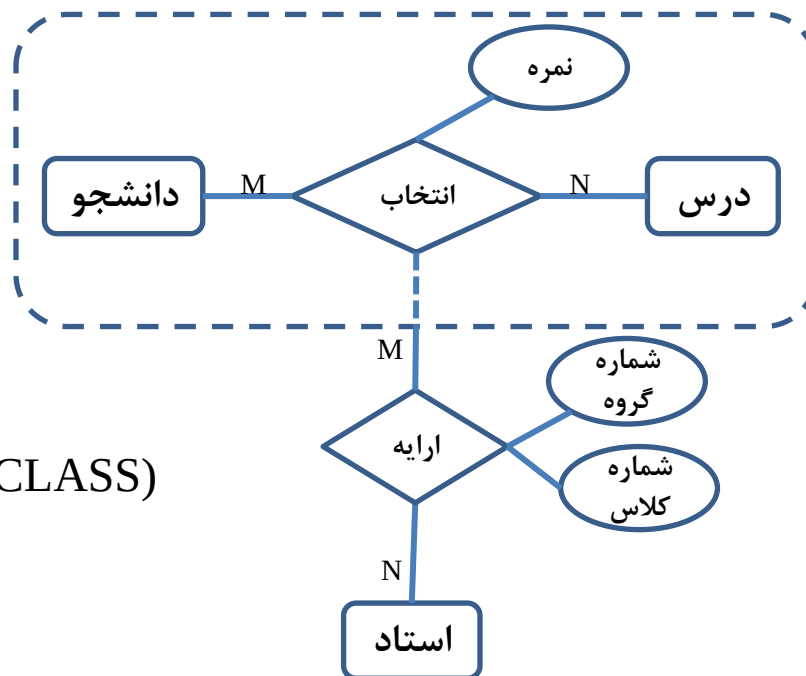
STUD (STID,)

COUR (COID,)

SCR (STID, COID, GR)

PROF (PRID,)

OFFERING (STID, COID, PROFID, GR#, CLASS)





حالت ۱۳: طراحی تکنیک Aggregation (ادامه)

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۳۱

این تکنیک چگونه کارایی سیستم را افزایش می‌دهد (نسبت به طراحی با یک ارتباط سه-تایی)؟ ☐

اگر مراجعه به ارتباط «انتخاب» بالا باشد و فرکانس ارجاع به ارتباط «ارائه» پایین باشد، سیستم با این ☐

طراحی کاراتر عمل می‌کند.



حالت ۱۴: طراحی با وجود چند ارتباط

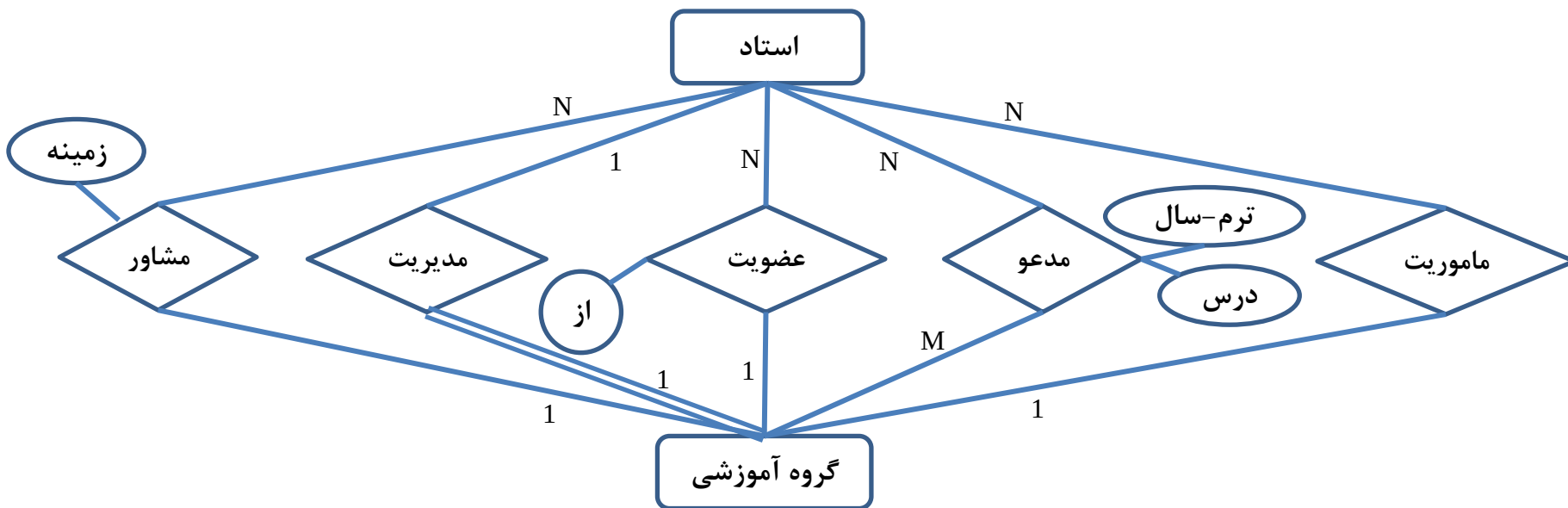
بخش هشتم: طراحی پایگاه داده رابطه‌ای

۳۲

حالت ۱۴

□ در صورتیکه چند ارتباط مثلاً بین دو نوع موجودیت برقرار باشد.

□ هر ارتباط را با توجه به وضع آن از نظر درجه و چندی ارتباط طراحی می‌کنیم. اما برای کاهش احتمال اشتباه در طراحی توصیه می‌شود اول ارتباطهای $M:N$ ، سپس $1:N$ و در آخر $1:1$ را طراحی نماییم.





حالت ۱۴: طراحی با وجود چند ارتباط (ادامه)

۳۳

بخش هشتم: طراحی پایگاه داده رابطه‌ای

DEPT (DEID,, DPHONE, PRID)

PROF (PRID,, PRRANK, MDEID, SUB, MEMDEID, FROM, CDEID, INT)

زمینه مشاور از عضویت موضوع ماموریت

سه کلید خارجی از یک دامنه

INVITED (DEID, PRID, YR, TR)

همین سیستم حداکثر با هفت رابطه نیز قابل طراحی است. □



طراحی RDB – روش سنتز یا نرمال‌تر سازی رابطه‌ها

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۳۴

□ **ایده اصلی:** یک رابطه، هر چند نرمال (با تعریفی که قبلاً دیدیم) ممکن است آنومالی (مشکل) داشته باشد

در عملیات ذخیره‌سازی (در درج، حذف یا بهنگام‌سازی).

□ **آنومالی در درج:** عدم امکان درج یک فقره اطلاع که منطقاً باید قابل درج باشد.

□ **آنومالی در حذف:** حذف یک اطلاع ناخواسته در پی حذف اطلاع خواسته.

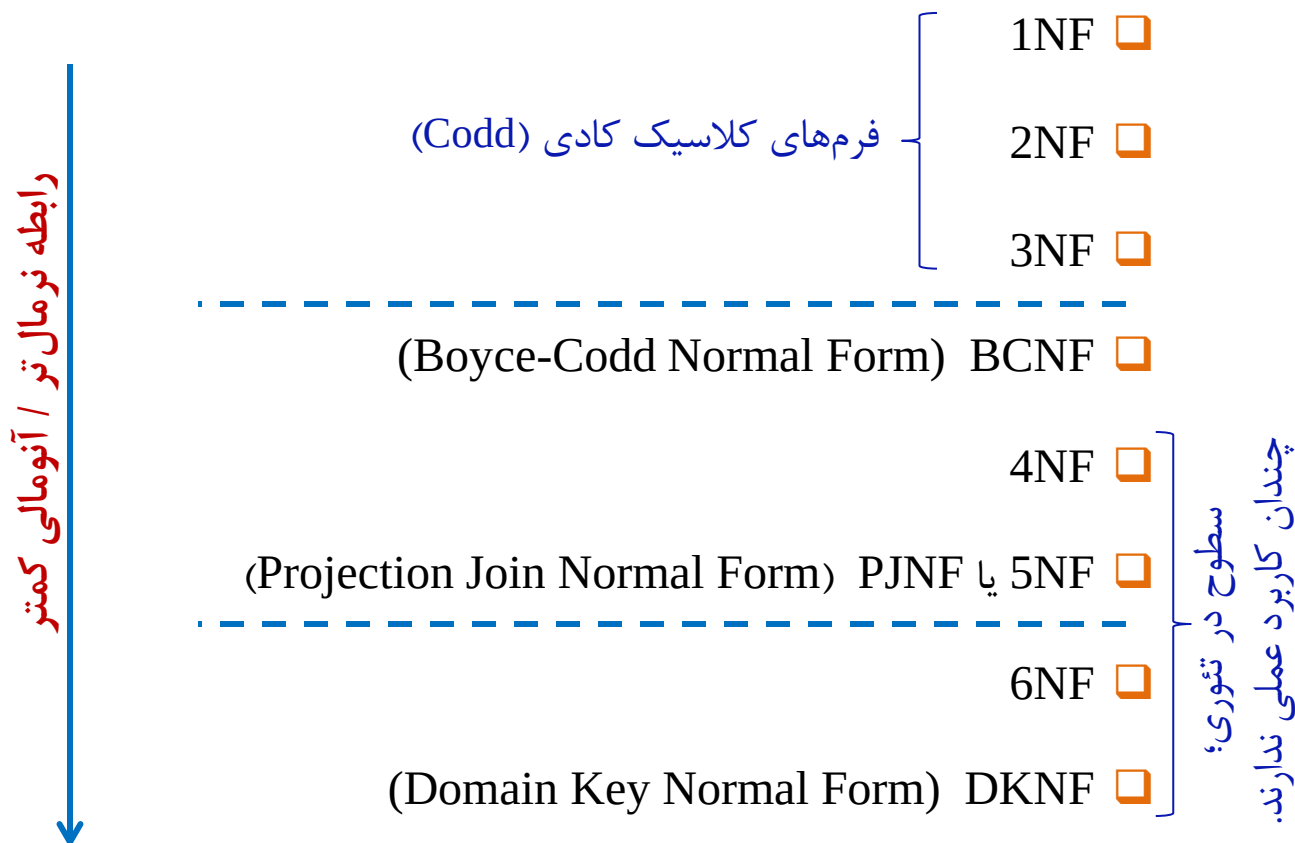
□ **آنومالی در بهنگام‌سازی:** بروز فزون کاری.

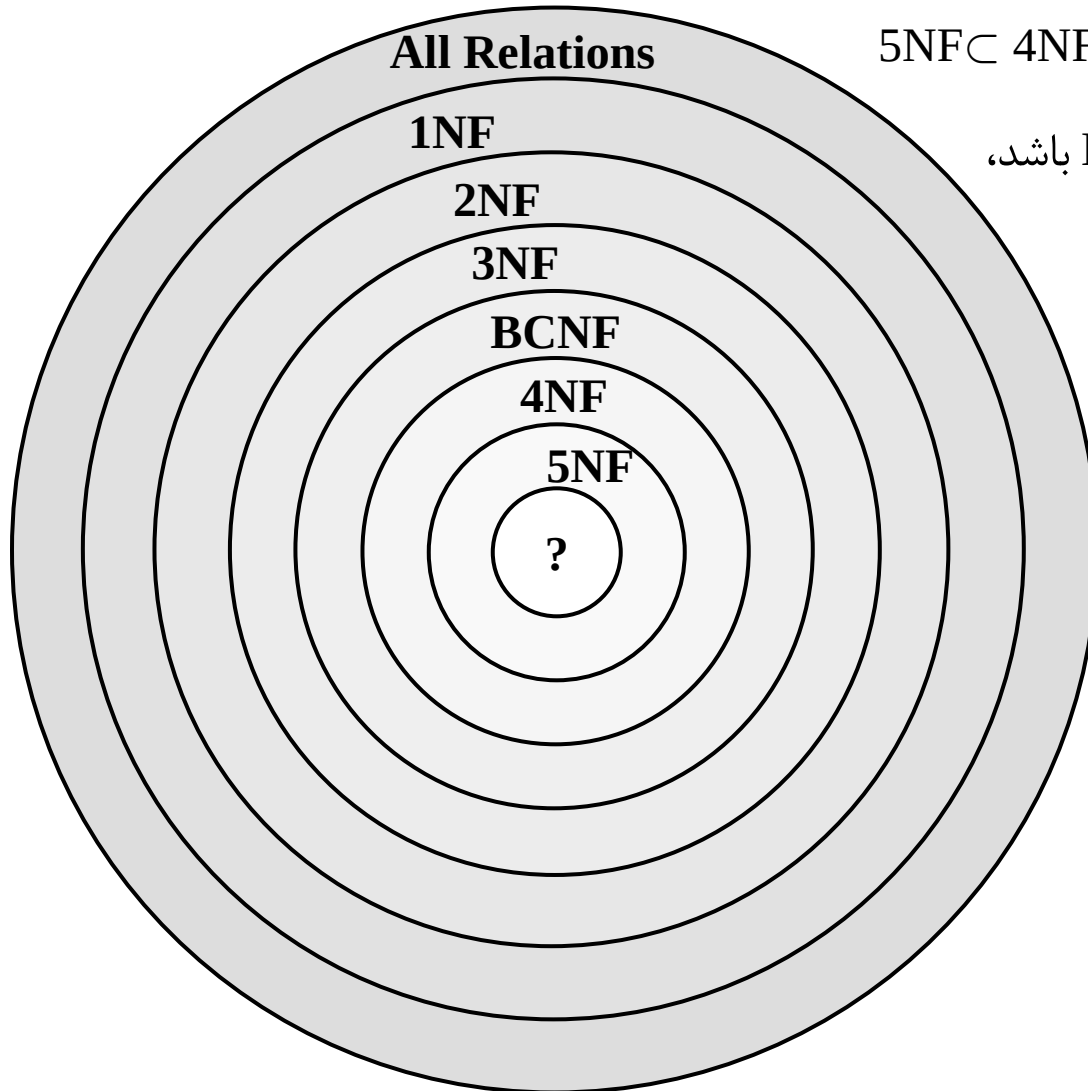
□ پس باید رابطه را نرمال‌تر کرد.



□ نرمال بودن رابطه (نرمالیتی)، فرم‌ها (صورت‌ها/ سطوح/ درجات) [NF: Normal Forms] مختلفی دارد.

□ فرم‌های نرمال:





$5NF \subset 4NF \subset BCNF \subset 3NF \subset 2NF \subset 1NF$ ☐

یعنی به طور مثال، رابطه‌ای که BCNF باشد، ☐

3NF هم هست.



□ برای بررسی فرم‌های نرمال، نیاز به مفاهیمی داریم از تئوری وابستگی (Dependency Theory).

□ مفاهیمی از تئوری وابستگی:

□ وابستگی تابعی (Functional Dependency)

□ وابستگی تابعی کامل [تام] (Fully Functional Dependency)

□ وابستگی تابعی با واسطه (Transitive Functional Dependency)



وابستگی تابعی (FD): صفت R.B به صفت R.A وابستگی تابعی دارد اگر و فقط اگر به ازای یک مقدار از A یک مقدار از B متناظر باشد. به عبارت دیگر اگر t_1 و t_2 دو تاپل از R باشند، در این صورت:

$$\text{IF } t_1.A = t_2.A \text{ THEN } t_1.B = t_2.B$$



با فرض اینکه کل تاپل‌های رابطه به صورت زیر باشد، آیا داریم:

R (A, B, C)

$a_1, b_1, c_1,$

a_1, b_1, c_2

a_2, b_2, c_2

a_3, b_3, c_3

a_4, b_2, c_3

$$a_1 \rightarrow b_1$$

$$a_1 \begin{cases} c_1 \\ c_2 \end{cases}$$

$A \rightarrow B$ ؟ بله

$A \rightarrow C$ ؟ خیر

$B \rightarrow A$ ؟ خیر

$B \rightarrow C$ ؟ خیر



نکات: 

(۱) صفات طرفین FD می‌توانند ساده یا مرکب باشند.

(۲) اگر $A \rightarrow B$ ، لزوماً نداریم: $B \rightarrow A$.

(۳) اگر $B \subseteq A$ ، به $A \rightarrow B$ ، FD نامهم یا بدیهی (Trivial) گوییم.

(۴) اگر K در رابطه R ، SK یا CK باشد و $G \subseteq H_R$ آنگاه داریم: $K \rightarrow G$.

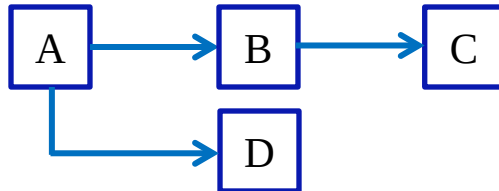


(۵) نمایش FDهای رابطه R به روشهای مختلف:

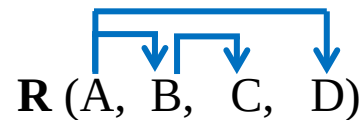
- به صورت یک مجموعه:

$$F = \{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$$

- با نمودار FDها:



- روی خود عنوان رابطه با استفاده از فلش‌هایی:





(۶) **تفسیر FD**: هر FD نمایشگر یک قاعده معنایی از محیط است: نوعی قاعده جامعیتی (که باید به نحوی به سیستم داده شود. خواهیم دید که در بحث طراحی، از طریق طراحی خوب به سیستم می‌دهیم).

□ **تمرین**: در رابطه $R(X, Y, Z)$ ، یک اظهار بنویسید که قاعده معنایی $X \rightarrow Y$ را پیاده‌سازی نماید.

(به طور مثال می‌توان از EXISTS استفاده کرد)

CREATE ASSERTION XTOYFD

CHECK (NOT EXISTS (SELECT X FROM R GROUP BY X HAVING MAX(Y) != MIN(Y)))

CONSTRAINT XTOYFD FORALL R1 (FORALL R2 IF R1.X=R2.X THEN R1.Y=R2.Y) حساب رابطه‌ای:

$STID \rightarrow STJ$: یک دانشجو فقط می‌تواند در یک رشته تحصیل کند.

$STJ \rightarrow STD$: یک رشته فقط در یک دانشکده ارائه می‌شود.

$STID \rightarrow STD$: یک دانشجو فقط در یک دانشکده تحصیل می‌کند.





قواعد استنتاج آرمسترانگ ☐

- 1- if $B \subseteq A$ then $A \rightarrow B \Rightarrow A \rightarrow A$ (قاعده انعکاسی)
- 2- if $A \rightarrow B$ and $B \rightarrow C$ then $A \rightarrow C$ (قاعده تعدی یا تراگذاری)
- 3- if $A \rightarrow B$ then $(A, C) \rightarrow (B, C)$ (قاعده افزایش)
-
- 4- if $A \rightarrow (B, C)$ then $A \rightarrow B$ and $A \rightarrow C$ (قاعده تجزیه)
- 5- if $A \rightarrow B$ and $C \rightarrow D$ then $(A, C) \rightarrow (B, D)$ (قاعده ترکیب)
- 6- if $A \rightarrow B$ and $A \rightarrow C$ then $A \rightarrow (B, C)$ (قاعده اجتماع)
- 7- if $A \rightarrow B$ and $(B, C) \rightarrow D$ then $(A, C) \rightarrow D$ (قاعده شبه تعدی)



□ سه قاعده اول **درست** و **کامل** هستند، بدین معنا که با داشتن یک مجموعه از وابستگی‌های تابعی F ،

تمام وابستگی‌های تابعی منطقاً قابل استنتاج از F ، با همین سه قاعده به دست می‌آیند و هیچ

وابستگی تابعی دیگر (که از F قابل استنتاج نباشد) نیز به دست نمی‌آید.

□ **توجه:** درستی سه قاعده اول به آسانی قابل اثبات است و قواعد دیگر از روی همانها اثبات می‌شوند.



تمرین: قاعده ۲ را اثبات کنید (با استفاده از برهان خلف).

اثبات: فرض خلف: گیریم که $A \not\rightarrow C$. در این صورت در رابطه R در حداقل دو تاپل، به ازای یک مقدار A ، دو مقدار متمایز از C داریم.

اما به ازای دو مقدار متمایز C ، مقدار B ممکن است دو مقدار متمایز با یک مقدار باشد.

R (A, B, C)			→	R (A, B, C)				R (A, B, C)		
a ₁	...	c ₁		a ₁	b ₁	c ₁		a ₁	b ₁	c ₁
		...				...				...
a ₁	...	c ₂		a ₁	b ₂	c ₂		a ₁	b ₁	c ₂

حالت اول

حالت دوم

در حالت اول، فرض $A \rightarrow B$ و در حالت دوم، فرض $B \rightarrow C$ نقض می‌شود. پس فرض خلف باطل است و حکم برقرار است.



❑ کاربردهای قواعد آرمسترانگ

۱- محاسبه بستار صفت A : A^+

مجموعه تمام صفاتی که با A ، وابستگی تابعی دارند.

نکته: اگر $A \Leftarrow A^+ = H_R$ سوپرکلید (الگوریتم تشخیص سوپرکلید و نه کلید کاندید)

۲- محاسبه بستار مجموعه وابستگی‌های تابعی یک رابطه: F^+

مجموعه تمام FDهایی که از F منطقاً استنتاج می‌شوند:

$$F = \{A \rightarrow B, B \rightarrow C\} \Rightarrow F^+ = \{A \rightarrow B, B \rightarrow C, A \rightarrow C, (A, C) \rightarrow (B, C), \dots\}$$



□ کاربردهای مهم F^+ :

۱- تشخیص معادل بودن دو مجموعه از FDهای رابطه‌ای R: به طور نمونه F و G

□ شرط معادل بودن: $F^+ = G^+$

هر FD که از F به دست آید، از G هم به دست می‌آید.

۲- تشخیص FD افزونه

□ ضابطه تشخیص: وابستگی تابعی $f \in F$ را افزونه گوئیم، هرگاه: $(F-f)^+ = F^+$

□ یعنی بود و نبود f در محاسبه F^+ تاثیری نداشته باشد.



۳- محاسبه مجموعه کاهش‌ناپذیر FD های یک رابطه

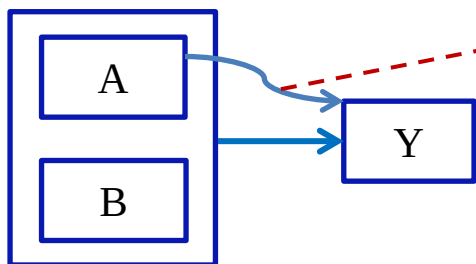
سه شرط دارد:

۱- هیچ FD در آن افزونه نباشد.

۲- سمت راست هر FD، صفت ساده باشد.

۳- سمت چپ هر FD، خود کاهش‌ناپذیر باشد: در وابستگی تابعی $X \rightarrow Y$ ، X را کاهش‌ناپذیر (و وابستگی $X \rightarrow Y$ را **کامل**) گوئیم، هرگاه Y با هیچ زیرمجموعه از X (غیر از خود X)، FD نداشته باشد. در غیر اینصورت X را کاهش‌پذیر گوئیم و وابستگی $X \rightarrow Y$ را **ناکامل** گوئیم.

اگر وجود داشته باشد، آنگاه X کاهش‌پذیر و $X \rightarrow Y$ یک FD ناکامل است.



$$\begin{cases} \overbrace{(A, B)}^X \rightarrow Y \\ A \rightarrow Y \end{cases} \Rightarrow \text{FD ناکامل}$$



□ **تمرین:** اگر یک FD کامل به صورت $A \rightarrow Y$ داشته باشیم، آنگاه FD ناکامل $(A,B) \rightarrow Y$ از آن قابل استنتاج است.

□ اثبات: با استفاده از قاعده افزایش از $A \rightarrow Y$ نتیجه می‌گیریم $(A,B) \rightarrow (Y,B)$

با استفاده از قاعده تجزیه داریم: $(A,B) \rightarrow B$ که یک FD بدیهی است و $(A,B) \rightarrow Y$ که همان حکم است.

کجای؟ مجموعه کاهش‌ناپذیر چه کاربردی دارد؟

تاریخ **وابستگی تابعی با واسطه (TFD):** اگر $A \rightarrow B$ ، $B \rightarrow C$ و $B \nrightarrow A$ ، می‌گوییم C با A، FD با واسطه از طریق B دارد.

اگر $B \rightarrow A$ هم برقرار باشد، آنگاه آن FD با واسطه، بدیهی (نامهم) است.



□ **توجه:** در سه فرم کلاسیک کادی، فقط با مفهوم کلید اصلی (PK) کار می‌کنیم و نه هر CK.

1NF: رابطه R در 1NF است اگر و فقط اگر تمام صفات آن تک‌مقداری باشد.

□ این تعریف می‌گوید هر رابطه نرمال در 1NF است.

2NF: رابطه R در 2NF است اگر و فقط اگر در 1NF باشد و هر صفت ناکلید (که خود PK یا CK نباشد و جزء CK یا PK هم نباشد) در آن، با کلید اصلی رابطه، FD کامل داشته باشد.

□ به بیان دیگر در این رابطه FD ناکامل با کلید اصلی نداشته باشیم.

□ الگوریتم تبدیل 1NF به 2NF: حذف FDهای ناکامل از طریق تجزیه عمودی رابطه به طور مناسب.

3NF: رابطه R در 3NF است اگر و فقط اگر در 2NF باشد و هر صفت ناکلید با کلید اصلی رابطه، فقط بی‌واسطه داشته باشد.

FD بی‌واسطه داشته باشد (FD با واسطه نداشته باشد).

□ الگوریتم تبدیل 2NF به 3NF: حذف FDهای با واسطه.



فرم‌های نرمال کلاسیک کادی (ادامه)

بخش هشتم: طراحی پایگاه داده رابطه‌ای

۵۰

مثالی قید می‌کنیم و در آن تا 3NF پیش می‌رویم.



در حالت کلی، تمام صفات دانشجو، درس و انتخاب در یک رابطه می‌توانند باشند.

قواعد محیط:

R (STID, COID, STJ, STD, GR)

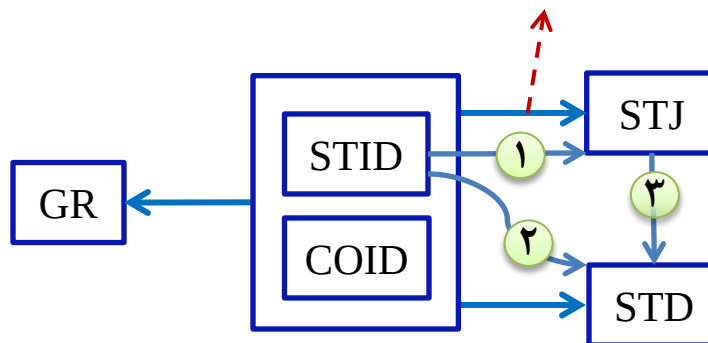
777	CO1	Phys	D11	19
777	CO2	Phys	D11	16
777	CO3	Phys	D11	11
888	CO1	Math	D12	16
888	CO2	Math	D12	18
444	CO1	Math	D12	13
555	CO1	Phys	D11	14
555	CO2	Phys	D11	12

۱- یک دانشجو در یک رشته تحصیل می‌کند.

۲- یک دانشجو در یک دانشکده تحصیل می‌کند.

۳- یک رشته در یک دانشکده ارائه می‌شود.

FDهای ناشی از PK (سمت چپ PK)





□ رابطه **R** در 1NF است (چون همه صفات تک مقداری هستند) ولی آنومالی دارد و باید نرمال‌تر شود.

□ آنومالی‌های رابطه **R**:

۱- در درج:

درج کن این فقره اطلاع درمورد یک دانشجو را: ⟨'666', 'chem', 'D16'⟩

درج ناممکن: تا ندانیم حداقل یک درسی که گرفته شده چیست.

۲- در حذف:

فرض می‌کنیم '444' در این لحظه فقط همین تک درس را داشته باشد.

حذف کن فقط این اطلاع را: ⟨'444', 'CO1', 13⟩

حذف انجام می‌شود اما اطلاع ناخواسته هم حذف می‌شود.

۳- در بهنگام‌سازی:

تغییر رشته تحصیلی دانشجو با شماره 777 به Chem.

برای انجام آن فزونکاری داریم؛ بهنگام‌سازی منتشرشونده (Propagating Update).



□ دلیل آنومالی‌های رابطه R:

□ از دیدگاه عملی: پدیده اختلاط اطلاعات، یعنی اطلاعات در مورد خود موجودیت دانشجو با اطلاعات در مورد انتخاب درس مخلوط شده است.

□ از دیدگاه تئوری: وجود FDهای ناکامل

$$\left\{ \begin{array}{l} (STID, COID) \rightarrow STJ \\ STID \rightarrow STJ \end{array} \right.$$

$$\left\{ \begin{array}{l} (STID, COID) \rightarrow STD \\ STID \rightarrow STD \end{array} \right.$$

□ این FDهای ناکامل باید از بین بروند. برای این منظور رابطه **R** را باید چنان تجزیه عمودی کنیم که در رابطه‌های حاصل، FD ناکامل نباشد.

□ برای این کار از عملگر **پرتو** استفاده می‌کنیم. پرتوی که منجر به یک **تجزیه خوب** شود.



بخش هشتم: طراحی پایگاه داده رابطه‌ای

$\Pi_{\langle \text{STID}, \text{COID}, \text{GR} \rangle}(\text{R})$



SCG (STID, COID, GR)

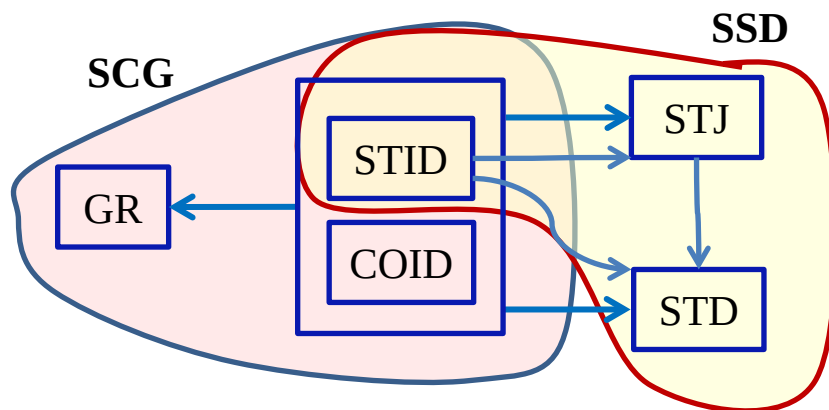
777	CO1	19
777	CO2	16
777	CO3	11
888	CO1	16
888	CO2	18
444	CO1	13
555	CO1	14
555	CO2	12

$\Pi_{\langle \text{STID}, \text{STJ}, \text{STD} \rangle}(\text{R})$

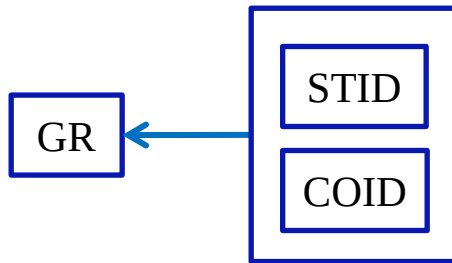


SSD (STID, STJ, STD)

777	Phys	D11
888	Math	D12
444	Math	D12
555	Phys	D11



SCG



رابطه‌های جدید آنومالی‌های R را ندارند: □

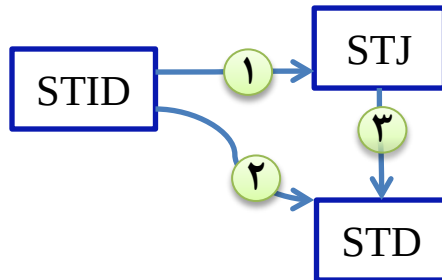
۱- درج کن: <'666', 'chem', 'D16'>

بدون مشکل در SSD درج می‌شود.

۲- حذف کن: <'444', 'CO1', 13>

بدون مشکل از SCG حذف می‌شود.

SSD



۳- بهنگام‌سازی کن: تغییر رشته دانشجوی 777 را به Chem

بدون مشکل در SSD بروز می‌شود.



فرم‌های نرمال کلاسیک کادی (ادامه)

۵۵

بخش هشتم: طراحی پایگاه داده رابطه‌ای

□ در طراحی جدید، FDهای ناکامل از بین رفتند. بنابراین SSD و SCG، 2NF هستند.

□ **تاکید:** رابطه R، 2NF است هرگاه اولاً در 1NF باشد و ثانیاً هر صفت ناکلید با کلید اصلی، FD کامل

داشته باشد (رابطه، FD ناکامل نداشته باشد).

□ **تمرین:** بررسی شود که آیا در این تجزیه همه FDها محفوظ می‌مانند؟

□ **نکته:** باید توجه کنیم که در تجزیه، FDای از دست نرود، چون هر FD یک قاعده جامعیت در محیط است.

□ توجه داشته باشید که در این تجزیه هیچ اطلاعی از دست نمی‌رود. یعنی اگر کاربر رابطه اصلی را به هر

$$R = SCG \bowtie SSD$$

دلیلی بخواهد، با پیوند دو رابطه جدید به دست می‌آید.



□ آیا رابطه‌های جدید (SSD و SCG) آنومالی ندارند؟

□ آنومالی‌های SSD:

۱- در درج:

اطلاع: «رشته IT در دانشکده D20 ارائه می‌شود.» به دلیل FD شماره ۳، این اطلاع منطقاً باید قابل درج باشد، اما درج ناممکن است. چون کلید ندارد، باید حداقل یک دانشجوی این رشته را بشناسیم.

۲- در حذف:

حذف کن «'Chem'، '666'» و با فرض اینکه تنها یک دانشجو در رشته Chem ثبت شده است. حذف انجام می‌شود ولی اطلاع «رشته شیمی در D16 ارائه می‌شود»، ناخواسته حذف می‌شود.

۳- در بهنگام‌سازی:

«شماره دانشکده رشته فیزیک را عوض کنید». به تعداد تمام دانشجویان این رشته باید بهنگام‌سازی شود.

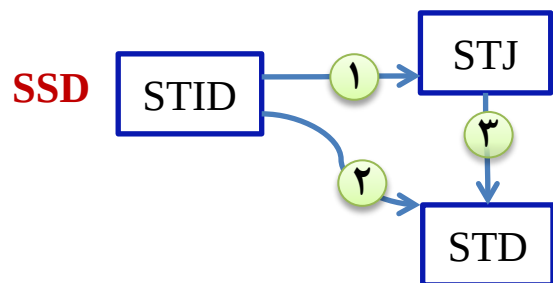
SSD باید نرمال‌تر شود.





□ دلیل آنومالی‌های SSD:

□ دلیل آنومالی‌های SSD، وجود FD با واسطه بین صفت ناکلید با کلید اصلی است (به دلیل FD شماره ۳).



□ این FD باید از بین برود.

□ فرض کنید SSD را به صورت زیر تجزیه کنیم:

$SJ (STID, STJ)$ و $SD (STJ, STD)$

777	Phys	Phys	D11
888	Math	Math	D12
444	Math		
555	Phys		

□ افزودن کم شد!

□ تمرین: بررسی شود که رابطه‌های جدید آنومالی‌های SSD را ندارند.

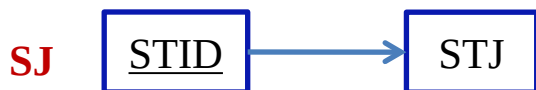


فرم‌های نرمال کلاسیک کادی (ادامه)

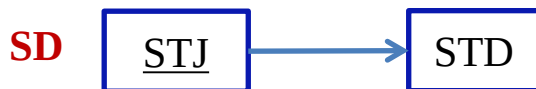
۵۸

بخش هشتم: طراحی پایگاه داده رابطه‌ای

این رابطه‌ها در 3NF هستند. ☐



اولاً در 2NF هستند. ☐



ثانیاً FD با واسطه نداریم. ☐

تمرین: بررسی شود که در این تجزیه هیچ اطلاعی از دست نمی‌رود و FDها هم حفظ می‌شوند. ☐

تاکید: رابطه R در 3NF است اگر و فقط اگر اولاً در 2NF باشد و ثانیاً هر صفت ناکلید با کلید اصلی FD ☐

بی‌واسطه داشته باشد (تمام FDها مستقیماً ناشی از PK باشد).

نتیجه: FDهای ناکامل و باواسطه مزاحم هستند و باید از بین بروند. ☐

در عمل رابطه‌ها باید حداقل تا 3NF نرمال شوند و خواهیم دید حتی‌الامکان در BCNF یا بیشتر باشند. ☐

در رابطه 3NF داریم که «یک بوده (واقعیت): یک رابطه» و یا «یک شیء: یک رابطه». ☐



□ در حالت کلی اگر $R_1, R_2, \dots, R_n$ پرتوهای دلخواه از R باشند، به شرط عدم وجود هیچمقدار داریم (ممکن است تاپل‌های افزونه بروز کند):

$$R \subseteq R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$$

□ **تجزیه بی حذف:** شرطش این است که **در صفات پیوند هیچمقدار (Null Value)** نداشته باشیم.

□ اگر در صفات پیوند هیچمقدار داشته باشیم، چه پیش می‌آید؟

$$T(\underline{A}, B, C, D, E) \Rightarrow T_1(A, B) \quad T_2(B, C, D, E)$$

تاپل‌هایی در پیوند از دست می‌روند. به این تاپل‌ها، تاپل‌های آونگان [معلق] (Dangling) گوییم.

□ در مباحث نرمالترسازی معمولاً فرض بر این است که **صفت (صفات) پیوند هیچمقدار ندارند**.



□ تجزیه خوب (Nonloss/Lossness Decomposition)

۱- بی‌حشو: در پیوند پرتوها، تاپل حشو [افزونه] بروز نکند.

۲- حافظ FDها: هیچ FDای در اثر تجزیه از دست نرود و همه FDهای رابطه اصلی حفظ شوند.

۳- بی‌حذف: در پیوند پرتوها هیچ تاپلی حذف نشود (صفت یا صفات پیوند هیچمقدار نباشند).

۴- حافظ صفات: $\bigcup_{i \in \{1, \dots, n\}} H_{R_i} = H_R$

پیش‌فرض یا بدیهی

□ در بیشتر متون کلاسیک، **تجزیه بی‌حشو** تحت عنوان **تجزیه بی‌کاست یا بی‌گمشدگی**

(Nonloss/Lossless Decomposition) مطرح شده است که به همراه خاصیت حفظ وابستگی‌های تابعی،

تجزیه خوب را شکل می‌دهد (دو ویژگی دیگر تجزیه خوب را پیش‌فرض تجزیه خوب می‌دانیم).

□ در واقع تاپلهای افزونه باعث از دست رفتن بخشی از اطلاعات می‌شوند.



□ قضیه ريسانن (Rissanen):

□ رابطه R به دو پرتوش $(R_1$ و $R_2)$ **تجزیه خوب** می‌شود، اگر R_1 و R_2 از یکدیگر مستقل باشند.

□ R_1 و R_2 **مستقل** از یکدیگرند اگر و فقط اگر:

- صفت مشترک، حداقل در یکی از آنها CK باشد $\Leftarrow$ بی‌حشو بودن

- تمام FDهای رابطه اصلی یا در مجموعه FDهای R_1 و R_2 وجود داشته باشند یا از آنها منطقاً

استنتاج شوند $\Leftarrow$ حافظ FDها

□ **نکته:** بر اساس ضوابط ريسانن، اگر در رابطه $R(A, B, C)$ ، وابستگی‌های $A \rightarrow B$ ، $B \rightarrow C$ و $A \rightarrow C$ برقرار

باشد، در اینصورت تجزیه خوب چنین است: $R_1(\underline{A}, B)$ و $R_2(B, \underline{C})$.

□ در اینجا صفت مشترک B در رابطه دوم کلید کاندید است، چون همه صفات به آن وابستگی تابعی

دارند و کاهش‌پذیر هم نیست.



□ مثال: رابطه SSD را در نظر می‌گیریم. این رابطه به سه شکل به پرتوهای دوگانی قابل تجزیه است.

I SS (STID, STJ) SD (STJ, STD)

II SS (STID, STJ) SD (STID, STD)

III SS(STID, STD) SJ (STJ, STD)

□ تجزیه I خوب است، چون هر دو شرط ریساین را دارد.

$$\left. \begin{array}{l} \text{STID} \rightarrow \text{STJ} \\ \text{STJ} \rightarrow \text{STD} \end{array} \right\} \Rightarrow \text{STID} \rightarrow \text{STD}$$

□ تجزیه II خوب نیست، چون FD از دست می‌دهد.

□ تجزیه III خوب نیست، چون FD از دست می‌دهد.



❑ **اصطلاح:** در وابستگی تابعی $A \rightarrow B$ (A Determines B) به A دترمینان گویند.

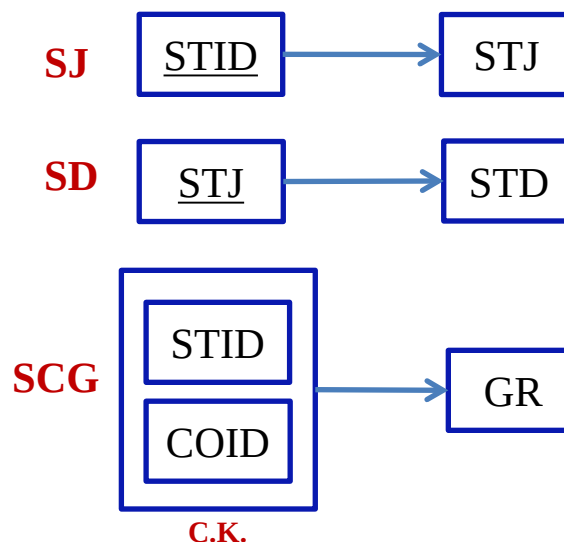
❑ **تعریف:** رابطه R در BCNF است اگر و فقط اگر در آن دترمینان هر FD مهم و کاهش‌ناپذیر، CK باشد.

❑ در 3NF، تنها باید دترمینان رابطه PK باشد.

❑ چون رابطه می‌تواند بیش از یک CK داشته باشد، BCNF از 3NF قوی‌تر است.

❑ **مثال:** رابطه‌های زیر در BCNF هستند.

SCGJD {
SCG(SID, COID, GR)
SJ (STID, STJ)
SD (STJ, STD)





☐ BCNF از 3NF قوی‌تر است. $\Leftarrow$ رابطه می‌تواند در 3NF باشد، اما در BCNF نباشد.

☐ **حالت I:** رابطه R فقط یک CK داشته باشد. $\Leftarrow$ اگر R در 3NF باشد، در BCNF هم هست (مثال دیده شده).

☐ **حالت II:** رابطه R بیش از یک CK داشته باشد.

☐ **II-1)** CKها مجزا باشند (صفت مشترک نداشته باشند). $\Leftarrow$ اگر R در 3NF باشد، در BCNF هم هست.

☐ **II-2)** CKها هم‌پوشا باشند. $\Leftarrow$ اگر R در 3NF باشد، لزوماً در BCNF نیست.



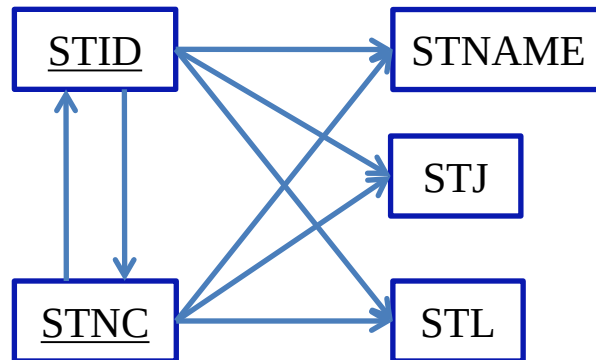
برای حالت II-1



ST (STID, STNAME, STNC, STJ, STL, ...)

C.K.

C.K.

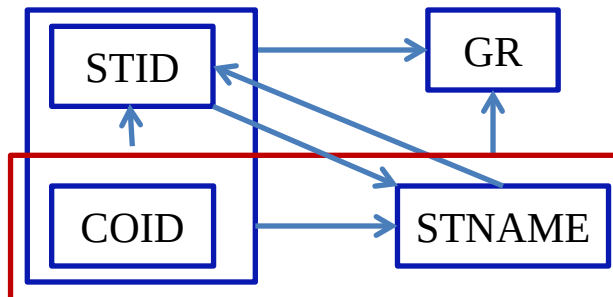


دو دترمینان، هر دو هم CK هستند.

SCNG (STID, COID, STNAME, GR)

C.K.

C.K.



برای حالت II-2



(فرض: هیچ دو دانشجویی نام یکسان ندارند.)



□ کافی است یک دترمینان در رابطه پیدا کنیم که CK نباشد. $\Leftarrow$ رابطه BCNF نیست.

□ پس در کدام فرم نرمال است؟

□ 1NF هست. چون صفت‌ها تک‌مقداری هستند.

□ 2NF هست. چون FD ناکامل نداریم. $\Leftarrow$ هر صفت ناکلید با کلید اصلی FD ناکامل نداشته باشد.

$\Leftarrow$ در اینجا STNAME صفت غیرکلید نیست، پس FD ناکامل نیست.

□ 3NF هست. چون FD با واسطه با کلید اصلی نداریم.

□ آیا این رابطه تجزیه می‌شود؟

$\left\{ \begin{array}{l} \text{SCG}(\text{STID}, \text{COID}, \text{GR}) \\ \text{SSN}(\text{STID}, \text{STNAME}) \end{array} \right. \Rightarrow \text{هر دو BCNF هستند.}$
C.K. C.K.

□ آیا طرز دیگر هم می‌شود تجزیه کرد؟ بله، به جای STID در SCG، STNAME بگذاریم.



☐ نشان دهید که این تجزیه خوب است؛ یعنی با پیوند پرتوها، رابطه اصلی به دست می‌آید و هیچ FD از دست نمی‌رود.

☐ چه پدیده‌ای در اینجا دیده می‌شود؟ این رابطه اختلاط اطلاعات دارد! با این همه 3NF است.

SCNG (STID, COID, STNAME, GR)

C.K.

C.K.

☐ **نکته:** صرف وجود اختلاط اطلاعات ایجاب می‌کند که رابطه در فرم نرمال ضعیفی باشد.

☐ **تمرین:** محیط دانشکده، قواعد معنایی:

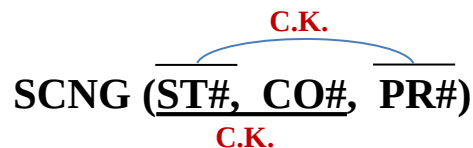
۱- یک دانشجو یک درس را با یک استاد انتخاب می‌کند.

۲- یک استاد فقط یک درس تدریس می‌کند.

۳- یک درس توسط بیش از یک استاد ارائه می‌شود.



□ فرض می‌کنیم طراح رابطه زیر را طراحی کرده است.



□ این رابطه در کدام فرم نرمال است؟

□ ابتدا باید با استفاده از قواعد، CKها را مشخص کنیم. سپس نمودار FD را رسم کنیم.

□ آیا این رابطه، تجزیه خوب دارد؟

□ **نکته:** اگر رابطه مثلاً 3NF باشد و تجزیه خوب نداشته باشد، نباید تجزیه کنیم تا رابطه‌های حاصل

BCNF باشد.

□ رابطه فوق در 3NF است و از نکته فوق این نتیجه مهم به دست می‌آید که این رابطه تجزیه خوب ندارد.



قضیه هیت (Heath): در رابطه $R(A, B, C)$ ، که در آن A ، B و C سه مجموعه از صفات هستند، اگر $A \rightarrow B$ (در F^+ باشد)، آنگاه تجزیه R به دو پرتو $R_1(A, B)$ و $R_2(A, C)$ ، **تجزیه بی کاست** (Nonloss) است.

دقت شود که برقراری شرایط **قضیه هیت**، یک تجزیه بی کاست (و نه لزوماً خوب که حافظ FD باشد) را تضمین می‌نماید اما برقراری شرایط **قضیه ريسانن**، یک تجزیه خوب را تضمین می‌نماید. واضح است که در قضیه ريسانن شرایط قضیه هیت نیز برقرار است. بیانی دیگر از **قضیه هیت** تحت عنوان **تست NJB** به صورت زیر است.

تست پیوند بی‌حشو برای تجزیه دودویی (NJB- Nonadditive Join Test for Binary Decompositions):

تجزیه دودویی $D = \{R_1, R_2\}$ از رابطه R خاصیت پیوند بی‌حشو دارد اگر و تنها اگر یکی از موارد زیر با توجه به مجموعه FDهای F برقرار باشد:

- وابستگی تابعی $(R_1 - R_2) \rightarrow (R_1 \cap R_2)$ در F^+ باشد یا

- وابستگی تابعی $(R_2 - R_1) \rightarrow (R_1 \cap R_2)$ در F^+ باشد.



4NF: رابطه R در 4NF است اگر و فقط اگر در BCNF باشد و وابستگی چندمقداری (MVD) مهم در آن وجود نداشته باشد.

وابستگی چندمقداری (MVD): در رابطه $R(A, B, C)$ (رابطه با سه صفت یا سه مجموعه صفت)، صفت B با صفت A، MVD دارد $(A \twoheadrightarrow B)$ اگر و فقط اگر به ازای یک مقدار A، مجموعه‌ای از مقادیر B متناظر باشد.

[یعنی به ازای هر جفت مشخص از (A,C)، مجموعه مقادیر B فقط با تغییرات A تغییر کند.]

به فرم نرمال
 $R(A, B, C)$

a_1	b_1	c_1
a_1	b_2	c_1
a_1	b_3	c_1
a_1	b_1	c_2
a_1	b_2	c_2
a_1	b_3	c_2
a_2	b_1	c_i
a_2	b_7	c_i

به فرم غیرنرمال
 $R(A, B, C)$

a_1	$\begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$	c_1
a_1	$\begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$	c_2
a_2	$\begin{bmatrix} b_1 \\ b_7 \end{bmatrix}$	c_i



$A \twoheadrightarrow B$



نکات: □

۱- اگر $B \subseteq A$ باشد، به $A \rightarrow \rightarrow B$ می‌گوییم MVD بدیهی [نامهم]

اگر $A \cup B = H_R$ باشد، به $A \rightarrow \rightarrow B$ می‌گوییم MVD بدیهی [نامهم]

۲- MVD در رابطه‌های با سه صفت [ساده یا مرکب] همیشه جفت است.

If $A \rightarrow \rightarrow B$ then $A \rightarrow \rightarrow (H - \{A, B\})$ یا $A \rightarrow \rightarrow C$

برای اثبات این نکته کافی است به جای یک جفت مقدار از (A, C) ، یک جفت (A, B) را بگیریم، آن مجموعه برای C تشکیل می‌شود.

۳- برای MVD هم قواعد آرمسترانگ وجود دارد که با قواعد مربوط به FDها متفاوت است.



استاد از دانشجو گزارش آزمایشگاه می‌گیرد.



رابطه غیرنرمال با صفت چندمقداری

NNPSR (PR#, ST#, RE#)

□ در این محیط یک قاعده معنایی خاص وجود دارد: یک استاد از هر یک از دانشجویان یک گروه، هر

یک از گزارش‌های یک مجموعه گزارش را می‌گیرد.

□ اگر این قاعده معنایی نباشد، این مجموعه‌ها شکل نمی‌گیرد.

NNPSR (PR#, ST#, RE#)

$PR_1 \left[\begin{matrix} 777 \\ 888 \\ 444 \end{matrix} \right] \left[\begin{matrix} R_1 \\ R_2 \end{matrix} \right]$

$PR_2 \left[\begin{matrix} 777 \\ 666 \end{matrix} \right] \left[R_3 \right]$

...



رابطه غیرنرمال با صفت چندمقداری

NNCTX (C#, T#, B#)

c ₁	t ₁	b ₁
	t ₂	b ₂
	t ₃	
c ₂	t ₄	b ₃
		b ₅
	t ₂	b ₇

CTX (C#, T#, B#)

c ₁	t ₁	b ₁
c ₁	t ₂	b ₁
c ₁	t ₃	b ₁
c ₁	t ₁	b ₂
c ₁	t ₂	b ₂
c ₁	t ₃	b ₂
c ₂	t ₄	b ₃
c ₂	t ₂	b ₃
c ₂	t ₄	b ₅
c ₂	t ₂	b ₅
c ₂	t ₄	b ₇
c ₂	t ₂	b ₇

درس C توسط استاد T از روی کتاب B ارائه می‌شود.



پدیده MVD بیان فرمال صفت چندمقداری است. □

فرم نرمال شده این مثال، افزونگی زیادی دارد.



رابطه تمام کلید است؛ یعنی هیچ یک به تنهایی و □

هیچ ترکیب دوتایی آن CK نیست.

رابطه تمام کلید حداقل BCNF است. □

زیرا یک دترمینان دارد که آن هم CK است.



با این همه رابطه اخیر آنومالی دارد. ☐

☐ **در درج:** در درس c_1 ، کتاب b_8 نیز به عنوان مرجع درس ثبت شود.

نمی‌توانیم بگوییم چون کلید نداریم نمی‌توانیم درج کنیم. باید قواعد معنایی رعایت شود.

باید درج کنیم: $\langle c_1, t_1, b_8 \rangle$

$\langle c_1, t_2, b_8 \rangle$

$\langle c_1, t_3, b_8 \rangle$

یعنی عمل منطقاً تاپلی تبدیل شده به عمل مجموعه‌ای

☐ در حذف و بهنگام‌سازی هم به دلیل وجود افزونگی، آنومالی داریم.

☐ رابطه CTB باید تجزیه شود تا رابطه‌های حاصل 4NF شود.



□ دلیل آنومالی این رابطه، وجود پدیده MVD است.

$$\left\{ \begin{array}{l} C\# \rightarrow\rightarrow B\# \\ C\# \rightarrow\rightarrow T\# \end{array} \right.$$

□ پس CTB را باید چنان تجزیه کنیم که در رابطه‌های حاصل، MVD وجود نداشته باشد.

□ برای این کار CTB را پرتوگیری می‌کنیم به نحوی که در عنوان هر پرتو، مبدأ MVD وجود داشته باشد.

CT (C#, T#)

c ₁	t ₁
c ₁	t ₂
c ₁	t ₃
c ₂	t ₄
c ₂	t ₂

CB (C#, B#)

c ₁	b ₁
c ₁	b ₂
c ₂	b ₃
c ₂	b ₅
<u>c₂</u>	<u>b₇</u>
c ₁	b ₈

درج به صورت عملاً تاپلی و نه مجموعه‌ای

□ رابطه‌های جدید آنومالی CTB را ندارند.

□ این دو رابطه جدید BCNF هستند، چون تمام کلید هستند. MVD مهم ندارند، پس 4NF هستند.

□ **تمرین:** نشان دهید با پیوند این دو رابطه، رابطه اصلی به دست می‌آید.



❑ قضیه فاگین (Fagin): رابطه $R(A, B, C)$ به دو پرتوش $R_1(A, B)$ و $R_2(A, C)$ تجزیه بی کاست

(Nonloss) می‌شود اگر و فقط اگر $A \rightarrow\rightarrow B$.

❑ قضیه فاگین (برای MVD) تعمیم قضیه هیث (برای FD) است.

❑ آیا می‌توان گفت مفهوم MVD تعمیم مفهوم FD است؟ آیا می‌توان گفت FD حالت خاصی از MVD است؟

❑ FD حالت خاصی از MVD است که در آن مجموعه مقادیر صفت وابسته، تک عنصری هستند.

❑ همچنین این استنتاج منطقی را هم داریم:

If $A \rightarrow B$ then $A \rightarrow\rightarrow B$



❑ **نکته:** بحث 4NF از یک دیدگاه می‌تواند اصلاً موضوعیت نداشته باشد. زیرا رابطه‌ای که BCNF باشد و

MVD داشته باشد قطعاً صفت چندمقداری دارد و می‌دانیم در طراحی برای صفات چندمقداری، از همان

ابتدا می‌توان رابطه‌های جداگانه طراحی کرد.

❑ با این همه مفهوم MVD به عنوان بیان فرمال صفت چندمقداری قابل توجه است.

تعریف زاینولو از 3NF، BCNF، 4NF و ... مطالعه شود.





وابستگی پیوندی (JD): رابطه R وابستگی پیوندی به n پرتو $R_1, R_2, \dots, R_n$ دارد اگر و فقط اگر R



حاصل پیوند بی‌حشو این n پرتو باشد.

$$R = [JD]^*(R_1, R_2, \dots, R_n)$$

$$CTB = [JD]^*(CT, CB)$$



JD را نامهم گوئیم هرگاه عنوان (Heading) یکی از R_i ها همان عنوان (Heading) رابطه R باشد. □

5NF [PJNF] - فرم نرمال پرتو پیوندی: رابطه R در 5NF است اگر و فقط اگر تمام JDهای آن ناشی



از CK باشد. $\Leftarrow$ ناشی از CK بودن یعنی عنوان همه پرتوها، در همه JDها، سوپرکلید باشد.

□ رابطه CTB در 5NF نیست، چون $(C\#, T\#)$ و $(C\#, B\#)$ سوپرکلید رابطه CTB نیستند.



STUD (STID, STNAME, STJ, STL)



□ فرض می‌کنیم که 3NF هست و FD مزاحم نداریم.

$$\left\{ \begin{array}{l} \text{STN} (\underline{\text{STID}}, \text{STNAME}) \\ \text{SJL} (\underline{\text{STID}}, \text{STJ}, \text{STL}) \end{array} \right. \Rightarrow \text{STUD} = [\text{JD}] * (\text{STN}, \text{SJL}) \quad \text{JD به دو پرتو}$$

$$\left\{ \begin{array}{l} \text{STN} (\underline{\text{STID}}, \text{STNAME}) \\ \text{SJ} (\underline{\text{STID}}, \text{STJ}) \\ \text{SL} (\underline{\text{STID}}, \text{STL}) \end{array} \right. \Rightarrow \text{STUD} = [\text{JD}] * (\text{STN}, \text{SJ}, \text{SL}) \quad \text{JD به سه پرتو}$$

□ رابطه STUD در 5NF است. چون عنوان همه پرتوها در همه JDهای آن، سوپرکلید هستند (ناشی از کلید کاندید هستند).



نکته: اگر رابطه‌ای در 3NF باشد و تمام CKهای آن ساده باشند، آن رابطه در 5NF است.

مثال PCD رابطه‌ای است که در 4NF است ولی در 5NF نیست. JD به دو پرتو ندارد، بلکه به سه پرتوش JD دارد، که هیچکدام سوپرکلید نیستند. این محدودیت، یک محدودیت دائمی و مستقل از زمان است که از محدودیت‌های محیط عملیاتی نشأت گرفته و همواره در بدنه رابطه برقرار است.

PCD (PR#, CO#, D#)

PR ₁	CO ₁	d ₂
PR ₁	CO ₂	d ₁
PR ₂	CO ₁	d ₁
PR ₁	CO ₁	d ₁

استاد PR# درس CO# را در دانشکده D# ارائه می‌دهد.

رابطه SPJ تمام کلید است. $\Leftarrow$ حداقل BCNF

SPJ (S#, P#, J#)

S ₁	P ₁	J ₁
S ₁	P ₁	J ₂
S ₁	P ₂	J ₁
S ₂	P ₁	J ₁

محدودیت موجود: هر گاه تولیدکننده S قطعه p را تولید کند و قطعه p در پروژه j استفاده شود و تولیدکننده S حداقل یک قطعه در پروژه j تولید کرده باشد، آنگاه تولید کننده S قطعه p را پروژه j تولید کرده است.



بخش هشتم: طراحی پایگاه داده رابطه‌ای

فرض می‌کنیم بخواهیم این رابطه را تجزیه کنیم:

SP (S#, P#)

S ₁	P ₁
S ₁	P ₂
S ₂	P ₁

PJ (P#, J#)

P ₁	J ₁
P ₁	J ₂
P ₂	J ₁



SPJ' (S#, P#, J#)

S ₁	P ₁	J ₁
S ₁	P ₁	J ₂
S ₁	P ₂	J ₁
S ₂	P ₁	J ₂
S ₂	P ₁	J ₁

تاپل حشو

SJ (S#, J#)

S ₁	J ₂
S ₁	J ₁
S ₂	J ₁

این رابطه JD به دو پرتوش ندارد.

یک پرتو دیگر هم می‌گیریم:



SPJ (S#, P#, J#)

۱	S ₁	P ₁	J ₁
۲	S ₁	P ₁	J ₂
۳	S ₁	P ₂	J ₁
۴	S ₂	P ₁	J ₁



$$SPJ = [JD] * (SP, PJ, SJ)$$

□ پس SPJ، JD دارد به سه پرتوش و نه کمتر:

و 5NF نیست چون عنوان (Heading) پرتوهایش سوپرکلید نیست.

□ در این مثال از سه فقره اطلاع دو موجودیتی، باید یک اطلاع سه موجودیتی را استنتاج کنیم، چرا که این

یک محدودیت جامعیتی حاکم بر محیط است (وجود وابستگی پیوندی).

□ توجه داشته باشید که در حالت کلی چنین استنتاجی درست نیست و پدیده دام پیوندی حلقه‌ای بروز

می‌کند، ولی در اینجا به دلیل وجود وابستگی پیوندی، چنین مشکلی بروز نمی‌کند.



نکته: در این رابطه یک محدودیت بسیار نادر، موسوم به محدودیت با **ماهیت چرخشی (CC)** وجود دارد.

□ با وجود تاپل‌های دوم تا چهارم در رابطه SPJ باید تاپل (S_1, P_1, J_1) نیز وجود داشته باشد.

□ این محدودیت ناشی از وجود (S_1, P_1) در تاپل دوم، (S_1, J_1) در تاپل سوم و (P_1, J_1) در تاپل چهارم است.

□ در واقع مقدار هر یک از سه صفت در سه تاپل از چهار تاپل رابطه SPJ یکسان است و در هر یک از سه پرتو دوتایی، یک صفت مشترک با دو پرتو دیگر وجود دارد.

□ اگر یک رابطه CC داشته باشد در فرم نرمال 5NF نیست.

□ برای تشخیص این محدودیت در رابطه درجه n دوتست انجام می‌دهیم:

۱- تعداد تاپل‌ها: $n+1$

۲- مقدار هر صفت، در n تاپل یکسان باشد.



در رابطه R هر ترکیب دوتایی CK است. لذا در فرم نرمال 5NF است زیرا:



R (A, B, C)

a ₁	b ₁	c ₂
a ₁	b ₁	c ₁
a ₂	b ₁	c ₁

□ سه دترمینان دارد که هر سه CK هستند. $\Leftarrow$ BCNF است.

□ MVD ندارد. $\Leftarrow$ 4NF است.

□ CC ندارد و همه JDهای آن ناشی از کلید کاندید هستند. $\Leftarrow$ 5NF است.



رابطه R در 6NF است هر گاه اصلاً JD نداشته باشد.



❑ **نکته:** در رابطه درجه n ، اگر غیر از کلید فقط یک صفت دیگر داشته باشد، در 6NF است.

به طور مثال رابطه SPJ که 5NF نبود را به سه رابطه SP، SJ و PJ تجزیه می‌کنیم. این سه رابطه در فرم نرمال 5NF و 6NF هستند.



فرم نرمال DKNF چیست؟





□ تئوری نرمال‌ترسازی به عنوان ابزار طراحی RDB، مزایا و معایبی دارد.

□ مزایای تئوری نرمال‌ترسازی:

۱- ارائه یک طراحی واضح از خُردجهان واقع (Clean Design)؛ یعنی با کمترین اختلاط اطلاعات.

یعنی در واقع رعایت یک اصل در عمل (one fact : one table).

۲- کاهش بعض افزونگی‌ها؛ آن افزونگی‌هایی که با پرتوگیری از بین می‌روند (کاهش می‌یابد).

۳- کاهش بعض آنومالی‌ها [ناشی از اختلاط اطلاعات].

۴- بعض قواعد جامعیت را اعمال می‌کنیم (ناشی از وابستگی بین صفات).

□ این تئوری به طراح کمک می‌کند تا تصمیم بگیرد چند رابطه داشته باشد و هر رابطه عنوانش چه باشد و

کلیدش چه باشد.



❑ معایب تئوری نرمال‌ترسازی:

- ۱- فزون‌کاری در بازیابی (اگر کاربر به هر دلیلی رابطه اصلی را بخواهد، عمل پیوند (Join) باید انجام شود که در حجم بالای داده، سربار زیادی دارد).
- به دلیل همین عیب، گاه در عمل لازم است غیرنرمال‌سازی (Denormalization) انجام دهیم. یعنی تبدیل حداقل دو رابطه $(i+1)NF$ به یک رابطه iNF .
- ۲- فرآیند نرمال‌ترسازی زمان‌گیر است به ویژه اگر مجموعه صفات محیط بزرگ باشد و نمودار FDها گسترده باشد.
- ۳- مبتنی است بر یک فرض نه چندان واقع‌بینانه [فرض: در آغاز مجموعه‌ای از صفات داریم در یک مجموعه Universal، آنگاه با روش سنتز صفات (دسته‌بندی صفات) به تعدادی رابطه می‌رسیم]. در حالیکه در عمل ابتدا روش بالا به پایین و رسیدن به تعدادی رابطه با درجه متعارف، آنگاه استفاده از ایده‌های این تئوری برای تست نرمالیتی (اول تست $3NF$ ، بعد $BCNF$ و $5NF$).



۴- همه وابستگی‌های بین صفات دیده نشده‌اند؛ مثلاً وابستگی شمول دیده نشده است.

۵- ایجاد میزانی افزونگی؛ چون اگر بخواهیم تجزیه خوبی داشته باشیم، یا CK باید در همه پرتوها تکرار شود یا پیوندهای CK-FK وجود داشته باشد!

۶- استفاده محدود از عملگرهای جبر رابطه‌ای. تجزیه ← پرتو بازسازی ← پیوند
حال آنکه در عمل گاه لازم است رابطه را تجزیه افقی کنیم:

$$ST_1 = \sigma_{STJ='Phys'}(STUD)$$

$$ST_2 = \sigma_{STJ='IT'}(STUD)$$

...

$$ST_n = \sigma_{STJ='Comp'}(STUD)$$

$$STUD = \bigcup_{i=1}^n (ST_i)$$



☐ به رابطه‌های ناشی از تجزیه افقی می‌گوییم:

فرم نرمال گزینش اجتماع (تحدید اجتماع) RUNF (Restriction Union Normal Form)

☐ RUNF لزوماً در امتداد فرم‌های نرمال نیست. به موازات آنها مطرح است. یعنی ممکن است رابطه 3NF باشد، تجزیه افقی کنیم و باز هم 3NF باشد.

☐ در چه شرایطی رابطه حاصل از تجزیه افقی از خود رابطه نرمال‌تر است؟



پرسش و پاسخ ...

amini@sharif.edu